

面向可核验双目标序列优化的确定性实验与证据链

摘要

针对具有序列相关准备时间与交期折损收益的工件选择问题，建立收益与时长的双目标模型。采用付费评价账本记录全部搜索观测，使用独立实现重算目标，并在当前小规模有限域上枚举参考前沿。确认阶段基准归一化超体积均值为 0.932828，开发阶段选定方法的确认均值为 0.999866，配对平均变化为 0.0670381。结论仅限当前合成实例与预算，不能外推为跨题目优势。

关键词：双目标优化；序列决策；独立评测；可复现实验

演示说明：本文是合成问题上的工程演示，不是当届国赛解答。模型调用与独立审查为显式测试夹具；数值计算为实际运行。

1 问题重述

在给定工件集合中选取一个非空子集并确定加工顺序。收益随交期延迟折损，总加工时长含相邻工件之间的准备时间。目标是得到收益与时长的折中解，并按事先确定的效用尺度给出代表性建议。该实例为合成数据，不是任何官方完整赛题。

2 假设、符号与数学模型

工件不可重复，设备串行加工，所有时间与收益参数在当前实验中视为确定量。以工件序列作为决策变量，用连续随机键表示可行非空子集。下式分别给出完成时刻、总收益和双目标最小化形式。

$$C_k = C_{k-1} + s_{\pi_{k-1}, \pi_k} + p_{\pi_k} \quad (1)$$

$$R = \sum_{k=1}^{|\pi|} \max\{0, v_{\pi_k} - w_{\pi_k} \max(C_k - d_{\pi_k}, 0)\} \quad (2)$$

$$\min(-R, C_{|\pi|}) \quad (3)$$

3 求解、控制试验与独立验证

随机搜索与定制优化器使用相同的函数评价预算、随机种子集合以及非支配后处理。独立评测器不导入优化器的目标函数，而是重新实现解码、收益计算与有限域枚举。搜索器不读取枚举参考前沿。消融只关闭代理预筛选开关，灵敏度试验只改变种群规模。开发试验用于选择候选，随后冻结候选才进行确认。

4 结果与适用范围

确认阶段的配对平均变化为 0.0670381。以下表格中的数值直接来自经过独立评价的运行记录，而非语言模型补写。代表性运行的推荐序列收益为 32，持续时间为 17。代表性运行按确认得分的中位排序选择，不按最优单次结果挑选。

5 经过计算的证据与结果

证据标识	数值	单位
mean_baseline	0.932828	normalized_hv
mean_c0	0.999866	normalized_hv
mean_improvement	0.0670381	normalized_hv
ci_low	0.0370704	normalized_hv
ci_high	0.084761	normalized_hv
result_hv	1	ratio
result_reward	32	reward_units
result_duration	17	minutes

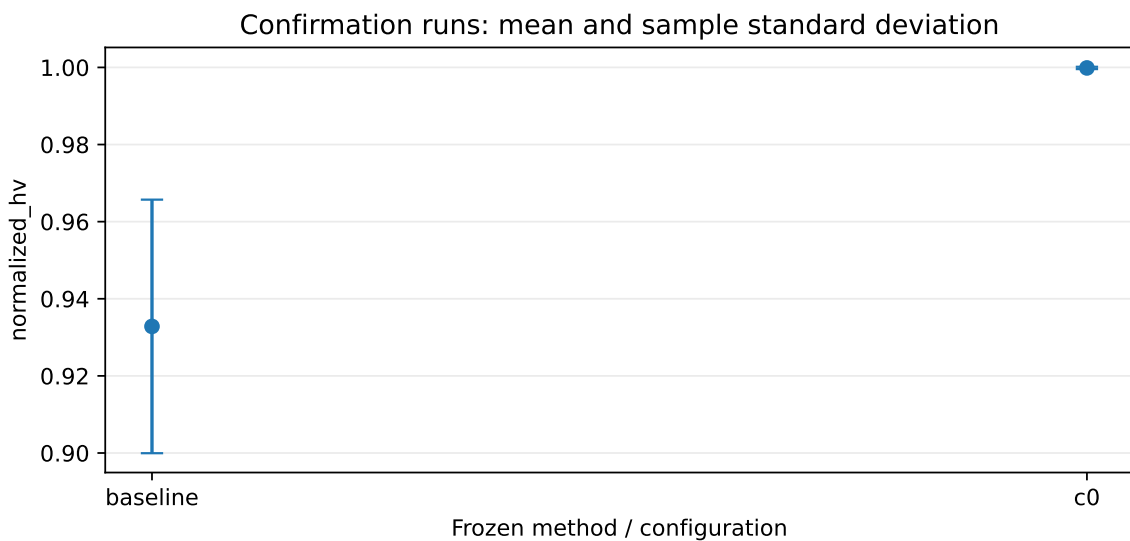


图 1: 确认阶段的实际得分。误差棒为样本标准差，不是置信区间。

6 适用范围与不足

随机种子是当前实例上的重复实验单位，不能视为独立任务样本。

工程演示的模型规划、建议和审查来自测试夹具；这些回执不是实际多模型审查。

精确枚举只覆盖当前声明的有限决策域，不证明现实模型假设正确或其他题目的最优性。

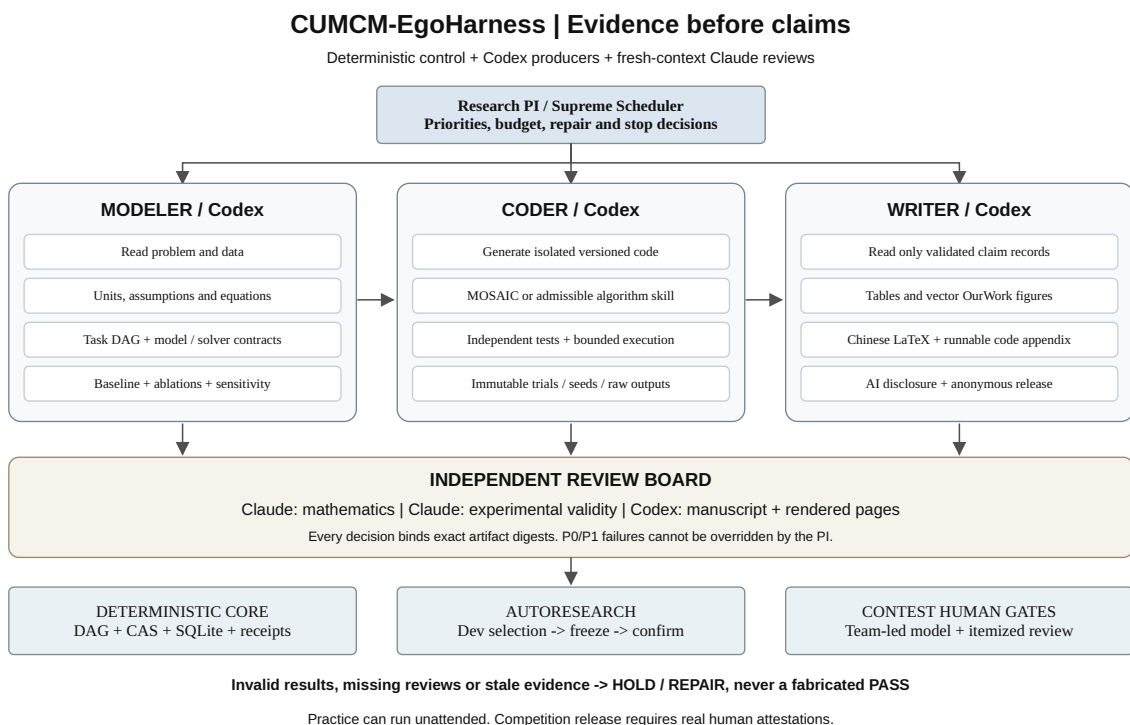


图 2: 基于所提供矢量绘图代码构建的控制、执行、审查与论文证据链。

AI 工具使用声明

本工程演示由人工智能辅助构建；本次演示的 Codex 与 Claude 子进程没有实际调用，审查夹具不能代替独立模型审查或人工核验。实际使用情况与状态见支撑材料。

参考文献

本演示的算法来源为随包提供的 MOSAIC 实现，问题为合成实例；未编造外部参考文献。正式研究应补入经过核验且在正文引用的文献。

A 附录：支撑材料与完整源程序

支撑材料包含运行配置、数据清单、实际数值结果、评测器、求解器、定制算法依赖、证据映射、图表源数据和人工智能使用详情。原始附件是否包含在支撑材料中，以数据来源配置与清单为准。

源程序文件清单

AI 工具使用详情.pdf

README.md

ai_usage.json

claims.json

code/baseline/main.py

code/c0/main.py

code/c1/main.py

code/dependencies/cumcm_harness/__init__.py

code/dependencies/cumcm_harness/algorithms.py

code/dependencies/cumcm_harness/common.py

code/dependencies/cumcm_harness/worker.py

code/dependencies/vendor/mosaic_v14/mosaic14/__init__.py

code/dependencies/vendor/mosaic_v14/mosaic14/bootstrap.py

code/dependencies/vendor/mosaic_v14/mosaic14/engine.py

code/dependencies/vendor/mosaic_v14/mosaic14/fast.py

code/dependencies/vendor/mosaic_v14/mosaic14/variants.py

code/dependencies/vendor/mosaic_v14/mosaic_solve.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/__init__.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/api.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/bootstrap.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/efficient.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/efficient_engine.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/metrics.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/model.py

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/problems.py
code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/residual.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/data/rgv2018_parameters.json
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/legacy/moo_core.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/legacy/mosaic_lqmoe.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/legacy/mosaic_sqmoe_v9.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/__init__.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/api.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/extended_core.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/extensions.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/final.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/graph_search.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/rgv.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/typed_baseline.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/__init__.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/bootstrap.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/fast_api.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/fast_v10.py
code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/problems.py
code/reproduce.py
code/verifier/evaluate.py
code/verifier/test_solver.py
config.json
figures/confirmation.data.json
figures/confirmation.pdf
figures/confirmation.png
figures/confirmation.provenance.json
figures/confirmation.svg
figures/ourwork.pdf
figures/ourwork.png
figures/ourwork.provenance.json

figures/ourwork.svg

input_manifest.json

inputs/confirmation/problem.json

inputs/development/problem.json

jobs/01b0e814cbcf4acd64f38b7475100d48b1661b9390359d17094df395b1381a8f/evaluation/custom_dependencies.json

jobs/01b0e814cbcf4acd64f38b7475100d48b1661b9390359d17094df395b1381a8f/evaluation/evaluation.json

jobs/01b0e814cbcf4acd64f38b7475100d48b1661b9390359d17094df395b1381a8f/receipt.json

jobs/01b0e814cbcf4acd64f38b7475100d48b1661b9390359d17094df395b1381a8f/solver/answer.json

jobs/01b0e814cbcf4acd64f38b7475100d48b1661b9390359d17094df395b1381a8f/solver/custom_dependencies.json

jobs/03b76d01e07afe68438604593ca62f709a90a416a8e95cda6870b9f0a47285b7/evaluation/custom_dependencies.json

jobs/03b76d01e07afe68438604593ca62f709a90a416a8e95cda6870b9f0a47285b7/evaluation/evaluation.json

jobs/03b76d01e07afe68438604593ca62f709a90a416a8e95cda6870b9f0a47285b7/receipt.json

jobs/03b76d01e07afe68438604593ca62f709a90a416a8e95cda6870b9f0a47285b7/solver/answer.json

jobs/03b76d01e07afe68438604593ca62f709a90a416a8e95cda6870b9f0a47285b7/solver/custom_dependencies.json

jobs/058585d4c9776e12a7f184e2379dbe352e697865fba38e07e1bcf9e9e5620b5e/evaluation/custom_dependencies.json

jobs/058585d4c9776e12a7f184e2379dbe352e697865fba38e07e1bcf9e9e5620b5e/evaluation/evaluation.json

jobs/058585d4c9776e12a7f184e2379dbe352e697865fba38e07e1bcf9e9e5620b5e/receipt.json

jobs/058585d4c9776e12a7f184e2379dbe352e697865fba38e07e1bcf9e9e5620b5e/solver/answer.json

jobs/058585d4c9776e12a7f184e2379dbe352e697865fba38e07e1bcf9e9e5620b5e/solver/custom_dependencies.json

jobs/087e07e7be406278afb29cafb075889d321d300ce627e79296600372c86f870e/evaluation/custom_dependencies.json

jobs/087e07e7be406278afb29cafb075889d321d300ce627e79296600372c86f870e/evaluation/evaluation.json

jobs/087e07e7be406278afb29cafb075889d321d300ce627e79296600372c86f870e/receipt.json

jobs/087e07e7be406278afb29cafb075889d321d300ce627e79296600372c86f870e/solver/answer.json

jobs/087e07e7be406278afb29cafb075889d321d300ce627e79296600372c86f870e/solver/custom_dependencies.json

jobs/12a7726c8cb41cdf239bd539026f95ba31c73da4f57044a0654597eb16de8301/evaluation/custom_dependencies.json

jobs/12a7726c8cb41cdf239bd539026f95ba31c73da4f57044a0654597eb16de8301/evaluation/evaluation.json

jobs/12a7726c8cb41cdf239bd539026f95ba31c73da4f57044a0654597eb16de8301/receipt.json

jobs/12a7726c8cb41cdf239bd539026f95ba31c73da4f57044a0654597eb16de8301/solver/answer.json

jobs/12a7726c8cb41cdf239bd539026f95ba31c73da4f57044a0654597eb16de8301/solver/custom_dependencies.json

jobs/24ee2a0f0460b81947fda159d71efa4701c1bfbca444a2f1646blac11bfe46ae/evaluation/custom_dependencies.json

jobs/24ee2a0f0460b81947fda159d71efa4701c1bfbca444a2f1646blac11bfe46ae/evaluation/evaluation.json

jobs/24ee2a0f0460b81947fda159d71efa4701c1bfbca444a2f1646blac11bfe46ae/receipt.json

jobs/24ee2a0f0460b81947fda159d71efa4701c1bfbca444a2f1646blac11bfe46ae/solver/answer.json

jobs/24ee2a0f0460b81947fda159d71efa4701c1bfbca444a2f1646blac11bfe46ae/solver/custom_dependencies.json

jobs/2f526bdd4616e72b3c8d8fe78b02004bc956b472e9eb9a83f4d7eea7163f85b2/evaluation/custom_dependencies.json

jobs/2f526bdd4616e72b3c8d8fe78b02004bc956b472e9eb9a83f4d7eea7163f85b2/evaluation/evaluation.json

jobs/2f526bdd4616e72b3c8d8fe78b02004bc956b472e9eb9a83f4d7eea7163f85b2/receipt.json

jobs/2f526bdd4616e72b3c8d8fe78b02004bc956b472e9eb9a83f4d7eea7163f85b2/solver/answer.json

jobs/2f526bdd4616e72b3c8d8fe78b02004bc956b472e9eb9a83f4d7eea7163f85b2/solver/custom_dependencies.json

jobs/32d7b9ba5dc02ca9fb793c82d980f51da4383b3bd4eba928fcd0ae82e033688/evaluation/custom_dependencies.json

jobs/32d7b9ba5dc02ca9fb793c82d980f51da4383b3bd4eba928fcd0ae82e033688/evaluation/evaluation.json

jobs/32d7b9ba5dc02ca9fb793c82d980f51da4383b3bd4eba928fcd0ae82e033688/receipt.json

jobs/32d7b9ba5dc02ca9fb793c82d980f51da4383b3bd4eba928fcd0ae82e033688/solver/answer.json

jobs/32d7b9ba5dc02ca9fb793c82d980f51da4383b3bd4eba928fcd0ae82e033688/solver/custom_dependencies.json

jobs/39b93122a9dc1841d07a537fc3efa356287590931b23711cf6dc3d11eb77d4e6/evaluation/custom_dependencies.json

jobs/39b93122a9dc1841d07a537fc3efa356287590931b23711cf6dc3d11eb77d4e6/evaluation/evaluation.json

jobs/39b93122a9dc1841d07a537fc3efa356287590931b23711cf6dc3d11eb77d4e6/receipt.json

jobs/39b93122a9dc1841d07a537fc3efa356287590931b23711cf6dc3d11eb77d4e6/solver/answer.json

jobs/39b93122a9dc1841d07a537fc3efa356287590931b23711cf6dc3d11eb77d4e6/solver/custom_dependencies.json

jobs/47a9c5f6759a74029a2b3229d2f454a0b6e75247955499a5fb7591d310b71e9d/evaluation/custom_dependencies.json

jobs/47a9c5f6759a74029a2b3229d2f454a0b6e75247955499a5fb7591d310b71e9d/evaluation/evaluation.json

jobs/47a9c5f6759a74029a2b3229d2f454a0b6e75247955499a5fb7591d310b71e9d/receipt.json

jobs/47a9c5f6759a74029a2b3229d2f454a0b6e75247955499a5fb7591d310b71e9d/solver/answer.json

jobs/47a9c5f6759a74029a2b3229d2f454a0b6e75247955499a5fb7591d310b71e9d/solver/custom_dependencies.json

jobs/4c4bb1b11ef462f04053c8dc59f770e094ac0d766b160d9533134ca12005f9c4/evaluation/custom_dependencies.json

jobs/4c4bb1b11ef462f04053c8dc59f770e094ac0d766b160d9533134ca12005f9c4/evaluation/evaluation.json

jobs/4c4bb1b11ef462f04053c8dc59f770e094ac0d766b160d9533134ca12005f9c4/receipt.json

jobs/4c4bb1b11ef462f04053c8dc59f770e094ac0d766b160d9533134ca12005f9c4/solver/answer.json

jobs/4c4bb1b11ef462f04053c8dc59f770e094ac0d766b160d9533134ca12005f9c4/solver/custom_dependencies.json

jobs/5033ce33f9865c1bfb189469cf9689684dfc075c3bd2a910dba0bc75db0b895e/evaluation/custom_dependencies.json

jobs/5033ce33f9865c1bfb189469cf9689684dfc075c3bd2a910dba0bc75db0b895e/evaluation/evaluation.json

jobs/5033ce33f9865c1bfb189469cf9689684dfc075c3bd2a910dba0bc75db0b895e/receipt.json

jobs/5033ce33f9865c1bfb189469cf9689684dfc075c3bd2a910dba0bc75db0b895e/solver/answer.json

jobs/5033ce33f9865c1bfb189469cf9689684dfc075c3bd2a910dba0bc75db0b895e/solver/custom_dependencies.json

jobs/5d98bcefab8ef1c33366a6f18ecf83bd0e7cf018e6490f622aba39ac3816ad78/evaluation/custom_dependencies.json

jobs/5d98bcefab8ef1c33366a6f18ecf83bd0e7cf018e6490f622aba39ac3816ad78/evaluation/evaluation.json

jobs/5d98bcefab8ef1c33366a6f18ecf83bd0e7cf018e6490f622aba39ac3816ad78/receipt.json

jobs/5d98bcefab8ef1c33366a6f18ecf83bd0e7cf018e6490f622aba39ac3816ad78/solver/answer.json

jobs/5d98bcefab8ef1c33366a6f18ecf83bd0e7cf018e6490f622aba39ac3816ad78/solver/custom_dependencies.json

jobs/72ccbdbee4aab327c2302bd2ad690e9d0a95f6e09b09b3950b0e21dd989a53b1/evaluation/custom_dependencies.json

jobs/72ccbdbee4aab327c2302bd2ad690e9d0a95f6e09b09b3950b0e21dd989a53b1/evaluation/evaluation.json

jobs/72ccbdbee4aab327c2302bd2ad690e9d0a95f6e09b09b3950b0e21dd989a53b1/receipt.json

jobs/72ccbdbee4aab327c2302bd2ad690e9d0a95f6e09b09b3950b0e21dd989a53b1/solver/answer.json

jobs/72ccbdbee4aab327c2302bd2ad690e9d0a95f6e09b09b3950b0e21dd989a53b1/solver/custom_dependencies.json

jobs/75021ee88408f4416e78ff4769c3c890e10dff9b1b8ac6462870d71114be3b2/evaluation/custom_dependencies.json

jobs/75021ee88408f4416e78ff4769c3c890e10dff9b1b8ac6462870d71114be3b2/evaluation/evaluation.json

jobs/75021ee88408f4416e78ff4769c3c890e10dff9b1b8ac6462870d71114be3b2/receipt.json

jobs/75021ee88408f4416e78ff4769c3c890e10dff9b1b8ac6462870d71114be3b2/solver/answer.json

jobs/75021ee88408f4416e78ff4769c3c890e10dff9b1b8ac6462870d71114be3b2/solver/custom_dependencies.json

jobs/752ceda387ba7b02613f814cde9922ee9d077170b90233f818d9f8e8665b45c2/evaluation/custom_dependencies.json

jobs/752ceda387ba7b02613f814cde9922ee9d077170b90233f818d9f8e8665b45c2/evaluation/evaluation.json

jobs/752ceda387ba7b02613f814cde9922ee9d077170b90233f818d9f8e8665b45c2/receipt.json

jobs/752ceda387ba7b02613f814cde9922ee9d077170b90233f818d9f8e8665b45c2/solver/answer.json

jobs/752ceda387ba7b02613f814cde9922ee9d077170b90233f818d9f8e8665b45c2/solver/custom_dependencies.json

jobs/7c2ab68c9248e7bd1f2a199cd1d8439a025b7cbc8b0cab20ca407be82786a82c/evaluation/custom_dependencies.json

jobs/7c2ab68c9248e7bd1f2a199cd1d8439a025b7cbc8b0cab20ca407be82786a82c/evaluation/evaluation.json

jobs/7c2ab68c9248e7bd1f2a199cd1d8439a025b7cbc8b0cab20ca407be82786a82c/receipt.json

jobs/7c2ab68c9248e7bd1f2a199cd1d8439a025b7cbc8b0cab20ca407be82786a82c/solver/answer.json

jobs/7c2ab68c9248e7bd1f2a199cd1d8439a025b7cbc8b0cab20ca407be82786a82c/solver/custom_dependencies.json

jobs/83671c4b0e9c975c08a28c3e512aa3747b006ff045760b676955caa7c9bb9951/evaluation/custom_dependencies.json

jobs/83671c4b0e9c975c08a28c3e512aa3747b006ff045760b676955caa7c9bb9951/evaluation/evaluation.json

jobs/83671c4b0e9c975c08a28c3e512aa3747b006ff045760b676955caa7c9bb9951/receipt.json

jobs/83671c4b0e9c975c08a28c3e512aa3747b006ff045760b676955caa7c9bb9951/solver/answer.json

jobs/83671c4b0e9c975c08a28c3e512aa3747b006ff045760b676955caa7c9bb9951/solver/custom_dependencies.json

jobs/844ee5faffe77c52b66d19c59081ed4ede1ac83fb5bf09ec2ed3d843c721a414/evaluation/custom_dependencies.json

jobs/844ee5faffe77c52b66d19c59081ed4ede1ac83fb5bf09ec2ed3d843c721a414/evaluation/evaluation.json

jobs/844ee5faffe77c52b66d19c59081ed4ede1ac83fb5bf09ec2ed3d843c721a414/receipt.json

jobs/844ee5faffe77c52b66d19c59081ed4ede1ac83fb5bf09ec2ed3d843c721a414/solver/answer.json

jobs/844ee5faffe77c52b66d19c59081ed4ede1ac83fb5bf09ec2ed3d843c721a414/solver/custom_dependencies.json

jobs/87a4fc61cf6e118dc9b1cbec877038fd3729d531af0c96ea0d5a8ec67b1cb8e4/evaluation/custom_dependencies.json

jobs/87a4fc61cf6e118dc9b1cbec877038fd3729d531af0c96ea0d5a8ec67b1cb8e4/evaluation/evaluation.json

jobs/87a4fc61cf6e118dc9b1cbec877038fd3729d531af0c96ea0d5a8ec67b1cb8e4/receipt.json

jobs/87a4fc61cf6e118dc9b1cbec877038fd3729d531af0c96ea0d5a8ec67b1cb8e4/solver/answer.json

jobs/87a4fc61cf6e118dc9b1cbec877038fd3729d531af0c96ea0d5a8ec67b1cb8e4/solver/custom_dependencies.json

jobs/95935ad503c742cd8ea2ae4dea4a8baa37c9022dbbalea25db41b9eb00d04a14/evaluation/custom_dependencies.json

jobs/95935ad503c742cd8ea2ae4dea4a8baa37c9022dbbalea25db41b9eb00d04a14/evaluation/evaluation.json

jobs/95935ad503c742cd8ea2ae4dea4a8baa37c9022dbbalea25db41b9eb00d04a14/receipt.json

jobs/95935ad503c742cd8ea2ae4dea4a8baa37c9022dbbalea25db41b9eb00d04a14/solver/answer.json

jobs/95935ad503c742cd8ea2ae4dea4a8baa37c9022dbbalea25db41b9eb00d04a14/solver/custom_dependencies.json

jobs/95cd71e01788d440442ff443b42bc182c50360abcfb7834b9521f9b988397e89/evaluation/custom_dependencies.json

jobs/95cd71e01788d440442ff443b42bc182c50360abcfb7834b9521f9b988397e89/evaluation/evaluation.json

jobs/95cd71e01788d440442ff443b42bc182c50360abcfb7834b9521f9b988397e89/receipt.json

jobs/95cd71e01788d440442ff443b42bc182c50360abcfb7834b9521f9b988397e89/solver/answer.json

jobs/95cd71e01788d440442ff443b42bc182c50360abcfb7834b9521f9b988397e89/solver/custom_dependencies.json

jobs/9a05a9324855790ed4f28de829333b53f86bad878d736ad68a853b3477680aac/evaluation/custom_dependencies.json

jobs/9a05a9324855790ed4f28de829333b53f86bad878d736ad68a853b3477680aac/evaluation/evaluation.json

jobs/9a05a9324855790ed4f28de829333b53f86bad878d736ad68a853b3477680aac/receipt.json

jobs/9a05a9324855790ed4f28de829333b53f86bad878d736ad68a853b3477680aac/solver/answer.json

jobs/9a05a9324855790ed4f28de829333b53f86bad878d736ad68a853b3477680aac/solver/custom_dependencies.json

jobs/b9d03e11ad9cb576c9db3ed97082866f4646d55714dbf1284c748f3d09bf1deb/evaluation/custom_dependencies.json

jobs/b9d03e11ad9cb576c9db3ed97082866f4646d55714dbf1284c748f3d09bf1deb/evaluation/evaluation.json

jobs/b9d03e11ad9cb576c9db3ed97082866f4646d55714dbf1284c748f3d09bf1deb/receipt.json

jobs/b9d03e11ad9cb576c9db3ed97082866f4646d55714dbf1284c748f3d09bf1deb/solver/answer.json

jobs/b9d03e11ad9cb576c9db3ed97082866f4646d55714dbf1284c748f3d09bf1deb/solver/custom_dependencies.json

jobs/be22552a24c95485523b9132b8356aa8d13d86de4a135b6908bf210010f497fe/evaluation/custom_dependencies.json

jobs/be22552a24c95485523b9132b8356aa8d13d86de4a135b6908bf210010f497fe/evaluation/evaluation.json

jobs/be22552a24c95485523b9132b8356aa8d13d86de4a135b6908bf210010f497fe/receipt.json

jobs/be22552a24c95485523b9132b8356aa8d13d86de4a135b6908bf210010f497fe/solver/answer.json

jobs/be22552a24c95485523b9132b8356aa8d13d86de4a135b6908bf210010f497fe/solver/custom_dependencies.json

jobs/ce4b7fad483ff5b6690193b4e68f6ced24633204ccd46733a70b7c1dbd379082/evaluation/custom_dependencies.json

jobs/ce4b7fad483ff5b6690193b4e68f6ced24633204ccd46733a70b7c1dbd379082/evaluation/evaluation.json

jobs/ce4b7fad483ff5b6690193b4e68f6ced24633204ccd46733a70b7c1dbd379082/receipt.json

jobs/ce4b7fad483ff5b6690193b4e68f6ced24633204ccd46733a70b7c1dbd379082/solver/answer.json

jobs/ce4b7fad483ff5b6690193b4e68f6ced24633204ccd46733a70b7c1dbd379082/solver/custom_dependencies.json

jobs/d2a80d28689ce044957aaf952c869272b05df4fdffe89a445e4f21362c94f091/evaluation/custom_dependencies.json

jobs/d2a80d28689ce044957aaf952c869272b05df4fdffe89a445e4f21362c94f091/evaluation/evaluation.json

jobs/d2a80d28689ce044957aaf952c869272b05df4fdffe89a445e4f21362c94f091/receipt.json

jobs/d2a80d28689ce044957aaf952c869272b05df4fdffe89a445e4f21362c94f091/solver/answer.json

jobs/d2a80d28689ce044957aaf952c869272b05df4fdffe89a445e4f21362c94f091/solver/custom_dependencies.json

jobs/dca49a718dd59a02a70bd0dbf8c48b4416f16071e2853bb8515d8c1258664837/evaluation/custom_dependencies.json

jobs/dca49a718dd59a02a70bd0dbf8c48b4416f16071e2853bb8515d8c1258664837/evaluation/evaluation.json

jobs/dca49a718dd59a02a70bd0dbf8c48b4416f16071e2853bb8515d8c1258664837/receipt.json

jobs/dca49a718dd59a02a70bd0dbf8c48b4416f16071e2853bb8515d8c1258664837/solver/answer.json

jobs/dca49a718dd59a02a70bd0dbf8c48b4416f16071e2853bb8515d8c1258664837/solver/custom_dependencies.json

jobs/e7649981ea4452d51a0a09fcb590cd795a386ebe220ddf548a88120d21f1c6bf/evaluation/custom_dependencies.json

jobs/e7649981ea4452d51a0a09fcb590cd795a386ebe220ddf548a88120d21f1c6bf/evaluation/evaluation.json

jobs/e7649981ea4452d51a0a09fcb590cd795a386ebe220ddf548a88120d21f1c6bf/receipt.json

jobs/e7649981ea4452d51a0a09fcb590cd795a386ebe220ddf548a88120d21f1c6bf/solver/answer.json

jobs/e7649981ea4452d51a0a09fcb590cd795a386ebe220ddf548a88120d21f1c6bf/solver/custom_dependencies.json

jobs/e933b96c4aef78ae30e6f9403ad5abba5153df8655887f76e10db8d6199379e4/evaluation/custom_dependencies.json

jobs/e933b96c4aef78ae30e6f9403ad5abba5153df8655887f76e10db8d6199379e4/evaluation/evaluation.json

jobs/e933b96c4aef78ae30e6f9403ad5abba5153df8655887f76e10db8d6199379e4/receipt.json

jobs/e933b96c4aef78ae30e6f9403ad5abba5153df8655887f76e10db8d6199379e4/solver/answer.json

jobs/e933b96c4aef78ae30e6f9403ad5abba5153df8655887f76e10db8d6199379e4/solver/custom_dependencies.json

jobs/f644024c8db58ba9f79681f8f95e0c22b7ce854c1258c40b4a852401f45f5eb5/evaluation/custom_dependencies.json

jobs/f644024c8db58ba9f79681f8f95e0c22b7ce854c1258c40b4a852401f45f5eb5/evaluation/evaluation.json
 jobs/f644024c8db58ba9f79681f8f95e0c22b7ce854c1258c40b4a852401f45f5eb5/receipt.json
 jobs/f644024c8db58ba9f79681f8f95e0c22b7ce854c1258c40b4a852401f45f5eb5/solver/answer.json
 jobs/f644024c8db58ba9f79681f8f95e0c22b7ce854c1258c40b4a852401f45f5eb5/solver/custom_dependencies.json
 manifest.json
 protocol.json
 reproduce.py
 results/confirmation.json
 results/development.json
 source_inventory.json
 support_inventory.json

源程序与运行资源

code/baseline/main.py

```

"""Synthetic ordered-subset scheduling demo, NOT an official CUMCM solution."""
from pathlib import Path
import argparse, json
import numpy as np
from cumcm_harness.algorithms import mosaic_modules, mosaic_solve, mosaic_infill_ablation
FULL_STRATEGY = 'fast_h0'

def nondominated(F):
    return np.array([not np.any(np.all(F<=f,axis=1)&np.any(F<f,axis=1)) for f in F])

def main():
    p=argparse.ArgumentParser();p.add_argument('--input',type=Path,required=True);p.add_argument('--out',type=Path,required=True)
    ↪ ue)
    p.add_argument('--seed',type=int,required=True);p.add_argument('--budget',type=int,required=True);p.add_argument('--variant',
    ↪ t',required=True)
    a=p.parse_args();spec=json.loads((a.input/'problem.json').read_text())
    mosaic_modules();from harnesslab.problems import make_problem
    problem=make_problem(spec)
    if a.variant=='baseline':
        rng=np.random.default_rng(a.seed);X=rng.random((a.budget,problem.n_var));F=problem.evaluate(X);spent=problem.calls;backen
        ↪ kend='uniform_random'
    elif a.variant in ('full','no_infill','population16'):
        if a.variant=='no_infill':run=mosaic_infill_ablation(problem,seed=a.seed,budget=a.budget,full_strategy=FULL_STRATEGY)
        else:run=mosaic_solve(problem,representation='ordered_subset',codec=problem.codec,constraints='feasible_decoder',
            strategy=FULL_STRATEGY,seed=a.seed,budget=a.budget,population=16 if a.variant=='population16' else
            ↪ 32,research_opt_in=True)
        X=np.array(run.ledger.X);F=np.array(run.ledger.F);spent=run.ledger.spent;backend=run.result.algorithm
    else:raise ValueError('unknown variant')
    # Identical postprocessing for every method: nondominated paid observations.
    keep=nondominated(F);frontX=X[keep];frontF=F[keep]
    _,ids=np.unique(frontF,axis=0,return_index=True);frontX=frontX[ids];frontF=frontF[ids]
    n=len(spec['values']);T=sum(spec['process'])+n*float(np.max(spec['setup']))+1
  
```

```

utility=-frontF[:,0]/sum(spec['values'])-frontF[:,1]/T;selected=int(np.argmax(utility))
answer={'X':frontX.tolist(),'F':frontF.tolist(),'ledger_X':X.tolist(),'ledger_F':F.tolist(),
        'selected_index':selected,'budget_spent':int(spent),'seed':a.seed,'backend':backend,
        'postprocessing':'nondominated paid observations only; no objective calls or reference-front access'}
a.out.mkdir(parents=True,exist_ok=True);(a.out/'answer.json').write_text(json.dumps(answer,allow_nan=False))
if __name__=='__main__':main()

```

源程序与运行资源

code/c0/main.py

```

"""Synthetic ordered-subset scheduling demo, NOT an official CUMCM solution."""
from pathlib import Path
import argparse,json
import numpy as np
from cumcm_harness.algorithms import mosaic_modules,mosaic_solve,mosaic_infill_ablation
FULL_STRATEGY = 'fast_h0'

def nondominated(F):
    return np.array([not np.any(np.all(F<=f,axis=1)&np.any(F<f,axis=1)) for f in F])
def main():
    p=argparse.ArgumentParser();p.add_argument('--input',type=Path,required=True);p.add_argument('--out',type=Path,required=True)
    ↪ ue)
    p.add_argument('--seed',type=int,required=True);p.add_argument('--budget',type=int,required=True);p.add_argument('--varian
    ↪ t',required=True)
    a=p.parse_args();spec=json.loads((a.input/'problem.json').read_text())
    mosaic_modules();from harnesslab.problems import make_problem
    problem=make_problem(spec)
    if a.variant=='baseline':
        rng=np.random.default_rng(a.seed);X=rng.random((a.budget,problem.n_var));F=problem.evaluate(X);spent=problem.calls;bac
        ↪ kend='uniform_random'
    elif a.variant in ('full','no_infill','population16'):
        if a.variant=='no_infill':run=mosaic_infill_ablation(problem,seed=a.seed,budget=a.budget,full_strategy=FULL_STRATEGY)
        else:run=mosaic_solve(problem,representation='ordered_subset',codec=problem.codec,constraints='feasible_decoder',
            strategy=FULL_STRATEGY,seed=a.seed,budget=a.budget,population=16 if a.variant=='population16' else
            ↪ 32,research_opt_in=True)
        X=np.array(run.ledger.X);F=np.array(run.ledger.F);spent=run.ledger.spent;backend=run.result.algorithm
    else:raise ValueError('unknown variant')
    # Identical postprocessing for every method: nondominated paid observations.
    keep=nondominated(F);frontX=X[keep];frontF=F[keep]
    _,ids=np.unique(frontF,axis=0,return_index=True);frontX=frontX[ids];frontF=frontF[ids]
    n=len(spec['values']);T=sum(spec['process'])+n*float(np.max(spec['setup']))+1
    utility=-frontF[:,0]/sum(spec['values'])-frontF[:,1]/T;selected=int(np.argmax(utility))
    answer={'X':frontX.tolist(),'F':frontF.tolist(),'ledger_X':X.tolist(),'ledger_F':F.tolist(),
            'selected_index':selected,'budget_spent':int(spent),'seed':a.seed,'backend':backend,
            'postprocessing':'nondominated paid observations only; no objective calls or reference-front access'}
    a.out.mkdir(parents=True,exist_ok=True);(a.out/'answer.json').write_text(json.dumps(answer,allow_nan=False))
    if __name__=='__main__':main()

```

源程序与运行资源

code/c1/main.py

```

"""Synthetic ordered-subset scheduling demo, NOT an official CUMCM solution."""
from pathlib import Path
import argparse, json
import numpy as np
from cumcm_harness.algorithms import mosaic_modules, mosaic_solve, mosaic_infill_ablation
FULL_STRATEGY = 'refit32'

def nondominated(F):
    return np.array([not np.any(np.all(F<=f,axis=1)&np.any(F<f,axis=1)) for f in F])

def main():
    p=argparse.ArgumentParser();p.add_argument('--input',type=Path,required=True);p.add_argument('--out',type=Path,required=True)
    p.add_argument('--seed',type=int,required=True);p.add_argument('--budget',type=int,required=True);p.add_argument('--variant',type=str,required=True)
    a=p.parse_args();spec=json.loads((a.input/'problem.json').read_text())
    mosaic_modules();from harnesslab.problems import make_problem
    problem=make_problem(spec)
    if a.variant=='baseline':
        rng=np.random.default_rng(a.seed);X=rng.random((a.budget,problem.n_var));F=problem.evaluate(X);spent=problem.calls;backend='uniform_random'
    elif a.variant in ('full','no_infill','population16'):
        if a.variant=='no_infill':run=mosaic_infill_ablation(problem,seed=a.seed,budget=a.budget,full_strategy=FULL_STRATEGY)
        else:run=mosaic_solve(problem,representation='ordered_subset',codec=problem.codec,constraints='feasible_decoder',
            strategy=FULL_STRATEGY,seed=a.seed,budget=a.budget,population=16 if a.variant=='population16' else 32,research_opt_in=True)
        X=np.array(run.ledger.X);F=np.array(run.ledger.F);spent=run.ledger.spent;backend=run.result.algorithm
    else:raise ValueError('unknown variant')
    # Identical postprocessing for every method: nondominated paid observations.
    keep=nondominated(F);frontX=X[keep];frontF=F[keep]
    _,ids=np.unique(frontF,axis=0,return_index=True);frontX=frontX[ids];frontF=frontF[ids]
    n=len(spec['values']);T=sum(spec['process'])+n*float(np.max(spec['setup']))+1
    utility=-frontF[:,0]/sum(spec['values'])-frontF[:,1]/T;selected=int(np.argmax(utility))
    answer={'X':frontX.tolist(),'F':frontF.tolist(),'ledger_X':X.tolist(),'ledger_F':F.tolist(),
        'selected_index':selected,'budget_spent':int(spent),'seed':a.seed,'backend':backend,
        'postprocessing':'nondominated paid observations only; no objective calls or reference-front access'}
    a.out.mkdir(parents=True,exist_ok=True);(a.out/'answer.json').write_text(json.dumps(answer,allow_nan=False))
if __name__=='__main__':main()

```

源程序与运行资源

code/dependencies/cumcm_harness/__init__.py

```

"""CUMCM-EgoHarness: deterministic orchestration, evidence before claims."""
__version__ = "0.1.0"

```

源程序与运行资源

code/dependencies/cumcm_harness/algorithms.py

```

"""Executable baseline library plus an unmodified MOSAIC adapter.
Cards are applicability checks, NOT a promise every implementation fits every task.
"""
from __future__ import annotations

```

```

import sys
import numpy as np
from .common import ROOT, IntegrityError

METHODS=[
    {'id':'linear-programming','family':'optimization','keywords':['线性','资源','linear','allocation'],'implementation':'scipy.」
    ↪ optimize.linprog(method="highs")','assumptions':['linear objective and constraints'],'checks':['primal residual','bound
    ↪ feasibility','solver status','dual information when available']},
    {'id':'mixed-integer','family':'optimization','keywords':['整数','调度','分配','integer','scheduling'],'implementation':'sci」
    ↪ py.optimize.milp','assumptions':['explicit integer variables; linear
    ↪ constraints'],'checks':['integrality','feasibility','solver bound and MIP gap']},
    {'id':'mosaic-multiobjective','family':'optimization','keywords':['多目标','帕累托','multiobjective','Pareto'],'implementati」
    ↪ on':'cumcm_harness.algorithms.mosaic_solve','assumptions':['deterministic objectives; continuous box or approved feasible
    ↪ ordered-subset codec'],'checks':['FE ledger','objective recomputation','Pareto dominance','equal-budget baseline','small
    ↪ exact oracle if finite']},
    {'id':'regression','family':'prediction','keywords':['回归','拟合','regression'],'implementation':'cumcm_harness.algorithms.」
    ↪ linear_fit','assumptions':['identified design; split before fitting preprocessing'],'checks':['rank','holdout
    ↪ error','residuals','leakage','coefficient sensitivity']},
    {'id':'time-series','family':'prediction','keywords':['时序','预测','forecast','trend'],'implementation':'cumcm_harness.algo」
    ↪ rithms.linear_forecast','assumptions':['ordered observations; linear extrapolation is only a
    ↪ baseline'],'checks':['rolling-origin split','last-value baseline','drift','no future leakage']},
    {'id':'topsis','family':'evaluation','keywords':['评价','排序','topsis'],'implementation':'cumcm_harness.algorithms.topsis,」
    ↪ 'assumptions':['explicit benefit/cost directions; justified nonnegative weights'],'checks':['constant columns','weight
    ↪ sensitivity','rank reversal','units']},
    {'id':'graph-path','family':'graph','keywords':['图论','路径','network','shortest'],'implementation':'scipy.sparse.csgraph.d」
    ↪ ijkstra','assumptions':['nonnegative edge weights; specified connectivity'],'checks':['route feasibility','total path
    ↪ cost','disconnected cases']},
    {'id':'ode','family':'dynamics','keywords':['微分','动力学','ode','dynamic'],'implementation':'scipy.integrate.solve_ivp','a」
    ↪ ssumptions':['specified initial conditions and physically meaningful RHS'],'checks':['convergence under step
    ↪ refinement','conservation','stability','units']},
    {'id':'monte-carlo','family':'simulation','keywords':['模拟','随机','Monte','simulation'],'implementation':'numpy.random.Gen」
    ↪ erator','assumptions':['specified distribution and independent replication unit'],'checks':['seed ledger','sampling
    ↪ error','variance reduction','rare-event coverage']},
    {'id':'statistics','family':'statistics','keywords':['统计','检验','聚类','statistics'],'implementation':'scipy.stats /
    ↪ sklearn clustering','assumptions':['explicit sampling unit and hypotheses'],'checks':['multiple testing','effect
    ↪ size','confounding','group/time split']},
]

def route_methods(query:str,top_k=6):
    text=query.casefold()
    ranked=sorted(METHODS,key=lambda m:(-sum(k.casefold() in text for k in m['keywords']),m['id']))
    return ranked[:top_k]

def linear_fit(X,y):
    X=np.asarray(X,float);y=np.asarray(y,float)
    if X.ndim==1:X=X[:,None]
    if X.ndim!=2 or y.shape!=(len(X),) or not (np.isfinite(X).all() and np.isfinite(y).all()):raise ValueError('Invalid
    ↪ regression data')
    A=np.c_[np.ones(len(X)),X];beta,res,rnk,s=np.linalg.lstsq(A,y,rcond=None)
    if rnk<A.shape[1]:raise ValueError('Unidentified/rank-deficient regression')
    return beta

def linear_forecast(y,horizon):
    y=np.asarray(y,float)
    if y.ndim!=1 or len(y)<3 or horizon<1:raise ValueError('At least three ordered observations required')
    beta=linear_fit(np.arange(len(y)),y)
    return beta[0]+beta[1]*np.arange(len(y),len(y)+horizon)

def topsis(X,weights,benefit):

```

```

X=np.asarray(X,float);w=np.asarray(weights,float);b=np.asarray(benefit,bool)
if X.ndim!=2 or X.shape[1]!=len(w) or len(b)!=len(w) or len(X)<2 or not np.isfinite(X).all():raise ValueError('Invalid
↳ TOPSIS input')
if not np.isfinite(w).all() or np.any(w<0) or w.sum()<=0:raise ValueError('Weights must be finite, nonnegative and
↳ nonzero')
norms=np.linalg.norm(X,axis=0)
if np.any(norms==0) or np.any(np.ptp(X,axis=0)==0):raise ValueError('Remove constant columns and renormalize weights
↳ explicitly')
V=X/norms*(w/w.sum());ideal=np.where(b,V.max(0),V.min(0));anti=np.where(b,V.min(0),V.max(0))
a=np.linalg.norm(V-ideal,axis=1);z=np.linalg.norm(V-anti,axis=1)
if np.any(a+z==0):raise ValueError('Indistinguishable alternatives')
return z/(a+z)

def mosaic_modules():
    path=ROOT/'vendor/mosaic_v14'
    if not path.is_dir():raise IntegrityError('Source distribution assets missing; use editable source install')
    if str(path) not in sys.path:sys.path.insert(0,str(path))
    import mosaic_solve as m
    return m

def mosaic_solve(problem, *, representation='continuous_box',strategy='incumbent',seed=1,budget=256,population=32,codec=None,d_j
↳ eterministic=True,constraints='box',research_opt_in=False):
    """No silent rounding/noise repair. V14 accelerated branches are bi-objective
    ordered subsets. Incumbent also retains the original continuous-box v9 backend.
    """
    if strategy in ('local24','refit32') and not research_opt_in:raise IntegrityError('Research-only branch requires explicit
↳ opt-in')
    if representation=='ordered_subset' and codec is None:raise IntegrityError('Ordered subsets require an explicit validated
↳ codec')
    m=mosaic_modules();contract=m.ProblemContract(representation=representation,codec=codec,deterministic=deterministic,constr_j
↳ aints=constraints)
    run=m.solve(problem,seed=seed,budget=budget,population=population,contract=contract,strategy=strategy)
    if run.ledger.spent!=run.result.evaluations or run.ledger.spent!=budget:raise IntegrityError('MOSAIC FE accounting
↳ mismatch')
    return run

def hv2(points,reference):
    """Exact 2D dominated hypervolume for minimization; fixed external reference."""
    F=np.asarray(points,float);r=np.asarray(reference,float)
    if F.ndim!=2 or F.shape[1]!=2 or r.shape!=(2,) or not np.isfinite(F).all() or not np.isfinite(r).all():raise
↳ ValueError('Invalid hypervolume input')
    if np.any(F>r):raise ValueError('Reference must weakly dominate all supplied points; do not silently discard')
    F=F[np.argsort(F[:,0],kind='stable')];y=r[1];area=0.
    for x,v in F:
        if v<y:area+=(r[0]-x)*(y-v);y=v
    return float(area)

def mosaic_infill_ablation(problem,*,seed=1,budget=192,population=32,full_strategy='fast_h0'):
    """Single-factor diagnostic: same deployed engine/config, enabled=False only.
    This is a local harness ablation, not an altered vendored production default.
    """
    mosaic_modules()
    from dataclasses import replace
    from mosaic14.bootstrap import EvaluationLedger,AuditedRun
    from mosaic14.engine import run_fast_h0
    from mosaic14.variants import CONFIGS,SearchHarness
    from harnesslab.problems import OrderedSubsetCodec
    from mosaic.rgv import RGVCodec

```



```

if full_strategy not in ('fast_h0', 'refit32') or type(problem.codec) not in (OrderedSubsetCodec, RGVCodec) or
↪ problem.n_obj!=2:
    raise IntegrityError('Unsupported ablation contract')
if budget<population or population<8:raise IntegrityError('Invalid ablation budget')
ledger=EvaluationLedger(problem,budget);cfg=replace(CONFIGS[full_strategy],enabled=False)
result=run_fast_h0(ledger,problem.codec,seed,population,budget,cfg,harness_type=SearchHarness)
if result.evaluations!=budget or ledger.spent!=budget:raise IntegrityError('Ablation ledger mismatch')
result.algorithm='MOSAIC-v14-ONE_FACTOR_ABLATION'
result.diagnostics['ablation']={'base_strategy':full_strategy,'enabled':False,'other_config_unchanged':True}
return AuditedRun(result,ledger)

```

源程序与运行资源

code/dependencies/cumcm_harness/common.py

```

"""Small trusted core: canonical JSON, safe paths, atomic files and digests."""
from __future__ import annotations
import hashlib, json, math, os, platform, re, tempfile
from pathlib import Path, PurePosixPath
from typing import Any

ROOT = Path(__file__).resolve().parents[1]
class HarnessError(RuntimeError): pass
class Blocked(HarnessError): pass
class IntegrityError(HarnessError): pass

def canonical(value: Any) -> bytes:
    return json.dumps(value, sort_keys=True, ensure_ascii=False, separators=(',', ':'), allow_nan=False).encode('utf-8')
def digest(value: Any) -> str:
    return hashlib.sha256(canonical(value)).hexdigest()
def file_hash(path: Path) -> str:
    h=hashlib.sha256()
    with Path(path).open('rb') as f:
        for b in iter(lambda:f.read(1024*1024),b''): h.update(b)
    return h.hexdigest()
def read_json(path: Path) -> Any:
    def reject(x): raise IntegrityError(f'Nonfinite JSON value: {x}')
    return json.loads(Path(path).read_text('utf-8'), parse_constant=reject)
def atomic_write(path: Path, data: str|bytes) -> None:
    path=Path(path); path.parent.mkdir(parents=True,exist_ok=True)
    b=data.encode('utf-8') if isinstance(data,str) else data
    fd,tmp=tempfile.mkstemp(prefix='.'+path.name+'.',dir=path.parent)
    try:
        with os.fdopen(fd,'wb') as f: f.write(b); f.flush(); os.fsync(f.fileno())
        os.replace(tmp,path)
    finally:
        if os.path.exists(tmp): os.unlink(tmp)
def write_json(path: Path,value: Any) -> None:
    atomic_write(path,json.dumps(value,indent=2,ensure_ascii=False,sort_keys=True,allow_nan=False)+'\n')
def safe_rel(s: str) -> str:
    p=PurePosixPath(s)
    if not s or '\\\\' in s or '\\x00' in s or ':' in s or p.is_absolute() or any(x in ('..','.') or x.startswith('.') for x in
↪ p.parts):
        raise IntegrityError(f'Unsafe relative path: {s!r}')
    if str(p)!=s or len(s)>240: raise IntegrityError(f'Noncanonical path: {s!r}')
    return s

```

```

def under(root: Path, rel: str) -> Path:
    safe_rel(rel); p=root/rel
    if not p.resolve().is_relative_to(root.resolve()): raise IntegrityError('Path or symlink escapes root')
    return p

def tree_manifest(root: Path) -> dict[str,str]:
    out={}
    for p in sorted(Path(root).rglob('*')):
        if p.is_symlink(): raise IntegrityError(f'Symlink forbidden: {p}')
        if p.is_file(): out[p.relative_to(root).as_posix()]=file_hash(p)
    return out

def verify_tree(root: Path, expected: dict[str,str]) -> None:
    if tree_manifest(root)!=expected: raise IntegrityError(f'Changed artifact tree: {root.name}')

def finite(x: Any) -> float:
    if isinstance(x,bool) or not isinstance(x,(int,float)) or not math.isfinite(x):
        raise IntegrityError('Expected finite numeric value')
    return float(x)

def environment() -> dict:
    import importlib.metadata
    versions={}
    for x in ('numpy','scipy','scikit-learn','pytest','numba','matplotlib','jsonschema','pandas'):
        try: versions[x]=importlib.metadata.version(x)
        except importlib.metadata.PackageNotFoundError: versions[x]='NOT_INSTALLED'
    return {'python':platform.python_version(),'platform':platform.platform(),'machine':platform.machine(),
            'cpu_count':os.cpu_count(),'packages':versions}

def verify_vendor():
    entries=read_json(ROOT/'vendor/MANIFEST.json')
    for entry in entries:
        if file_hash(under(ROOT,entry['path']))!=entry['sha256']:raise IntegrityError('Vendor mismatch: '+entry['path'])
    return {'status':'PASS','files':len(entries),'manifest_sha256':file_hash(ROOT/'vendor/MANIFEST.json')}

```

源程序与运行资源

code/dependencies/cumcm_harness/worker.py

"""Sandbox entry wrapper; records imported code AND runtime-read custom assets.

The Python audit hook is dependency observation, not a security boundary. Native extensions may read files without emitting a Python open event; isolated support bundle reproduction remains a required acceptance check.

"""

```

from __future__ import annotations
import os, runpy, sys
from pathlib import Path
from .common import ROOT, Blocked, file_hash, write_json

```

```

class DependencyReads:
    def __init__(self, root: Path):
        self.root=root.resolve();self.paths=set();self.active=True
    def __call__(self,event,args):
        if not self.active or event!='open' or not args or not isinstance(args[0],(str,bytes,os.PathLike)):return

```

```

# Do not treat generated outputs or cache writes as input dependencies.
mode=args[1] if len(args)>1 else None
flags=args[2] if len(args)>2 else 0
if isinstance(mode,str) and any(c in mode for c in 'wax+'):return
if isinstance(flags,int) and flags&(os.O_WRONLY|os.O_RDWR|os.O_CREAT|os.O_TRUNC|os.O_APPEND):return
try:
    p=Path(os.fsdecode(args[0])).resolve()
    if not p.is_relative_to(self.root):return
    rel=p.relative_to(self.root).as_posix()
    if not rel.startswith(('vendor/mosaic_v14/', 'cumcm_harness/')):return
    if '__pycache__' in p.parts or p.suffix in ('.pyc', '.pyo', '.nbc', '.nbi'):return
    if p.suffix.lower() in ('.ttf', '.otf', '.ttc', '.woff', '.woff2', '.eot'):
        raise Blocked('Runtime font dependency cannot be distributed; install fonts externally')
    self.paths.add(p)
except (OSError, ValueError):return

def main():
    entry=Path(sys.argv[1]).resolve();args=sys.argv[2:]
    if '--out' not in args:raise SystemExit('missing output path')
    out=Path(args[args.index('--out')+1]);sys.argv=[str(entry),*args]
    sys.path.insert(0, str(entry.parent))
    reads=DependencyReads(ROOT);sys.addaudithook(reads)
    try:
        try:runpy.run_path(str(entry), run_name='__main__')
        except SystemExit as e:
            if e.code not in (None, 0):raise
    finally:reads.active=False
    paths=set(reads.paths)
    for module in list(sys.modules.values()):
        value=getattr(module, '__file__', None)
        if not value:continue
        path=Path(value).resolve()
        if path.suffix=='.py' and path.is_relative_to(ROOT) and path.is_file():
            rel=path.relative_to(ROOT).as_posix()
            if rel.startswith(('vendor/mosaic_v14/', 'cumcm_harness/')):paths.add(path)
    refs={p.relative_to(ROOT).as_posix():file_hash(p) for p in sorted(paths) if p.is_file()}
    write_json(out/ 'custom_dependencies.json', refs)
if __name__=='__main__':main()

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/mosaic14/__init__.py

```

"""MOSAIC v14 efficiency research; no hidden oracle or previous-run training."""

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/mosaic14/bootstrap.py

```

from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
V13=ROOT/ 'vendor' / 'v13'

```

```

if str(V13) not in sys.path: sys.path.insert(0,str(V13))
from harnesslab.bootstrap import core,EvaluationLedger,AuditedRun,ProblemContract,run_fast_v10,run_typed_nsga2_unique
from harnesslab.problems import make_problem,enumerate_reference
from harnesslab.metrics import score_trace

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/mosaic14/engine.py

```

"""Behavior-diverse archive search with sparse, real-evaluation-only expert credit.

Preserves distinct decision policies even at exactly equal objective vectors.
The idea is a structured search extension, not a claim to reproduce a neural MoE.
"""

from __future__ import annotations
from collections import defaultdict,deque
import numpy as np
from scipy.special import softmax
from .bootstrap import core
from .fast import PolicyCodec,FastHarness

def key(codec,x): return tuple(map(int,codec.decode(x)))

def run_fast_h0(problem,codec,seed,pop_size,max_evals,config,mode='no_memory',ties_per_objective=1,harness_type=FastHarness):
    assert mode=='no_memory' and ties_per_objective==1, 'Fast path certified only for R1 single representative'
    codec=codec if isinstance(codec,PolicyCodec) else PolicyCodec(codec)
    key_cache={}
    def key(codec,x):
        raw=np.asarray(x,dtype=float).tobytes()
        if raw not in key_cache:key_cache[raw]=codec.key(x)
        return key_cache[raw]
    rng,X,F=core.initialize(problem,pop_size,seed)
    harness=harness_type(codec,seed,max_evals,config);harness.initialize(X,F)
    # Index by exact phenotype, NOT by floating-point random-key genotype.
    seen=set(key(codec,x) for x in X)
    visits=defaultdict(int)
    a=np.ones((3,4));b=np.ones((3,4));counts=np.zeros(4,int);rewards=np.zeros(4)
    bias=np.zeros(4);history=deque(maxlen=48);score_history=deque(maxlen=48)
    X=[x.copy() for x in X];F=[f.copy() for f in F]
    evals=pop_size;log=[];rejected_duplicates=0

    def trim_pool(X,F):
        arr=np.asarray(F)
        groups={}
        for i,f in enumerate(F):groups.setdefault(tuple(f),[]).append(i)
        ordered=sorted(groups);unique=np.asarray(ordered)
        valid=core.nondominated_mask(unique)
        kept=[]
        for u in np.flatnonzero(valid):
            ids=groups[ordered[u]]
            # Retain underexplored phenotypes; zero visits to newly admitted point.
            if mode=='no_memory':limit=1
            else:limit=ties_per_objective
            ids=sorted(ids,key=lambda i:(visits[key(codec,X[i])],rng.random()))
            distinct=[];local_seen=set()
            for i in ids:

```

```

        k=key(codec,X[i])
        if k not in local_seen:
            distinct.append(i);local_seen.add(k)
            if len(distinct)>=limit:break
        kept.extend(distinct)
    if len(kept)>4*pop_size:
        loc=core.select_indices_farthest(arr[kept],4*pop_size)
        kept=[kept[i] for i in loc]
    return [X[i] for i in kept],[F[i] for i in kept]

X,F=trim_pool(X,F)
while evals<max_evals:
    arr=np.asarray(F);unique=arr;inv=np.arange(len(F))
    # Regions (objective vectors) have equal parent probability, irrespective
    # of how many behavior variants happen to have accumulated there.
    group=int(rng.integers(len(unique))); ids=np.flatnonzero(inv==group)
    nvis=np.array([visits[key(codec,X[i])] for i in ids],float)
    parent_id=int(rng.choice(ids,p=(1/(1+nvis))/(1/(1+nvis)).sum()))
    donor_id=int(rng.integers(len(X)));parent=X[parent_id];pf=F[parent_id];donor=X[donor_id]
    state=min(2,int(3*evals/max_evals))
    sample=rng.beta(a[state],b[state])
    if mode in ('uniform','no_memory','no_novelty'):sample=rng.random(4)
    if mode=='single_expert':sample=np.array([1.,0.,0.,0.])
    if mode=='quantile' and len(score_history)>=16 and evals%16==0:
        S=np.asarray(score_history)
        quality=(a.sum(0)/(a+b).sum(0))
        q=.20/4+.80*softmax((quality-quality.max())/-.20)
        candidates=[]
        for e in range(4):
            margin=np.max(np.delete(S+bias,e,axis=1),axis=1)-S[:,e]
            candidates.append(np.quantile(margin,1-q[e]))
        correction=np.asarray(candidates);correction-=correction.mean()
        bias=np.clip(.75*bias+.25*correction,-.08,.08)
    score_history.append(sample.copy())
    if mode=='quantile':sample+=bias
    ranked=np.argsort(-sample,kind='stable')
    child=None;chosen=None
    # Pure generation can be retried without a function call. The same rule
    # is provided to the duplicate-aware NSGA-II control below.
    for e in ranked:
        attempts=1 if mode=='no_novelty' else 12
        for _ in range(attempts):
            cand=codec.mutate(parent,donor,int(e),rng)
            if mode=='no_novelty' or key(codec,cand) not in seen:
                child=cand;chosen=int(e);break
            rejected_duplicates+=1
        if child is not None:break
    if child is None:
        child=problem.xl+rng.random(problem.n_var)*(problem.xu-problem.xl);chosen=4
    child,chosen=harness.intervene(child,chosen,parent,donor,seen,arr,evals)
    if getattr(harness,'selected_parent_is_donor',False):
        parent=donor;pf=F[donor_id]
    cf=problem.evaluate(child)[0];evals+=1
    harness.observe(child,cf,evals)
    visits[key(codec,parent)]+=1;seen.add(key(codec,child))
    nd=not np.any(np.all(arr<=cf,axis=1)&np.any(arr<cf,axis=1))
    dominates=bool(np.all(cf<=pf) and np.any(cf<pf))
    novelty=not np.any(np.all(arr==cf,axis=1))

```

```

reward=float(.50*dominates+.35*(nd and novelty)+.15*nd)
if chosen<4:
    # Only the actually evaluated expert receives evidence.
    a[state,chosen]=1+.995*(a[state,chosen]-1)+reward
    b[state,chosen]=1+.995*(b[state,chosen]-1)+1-reward
    counts[chosen]+=1;rewards[chosen]+=reward
if nd:
    X.append(child.copy());F.append(cf.copy());X,F=trim_pool(X,F)
log.append({'fe':evals,'expert':chosen,'reward':reward,'pool_size':len(X)})
AX=np.asarray(X);AF=np.asarray(F)
_,idx=np.unique(AF,axis=0,return_index=True);AX=AX[idx];AF=AF[idx]
if len(AF)>pop_size:
    idx=core.select_indices_farthest(AF,pop_size);AX,AF=AX[idx],AF[idx]
return core.RunResult('graph_'+mode,problem.name,seed,AX,AF,evals,[],
    {'counts':counts.tolist(),'reward_sum':rewards.tolist(),'route_log':log,
    'duplicate_proposals_rejected':rejected_duplicates,'unique_evaluated':len(seen),
    'behavior_memory':False,'novelty_filter':True,'key_cache_size':len(key_cache),'harness':harness.diagnostics()})

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/mosaic14/fast.py

"""Pure execution optimizations: policy-key/feature/front caches; same RNG calls.

No objective values cached. The objective evaluator remains untouched. Cached encoding and feature arrays are immutable internally; public returns are copies.
"""

```

from __future__ import annotations
import numpy as np
from .bootstrap import core
from harnesslab.residual import ResidualHarness
from harnesslab.efficient import EfficientModel
from harnesslab.model import features as reference_features, nd_indices

class PolicyCodec:
    def __init__(self,base):
        self.base=base; self.n_var=base.n_var; self.n=getattr(base,'n',self.n_var//2)
        self._keys={};self._encoded={};self._features={};self.feature_hits=0;self.feature_misses=0
    def key(self,x):
        raw=np.asarray(x,dtype=float).tobytes()
        k=self._keys.get(raw)
        if k is None:
            k=tuple(map(int,self.base.decode(x)));self._keys[raw]=k
        return k
    def decode(self,x):return np.asarray(self.key(x),dtype=np.int64)
    def encode(self,order):
        k=tuple(map(int,order));old=self._encoded.get(k)
        if old is None:
            if not k or len(k)!=len(set(k)) or min(k)<0 or max(k)>=self.n:raise ValueError('Invalid policy')
            old=np.zeros(self.n_var);old[list(k)]=.75;old[self.n]=.99
            for i,j in enumerate(k):old[self.n+j]=(i+.5)/self.n
            old.setflags(write=False);self._encoded[k]=old;self._keys[old.tobytes()]=k
        return old.copy()
    def mutate(self,parent,donor,expert,rng):
        # Matches every RNG call and policy update in inherited codecs.
        p=list(self.key(parent));q=list(self.key(donor));absent=[j for j in range(self.n) if j not in p]

```

```

if expert==0:
    if len(p)>1:
        a,b=rng.choice(len(p),2,replace=False);p[a],p[b]=p[b],p[a]
        elif absent:p.append(int(rng.choice(absent)))
elif expert==1:
    if len(p)>1:
        a,b=map(int,rng.choice(len(p),2,replace=False));j=p.pop(a);p.insert(b,j)
        elif absent:p.insert(0,int(rng.choice(absent)))
elif expert==2:
    if absent and (len(p)==1 or rng.random()<.5):p.insert(int(rng.integers(len(p)+1)),int(rng.choice(absent)))
    elif len(p)>1:p.pop(int(rng.integers(len(p))))
elif expert==3:
    if len(q)>1:
        a,b=sorted(map(int,rng.choice(len(q)+1,2,replace=False)));piece=q[a:b]
        else:piece=q[:]
        p=[j for j in p if j not in piece];at=int(rng.integers(len(p)+1));p[at:at]=piece
    else:raise ValueError('Unknown expert')
    return self.encode(p)
def feature_matrix(self,X,raw=False):
    X=np.atleast_2d(X)
    if raw:return X.copy()
    p=3*self.n+(self.n+1)**2;out=np.empty((len(X),p))
    for i,x in enumerate(X):
        k=self.key(x);row=self._features.get(k)
        if row is None:
            row=reference_features(x,self)[0];row.setflags(write=False);self._features[k]=row;self.feature_misses+=1
        else:self.feature_hits+=1
        out[i]=row
    return out

class CachedHV:
    def __init__(self):self.signature=None;self.rebuilds=0;self.hits=0
    def __call__(self,points,front,ref):
        points=np.atleast_2d(points);front=np.asarray(front);ref=np.asarray(ref)
        signature=(front.shape,front.tobytes(),ref.tobytes())
        if signature!=self.signature:
            f=front[np.all(front<ref,axis=1)]
            if len(f):f=f[nd_indices(f)]
            self.left=np.r_[-np.inf,f[:,0]] if len(f) else []
            self.right=np.r_[f[:,0]] if len(f) else [],ref[0]]
            self.height=np.r_[ref[1],f[:,1]] if len(f) else []
            self.signature=signature;self.rebuilds+=1
        else:self.hits+=1
        widths=np.maximum(self.right[None,:]-np.maximum(self.left[None,:],points[:,0,None]),0.)
        heights=np.maximum(self.height[None,:]-points[:,1,None],0.)
        return (widths*heights).sum(1)

from dataclasses import asdict
from collections import deque

class FastHarness(ResidualHarness):
    def __init__(self,codec,seed,budget,cfg):
        self.codec=codec;self.cfg=cfg;self.budget=budget;self.rng=np.random.default_rng(np.random.SeedSequence([seed,139831]))
        self.dataX=[];self.dataF=[];self.Z=[];self.forecasts=[];self.events=[];self.errors=deque(maxlen=16);self.calls=0;self.
        ↪ trials=0;self.override=0;self.pending=None
        self.model=EfficientModel(cfg,seed+13);self.fit_at=-1;self.key=codec.key;self.hvi=CachedHV()
    def initialize(self,X,F):

```

```

self.dataX=list(X.copy());self.dataF=list(F.copy());self.low=F.min(0);self.scale=np.maximum(np.ptp(F,axis=0),1e-9);self.ref=(F.max(0)-self.low)/self.scale+.2
↪ f.ref=(F.max(0)-self.low)/self.scale+.2
def intervene(self,child,expert,parent,donor,seen,AF,fe):
    self.pending=None;c=self.cfg
    if not c.enabled or fe<c.warmup:return child,expert
    if c.model!='random':
        if self.fit_at<0 or fe-self.fit_at>=c.refit:
            self.model.fit(self.codec.feature_matrix(np.asarray(self.dataX),c.raw_features),(np.asarray(self.dataF)-self.low)/self.scale);self.fit_at=fe
            allowed=not c.guarded or (self.model.rank>=c.rank_threshold and self.model.mae<=c.error_threshold and
            ↪ (len(self.errors)<8 or np.mean(self.errors)<=.35))
            mu,std=self.model.predict(self.codec.feature_matrix(child,c.raw_features));self.calls+=1
            # Predictions are never promoted into verified observations.
            if not np.isfinite(mu).all() or not np.isfinite(std).all():
                self.events.append(dict(fe=fe+1,version=self.model.version,reason='nonfinite_prediction_fallback'))
                return child,expert
            self.pending=dict(fe=fe+1,version=self.model.version,train_fe=self.fit_at,prediction=mu[0].tolist(),std=std[0].tolist(),override=False,reason='base')
            ↪ ist(),override=False,reason='base')
        else:allowed=True
    if not allowed:
        if self.pending:self.pending['reason']='audit_fallback'
        return child,expert
    if self.rng.random()<c.exploration:
        if self.pending:self.pending['reason']='baseline_exploration'
        return child,expert
    candidates=[child.copy()];experts=[expert];local={self.key(child)}
    # No objective function is available to this object. The default baseline
    # proposal is always in the pool; the four operator definitions do not change.
    for _ in range(c.pool-1):
        for tries in range(12):
            ee=int(self.rng.integers(4));cc=self.codec.mutate(parent,donor,ee,self.rng);self.trials+=1;k=self.key(cc)
            if k not in local and k not in seen:candidates.append(cc);experts.append(ee);local.add(k);break
    X=np.asarray(candidates)
    if c.model=='random':pick=int(self.rng.integers(len(X)))
    else:
        mu,std=self.model.predict(self.codec.feature_matrix(X,c.raw_features));self.calls+=len(X)
        if not np.isfinite(mu).all() or not np.isfinite(std).all():
            self.pending['reason']='nonfinite_prediction';return child,expert
        scoreY=mu-c.uncertainty*std
        if c.acquisition=='scalar':
            w=self.rng.dirichlet(np.ones(2));scores=-np.max(scoreY*w,axis=1)
        else:
            scores=self.hvi(scoreY,(AF-self.low)/self.scale,self.ref)
            if scores.max()<=1e-14:
                # No predicted improvement: preserve the baseline proposal.
                pick=0
            else:pick=int(np.argmax(scores))
        if c.acquisition=='scalar':pick=int(np.argmax(scores))
        self.pending=dict(fe=fe+1,version=self.model.version,train_fe=self.fit_at,prediction=mu[pick].tolist(),std=std[pick].tolist(),override=bool(pick),reason='residual' if pick else
        ↪ 'base_wins',candidate_count=len(X),baseline_prediction=mu[0].tolist())
    self.override+=bool(pick)
    return X[pick].copy(),int(experts[pick])
def observe(self,child,cf,fe):
    self.dataX.append(child.copy());self.dataF.append(cf.copy())
    if self.pending is not None:
        pred=np.asarray(self.pending['prediction']);err=float(np.mean(np.abs(pred-(cf-self.low)/self.scale)))
        if np.isfinite(err):self.errors.append(err)

```



```

        self.pending['actual']=cf.tolist();self.pending['error']=err;self.forecasts.append(self.pending);self.pending=None
def diagnostics(self):return
↪ dict(config=asdict(self.cfg),model_fits=self.model.model_fits,forecasts=self.forecasts,overrides=self.override,surroga
↪ te_predictions=self.calls,extra_pure_proposals=self.trials,events=self.events,feature_cache_hits=self.codec.feature_hi
↪ ts,feature_cache_misses=self.codec.feature_misses,hv_cache_hits=self.hvi.hits,hv_cache_rebuilds=self.hvi.rebuilds)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/mosaic14/variants.py

```

"""Explicit experimental policies. No prior-run labels or oracle access."""
from dataclasses import dataclass,asdict
import numpy as np
from .fast import FastHarness,PolicyCodec
from harnesslab.residual import ResidualConfig

@dataclass(frozen=True)
class SearchConfig(ResidualConfig):
    proposal_mode:str='legacy'    # legacy, bounded, local
    trial_cap:int=24
    local_cap:int=24

CONFIGS={
'dual24':SearchConfig(model='linear',uncertainty=0.,proposal_mode='dual',local_cap=24),
'dual48':SearchConfig(model='linear',uncertainty=0.,proposal_mode='dual',local_cap=48),
'dual24_refit32':SearchConfig(model='linear',uncertainty=0.,proposal_mode='dual',local_cap=24,refit=32),
'local24_refit128':SearchConfig(model='linear',uncertainty=0.,proposal_mode='local',local_cap=24,refit=128),
'fast_h0':SearchConfig(model='linear',uncertainty=0.),
'bounded24':SearchConfig(model='linear',uncertainty=0.,proposal_mode='bounded',trial_cap=24),
'pool4':SearchConfig(model='linear',uncertainty=0.,pool=4),
'local24':SearchConfig(model='linear',uncertainty=0.,proposal_mode='local',local_cap=24),
'local48':SearchConfig(model='linear',uncertainty=0.,proposal_mode='local',local_cap=48),
'local_all':SearchConfig(model='linear',uncertainty=0.,proposal_mode='local',local_cap=10000),
'refit32':SearchConfig(model='linear',uncertainty=0.,refit=32),
'local24_refit32':SearchConfig(model='linear',uncertainty=0.,proposal_mode='local',local_cap=24,refit=32),
'local24_random':SearchConfig(model='random',guarded=False,uncertainty=0.,proposal_mode='local',local_cap=24),
'local24_no_cache':SearchConfig(model='linear',uncertainty=0.,proposal_mode='local_no_cache',local_cap=24),
}

class SearchHarness(FastHarness):
    def __init__(self,codec,seed,budget,cfg):
        super().__init__(codec,seed,budget,cfg)
        self.neighborhood_cache={};self.neighbor_builds=0;self.neighbor_cache_hits=0
        self.neighbor_filter_checks=0;self.pool_counts=[]
    def neighbors(self,parent):
        k=self.codec.key(parent)
        if self.cfg.proposal_mode!='local_no_cache' and k in self.neighborhood_cache:
            self.neighbor_cache_hits+=1;return self.neighborhood_cache[k]
        p=list(k);n=self.codec.n;out={}
        def add(q,e):
            z=tuple(q)
            if z!=k:out.setdefault(z,e)
        for a in range(len(p)):
            for b in range(a+1,len(p)):
                q=p.copy();q[a],q[b]=q[b],q[a];add(q,0)
        for a in range(len(p)):

```

```

        for b in range(len(p)):
            if a==b:continue
            q=p.copy();x=q.pop(a);q.insert(b,x);add(q,1)
    for j in range(n):
        if j in k:continue
        for a in range(len(p)+1):
            q=p.copy();q.insert(a,j);add(q,2)
    if len(p)>1:
        for a in range(len(p)):
            q=p.copy();q.pop(a);add(q,2)
    # Some moves have several representations. Preserve deterministic first
    # producer; records report it as a local-pool generator, not causal credit.
    ans=list(out.items());self.neighbor_builds+=1
    if self.cfg.proposal_mode!='local_no_cache':self.neighborhood_cache[k]=ans
    return ans

def make_candidates(self,child,expert,parent,donor,seen):
    c=self.cfg;candidates=[child.copy()];experts=[expert];local={self.key(child)};origins=[0]
    self.selected_parent_is_donor=False
    if c.proposal_mode=='dual':
        available=[];seen_pool=set(local)
        # Combine two actual evaluated parent neighborhoods. Deduplicate
        # with deterministic first-source provenance, then sample a bounded
        # pool. No target-specific simulator parameters are accessed.
        for origin,base in enumerate([parent,donor]):
            nb=self.neighbors(base);self.neighbor_filter_checks+=len(nb)
            for k,e in nb:
                if k not in seen and k not in seen_pool:
                    available.append((k,e,origin));seen_pool.add(k)
            take=min(c.local_cap,len(available))
            ids=self.rng.choice(len(available),take,replace=False) if take else []
            for i in ids:
                k,e,origin=available[int(i)];candidates.append(self.codec.encode(k));experts.append(e);origins.append(origin)
    elif c.proposal_mode.startswith('local'):
        nb=self.neighbors(parent);self.neighbor_filter_checks+=len(nb)
        available=[(k,e) for k,e in nb if k not in seen and k not in local]
        take=min(c.local_cap,len(available))
        # Same random sampling rule for cached and uncached ablation.
        ids=self.rng.choice(len(available),take,replace=False) if take else []
        for i in ids:
            k,e=available[int(i)];candidates.append(self.codec.encode(k));experts.append(e);local.add(k)
        # The incumbent x0 still includes the donor-segment operator. Do not
        # silently generate new donor-search operators in this experiment.
    elif c.proposal_mode=='bounded':
        for _ in range(c.trial_cap):
            if len(candidates)>=c.pool:break
            ee=int(self.rng.integers(4));cc=self.codec.mutate(parent,donor,ee,self.rng);self.trials+=1;k=self.key(cc)
            if k not in local and k not in seen:candidates.append(cc);experts.append(ee);local.add(k)
    else:
        for _ in range(c.pool-1):
            for tries in range(12):
                ee=int(self.rng.integers(4));cc=self.codec.mutate(parent,donor,ee,self.rng);self.trials+=1;k=self.key(cc)
                if k not in local and k not in seen:candidates.append(cc);experts.append(ee);local.add(k);break
        self.pool_counts.append(len(candidates));self.pool_origins=origins if c.proposal_mode=='dual' else
        ↪ [0]*len(candidates);return np.asarray(candidates),experts

def intervene(self,child,expert,parent,donor,seen,AF,fe):
    self.pending=None;c=self.cfg;self.selected_parent_is_donor=False
    if not c.enabled or fe<c.warmup:return child,expert
    if c.model!='random':

```

```

if self.fit_at<0 or fe-self.fit_at>=c.refit:
    self.model.fit(self.codec.feature_matrix(np.asarray(self.dataX),c.raw_features),(np.asarray(self.dataF)-self.l_j
    ↪ ow)/self.scale);self.fit_at=fe
allowed=not c.guarded or (self.model.rank>=c.rank_threshold and self.model.mae<=c.error_threshold and
    ↪ (len(self.errors)<8 or np.mean(self.errors)<=.35))
mu,std=self.model.predict(self.codec.feature_matrix(child,c.raw_features));self.calls+=1
# Predictions are never promoted into verified observations.
if not np.isfinite(mu).all() or not np.isfinite(std).all():
    self.events.append(dict(fe=fe+1,version=self.model.version,reason='nonfinite_prediction_fallback'))
    return child,expert
self.pending=dict(fe=fe+1,version=self.model.version,train_fe=self.fit_at,prediction=mu[0].tolist(),std=std[0].tol
    ↪ ist(),override=False,reason='base')
else:allowed=True
if not allowed:
    if self.pending:self.pending['reason']='audit_fallback'
    return child,expert
if self.rng.random()<c.exploration:
    if self.pending:self.pending['reason']='baseline_exploration'
    return child,expert
X,experts=self.make_candidates(child,expert,parent,donor,seen)
if c.model=='random':pick=int(self.rng.integers(len(X)))
else:
    mu,std=self.model.predict(self.codec.feature_matrix(X,c.raw_features));self.calls+=len(X)
    if not np.isfinite(mu).all() or not np.isfinite(std).all():
        self.pending['reason']='nonfinite_prediction';return child,expert
    scoreY=mu-c.uncertainty*std
    if c.acquisition=='scalar':
        w=self.rng.dirichlet(np.ones(2));scores=-np.max(scoreY*w,axis=1)
    else:
        scores=self.hvi(scoreY,(AF-self.low)/self.scale,self.ref)
        if scores.max()<=1e-14:
            # No predicted improvement: preserve the baseline proposal.
            pick=0
        else:pick=int(np.argmax(scores))
    if c.acquisition=='scalar':pick=int(np.argmax(scores))
    self.pending=dict(fe=fe+1,version=self.model.version,train_fe=self.fit_at,prediction=mu[pick].tolist(),std=std[pic
    ↪ k].tolist(),override=bool(pick),reason='residual' if pick else
    ↪ 'base_wins',candidate_count=len(X),baseline_prediction=mu[0].tolist())
self.selected_parent_is_donor=bool(self.pool_origins[pick])
if self.pending:self.pending['parent_is_donor']=self.selected_parent_is_donor
self.override+=bool(pick)
return X[pick].copy(),int(experts[pick])
def diagnostics(self):
    d=super().diagnostics();d.update(neighbor_builds=self.neighbor_builds,neighbor_cache_hits=self.neighbor_cache_hits,nei
    ↪ ghbor_filter_checks=self.neighbor_filter_checks,pool_counts=self.pool_counts);return d

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/mosaic_solve.py

"""Deployment wrapper. Search code/configuration is separately frozen.

No objective caching, cross-run training data, hidden enumeration, or automatic
activation of unsupported variable/constraint types.
"""

from mosaic14.bootstrap import EvaluationLedger,AuditedRun,ProblemContract

```

from mosaic14.engine import run_fast_h0
from mosaic14.fast import PolicyCodec
from mosaic14.bootstrap import run_fast_v10
from mosaic14.variants import CONFIGS, SearchHarness
from harnesslab.api import solve as previous_solve
from harnesslab.problems import OrderedSubsetCodec
from mosaic.rgv import RGVCodec

VERSION='MOSAIC-v14-Efficiency-Lab'

def solve(problem, *, seed:int=1, budget:int=1024, population:int=32,
          contract:ProblemContract=ProblemContract(), strategy:str='incumbent')->AuditedRun:
    """Use fast_h0 for execution-equivalent accelerated v13 H0.

    incumbent: unchanged Fast-v10 dispatch, including its continuous fallback.
    fast_h0: exact-trace engineering candidate (tested numerical environment).
    local24/refit32: explicitly experimental policy changes, not auto-selected.
    """

    if strategy=='incumbent':
        return previous_solve(problem, seed, budget, population, contract, strategy='incumbent')
    if strategy not in ('incumbent_cached', 'fast_h0', 'local24', 'refit32'):
        raise ValueError('Choose incumbent, incumbent_cached, fast_h0, local24, or refit32')
    if budget < population or population < 8: raise ValueError('Require budget >= population >= 8')
    if not contract.deterministic or contract.constraints not in ('box', 'feasible_decoder'):
        raise NotImplementedError('Noise and general constraints need separately validated adapters')
    if contract.representation != 'ordered_subset' or contract.codec is None or problem.n_obj != 2:
        raise NotImplementedError('This extension supports deterministic bi-objective ordered subsets only')
    if type(contract.codec) not in (RGVCodec, OrderedSubsetCodec):
        raise NotImplementedError('The exact codec rewrite is certified only for the two included codecs')
    ledger = EvaluationLedger(problem, budget)
    if strategy == 'incumbent_cached':
        result = run_fast_v10(ledger, PolicyCodec(contract.codec), seed, population, budget)
    else:
        result = run_fast_h0(ledger, contract.codec, seed, population, budget, CONFIGS[strategy], harness_type=SearchHarness)
    if result.evaluations != ledger.spent or ledger.spent != budget: raise AssertionError('Budget mismatch')
    result.algorithm = VERSION + '-' + strategy
    result.diagnostics.update(strategy=strategy, engineering_equivalent_to=('v13-H0' if strategy == 'fast_h0' else 'Fast-v10' if
    ↪ strategy == 'incumbent_cached' else None),
                             no_universal_sota_claim=True, empirical_scope_only=True)
    return AuditedRun(result, ledger)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/__init__.py

"""Research harness; no candidate is a default replacement until tested."""

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/api.py

"""Validated opt-in research API. No automatic claim of universal superiority."""
from .bootstrap import ProblemContract, AuditedRun, EvaluationLedger

```

from src.fast_api import solve_fast
from .efficient import FAST_CONFIGS
from .efficient_engine import run_efficient

VERSION='MOSAIC-v13-Audited-Infill'

def solve(problem,seed:int=1,budget:int=1024,population:int=32,
          contract:ProblemContract=ProblemContract(),strategy:str='incumbent')->AuditedRun:
    """Default stays Fast-v10. Set strategy='audited_infill' explicitly.

    New branch supports deterministic, feasible ordered subsets with exactly two
    minimization objectives. Its predictor learns ONLY the current run's paid
    evaluations. Reference fronts and model internals are never an input.
    """
    if strategy=='incumbent':return solve_fast(problem,seed,budget,population,contract)
    if strategy!='audited_infill':raise ValueError("strategy must be 'incumbent' or 'audited_infill'")
    if budget<population or population<8:raise ValueError('Require budget >= population >= 8')
    if not contract.deterministic or contract.constraints not in ('box','feasible_decoder'):
        raise NotImplementedError('Noise/general constraints need separately validated evaluators')
    if contract.representation!='ordered_subset' or contract.codec is None or problem.n_obj!=2:
        raise NotImplementedError('Audited infill currently requires deterministic bi-objective ordered-subset encoding')
    ledger=EvaluationLedger(problem,budget)
    result=run_efficient(ledger,contract.codec,seed,population,budget,FAST_CONFIGS['no_uncertainty_linear'])
    if result.evaluations!=ledger.spent or ledger.spent!=budget:raise AssertionError('Evaluation accounting mismatch')
    result.algorithm=VERSION
    result.diagnostics.update(research_opt_in=True,official_sota=False,guaranteed_nonregression=False)
    return AuditedRun(result,ledger)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/bootstrap.py

```

from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
VENDOR=ROOT/'vendor'
if str(VENDOR) not in sys.path:sys.path.insert(0,str(VENDOR))
from src.bootstrap import core,EvaluationLedger,ProblemContract,AuditedRun
from src.fast_v10 import run_fast_v10
from src.problems import make_problem as old_make_problem
from mosaic.typed_baseline import run_typed_nsga2_unique

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/efficient.py

```

"""Memoize deterministic codec operations, never objective evaluations.
Operator code and all RNG calls are delegated unchanged. Cached outputs are copies.
"""
import numpy as np
from .residual import ResidualHarness,ResidualConfig,ObjectiveModel

FAST_CONFIGS={

```

```

'guarded_linear':ResidualConfig(model='linear'),
'unguarded_linear':ResidualConfig(model='linear',guarded=False),
'random_residual':ResidualConfig(model='random',guarded=False),
'scalar_linear':ResidualConfig(model='linear',acquisition='scalar'),
'raw_linear':ResidualConfig(model='linear',raw_features=True),
'no_uncertainty_linear':ResidualConfig(model='linear',uncertainty=0.),
'guarded_hybrid':ResidualConfig(),
'off_fast':ResidualConfig(enabled=False),
}

class CachedCodec:
    def __init__(self,base):
        self.base=base;self.n_var=base.n_var;self.n=getattr(base,'n',self.n_var//2)
        self._dec={};self._enc={}
    def decode(self,x):
        k=np.asarray(x,dtype=float).tobytes()
        if k not in self._dec:self._dec[k]=self.base.decode(x)
        return self._dec[k].copy()
    def encode(self,p):
        k=tuple(map(int,p))
        if k not in self._enc:self._enc[k]=self.base.encode(p)
        return self._enc[k].copy()
    def mutate(self,parent,donor,expert,rng):
        return type(self.base).mutate(self,parent,donor,expert,rng)

class EfficientModel(ObjectiveModel):
    def predict(self,Z):
        r=Z@self.ridge.coef_.T+self.ridge.intercept_
        if self.tree is None:return r,np.tile(self.error,(len(Z),1))
        ps=np.asarray([t.predict(Z) for t in self.tree.estimators_])
        return np.where(self.choices,r,ps.mean(0)),np.where(self.choices,self.error,ps.std(0))

class EfficientHarness(ResidualHarness):
    def __init__(self,codec,seed,budget,cfg):
        super().__init__(CachedCodec(codec),seed,budget,cfg)
        self.model=EfficientModel(cfg,seed+13)
    def diagnostics(self):
        d=super().diagnostics();d['codec_cached']=True;d['codec_cache_sizes']=[len(self.codec._enc),len(self.codec._dec)];return d
↪ rn d

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/efficient_engine.py

"""Behavior-diverse archive search with sparse, real-evaluation-only expert credit.

Preserves distinct decision policies even at exactly equal objective vectors.

The idea is a structured search extension, not a claim to reproduce a neural MoE.

"""

```

from __future__ import annotations
from collections import defaultdict,deque
import numpy as np
from scipy.special import softmax
from .bootstrap import core
from .efficient import EfficientHarness as ResidualHarness

```

```

def key(codec,x): return tuple(map(int,codec.decode(x)))

def run_efficient(problem,codec,seed,pop_size,max_evals,config,mode='no_memory',ties_per_objective=1):
    assert mode=='no_memory' and ties_per_objective==1, 'Fast path certified only for R1 single representative'
    key_cache={}
    def key(codec,x):
        raw=np.asarray(x,dtype=float).tobytes()
        if raw not in key_cache:key_cache[raw]=tuple(map(int,codec.decode(x)))
        return key_cache[raw]
    rng,X,F=core.initialize(problem,pop_size,seed)
    harness=ResidualHarness(codec,seed,max_evals,config);harness.initialize(X,F)
    # Index by exact phenotype, NOT by floating-point random-key genotype.
    seen=set(key(codec,x) for x in X)
    visits=defaultdict(int)
    a=np.ones((3,4));b=np.ones((3,4));counts=np.zeros(4,int);rewards=np.zeros(4)
    bias=np.zeros(4);history=deque(maxlen=48);score_history=deque(maxlen=48)
    X=[x.copy() for x in X];F=[f.copy() for f in F]
    evals=pop_size;log=[];rejected_duplicates=0

    def trim_pool(X,F):
        arr=np.asarray(F)
        groups={}
        for i,f in enumerate(F):groups.setdefault(tuple(f),[]).append(i)
        ordered=sorted(groups);unique=np.asarray(ordered)
        valid=core.nondominated_mask(unique)
        kept=[]
        for u in np.flatnonzero(valid):
            ids=groups[ordered[u]]
            # Retain underexplored phenotypes; zero visits to newly admitted point.
            if mode=='no_memory':limit=1
            else:limit=ties_per_objective
            ids=sorted(ids,key=lambda i:(visits[key(codec,X[i])],rng.random()))
            distinct=[];local_seen=set()
            for i in ids:
                k=key(codec,X[i])
                if k not in local_seen:
                    distinct.append(i);local_seen.add(k)
                if len(distinct)>=limit:break
            kept.extend(distinct)
        if len(kept)>4*pop_size:
            loc=core.select_indices_farthest(arr[kept],4*pop_size)
            kept=[kept[i] for i in loc]
        return [X[i] for i in kept],[F[i] for i in kept]

    X,F=trim_pool(X,F)
    while evals<max_evals:
        arr=np.asarray(F);unique=arr;inv=np.arange(len(F))
        # Regions (objective vectors) have equal parent probability, irrespective
        # of how many behavior variants happen to have accumulated there.
        group=int(rng.integers(len(unique))); ids=np.flatnonzero(inv==group)
        nvis=np.array([visits[key(codec,X[i])] for i in ids],float)
        parent_id=int(rng.choice(ids,p=(1/(1+nvis))/(1/(1+nvis)).sum()))
        donor_id=int(rng.integers(len(X)));parent=X[parent_id];pf=F[parent_id];donor=X[donor_id]
        state=min(2,int(3*evals/max_evals))
        sample=rng.beta(a[state],b[state])
        if mode in ('uniform','no_memory','no_novelty'):sample=rng.random(4)
        if mode=='single_expert':sample=np.array([1.,0.,0.,0.])
        if mode=='quantile' and len(score_history)>=16 and evals%16==0:

```

```

S=np.asarray(score_history)
quality=(a.sum(0)/(a+b).sum(0))
q=.20/4+.80*softmax((quality-quality.max())/20)
candidates=[]
for e in range(4):
    margin=np.max(np.delete(S+bias,e,axis=1),axis=1)-S[:,e]
    candidates.append(np.quantile(margin,1-q[e]))
correction=np.asarray(candidates);correction-=correction.mean()
bias=np.clip(.75*bias+.25*correction,-.08,.08)
score_history.append(sample.copy())
if mode=='quantile':sample+=bias
ranked=np.argsort(-sample,kind='stable')
child=None;chosen=None
# Pure generation can be retried without a function call. The same rule
# is provided to the duplicate-aware NSGA-II control below.
for e in ranked:
    attempts=1 if mode=='no_novelty' else 12
    for _ in range(attempts):
        cand=codec.mutate(parent,donor,int(e),rng)
        if mode=='no_novelty' or key(codec,cand) not in seen:
            child=cand;chosen=int(e);break
        rejected_duplicates+=1
    if child is not None:break
if child is None:
    child=problem.xl+rng.random(problem.n_var)*(problem.xu-problem.xl);chosen=4
child,chosen=harness.intervene(child,chosen,parent,donor,seen,arr,evals)
cf=problem.evaluate(child)[0];evals+=1
harness.observe(child,cf,evals)
visits[key(codec,parent)]+=1;seen.add(key(codec,child))
nd=not np.any(np.all(arr<=cf,axis=1)&np.any(arr<cf,axis=1))
dominates=bool(np.all(cf<=pf) and np.any(cf<pf))
novelty=not np.any(np.all(arr==cf,axis=1))
reward=float(.50*dominates+.35*(nd and novelty)+.15*nd)
if chosen<4:
    # Only the actually evaluated expert receives evidence.
    a[state,chosen]=1+.995*(a[state,chosen]-1)+reward
    b[state,chosen]=1+.995*(b[state,chosen]-1)+1-reward
    counts[chosen]+=1;rewards[chosen]+=reward
if nd:
    X.append(child.copy());F.append(cf.copy());X,F=trim_pool(X,F)
log.append({'fe':evals,'expert':chosen,'reward':reward,'pool_size':len(X)})
AX=np.asarray(X);AF=np.asarray(F)
_,idx=np.unique(AF,axis=0,return_index=True);AX=AX[idx];AF=AF[idx]
if len(AF)>pop_size:
    idx=core.select_indices_farthest(AF,pop_size);AX,AF=AX[idx],AF[idx]
return core.RunResult('graph_'+mode,problem.name,seed,AX,AF,evals,[],
    {'counts':counts.tolist(),'reward_sum':rewards.tolist(),'route_log':log,
    'duplicate_proposals_rejected':rejected_duplicates,'unique_evaluated':len(seen),
    'behavior_memory':False,'novelty_filter':True,'key_cache_size':len(key_cache),'harness':harness.diagnostics()})

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/metrics.py

```

"""Independent finite-front scoring, callable only after an optimizer finishes."""
import numpy as np

```



```

from scipy.spatial.distance import cdist

def nd2(F):
    F=np.unique(np.atleast_2d(F),axis=0)
    ids=np.lexsort((F[:,1],F[:,0]));last=np.inf;out=[]
    for i in ids:
        if F[i,1]<last:out.append(i);last=F[i,1]
    return F[out]

def hv2(F,ref=(1.1,1.1)):
    F=np.atleast_2d(F);r=np.asarray(ref)
    F=F[np.all(F<r,axis=1)]
    last=r[1];v=0.
    for f in F[np.argsort(F[:,0],kind='stable')]:
        if f[1]<last:v+=(r[0]-f[0])*(last-f[1]);last=f[1]
    return float(v)

def metrics(F,R,rounding=True):
    low=np.min(R,axis=0);scale=np.maximum(np.ptp(R,axis=0),1e-12)
    y=(np.atleast_2d(F)-low)/scale;ref=(np.atleast_2d(R)-low)/scale
    if rounding:y=np.round(y,10);ref=np.round(ref,10)
    y=nd2(y);ref=nd2(ref);hvr=hv2(ref);hvy=hv2(y)
    igd=np.sqrt(np.sum(np.maximum(y[None,:]-ref[:,None,:],0.))**2,axis=-1)).min(axis=1).mean()
    covered=(cdist(ref,y,metric='chebyshev').min(axis=1)<=1e-9)
    return dict(hv_gap=float(max(0.,hvr-hvy)/max(hvr,1e-15)),igd_plus=float(igd),
                recall=float(covered.mean()),numeric_front_hit=bool(covered.all()),front_size=len(y),reference_size=len(ref))

def score_trace(F,outF,R,budget):
    cs=[v for v in [32,64,128,256,512,1024,2048] if v<=budget]
    if cs[-1]!=budget:cs.append(budget)
    trace=[dict(fe=c,**metrics(F[:c],R)) for c in cs]
    auc=float(np.trapezoid([x['hv_gap'] for x in trace],np.log2(cs))/(np.log2(cs[-1])-np.log2(cs[0])))
    end=metrics(outF,R);raw=metrics(outF,R,rounding=False)
    return dict(auc=auc,final_gap=end['hv_gap'],igd=end['igd_plus'],recall=end['recall'],
                hit=end['numeric_front_hit'],front_size=end['front_size'],reference_size=end['reference_size'],
                observed_gap=trace[-1]['hv_gap'],raw_final_gap=raw['hv_gap'],raw_igd=raw['igd_plus'],trace=trace)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/model.py

```

"""Pure prediction: cannot evaluate objective or write verified archives."""
import numpy as np
from sklearn.ensemble import ExtraTreesRegressor
from scipy.stats import spearmanr

def features(X,codec,raw=False):
    X=np.atleast_2d(X)
    if raw:return X.copy()
    n=X.shape[1]//2;p=2*n*(n+1)**2+n
    out=np.zeros((len(X),p))
    for i,x in enumerate(X):
        order=codec.decode(x);out[i,order]=1
        out[i,n+order]=(np.arange(len(order))+1)/n
        nodes=np.r_[n,order,n];a=nodes[:-1];b=nodes[1:]
        out[i,2*n+a*(n+1)+b]=1

```

```

        out[i,2*n+(n+1)**2+len(order)-1]=1
    return out

class PredictionModel:
    def __init__(self,seed):self.seed=seed;self.version=0;self.validation_score=0.;self.validation_error=float('inf')
    def _new(self):return
    ↪ ExtraTreesRegressor(n_estimators=16,max_depth=14,min_samples_leaf=2,max_features=1.,n_jobs=1,random_state=self.seed)
    def fit(self,Z,Y):
        # Time-ordered validation: the most recent evaluated points were not in
        # the fit used to score validity. All these labels have already cost FE.
        hold=min(32,max(12,len(Z)//4));m=self._new();m.fit(Z[:-hold],Y[:-hold]);pred=m.predict(Z[-hold:])
        cor=[]
        for j in range(Y.shape[1]):
            if np.std(Y[:-hold:,j])<1e-12 or np.std(pred[:-hold:,j])<1e-12:cor.append(0.)
            else:cor.append(float(spearmanr(Y[:-hold:,j],pred[:-hold:,j]).statistic))
        self.validation_score=float(min(cor));self.validation_error=float(np.mean(np.abs(pred-Y[:-hold:])))
        self.model=self._new();self.model.fit(Z,Y);self.version+=1;self.fitted_until=len(Z)
        return dict(version=self.version,fe=len(Z),validation_rank=self.validation_score,validation_mae=self.validation_error)
    def predict(self,Z):
        ps=np.asarray([m.predict(Z) for m in self.model.estimators_])
        return ps.mean(0),ps.std(0)

def nd_indices(F):
    F=np.asarray(F);ids=np.lexsort((F[:,1],F[:,0]));best=np.inf;keep=[]
    for i in ids:
        if F[i,1]<best:keep.append(i);best=F[i,1]
    return np.asarray(keep,int)

def hypervolume_improvement(points,front,ref):
    """Exact 2D HVI for predicted points, independently versus the observed front."""
    points=np.atleast_2d(points);front=np.asarray(front);ref=np.asarray(ref)
    front=front[np.all(front<ref,axis=1)]
    if len(front):front=front[nd_indices(front)]
    left=np.r_[-np.inf,front[:,0] if len(front) else []]
    right=np.r_[front[:,0] if len(front) else [],ref[0]]
    height=np.r_[ref[1],front[:,1] if len(front) else []]
    widths=np.maximum(right[None,:]-np.maximum(left[None,:],points[:,0,None]),0.)
    heights=np.maximum(height[None,:]-points[:,1,None],0.)
    return (widths*heights).sum(1)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/problems.py

```

"""Inherited RGV; variable-sized tours and a NEW synthetic due-date job model.
All dimensions, tasks and objective semantics are explicitly declared. The job
model is not a claim to solve another official competition problem.
"""

from __future__ import annotations
import numpy as np
from itertools import permutations
from numba import njit
from .bootstrap import old_make_problem

class OrderedSubsetCodec:
    def __init__(self,n):self.n=int(n);self.n_var=2*self.n

```

```

def decode(self,x):
    x=np.asarray(x);active=x[:self.n]>=.5
    if not np.any(active):active[int(np.argmax(x[:self.n]))]=True
    p=np.argsort(x[self.n:],kind='stable');return p[active[p]].astype(np.int64)
def encode(self,p):
    p=np.asarray(p,dtype=int)
    if len(p)==0 or len(np.unique(p))!=len(p) or p.min()<0 or p.max()>=self.n:raise ValueError('Invalid policy')
    x=np.zeros(2*self.n);x[p]=.75;x[self.n:]=.99
    for k,j in enumerate(p):x[self.n+j]=(k+.5)/self.n
    return x
def mutate(self,parent,donor,expert,rng):
    p=list(map(int,self.decode(parent)));q=list(map(int,self.decode(donor)))
    absent=[j for j in range(self.n) if j not in p]
    if expert==0:
        if len(p)>1:
            a,b=rng.choice(len(p),2,replace=False);p[a],p[b]=p[b],p[a]
            elif absent:p.append(int(rng.choice(absent)))
    elif expert==1:
        if len(p)>1:
            a,b=map(int,rng.choice(len(p),2,replace=False));j=p.pop(a);p.insert(b,j)
            elif absent:p.insert(0,int(rng.choice(absent)))
    elif expert==2:
        if absent and (len(p)==1 or rng.random()<.5):p.insert(int(rng.integers(len(p)+1)),int(rng.choice(absent)))
        elif len(p)>1:p.pop(int(rng.integers(len(p))))
    elif expert==3:
        if len(q)>1:
            a,b=sorted(map(int,rng.choice(len(q)+1,2,replace=False)));piece=q[a:b]
            else:piece=q[:]
            p=[j for j in p if j not in piece];at=int(rng.integers(len(p)+1));p[at:at]=piece
        else:raise ValueError('Unknown operator')
    return self.encode(p)

@jit(cache=True)
def tour_batch(orders,sizes,dist,values,service,discount):
    n=len(values);out=np.zeros((len(orders),2))
    for i in range(len(orders)):
        t=0.;reward=0.;prev=n
        for k in range(sizes[i]):
            j=orders[i,k];t+=dist[prev,j]+service[j]
            reward+=values[j]*np.exp(-t/discount[j]);prev=j
        out[i,0]=-reward;out[i,1]=t+dist[prev,n]
    return out

@jit(cache=True)
def job_batch(orders,sizes,setup,values,process,dues,penalty):
    n=len(values);out=np.zeros((len(orders),2))
    for i in range(len(orders)):
        t=0.;reward=0.;prev=n
        for k in range(sizes[i]):
            j=orders[i,k];t+=setup[prev,j]+process[j]
            reward+=max(0.,values[j]-penalty[j]*max(t-dues[j],0.));prev=j
        out[i,0]=-reward;out[i,1]=t
    return out

class SequenceProblem:
    def __init__(self,spec):
        self.spec=spec;self.name=spec['id'];self.n=len(spec['values']);self.codec=OrderedSubsetCodec(self.n)
        self.n_var=2*self.n;self.n_obj=2;self.xl=np.zeros(self.n_var);self.xu=np.ones(self.n_var);self.calls=0

```

```

def evaluate(self,X):
    X=np.atleast_2d(np.asarray(X,float));orders=np.zeros((len(X),self.n),np.int64);sizes=np.zeros(len(X),np.int64)
    for i,x in enumerate(X):
        p=self.codec.decode(x);orders[i,:len(p)]=p;sizes[i]=len(p)
    self.calls+=len(X);return self.evaluate_orders(orders,sizes)
def evaluate_orders(self,o,s):
    d=self.spec
    if d['kind']=='tour':return
    ↪ tour_batch(o,s,np.array(d['dist']),np.array(d['values']),np.array(d['service']),np.array(d['discount']))
    return job_batch(o,s,np.array(d['setup']),np.array(d['values']),np.array(d['process']),np.array(d['dues']),np.array(d[
    ↪ 'penalty']))
def pareto_front(self,*a,**kw):raise RuntimeError('Reference front forbidden during search')

def make_problem(spec):
    return old_make_problem(spec) if spec['kind']=='rgv' else SequenceProblem(spec)

def enumerate_reference(spec):
    p=make_problem(spec);n=p.n_var//2
    if n>8:raise ValueError('Exhaustive certificate deliberately limited to n<=8')
    orders=[];sizes=[]
    for k in range(1,n+1):
        for perm in permutations(range(n),k):orders.append(perm+(-1,)*(n-k));sizes.append(k)
    orders=np.asarray(orders,np.int64);sizes=np.asarray(sizes,np.int64)
    if spec['kind']=='rgv':
        from mosaic.rgv import batch_simulate
        F=batch_simulate(orders,sizes,p.travel,p.process,p.loads,p.wash,p.horizon)
    else:F=p.evaluate_orders(orders,sizes)
    ids=np.lexsort((F[:,1],F[:,0]));best=np.inf;keep=[]
    for j in ids:
        if F[j,1]<best:best=F[j,1];keep.append(j)
    return F[keep],orders[keep],sizes[keep],len(F)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/harnesslab/residual.py

```

"""Second development family: auditable residual preselection, not an LLM.
Preserves the incumbent's proposed point and its RNG when no intervention occurs.
Per-objective model choice is checked against already-paid chronological holdout.
"""

```

```

from __future__ import annotations
from dataclasses import dataclass,asdict
from collections import deque
import numpy as np
from sklearn.linear_model import Ridge
from sklearn.ensemble import ExtraTreesRegressor
from scipy.stats import spearmanr
from .model import features,hypervolume_improvement

```

```
@dataclass(frozen=True)
```

```

class ResidualConfig:
    model:str='hybrid'
    guarded:bool=True
    enabled:bool=True
    pool:int=8
    warmup:int=64

```

```

refit:int=64
exploration:float=.20
uncertainty:float=.15
rank_threshold:float=.35
error_threshold:float=.25
acquisition:str='hvi'
raw_features:bool=False

RESIDUAL_CONFIGS={
    'residual':ResidualConfig(),
    'residual_no_gate':ResidualConfig(guarded=False),
    'residual_linear':ResidualConfig(model='linear'),
    'residual_trees':ResidualConfig(model='trees'),
    'residual_random':ResidualConfig(model='random',guarded=False),
    'residual_scalar':ResidualConfig(acquisition='scalar'),
    'residual_strict':ResidualConfig(rank_threshold=.60,error_threshold=.15),
    'residual_off':ResidualConfig(enabled=False),
}

class ObjectiveModel:
    def __init__(self,cfg,seed):self.cfg=cfg;self.seed=seed;self.version=0;self.model_fits=[]
    def newtree(self):return
    ↪ ExtraTreesRegressor(n_estimators=12,min_samples_leaf=2,max_depth=14,n_jobs=1,random_state=self.seed)
    def fit(self,Z,Y):
        n=len(Z);h=min(32,max(12,n//4));train,test=Z[:~h],Z[~h:];yt,yv=Y[:~h],Y[~h:]
        r=Ridge(alpha=.10).fit(train,yt);rp=r.predict(test)
        if self.cfg.model!='linear':
            tree=self.newtree().fit(train,yt);tp=tree.predict(test)
        else:tp=rp
        re=np.mean(np.abs(rp-yv),axis=0);te=np.mean(np.abs(tp-yv),axis=0)
        choices=re<=te
        if self.cfg.model=='linear':choices[:]=True
        if self.cfg.model=='trees':choices[:]=False
        pred=np.where(choices,rp,tp);ranks=[]
        for j in range(Y.shape[1]):
            if np.std(yv[:,j])<1e-12 or np.std(pred[:,j])<1e-12:ranks.append(0.)
            else:ranks.append(float(spearmanr(yv[:,j],pred[:,j]).statistic))
        self.rank=float(min(ranks));self.mae=float(np.mean(np.abs(pred-yv)));self.error=np.sqrt(np.mean((pred-yv)**2,axis=0))
        self.choices=choices;self.ridge=Ridge(alpha=.1).fit(Z,Y)
        self.tree=self.newtree().fit(Z,Y) if not choices.all() else None
        self.version+=1;self.fitted_fe=n
        rec=dict(version=self.version,fe=n,rank=self.rank,mae=self.mae,per_objective=['linear'if c else 'trees' for c in
        ↪ choices],linear_holdout_mae=re.tolist(),trees_holdout_mae=te.tolist())
        self.model_fits.append(rec)
    def predict(self,Z):
        r=self.ridge.predict(Z)
        if self.tree is None:return r,np.tile(self.error,(len(Z),1))
        ps=np.asarray([t.predict(Z) for t in
        ↪ self.tree.estimators_]);mu=np.where(self.choices,r,ps.mean(0));sigma=np.where(self.choices,self.error,ps.std(0))
        return mu,sigma

class ResidualHarness:
    def __init__(self,codec,seed,budget,cfg):
        self.codec=codec;self.cfg=cfg;self.budget=budget;self.rng=np.random.default_rng(np.random.SeedSequence([seed,139831]))
        self.dataX=[];self.dataF=[];self.Z=[];self.forecasts=[];self.events=[];self.errors=deque(maxlen=16);self.calls=0;self.
        ↪ trials=0;self.override=0;self.pending=None
        self.model=ObjectiveModel(cfg,seed+13);self.fit_at=-1;self.key=lambda x:tuple(map(int,codec.decode(x)))
    def initialize(self,X,F):

```

```

self.dataX=list(X.copy());self.dataF=list(F.copy());self.low=F.min(0);self.scale=np.maximum(np.ptp(F,axis=0),1e-9);self.ref=(F.max(0)-self.low)/self.scale+.2
↪ f.ref=(F.max(0)-self.low)/self.scale+.2
def intervene(self,child,expert,parent,donor,seen,AF,fe):
    self.pending=None;c=self.cfg
    if not c.enabled or fe<c.warmup:return child,expert
    if c.model!='random':
        if self.fit_at<0 or fe-self.fit_at>=c.refit:
            self.model.fit(features(np.asarray(self.dataX),self.codec,c.raw_features),(np.asarray(self.dataF)-self.low)/self.scale);self.fit_at=fe
        allowed=not c.guarded or (self.model.rank>=c.rank_threshold and self.model.mae<=c.error_threshold and
↪ (len(self.errors)<8 or np.mean(self.errors)<=.35))
        mu,std=self.model.predict(features(child,self.codec,c.raw_features));self.calls+=1
        # Predictions are never promoted into verified observations.
        if not np.isfinite(mu).all() or not np.isfinite(std).all():
            self.events.append(dict(fe=fe+1,version=self.model.version,reason='nonfinite_prediction_fallback'))
            return child,expert
        self.pending=dict(fe=fe+1,version=self.model.version,train_fe=self.fit_at,prediction=mu[0].tolist(),std=std[0].tolist(),override=False,reason='base')
↪ ist(),override=False,reason='base')
    else:allowed=True
    if not allowed:
        if self.pending:self.pending['reason']='audit_fallback'
        return child,expert
    if self.rng.random()<c.exploration:
        if self.pending:self.pending['reason']='baseline_exploration'
        return child,expert
    candidates=[child.copy()];experts=[expert];local={self.key(child)}
    # No objective function is available to this object. The default baseline
    # proposal is always in the pool; the four operator definitions do not change.
    for _ in range(c.pool-1):
        for tries in range(12):
            ee=int(self.rng.integers(4));cc=self.codec.mutate(parent,donor,ee,self.rng);self.trials+=1;k=self.key(cc)
            if k not in local and k not in seen:candidates.append(cc);experts.append(ee);local.add(k);break
    X=np.asarray(candidates)
    if c.model=='random':pick=int(self.rng.integers(len(X)))
    else:
        mu,std=self.model.predict(features(X,self.codec,c.raw_features));self.calls+=len(X)
        if not np.isfinite(mu).all() or not np.isfinite(std).all():
            self.pending['reason']='nonfinite_prediction';return child,expert
        scoreY=mu-c.uncertainty*std
        if c.acquisition=='scalar':
            w=self.rng.dirichlet(np.ones(2));scores=-np.max(scoreY*w,axis=1)
        else:
            scores=hypervolume_improvement(scoreY,(AF-self.low)/self.scale,self.ref)
            if scores.max()<=1e-14:
                # No predicted improvement: preserve the baseline proposal.
                pick=0
            else:pick=int(np.argmax(scores))
        if c.acquisition=='scalar':pick=int(np.argmax(scores))
        self.pending=dict(fe=fe+1,version=self.model.version,train_fe=self.fit_at,prediction=mu[pick].tolist(),std=std[pick].tolist(),override=bool(pick),reason='residual' if pick else
↪ 'base_wins',candidate_count=len(X),baseline_prediction=mu[0].tolist())
    self.override+=bool(pick)
    return X[pick].copy(),int(experts[pick])
def observe(self,child,cf,fe):
    self.dataX.append(child.copy());self.dataF.append(cf.copy())
    if self.pending is not None:
        pred=np.asarray(self.pending['prediction']);err=float(np.mean(np.abs(pred-(cf-self.low)/self.scale)))
        if np.isfinite(err):self.errors.append(err)

```

```

        self.pending['actual']=cf.tolist();self.pending['error']=err;self.forecasts.append(self.pending);self.pending=None
    def diagnostics(self):return dict(config=asdict(self.cfg),model_fits=self.model.model_fits,forecasts=self.forecasts,overri_
↪ des=self.override,surrogate_predictions=self.calls,extra_pure_proposals=self.trials,events=self.events)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/data/rgv2018_parameters.json

```

{
  "source_official_page": "https://www.mcm.edu.cn/html_cn/node/7cec7725b9a0ea07b4dfd175e8042c33.html",
  "source_transcription": "https://github.com/zouwx2cs/2018_CUMCM_B/blob/master/README.md",
  "source_blob_sha": "b459d2ea2de21d099287b5689c1bd1ee9fb4921a",
  "units": "seconds",
  "shift_duration": 28800,
  "station_positions": [0,0,1,1,2,2,3,3],
  "groups": [
    {"id":1,"travel":[0,20,33,46],"process":560,"odd_load":28,"even_load":31,"wash":25},
    {"id":2,"travel":[0,23,41,59],"process":580,"odd_load":30,"even_load":35,"wash":30},
    {"id":3,"travel":[0,18,32,46],"process":545,"odd_load":27,"even_load":32,"wash":25}
  ],
  "scope": "Official case (1): single-process, no failure. Restrict policies to a repeated ordered nonempty subset of CNCs,
↪ with no repeated CNC within a cycle. This is not the complete space of dynamic policies.",
  "extension": "Minimize (-completed washed products, total RGV travel seconds). The travel objective is our transparent
↪ multi-objective extension, not a separately required original contest objective.",
  "explicit_model_assumptions": ["RGV begins at the first track position; all CNCs are initially empty.", "A load/unload
↪ service occupies the RGV and a CNC for the stated service time. A new part starts processing at the end of service.", "An
↪ unloaded completed part is washed immediately, before the next RGV move; no washing is needed after the initial empty-CNC
↪ load.", "A product is counted only when washing finishes by the shift horizon.", "Only complete move/service/wash blocks
↪ finishing within the horizon are executed; no terminal travel or partial operation is charged.", "No process failures,
↪ two-stage processing, or noncyclic adaptive policy is included."]
}

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/legacy/moo_core.py

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable, Dict, List, Optional, Sequence, Tuple
import math
import numpy as np
from scipy.stats import qmc

Array = np.ndarray

@dataclass(frozen=True)
class Problem:
    name: str
    n_var: int
    n_obj: int
    xl: Array

```

```

xu: Array
evaluate: Callable[[Array], Array]
pareto_front: Callable[[int], Array]

# -----
# Benchmark problems
# -----

def _as_2d(x: Array) -> Array:
    x = np.asarray(x, dtype=float)
    return x[None, :] if x.ndim == 1 else x

def make_zdt(name: str, n_var: int = 30) -> Problem:
    name = name.lower()
    if name == "zdt4":
        n_var = 10
        xl = np.r_[0.0, np.full(n_var - 1, -5.0)]
        xu = np.r_[1.0, np.full(n_var - 1, 5.0)]
    else:
        xl = np.zeros(n_var)
        xu = np.ones(n_var)

    def evaluate(x: Array) -> Array:
        X = _as_2d(x)
        if name == "zdt1":
            f1 = X[:, 0]
            g = 1.0 + 9.0 * X[:, 1:].mean(axis=1)
            f2 = g * (1.0 - np.sqrt(f1 / g))
        elif name == "zdt2":
            f1 = X[:, 0]
            g = 1.0 + 9.0 * X[:, 1:].mean(axis=1)
            f2 = g * (1.0 - (f1 / g) ** 2)
        elif name == "zdt3":
            f1 = X[:, 0]
            g = 1.0 + 9.0 * X[:, 1:].mean(axis=1)
            r = f1 / g
            f2 = g * (1.0 - np.sqrt(r) - r * np.sin(10.0 * np.pi * f1))
        elif name == "zdt4":
            f1 = X[:, 0]
            z = X[:, 1:]
            g = 1.0 + 10.0 * (n_var - 1) + np.sum(z * z - 10.0 * np.cos(4.0 * np.pi * z), axis=1)
            f2 = g * (1.0 - np.sqrt(f1 / g))
        elif name == "zdt6":
            x1 = X[:, 0]
            f1 = 1.0 - np.exp(-4.0 * x1) * np.sin(6.0 * np.pi * x1) ** 6
            g = 1.0 + 9.0 * (X[:, 1:].mean(axis=1) ** 0.25)
            f2 = g * (1.0 - (f1 / g) ** 2)
        else:
            raise KeyError(name)
        return np.column_stack([f1, f2])

    def pareto_front(n: int = 2000) -> Array:
        x = np.linspace(0.0, 1.0, max(n * 3, 4000))
        if name in ("zdt1", "zdt4"):
            f1 = x
            f2 = 1.0 - np.sqrt(x)

```



```

elif name == "zdt2":
    f1 = x
    f2 = 1.0 - x**2
elif name == "zdt3":
    intervals = np.array([
        [0.0, 0.0830015349],
        [0.1822287280, 0.2577623634],
        [0.4093136748, 0.4538821041],
        [0.6183967944, 0.6525117038],
        [0.8233317983, 0.8518328654],
    ])
    per = max(20, int(math.ceil(n / len(intervals))))
    f1 = np.concatenate([np.linspace(a, b, per) for a, b in intervals][:n])
    f2 = 1.0 - np.sqrt(f1) - f1 * np.sin(10.0 * np.pi * f1)
    return np.column_stack([f1, f2])
elif name == "zdt6":
    f1 = 1.0 - np.exp(-4.0 * x) * np.sin(6.0 * np.pi * x) ** 6
    f2 = 1.0 - f1**2
    F = np.column_stack([f1, f2])
    F = F[nondominated_mask(F)]
    F = F[np.argsort(F[:, 0])]
    # Uniformize by arc length rather than x1 because ZDT6 density is strongly biased.
    d = np.r_[0.0, np.cumsum(np.linalg.norm(np.diff(F, axis=0), axis=1))]
    targets = np.linspace(0, d[-1], min(n, len(F)))
    idx = np.searchsorted(d, targets)
    return F[np.clip(idx, 0, len(F) - 1)]
else:
    raise KeyError(name)
return np.column_stack([f1, f2])

return Problem(name.upper(), n_var, 2, xl, xu, evaluate, pareto_front)

def _dirichlet_reference_points(n: int, m: int, seed: int = 12345, pool_mult: int = 80) -> Array:
    """Deterministic maximin simplex design with exact extremes."""
    rng = np.random.default_rng(seed + 97 * m + n)
    if m == 2:
        t = np.linspace(0.0, 1.0, n)
        return np.column_stack([t, 1.0 - t])
    if n > 500:
        # Fast deterministic quasi-random Dirichlet design for dense metric fronts.
        power = int(math.ceil(math.log2(max(n - m, 1))))
        U = qmc.Sobol(d=m, scramble=True, seed=seed + 97 * m).random_base2(power)
        U = np.clip(U[:, max(n - m, 1)], 1e-12, 1.0 - 1e-12)
        E = -np.log(U)
        W = E / E.sum(axis=1, keepdims=True)
        W = np.vstack([np.eye(m), W])[:n]
        return W
    pool_n = max(n * pool_mult, 5000)
    pool = rng.dirichlet(np.ones(m), size=pool_n)
    extremes = np.eye(m)
    chosen = [e for e in extremes]
    C = np.vstack(chosen)
    # Maximin farthest-point sampling in Euclidean simplex geometry.
    min_d2 = ((pool[:, None, :] - C[None, :, :]) ** 2).sum(axis=2).min(axis=1)
    while len(chosen) < n:
        j = int(np.argmax(min_d2))
        p = pool[j]

```

```

        chosen.append(p)
        d2 = ((pool - p) ** 2).sum(axis=1)
        min_d2 = np.minimum(min_d2, d2)
        min_d2[j] = -1.0
    W = np.vstack(chosen[:n])
    W = np.maximum(W, 1e-12)
    W /= W.sum(axis=1, keepdims=True)
    return W

def make_dtlz(name: str, n_obj: int = 3, n_var: Optional[int] = None) -> Problem:
    name = name.lower()
    if n_var is None:
        k = 5 if name == "dtlz1" else 10
        n_var = n_obj + k - 1
    xl = np.zeros(n_var)
    xu = np.ones(n_var)

    def evaluate(x: Array) -> Array:
        X = _as_2d(x)
        M = n_obj
        k = n_var - M + 1
        tail = X[:, M - 1 :]
        if name in ("dtlz1", "dtlz3"):
            g = 100.0 * (k + np.sum((tail - 0.5) ** 2 - np.cos(20.0 * np.pi * (tail - 0.5)), axis=1))
        elif name in ("dtlz2", "dtlz4", "dtlz5"):
            g = np.sum((tail - 0.5) ** 2, axis=1)
        elif name == "dtlz6":
            g = np.sum(tail ** 0.1, axis=1)
        elif name == "dtlz7":
            g = 1.0 + 9.0 / k * np.sum(tail, axis=1)
        else:
            raise KeyError(name)

        if name == "dtlz1":
            F = np.empty((len(X), M))
            for i in range(M):
                value = 0.5 * (1.0 + g)
                for j in range(M - i - 1):
                    value *= X[:, j]
                if i > 0:
                    value *= 1.0 - X[:, M - i - 1]
                F[:, i] = value
            return F

        if name in ("dtlz2", "dtlz3", "dtlz4"):
            alpha = 100.0 if name == "dtlz4" else 1.0
            A = X[:, : M - 1] ** alpha
            F = np.empty((len(X), M))
            for i in range(M):
                value = 1.0 + g
                for j in range(M - i - 1):
                    value *= np.cos(A[:, j] * np.pi / 2.0)
                if i > 0:
                    value *= np.sin(A[:, M - i - 1] * np.pi / 2.0)
                F[:, i] = value
            return F

```

```

if name in ("dtlz5", "dtlz6"):
    theta = np.empty((len(X), M - 1))
    theta[:, 0] = X[:, 0] * np.pi / 2.0
    if M > 2:
        theta[:, 1:] = (np.pi / (4.0 * (1.0 + g)))[:, None] * (
            1.0 + 2.0 * g[:, None] * X[:, 1 : M - 1]
        )
    F = np.empty((len(X), M))
    for i in range(M):
        value = 1.0 + g
        for j in range(M - i - 1):
            value *= np.cos(theta[:, j])
        if i > 0:
            value *= np.sin(theta[:, M - i - 1])
        F[:, i] = value
    return F

# DTLZ7
F = np.empty((len(X), M))
F[:, : M - 1] = X[:, : M - 1]
h = M - np.sum(
    (F[:, : M - 1] / (1.0 + g[:, None]))
    * (1.0 + np.sin(3.0 * np.pi * F[:, : M - 1])),
    axis=1,
)
F[:, M - 1] = (1.0 + g) * h
return F

def pareto_front(n: int = 3000) -> Array:
    M = n_obj
    if name == "dtlz1":
        return 0.5 * _dirichlet_reference_points(n, M, seed=211)
    if name in ("dtlz2", "dtlz3", "dtlz4"):
        W = _dirichlet_reference_points(n, M, seed=223)
        return W / np.linalg.norm(W, axis=1, keepdims=True)
    if name in ("dtlz5", "dtlz6"):
        # Degenerate one-dimensional Pareto front. Only the first angle varies;
        # the remaining angles are pi/4 at g=0.
        theta0 = np.linspace(0.0, np.pi / 2.0, n)
        theta = np.full((n, M - 1), np.pi / 4.0)
        theta[:, 0] = theta0
        F = np.empty((n, M))
        for i in range(M):
            value = np.ones(n)
            for j in range(M - i - 1):
                value *= np.cos(theta[:, j])
            if i > 0:
                value *= np.sin(theta[:, M - i - 1])
            F[:, i] = value
        return F
    if name == "dtlz7":
        # The DTLZ7 Pareto set is disconnected. For each free objective, the
        # efficient intervals are approximately [0,0.2514] and
        # [0.6316,0.8594]. Sampling their Cartesian union avoids an
        # 0(pool^2) skyline filter and yields a stable reference front.
        rng = np.random.default_rng(239 + M + n)
        pool_n = max(30 * n, 12000)
        choose_hi = rng.random((pool_n, M - 1)) < 0.5

```

```

        U = rng.random((pool_n, M - 1))
        low = 0.251411836
        hi0, hi1 = 0.631626531, 0.859400856
        X = np.where(choose_hi, hi0 + (hi1 - hi0) * U, low * U)
        h = M - np.sum((X / 2.0) * (1.0 + np.sin(3.0 * np.pi * X)), axis=1)
        F = np.column_stack([X, 2.0 * h])
        return farthest_point_select(F, min(n, len(F)), normalize=True)
    raise KeyError(name)

return Problem(name.upper(), n_var, n_obj, xl, xu, evaluate, pareto_front)

# -----
# WFG benchmark suite (WFG1-WFG9)
# -----

def _wfg_correct01(x: Array, eps: float = 1e-10) -> Array:
    y = np.asarray(x, dtype=float).copy()
    y[(y < 0.0) & (y >= -eps)] = 0.0
    y[(y > 1.0) & (y <= 1.0 + eps)] = 1.0
    return np.clip(y, 0.0, 1.0)

def _wfg_shift_linear(y: Array, a: float = 0.35) -> Array:
    return _wfg_correct01(np.abs(y - a) / np.abs(np.floor(a - y) + a))

def _wfg_shift_deceptive(y: Array, a: float = 0.35, b: float = 0.001, c: float = 0.05) -> Array:
    t1 = np.floor(y - a + b) * (1.0 - c + (a - b) / b) / (a - b)
    t2 = np.floor(a + b - y) * (1.0 - c + (1.0 - a - b) / b) / (1.0 - a - b)
    return _wfg_correct01(1.0 + (np.abs(y - a) - b) * (t1 + t2 + 1.0 / b))

def _wfg_shift_multimodal(y: Array, a: float, b: float, c: float) -> Array:
    t1 = np.abs(y - c) / (2.0 * (np.floor(c - y) + c))
    t2 = (4.0 * a + 2.0) * np.pi * (0.5 - t1)
    return _wfg_correct01((1.0 + np.cos(t2) + 4.0 * b * t1**2) / (b + 2.0))

def _wfg_param_dependent(y: Array, u: Array, a: float = 0.98 / 49.98,
                        b: float = 0.02, c: float = 50.0) -> Array:
    aux = a - (1.0 - 2.0 * u) * np.abs(np.floor(0.5 - u) + a)
    return _wfg_correct01(y ** (b + (c - b) * aux))

def _wfg_bias_flat(y: Array, a: float, b: float, c: float) -> Array:
    ret = (
        a
        + np.minimum(0.0, np.floor(y - b)) * (a * (b - y) / b)
        - np.minimum(0.0, np.floor(c - y)) * ((1.0 - a) * (y - c) / (1.0 - c))
    )
    return _wfg_correct01(ret)

def _wfg_bias_poly(y: Array, alpha: float) -> Array:
    return _wfg_correct01(y ** alpha)

```

```

def _wfg_reduce_weighted(y: Array, w: Array) -> Array:
    return _wfg_correct01(np.dot(y, w) / np.sum(w))

def _wfg_reduce_uniform(y: Array) -> Array:
    return _wfg_correct01(np.mean(y, axis=1))

def _wfg_reduce_nonsep(y: Array, a: int) -> Array:
    n, m = y.shape
    val = np.ceil(a / 2.0)
    num = np.zeros(n)
    for j in range(m):
        num += y[:, j]
        for k in range(a - 1):
            num += np.abs(y[:, j] - y[:, (1 + j + k) % m])
    den = m * val * (1.0 + 2.0 * a - 2.0 * val) / a
    return _wfg_correct01(num / den)

def _wfg_shape_concave(x: Array, m: int) -> Array:
    M = x.shape[1]
    if m == 1:
        out = np.prod(np.sin(0.5 * x[:, :M] * np.pi), axis=1)
    elif 1 < m <= M:
        out = np.prod(np.sin(0.5 * x[:, : M - m + 1] * np.pi), axis=1)
        out *= np.cos(0.5 * x[:, M - m + 1] * np.pi)
    else:
        out = np.cos(0.5 * x[:, 0] * np.pi)
    return _wfg_correct01(out)

def _wfg_shape_convex(x: Array, m: int) -> Array:
    M = x.shape[1]
    if m == 1:
        out = np.prod(1.0 - np.cos(0.5 * x[:, :M] * np.pi), axis=1)
    elif 1 < m <= M:
        out = np.prod(1.0 - np.cos(0.5 * x[:, : M - m + 1] * np.pi), axis=1)
        out *= 1.0 - np.sin(0.5 * x[:, M - m + 1] * np.pi)
    else:
        out = 1.0 - np.sin(0.5 * x[:, 0] * np.pi)
    return _wfg_correct01(out)

def _wfg_shape_linear(x: Array, m: int) -> Array:
    M = x.shape[1]
    if m == 1:
        out = np.prod(x, axis=1)
    elif 1 < m <= M:
        out = np.prod(x[:, : M - m + 1], axis=1)
        out *= 1.0 - x[:, M - m + 1]
    else:
        out = 1.0 - x[:, 0]
    return _wfg_correct01(out)

def _wfg_shape_mixed(x: Array, a: float = 5.0, alpha: float = 1.0) -> Array:
    aux = 2.0 * a * np.pi

```

```

    return _wfg_correct01((1.0 - x - np.cos(aux * x + 0.5 * np.pi) / aux) ** alpha)

def _wfg_shape_disconnected(x: Array, alpha: float = 1.0, beta: float = 1.0,
                             a: float = 5.0) -> Array:
    return _wfg_correct01(1.0 - x**alpha * np.cos(a * np.pi * x**beta) ** 2)

def _wfg_post(t: Array, a: Array) -> Array:
    cols = [np.maximum(t[:, -1], a[i]) * (t[:, i] - 0.5) + 0.5
             for i in range(t.shape[1] - 1)]
    cols.append(t[:, -1])
    return np.column_stack(cols)

def make_wfg(name: str, n_obj: int = 3, n_var: Optional[int] = None, k: Optional[int] = None) -> Problem:
    name = name.lower()
    valid = {f"wfg{i}" for i in range(1, 10)}
    if name not in valid:
        raise KeyError(name)
    if k is None:
        k = 4 if n_obj == 2 else 2 * (n_obj - 1)
    if n_var is None:
        n_var = k + 20
    l = n_var - k
    if k % (n_obj - 1) != 0 or k < 4 or l <= 0:
        raise ValueError("invalid WFG dimensions")
    if name in {"wfg2", "wfg3"} and l % 2:
        raise ValueError("WFG2/WFG3 require an even number of distance variables")
    xl = np.zeros(n_var)
    xu = 2.0 * np.arange(1, n_var + 1, dtype=float)
    S = 2.0 * np.arange(1, n_obj + 1, dtype=float)
    A = np.ones(n_obj - 1)
    if name == "wfg3" and len(A) > 1:
        A[1:] = 0.0
    gap = k // (n_obj - 1)

    def reduce_uniform_blocks(y: Array, tail_end: Optional[int] = None) -> Array:
        cols = [_wfg_reduce_uniform(y[:, j * gap : (j + 1) * gap])
                 for j in range(n_obj - 1)]
        stop = y.shape[1] if tail_end is None else tail_end
        cols.append(_wfg_reduce_uniform(y[:, k:stop]))
        return np.column_stack(cols)

    def reduce_nonsep_blocks(y: Array) -> Array:
        cols = [_wfg_reduce_nonsep(y[:, j * gap : (j + 1) * gap], gap)
                 for j in range(n_obj - 1)]
        cols.append(_wfg_reduce_nonsep(y[:, k:], y.shape[1] - k))
        return np.column_stack(cols)

    def evaluate(x: Array) -> Array:
        y = _as_2d(x) / xu
        apply_post = True
        shape_kind = "concave"

        if name == "wfg1":
            z = y.copy()
            z[:, k:] = _wfg_shift_linear(z[:, k:], 0.35)

```

```

z[:, k:] = _wfg_bias_flat(z[:, k:], 0.8, 0.75, 0.85)
z = _wfg_bias_poly(z, 0.02)
weights = 2.0 * np.arange(1, n_var + 1, dtype=float)
cols = [_wfg_reduce_weighted(z[:, j * gap : (j + 1) * gap],
                             weights[j * gap : (j + 1) * gap])
        for j in range(n_obj - 1)]
cols.append(_wfg_reduce_weighted(z[:, k:], weights[k:]))
t = np.column_stack(cols)
shape_kind = "wfg1"

elif name in {"wfg2", "wfg3"}:
    z = y.copy()
    z[:, k:] = _wfg_shift_linear(z[:, k:], 0.35)
    pair_tail = np.column_stack([
        _wfg_reduce_nonsep(z[:, k + 2 * q : k + 2 * q + 2], 2)
        for q in range(l // 2)
    ])
    z = np.column_stack([z[:, :k], pair_tail])
    t = reduce_uniform_blocks(z, tail_end=k + l // 2)
    shape_kind = "linear" if name == "wfg3" else "wfg2"

elif name == "wfg4":
    z = _wfg_shift_multimodal(y, 30.0, 10.0, 0.35)
    t = reduce_uniform_blocks(z)

elif name == "wfg5":
    z = _wfg_shift_deceptive(y, 0.35, 0.001, 0.05)
    t = reduce_uniform_blocks(z)

elif name == "wfg6":
    z = y.copy()
    z[:, k:] = _wfg_shift_linear(z[:, k:], 0.35)
    t = reduce_nonsep_blocks(z)

elif name == "wfg7":
    z = y.copy()
    for i in range(k):
        u = _wfg_reduce_uniform(z[:, i + 1:])
        z[:, i] = _wfg_param_dependent(z[:, i], u)
    z[:, k:] = _wfg_shift_linear(z[:, k:], 0.35)
    t = reduce_uniform_blocks(z)

elif name == "wfg8":
    z = y.copy()
    dependent_tail = []
    for i in range(k, n_var):
        u = _wfg_reduce_uniform(y[:, :i])
        dependent_tail.append(_wfg_param_dependent(y[:, i], u))
    z[:, k:] = np.column_stack(dependent_tail)
    z[:, k:] = _wfg_shift_linear(z[:, k:], 0.35)
    t = reduce_uniform_blocks(z)

else: # WFG9: parameter-dependent, deceptive, multimodal, non-separable
    z = y.copy()
    for i in range(n_var - 1):
        u = _wfg_reduce_uniform(z[:, i + 1:])
        z[:, i] = _wfg_param_dependent(z[:, i], u)
    a_cols = [_wfg_shift_deceptive(z[:, i], 0.35, 0.001, 0.05) for i in range(k)]

```

```

        b_cols = [_wfg_shift_multimodal(z[:, i], 30.0, 95.0, 0.35)
                    for i in range(k, n_var)]
        z = np.column_stack(a_cols + b_cols)
        t = reduce_nonsep_blocks(z)
        apply_post = False

    if apply_post:
        t = _wfg_post(t, A)
    if shape_kind == "wfg1":
        h = [_wfg_shape_convex(t[:, :-1], j + 1) for j in range(n_obj - 1)]
        h.append(_wfg_shape_mixed(t[:, 0], alpha=1.0, a=5.0))
    elif shape_kind == "wfg2":
        h = [_wfg_shape_convex(t[:, :-1], j + 1) for j in range(n_obj - 1)]
        h.append(_wfg_shape_disconnected(t[:, 0], alpha=1.0, beta=1.0, a=5.0))
    elif shape_kind == "linear":
        h = [_wfg_shape_linear(t[:, :-1], j + 1) for j in range(n_obj)]
    else:
        h = [_wfg_shape_concave(t[:, :-1], j + 1) for j in range(n_obj)]
    return t[:, -1][:, None] + S[None, :] * np.column_stack(h)

def optimal_decisions(position: Array) -> Array:
    K = np.asarray(position, dtype=float).copy()
    if name == "wfg8":
        X = K.copy()
        for _ in range(l):
            u = X.mean(axis=1)
            tmp1 = np.abs(np.floor(0.5 - u) + 0.98 / 49.98)
            tmp2 = 0.02 + 49.98 * (0.98 / 49.98 - (1.0 - 2.0 * u) * tmp1)
            suffix = 0.35 ** (tmp2 ** -1.0)
            X = np.column_stack([X, suffix])
    elif name == "wfg9":
        X = np.column_stack([K, np.zeros((len(K), l))])
        X[:, -1] = 0.35
        for i in range(n_var - 2, k - 1, -1):
            u = X[:, i + 1:].mean(axis=1)
            X[:, i] = 0.35 ** ((0.02 + 1.96 * u) ** -1.0)
    else:
        X = np.column_stack([K, np.full((len(K), l), 0.35)])
    return X * xu

def pareto_front(n: int = 3000) -> Array:
    if name in {"wfg4", "wfg5", "wfg6", "wfg7", "wfg8", "wfg9"}:
        W = _dirichlet_reference_points(n, n_obj, seed=307 + int(name[-1]))
        return (W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)) * S[None, :]
    # WFG1-WFG3 have mixed/disconnected/degenerate fronts. Generate an
    # official-form Pareto set in decision space, evaluate it, then retain
    # a deterministic maximin objective-space reference design.
    pool_n = max(4 * n, 4000)
    sobol_power = int(math.ceil(math.log2(pool_n)))
    U = qmc.Sobol(d=k, scramble=True, seed=331 + int(name[-1])).random_base2(sobol_power)[:pool_n]
    if name == "wfg1":
        U = U ** 50.0
    binary = np.array([[float((mask >> j) & 1) for j in range(k)]
                        for mask in range(2**k)], dtype=float)
    K = np.vstack([binary, U])
    PF = evaluate(optimal_decisions(K))
    target = min(n, len(PF))
    lo, hi = PF.min(axis=0), PF.max(axis=0)

```



```

G = (PF - lo) / (hi - lo + 1e-12)
if name == "wfg3":
    # The WFG3 front is degenerate. Order by its principal geodesic and
    # sample uniformly in cumulative arc length instead of forcing a
    # full-dimensional reference simplex onto a line.
    _, _, vt = np.linalg.svd(G - G.mean(axis=0), full_matrices=False)
    order = np.argsort((G - G.mean(axis=0)) @ vt[0])
    Q = PF[order]
    arc = np.r_[0.0, np.cumsum(np.linalg.norm(np.diff(Q, axis=0), axis=1))]
    idx = np.searchsorted(arc, np.linspace(0.0, arc[-1], target))
    return Q[np.clip(idx, 0, len(Q) - 1)]
# Match candidate Pareto points to deterministic simplex directions in
# chunks. This mirrors the WFG toolkit's reference-direction sampling
# while avoiding an O(pool*n) Python maximin loop.
R = _dirichlet_reference_points(target, n_obj, seed=347 + int(name[-1]))
R = R / (np.linalg.norm(R, axis=1, keepdims=True) + 1e-12)
g2 = np.sum(G * R, axis=1)
selected: List[int] = []
seen = set()
for start_ref in range(0, target, 128):
    Rc = R[start_ref : start_ref + 128]
    dot = G @ Rc.T
    d2 = np.maximum(g2[:, None] - dot * dot, 0.0)
    for idx in np.argmin(d2, axis=0):
        ii = int(idx)
        if ii not in seen:
            seen.add(ii)
            selected.append(ii)
if len(selected) < target:
    # Disconnected WFG2 can map several directions to one component.
    # Fill only the missing slots by exact maximin selection.
    chosen = np.asarray(selected, dtype=int)
    if len(chosen) == 0:
        chosen = np.array([int(np.argmin(G[:, 0]))])
        selected = chosen.tolist(); seen = set(selected)
    min_d2 = ((G[:, None, :] - G[chosen][None, :, :]) ** 2).sum(axis=2).min(axis=1)
    min_d2[chosen] = -1.0
    while len(selected) < target:
        ii = int(np.argmax(min_d2))
        selected.append(ii); seen.add(ii)
        min_d2 = np.minimum(min_d2, np.sum((G - G[ii]) ** 2, axis=1))
        min_d2[np.asarray(selected, dtype=int)] = -1.0
    return PF[np.asarray(selected[:target], dtype=int)]

return Problem(name.upper(), n_var, n_obj, xl, xu, evaluate, pareto_front)

def get_problem(name: str, n_obj: int = 3) -> Problem:
    lname = name.lower()
    if lname.startswith("zdt"):
        return make_zdt(lname)
    if lname.startswith("dtlz"):
        return make_dtlz(lname, n_obj=n_obj)
    if lname.startswith("wfg"):
        return make_wfg(lname, n_obj=n_obj)
    raise KeyError(name)

```

```

# -----
# Pareto utilities and indicators
# -----

def dominates(a: Array, b: Array) -> bool:
    return bool(np.all(a <= b) and np.any(a < b))

def nondominated_mask(F: Array) -> Array:
    F = np.asarray(F, dtype=float)
    n = len(F)
    if n == 0:
        return np.zeros(0, dtype=bool)
    if F.shape[1] == 2:
        # O(N log N) skyline for minimization, robust to ties.
        order = np.lexsort((F[:, 1], F[:, 0]))
        keep = np.zeros(n, dtype=bool)
        best_y = np.inf
        best_x = None
        i = 0
        while i < n:
            xval = F[order[i], 0]
            j = i
            group = []
            while j < n and abs(F[order[j], 0] - xval) <= 1e-14:
                group.append(order[j])
                j += 1
            min_y = min(F[g, 1] for g in group)
            if min_y < best_y - 1e-14:
                # For identical x, only the smallest y is nondominated; preserve one duplicate.
                candidates = [g for g in group if abs(F[g, 1] - min_y) <= 1e-14]
                keep[candidates[0]] = True
                best_y = min_y
            i = j
        return keep
    # Vectorized dominance matrix for the moderate population sizes used here.
    le = np.all(F[:, None, :] <= F[None, :, :], axis=2)
    lt = np.any(F[:, None, :] < F[None, :, :], axis=2)
    dom = le & lt
    return ~np.any(dom, axis=0)

def fast_nondominated_sort(F: Array) -> Tuple[List[Array], Array]:
    F = np.asarray(F, dtype=float)
    n = len(F)
    if n == 0:
        return [], np.zeros(0, dtype=int)
    le = np.all(F[:, None, :] <= F[None, :, :], axis=2)
    lt = np.any(F[:, None, :] < F[None, :, :], axis=2)
    dom = le & lt
    remaining = np.ones(n, dtype=bool)
    ranks = np.full(n, -1, dtype=int)
    fronts: List[Array] = []
    rank = 0
    while np.any(remaining):
        idx = np.flatnonzero(remaining)
        sub = dom[np.ix_(idx, idx)]
        front_local = ~np.any(sub, axis=0)
        front = idx[front_local]

```

```

        if len(front) == 0: # numerical safety fallback
            front = idx[:1]
        ranks[front] = rank
        fronts.append(front)
        remaining[front] = False
        rank += 1
    return fronts, ranks

def crowding_distance(F: Array) -> Array:
    F = np.asarray(F, dtype=float)
    n, m = F.shape
    if n == 0:
        return np.zeros(0)
    if n <= 2:
        return np.full(n, np.inf)
    d = np.zeros(n)
    for j in range(m):
        order = np.argsort(F[:, j], kind="mergesort")
        d[order[0]] = d[order[-1]] = np.inf
        span = F[order[-1], j] - F[order[0], j]
        if span <= 1e-15:
            continue
        d[order[1:-1]] += (F[order[2:], j] - F[order[:-2], j]) / span
    return d

def farthest_point_select(F: Array, n: int, normalize: bool = True) -> Array:
    """Return selected points, preserving each objective extreme then maximin filling."""
    F = np.asarray(F, dtype=float)
    if len(F) <= n:
        return F.copy()
    if normalize:
        lo = F.min(axis=0)
        hi = F.max(axis=0)
        G = (F - lo) / (hi - lo + 1e-12)
    else:
        G = F
    selected: List[int] = []
    for j in range(G.shape[1]):
        idx = int(np.argmin(G[:, j]))
        if idx not in selected:
            selected.append(idx)
    if not selected:
        selected = [int(np.argmin(np.linalg.norm(G, axis=1)))]
    min_d2 = ((G[:, None, :] - G[np.asarray(selected)][None, :, :]) ** 2).sum(axis=2).min(axis=1)
    min_d2[np.asarray(selected)] = -1.0
    while len(selected) < n:
        j = int(np.argmax(min_d2))
        selected.append(j)
        d2 = ((G - G[j]) ** 2).sum(axis=1)
        min_d2 = np.minimum(min_d2, d2)
        min_d2[np.asarray(selected)] = -1.0
    return F[np.asarray(selected)]

def select_indices_farthest(F: Array, n: int) -> Array:
    if len(F) <= n:
        return np.arange(len(F), dtype=int)

```

```

lo = F.min(axis=0)
hi = F.max(axis=0)
G = (F - lo) / (hi - lo + 1e-12)
selected: List[int] = []
for j in range(G.shape[1]):
    idx = int(np.argmax(G[:, j]))
    if idx not in selected:
        selected.append(idx)
min_d2 = ((G[:, None, :] - G[np.asarray(selected)][None, :, :]) ** 2).sum(axis=2).min(axis=1)
min_d2[np.asarray(selected)] = -1.0
while len(selected) < n:
    idx = int(np.argmax(min_d2))
    selected.append(idx)
    min_d2 = np.minimum(min_d2, ((G - G[idx]) ** 2).sum(axis=1))
    min_d2[np.asarray(selected)] = -1.0
return np.asarray(selected, dtype=int)

```

class Archive:

```

def __init__(self, max_size: int = 500):
    self.max_size = int(max_size)
    self.X = np.empty((0, 0))
    self.F = np.empty((0, 0))

def update(self, X: Array, F: Array) -> Tuple[bool, float]:
    X = _as_2d(X)
    F = _as_2d(F)
    any_entered = False
    max_novelty = 0.0
    for x, f in zip(X, F):
        if self.F.size == 0:
            self.X = x[None, :].copy()
            self.F = f[None, :].copy()
            any_entered = True
            max_novelty = 1.0
            continue

        # Exact incremental Pareto verifier.
        equal = np.all(np.isclose(self.F, f, atol=1e-11, rtol=1e-10), axis=1)
        if np.any(equal):
            continue

        old_dominates = np.all(self.F <= f, axis=1) & np.any(self.F < f, axis=1)
        if np.any(old_dominates):
            continue

        f_dominates = np.all(f <= self.F, axis=1) & np.any(f < self.F, axis=1)
        lo = np.minimum(self.F.min(axis=0), f)
        hi = np.maximum(self.F.max(axis=0), f)
        fn = (f - lo) / (hi - lo + 1e-12)
        oldn = (self.F - lo) / (hi - lo + 1e-12)
        nearest = float(np.linalg.norm(oldn - fn, axis=1).min())
        novelty = float(np.clip(nearest / 0.15, 0.0, 1.0))
        keep = ~f_dominates
        self.X = np.vstack([self.X[keep], x])
        self.F = np.vstack([self.F[keep], f])
        any_entered = True
        max_novelty = max(max_novelty, novelty)
    if len(self.F) > self.max_size + max(20, self.max_size // 10):
        # Cheap internal compression; final reporting still uses maximin selection.
        cd = crowding_distance(self.F)

```

```

        idx = np.argsort(-cd, kind="mergesort")[: self.max_size]
        self.X, self.F = self.X[idx], self.F[idx]
    return any_entered, max_novelty

def output(self, n: int) -> Tuple[Array, Array]:
    if len(self.F) <= n:
        return self.X.copy(), self.F.copy()
    idx = select_indices_farthest(self.F, n)
    return self.X[idx].copy(), self.F[idx].copy()

def igd(reference: Array, approx: Array) -> float:
    R, A = np.asarray(reference), np.asarray(approx)
    d2 = ((R[:, None, :] - A[None, :, :]) ** 2).sum(axis=2)
    return float(np.sqrt(d2.min(axis=1)).mean())

def igd_plus(reference: Array, approx: Array) -> float:
    R, A = np.asarray(reference), np.asarray(approx)
    # d+(r,a) for minimization: only components where approximation is worse count.
    D = np.maximum(A[None, :, :] - R[:, None, :], 0.0)
    dist = np.sqrt((D * D).sum(axis=2))
    return float(dist.min(axis=1).mean())

def _hv_2d(F: Array, ref: Array) -> float:
    if len(F) == 0:
        return 0.0
    F = F[np.all(F < ref, axis=1)]
    if len(F) == 0:
        return 0.0
    F = F[nondominated_mask(F)]
    F = F[np.argsort(F[:, 0])]
    hv = 0.0
    prev_y = ref[1]
    for x, y in F:
        if y < prev_y:
            hv += max(ref[0] - x, 0.0) * (prev_y - y)
            prev_y = y
    return float(max(hv, 0.0))

def _hv_3d(F: Array, ref: Array) -> float:
    # Dimension sweep along first objective, integrating exact 2-D dominated area.
    if len(F) == 0:
        return 0.0
    F = F[np.all(F < ref, axis=1)]
    if len(F) == 0:
        return 0.0
    F = F[nondominated_mask(F)]
    xs = np.unique(np.r_[F[:, 0], ref[0]])
    xs.sort()
    hv = 0.0
    for i in range(len(xs) - 1):
        left, right = xs[i], xs[i + 1]
        active = F[F[:, 0] <= left + 1e-15][:, 1:]
        if len(active):
            hv += (right - left) * _hv_2d(active, ref[1:])

```

```

    return float(max(hv, 0.0))

def hypervolume(F: Array, ref: Array) -> float:
    F, ref = np.asarray(F, dtype=float), np.asarray(ref, dtype=float)
    if F.shape[1] == 2:
        return _hv_2d(F, ref)
    if F.shape[1] == 3:
        return _hv_3d(F, ref)
    # Deterministic quasi-Monte Carlo approximation for >3 objectives.
    # Used only as a secondary diagnostic, not for algorithm decisions.
    mask = np.all(F < ref, axis=1)
    F = F[mask]
    if len(F) == 0:
        return 0.0
    lo = np.minimum(F.min(axis=0), 0.0)
    rng = np.random.default_rng(99173 + F.shape[1] + len(F))
    samples = rng.random((50000, F.shape[1])) * (ref - lo) + lo
    dominated = np.any(np.all(F[:, None, :] <= samples[None, :, :], axis=2), axis=0)
    return float(dominated.mean() * np.prod(ref - lo))

def normalize_with_reference(F: Array, reference_pf: Array) -> Tuple[Array, Array, Array]:
    lo = reference_pf.min(axis=0)
    hi = reference_pf.max(axis=0)
    return (F - lo) / (hi - lo + 1e-12), lo, hi

def metric_bundle(F: Array, reference_pf: Array) -> Dict[str, float]:
    Fn, lo, hi = normalize_with_reference(F, reference_pf)
    Rn = (reference_pf - lo) / (hi - lo + 1e-12)
    ref = np.full(F.shape[1], 1.1)
    return {
        "igd": igd(Rn, Fn),
        "igd_plus": igd_plus(Rn, Fn),
        "hv": hypervolume(Fn, ref),
    }

# -----
# Sampling and variation
# -----

def latin_hypercube(n: int, d: int, rng: np.random.Generator) -> Array:
    U = rng.random((n, d))
    X = np.empty((n, d))
    for j in range(d):
        perm = rng.permutation(n)
        X[:, j] = (perm + U[:, j]) / n
    return X

def initialize(problem: Problem, n: int, seed: int) -> Tuple[np.random.Generator, Array, Array]:
    rng = np.random.default_rng(seed)
    U = latin_hypercube(n, problem.n_var, rng)
    X = problem.xl + U * (problem.xu - problem.xl)
    F = problem.evaluate(X)
    return rng, X, F

```

```

def repair_bounds(x: Array, xl: Array, xu: Array, parent: Optional[Array] = None) -> Array:
    y = np.asarray(x, dtype=float).copy()
    if parent is None:
        return np.clip(y, xl, xu)
    low = y < xl
    high = y > xu
    y[low] = 0.5 * (xl[low] + parent[low])
    y[high] = 0.5 * (xu[high] + parent[high])
    return np.clip(y, xl, xu)

def polynomial_mutation(x: Array, xl: Array, xu: Array, rng: np.random.Generator,
                        eta: float = 20.0, prob: Optional[float] = None) -> Array:
    y = np.asarray(x, dtype=float).copy()
    d = len(y)
    if prob is None:
        prob = 1.0 / d
    span = xu - xl
    for j in range(d):
        if rng.random() > prob or span[j] <= 0:
            continue
        delta1 = (y[j] - xl[j]) / span[j]
        delta2 = (xu[j] - y[j]) / span[j]
        r = rng.random()
        mut_pow = 1.0 / (eta + 1.0)
        if r <= 0.5:
            xy = 1.0 - delta1
            val = 2.0 * r + (1.0 - 2.0 * r) * (xy ** (eta + 1.0))
            deltaq = val ** mut_pow - 1.0
        else:
            xy = 1.0 - delta2
            val = 2.0 * (1.0 - r) + 2.0 * (r - 0.5) * (xy ** (eta + 1.0))
            deltaq = 1.0 - val ** mut_pow
        y[j] += deltaq * span[j]
    return np.clip(y, xl, xu)

def sbx_one(p1: Array, p2: Array, xl: Array, xu: Array, rng: np.random.Generator,
            eta: float = 20.0, prob_var: float = 0.5) -> Array:
    c = p1.copy()
    for j in range(len(p1)):
        if rng.random() > prob_var or abs(p1[j] - p2[j]) < 1e-14:
            continue
        x1, x2 = sorted((p1[j], p2[j]))
        rand = rng.random()
        beta = 1.0 + 2.0 * (x1 - xl[j]) / (x2 - x1)
        alpha = 2.0 - beta ** (-(eta + 1.0))
        if rand <= 1.0 / alpha:
            betaq = (rand * alpha) ** (1.0 / (eta + 1.0))
        else:
            betaq = (1.0 / (2.0 - rand * alpha)) ** (1.0 / (eta + 1.0))
        c1 = 0.5 * ((x1 + x2) - betaq * (x2 - x1))
        beta = 1.0 + 2.0 * (xu[j] - x2) / (x2 - x1)
        alpha = 2.0 - beta ** (-(eta + 1.0))
        if rand <= 1.0 / alpha:
            betaq = (rand * alpha) ** (1.0 / (eta + 1.0))

```

```

        else:
            betaq = (1.0 / (2.0 - rand * alpha)) ** (1.0 / (eta + 1.0))
            c2 = 0.5 * ((x1 + x2) + betaq * (x2 - x1))
            c[j] = c1 if rng.random() < 0.5 else c2
        return np.clip(c, xl, xu)

def de_binomial(target: Array, mutant: Array, cr: float, rng: np.random.Generator) -> Array:
    d = len(target)
    mask = rng.random(d) < cr
    mask[rng.integers(d)] = True
    return np.where(mask, mutant, target)

# -----
# Selection helpers
# -----

def nsga2_environmental_selection(X: Array, F: Array, n: int) -> Tuple[Array, Array, Array, Array]:
    fronts, ranks = fast_nondominated_sort(F)
    selected: List[int] = []
    crowd = np.zeros(len(F))
    for front in fronts:
        cd = crowding_distance(F[front])
        crowd[front] = cd
        if len(selected) + len(front) <= n:
            selected.extend(front.tolist())
        else:
            order = np.argsort(-cd, kind="mergesort")
            selected.extend(front[order[: n - len(selected)]].tolist())
            break
    idx = np.asarray(selected, dtype=int)
    new_fronts, new_ranks = fast_nondominated_sort(F[idx])
    new_crowd = np.zeros(len(idx))
    for front in new_fronts:
        new_crowd[front] = crowding_distance(F[idx][front])
    return X[idx], F[idx], new_ranks, new_crowd

def tournament(ranks: Array, crowd: Array, rng: np.random.Generator) -> int:
    a, b = rng.integers(0, len(ranks), size=2)
    if ranks[a] < ranks[b]:
        return int(a)
    if ranks[b] < ranks[a]:
        return int(b)
    if crowd[a] > crowd[b]:
        return int(a)
    if crowd[b] > crowd[a]:
        return int(b)
    return int(a if rng.random() < 0.5 else b)

def associate_to_reference(F: Array, W: Array) -> Tuple[Array, Array]:
    # Objective normalization then perpendicular-angle association.
    z = F.min(axis=0)
    nadir = F.max(axis=0)
    Y = (F - z) / (nadir - z + 1e-12)
    Yn = Y / (np.linalg.norm(Y, axis=1, keepdims=True) + 1e-12)

```



```

Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)
cos = np.clip(Yn @ Wn.T, -1.0, 1.0)
assoc = np.argmax(cos, axis=1)
angle = np.arccos(cos[np.arange(len(F)), assoc])
return assoc, angle

def nsga3_environmental_selection(X: Array, F: Array, n: int, W: Array,
                                rng: np.random.Generator) -> Tuple[Array, Array]:
    fronts, _ = fast_nondominated_sort(F)
    selected: List[int] = []
    last: Optional[Array] = None
    for front in fronts:
        if len(selected) + len(front) <= n:
            selected.extend(front.tolist())
        else:
            last = front
            break
    if len(selected) == n or last is None:
        idx = np.asarray(selected[:n], dtype=int)
        return X[idx], F[idx]

    all_idx = np.asarray(selected + last.tolist(), dtype=int)
    Fall = F[all_idx]
    assoc, angle = associate_to_reference(Fall, W)
    k_selected = len(selected)
    rho = np.bincount(assoc[:k_selected], minlength=len(W)).astype(int)
    candidates = {j: [] for j in range(len(W))}
    for local_idx in range(k_selected, len(all_idx)):
        candidates[int(assoc[local_idx])].append(local_idx)
    chosen_local = list(range(k_selected))
    while len(chosen_local) < n:
        available = [j for j, vals in candidates.items() if vals]
        if not available:
            remaining = [i for i in range(k_selected, len(all_idx)) if i not in chosen_local]
            rng.shuffle(remaining)
            chosen_local.extend(remaining[: n - len(chosen_local)])
            break
        min_rho = min(rho[j] for j in available)
        refs = [j for j in available if rho[j] == min_rho]
        j = int(rng.choice(refs))
        vals = candidates[j]
        if rho[j] == 0:
            pick = min(vals, key=lambda i: angle[i])
        else:
            pick = int(rng.choice(vals))
        chosen_local.append(pick)
        vals.remove(pick)
        rho[j] += 1
    idx = all_idx[np.asarray(chosen_local[:n], dtype=int)]
    return X[idx], F[idx]

# -----
# Baseline algorithms
# -----
@dataclass
class RunResult:

```

```

algorithm: str
problem: str
seed: int
X: Array
F: Array
evaluations: int
trace: List[Dict[str, float]]
diagnostics: Dict[str, object]

def run_nsga2(problem: Problem, seed: int, pop_size: int = 100, max_evals: int = 10100,
             trace_every: int = 10) -> RunResult:
    rng, X, F = initialize(problem, pop_size, seed)
    archive = Archive(max_size=5 * pop_size)
    archive.update(X, F)
    fronts, ranks = fast_nondominated_sort(F)
    crowd = np.zeros(pop_size)
    for front in fronts:
        crowd[front] = crowding_distance(F[front])
    evals = pop_size
    gen = 0
    trace: List[Dict[str, float]] = []
    while evals + pop_size <= max_evals:
        off = []
        for _ in range(pop_size):
            i = tournament(ranks, crowd, rng)
            j = tournament(ranks, crowd, rng)
            child = sbx_one(X[i], X[j], problem.xl, problem.xu, rng, eta=20.0)
            child = polynomial_mutation(child, problem.xl, problem.xu, rng, eta=20.0)
            off.append(child)
        Q = np.asarray(off)
        QF = problem.evaluate(Q)
        evals += pop_size
        archive.update(Q, QF)
        X, F, ranks, crowd = nsga2_environmental_selection(np.vstack([X, Q]), np.vstack([F, QF]), pop_size)
        gen += 1
        if gen % trace_every == 0:
            trace.append({"gen": gen, "evals": evals, "archive_size": len(archive.F)})
    outX, outF = archive.output(pop_size)
    return RunResult("NSGA-II", problem.name, seed, outX, outF, evals, trace, {})

def _tchebycheff(F: Array, W: Array, ideal: Array, nadir: Array) -> Array:
    del nadir
    Y = np.abs(F - ideal)
    ww = np.maximum(W, 1e-4)
    return np.max(ww * Y, axis=-1)

def run_moea(problem: Problem, seed: int, pop_size: int = 100, max_evals: int = 10100,
             T: int = 20, delta: float = 0.9, nr: int = 2, trace_every: int = 10) -> RunResult:
    rng, X, F = initialize(problem, pop_size, seed)
    W = _dirichlet_reference_points(pop_size, problem.n_obj, seed=541)
    d = ((W[:, None, :] - W[None, :, :]) ** 2).sum(axis=2)
    B = np.argsort(d, axis=1)[:T, : min(T, pop_size)]
    ideal = F.min(axis=0)
    nadir = F.max(axis=0)
    archive = Archive(max_size=5 * pop_size)

```

```

archive.update(X, F)
evals = pop_size
gen = 0
trace: List[Dict[str, float]] = []
while evals < max_evals:
    for i in rng.permutation(pop_size):
        if evals >= max_evals:
            break
        pool = B[i] if rng.random() < delta else np.arange(pop_size)
        replace = len(pool) < 3
        if rng.random() < 0.5:
            r1, r2 = rng.choice(pool, size=2, replace=len(pool) < 2)
            child = sbx_one(X[r1], X[r2], problem.xl, problem.xu, rng, eta=20.0)
        else:
            r1, r2 = rng.choice(pool, size=2, replace=len(pool) < 2)
            mutant = X[i] + 0.5 * (X[r1] - X[r2])
            child = de_binomial(X[i], mutant, cr=0.9, rng=rng)
            child = repair_bounds(child, problem.xl, problem.xu, parent=X[i])
        child = polynomial_mutation(child, problem.xl, problem.xu, rng, eta=20.0)
        childF = problem.evaluate(child)[0]
        evals += 1
        archive.update(child, childF)
        ideal = np.minimum(ideal, childF)
        nadir = np.maximum(nadir, childF)
        order = rng.permutation(B[i])
        replaced = 0
        for j in order:
            oldg = _tchebycheff(F[j][None, :], W[j][None, :], ideal, nadir)[0]
            newg = _tchebycheff(childF[None, :], W[j][None, :], ideal, nadir)[0]
            if newg <= oldg:
                X[j], F[j] = child.copy(), childF.copy()
                replaced += 1
                if replaced >= nr:
                    break
        gen += 1
    if gen % trace_every == 0:
        trace.append({"gen": gen, "evals": evals, "archive_size": len(archive.F)})
    outX, outF = archive.output(pop_size)
    return RunResult("MOEA/D-DE", problem.name, seed, outX, outF, evals, trace, {})

```

```

def run_nsga3(problem: Problem, seed: int, pop_size: int = 100, max_evals: int = 10100,
              trace_every: int = 10) -> RunResult:
    rng, X, F = initialize(problem, pop_size, seed)
    W = _dirichlet_reference_points(pop_size, problem.n_obj, seed=557)
    archive = Archive(max_size=5 * pop_size)
    archive.update(X, F)
    evals = pop_size
    gen = 0
    trace: List[Dict[str, float]] = []
    # Reuse NSGA-II rank/crowding tournament for mating; NSGA-III differs in environmental selection.
    fronts, ranks = fast_nondominated_sort(F)
    crowd = np.zeros(pop_size)
    for front in fronts:
        crowd[front] = crowding_distance(F[front])
    while evals + pop_size <= max_evals:
        Q = []
        for _ in range(pop_size):

```

```

        i = tournament(ranks, crowd, rng)
        j = tournament(ranks, crowd, rng)
        child = sbx_one(X[i], X[j], problem.xl, problem.xu, rng, eta=30.0)
        child = polynomial_mutation(child, problem.xl, problem.xu, rng, eta=20.0)
        Q.append(child)
    Q = np.asarray(Q)
    QF = problem.evaluate(Q)
    evals += pop_size
    archive.update(Q, QF)
    X, F = nsga3_environmental_selection(np.vstack([X, Q]), np.vstack([F, QF]), pop_size, W, rng)
    fronts, ranks = fast_nondominated_sort(F)
    crowd = np.zeros(pop_size)
    for front in fronts:
        crowd[front] = crowding_distance(F[front])
    gen += 1
    if gen % trace_every == 0:
        trace.append({"gen": gen, "evals": evals, "archive_size": len(archive.F)})
    outX, outF = archive.output(pop_size)
    return RunResult("NSGA-III", problem.name, seed, outX, outF, evals, trace, {})

def _rdex_cal_fitness(F: Array, kappa: float = 0.05) -> Tuple[Array, Array, Array]:
    F = np.asarray(F, dtype=float)
    lo, hi = F.min(axis=0), F.max(axis=0)
    G = (F - lo) / (hi - lo + 1e-12)
    I = np.max(G[:, None, :] - G[None, :, :], axis=2)
    C = np.max(np.abs(I), axis=0)
    C = np.maximum(C, 1e-12)
    fitness = 1.0 - np.sum(np.exp(-I / C[None, :] / kappa), axis=0)
    return fitness, I, C

def _rdex_environmental(X: Array, F: Array, n: int, kappa: float = 0.05) -> Tuple[Array, Array]:
    if len(F) <= n:
        return X.copy(), F.copy()
    fitness, I, C = _rdex_cal_fitness(F, kappa)
    active = list(range(len(F)))
    while len(active) > n:
        values = fitness[np.asarray(active)]
        pos = int(np.argmin(values))
        worst = active[pos]
        # Faithful to the released MATLAB code: C is indexed by the removed solution.
        fitness += np.exp(-I[worst, :] / C[worst] / kappa)
        active.pop(pos)
    idx = np.asarray(active, dtype=int)
    return X[idx], F[idx]

def _rdex_selection(X: Array, F: Array, n: int, rng: np.random.Generator) -> Tuple[Array, Array, int]:
    nd = nondominated_mask(F)
    X, F = X[nd], F[nd]
    if len(F) == 0:
        return X, F, 0
    perm = rng.permutation(len(F))
    X, F = X[perm], F[perm]
    n_nd = len(F)
    if len(F) <= n:
        return X, F, n_nd
    lo, hi = F.min(axis=0), F.max(axis=0)

```

```

G = (F - lo) / (hi - lo + 1e-12)
d = np.linalg.norm(G[:, None, :] - G[None, :, :], axis=2)
np.fill_diagonal(d, np.inf)
sd = np.sort(d, axis=1)
kth = min(2, sd.shape[1] - 1)
radius = max(float(np.mean(sd[:, kth])), 1e-12)
R = np.minimum(d / radius, 1.0)
np.fill_diagonal(R, 1.0)
active = np.arange(len(F))
while len(active) > n:
    crowd_score = 1.0 - np.prod(R, axis=1)
    worst = int(np.argmax(crowd_score))
    active = np.delete(active, worst)
    R = np.delete(np.delete(R, worst, axis=0), worst, axis=1)
return X[active], F[active], n_nd

def _rdex_tournament(fitness: Array, n: int, rng: np.random.Generator) -> Array:
    # Released code passes -CalFitness to a minimization tournament; equivalent to
    # choosing the larger CalFitness value from each random pair.
    cand = rng.integers(0, len(fitness), size=(n, 2))
    choose_left = fitness[cand[:, 0]] >= fitness[cand[:, 1]]
    return np.where(choose_left, cand[:, 0], cand[:, 1])

def _rdex_operator(P: Array, fitness: Array, problem: Problem, progress: float,
                  rng: np.random.Generator) -> Array:
    N, D = P.shape
    order = np.argsort(-fitness, kind="mergesort")
    Fm = np.array([0.6, 0.8, 1.0])
    CRm = np.array([0.1, 0.2, 1.0])
    scale = rng.choice(Fm, size=N)
    cr = rng.choice(CRm, size=N)
    p = max(int(np.floor(0.17 * N * (1.0 - 0.9 * progress) + 0.5)), 2)
    p = min(p, N)
    pbest = P[order[rng.integers(0, p, size=N)]]
    r1 = np.empty(N, dtype=int)
    r2 = np.empty(N, dtype=int)
    for i in range(N):
        pool1 = np.delete(np.arange(N), i)
        r1[i] = int(rng.choice(pool1))
        pool2 = np.delete(np.arange(N), np.sort([i, r1[i]]))
        r2[i] = int(rng.choice(pool2))
    V = P + scale[:, None] * (pbest - P + P[r1] - P[r2])
    mask = rng.random((N, D)) < cr[:, None]
    mask[np.arange(N), rng.integers(0, D, size=N)] = True
    Q = np.where(mask, V, P)
    perturb = rng.random((N, D)) < 0.2
    Q[perturb] += 0.2 * rng.standard_cauchy(np.count_nonzero(perturb))
    return np.clip(Q, problem.xl, problem.xu)

def _rdex_exploration(PCX: Array, PCF: Array, workX: Array, workF: Array,
                    n_nd: int, pop_size: int, problem: Problem,
                    rng: np.random.Generator) -> Array:
    if len(PCF) == 0:
        return np.empty((0, problem.n_var))
    lo, hi = PCF.min(axis=0), PCF.max(axis=0)

```

```

A = (PCF - lo) / (hi - lo + 1e-12)
B = (workF - lo) / (hi - lo + 1e-12)
d_pc = np.linalg.norm(A[:, None, :] - A[None, :, :], axis=2)
np.fill_diagonal(d_pc, np.inf)
if len(A) == 1:
    r0 = 0.0
else:
    sd = np.sort(d_pc, axis=1)
    r0 = float(np.mean(sd[:, min(2, sd.shape[1] - 1)]))
radius = (n_nd / pop_size) * r0
d = np.linalg.norm(A[:, None, :] - B[None, :, :], axis=2)
sparse = np.flatnonzero(np.sum(d <= radius + 1e-15, axis=1) <= 1)
if len(sparse) == 0:
    return np.empty((0, problem.n_var))
base = PCX[sparse]
guide = PCX[rng.integers(0, len(PCX), size=len(sparse))]
n, D = base.shape
scale = np.clip(0.7 + 0.2 * rng.standard_cauchy(n), 0.0, 1.0)
cr = np.clip(rng.normal(0.5, 0.1, size=n), 0.0, 1.0)
perm = rng.permutation(n)
V = base + scale[:, None] * (base[perm] - guide)
mask = rng.random((n, D)) < cr[:, None]
mask[np.arange(n), rng.integers(0, D, size=n)] = True
Q = np.where(mask, V, base)
return np.clip(Q, problem.xl, problem.xu)

def run_rdex(problem: Problem, seed: int, pop_size: int = 100, max_evals: int = 10100,
            kappa: float = 0.05, trace_every: int = 10) -> RunResult:
    """Mechanism-faithful NumPy port of the released CEC-2025 RDEx-MOP code.

    This is a translation for controlled cross-problem comparison, not the official
    MATLAB/PlatEMO executable or an official competition score reproduction.
    """
    rng, workX, workF = initialize(problem, pop_size, seed)
    archive = Archive(max_size=5 * pop_size)
    archive.update(workX, workF)
    pcX, pcF, n_nd = _rdex_selection(workX, workF, pop_size, rng)
    auxX, auxF = pcX.copy(), pcF.copy()
    evals = pop_size
    gen = 0
    trace: List[Dict[str, float]] = []

    while evals < max_evals:
        exploreX = _rdex_exploration(pcX, pcF, workX, workF, n_nd, pop_size, problem, rng)
        remaining = max_evals - evals
        if len(exploreX) > remaining:
            exploreX = exploreX[:remaining]
        exploreF = problem.evaluate(exploreX) if len(exploreX) else np.empty((0, problem.n_obj))
        evals += len(exploreX)
        if len(exploreX):
            archive.update(exploreX, exploreF)
        if evals >= max_evals:
            pcX, pcF, n_nd = _rdex_selection(
                np.vstack([pcX, exploreX]), np.vstack([pcF, exploreF]), pop_size, rng
            )
        break

```

```

use_aux = evals >= 0.5 * max_evals and rng.random() < 0.5
if use_aux:
    if len(exploreX):
        auxX, auxF = _rdex_environmental(
            np.vstack([auxX, exploreX]), np.vstack([auxF, exploreF]), pop_size, kappa
        )
        sourceX, sourceF = auxX, auxF
    else:
        if len(exploreX):
            workX, workF = _rdex_environmental(
                np.vstack([workX, exploreX]), np.vstack([workF, exploreF]), pop_size, kappa
            )
            sourceX, sourceF = workX, workF

fit, _, _ = _rdex_cal_fitness(sourceF, kappa)
mating = _rdex_tournament(fit, pop_size, rng)
parentsX = sourceX[mating]
parent_fit, _, _ = _rdex_cal_fitness(sourceF[mating], kappa)
newX = _rdex_operator(parentsX, parent_fit, problem, evals / max_evals, rng)
remaining = max_evals - evals
if len(newX) > remaining:
    newX = newX[:remaining]
newF = problem.evaluate(newX)
evals += len(newX)
archive.update(newX, newF)

if use_aux:
    auxX, auxF = _rdex_environmental(
        np.vstack([auxX, newX]), np.vstack([auxF, newF]), pop_size, kappa
    )
else:
    workX, workF = _rdex_environmental(
        np.vstack([workX, newX]), np.vstack([workF, newF]), pop_size, kappa
    )

pcX, pcF, n_nd = _rdex_selection(
    np.vstack([pcX, newX, exploreX, auxX]),
    np.vstack([pcF, newF, exploreF, auxF]),
    pop_size, rng,
)
auxX, auxF = pcX.copy(), pcF.copy()
gen += 1
if gen % trace_every == 0:
    trace.append({"gen": gen, "evals": evals, "archive_size": len(archive.F), "pc_size": len(pcF)})

outX, outF = archive.output(pop_size)
return RunResult("RDEx-MOP-port", problem.name, seed, outX, outF, evals, trace,
    {"translation": "released MATLAB mechanisms, not official PlatEMO score"})

def _rvea_environmental_selection(X: Array, F: Array, n: int, W: Array,
    progress: float, alpha: float = 2.0) -> Tuple[Array, Array]:
    """Reference-vector guided selection using the angle-penalized distance (APD)."""
    ideal = F.min(axis=0)
    Y = np.maximum(F - ideal, 0.0)
    mag = np.linalg.norm(Y, axis=1)
    Yn = Y / (mag[:, None] + 1e-12)
    Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)

```

```

cosine = np.clip(Yn @ Wn.T, -1.0, 1.0)
assoc = np.argmax(cosine, axis=1)
angles = np.arccos(cosine[np.arange(len(F)), assoc])

ww_cos = np.clip(Wn @ Wn.T, -1.0, 1.0)
np.fill_diagonal(ww_cos, -1.0)
gamma = np.arccos(np.max(ww_cos, axis=1))
gamma = np.maximum(gamma, 1e-12)
penalty = F.shape[1] * (float(progress) ** alpha) * angles / gamma[assoc]
apd = mag * (1.0 + penalty)

selected: List[int] = []
for j in range(len(W)):
    idx = np.flatnonzero(assoc == j)
    if len(idx):
        selected.append(int(idx[np.argmin(apd[idx])]))
# The canonical RVEA population can temporarily shrink when reference cells are
# empty. For a fair fixed-population comparison, deterministically fill from the
# remaining lowest-APD candidates while avoiding duplicates.
if len(selected) < n:
    selected_set = set(selected)
    for idx in np.argsort(apd, kind="mergesort"):
        ii = int(idx)
        if ii not in selected_set:
            selected.append(ii)
            selected_set.add(ii)
            if len(selected) == n:
                break
if len(selected) > n:
    selected = selected[:n]
idx = np.asarray(selected, dtype=int)
return X[idx], F[idx]

def run_rvea(problem: Problem, seed: int, pop_size: int = 100, max_evals: int = 10100,
            alpha: float = 2.0, adapt_fraction: float = 0.1, trace_every: int = 10) -> RunResult:
    rng, X, F = initialize(problem, pop_size, seed)
    W0 = _dirichlet_reference_points(pop_size, problem.n_obj, seed=563)
    W = W0 / (np.linalg.norm(W0, axis=1, keepdims=True) + 1e-12)
    archive = Archive(max_size=5 * pop_size)
    archive.update(X, F)
    evals = pop_size
    gen = 0
    max_gen = max(1, (max_evals - pop_size) // pop_size)
    adapt_every = max(1, int(round(adapt_fraction * max_gen)))
    trace: List[Dict[str, float]] = []
    fronts, ranks = fast_nondominated_sort(F)
    crowd = np.zeros(len(F))
    for front in fronts:
        crowd[front] = crowding_distance(F[front])

    while evals + pop_size <= max_evals:
        Q = []
        for _ in range(pop_size):
            i = tournament(ranks, crowd, rng)
            j = tournament(ranks, crowd, rng)
            child = sbx_one(X[i], X[j], problem.xl, problem.xu, rng, eta=30.0)
            child = polynomial_mutation(child, problem.xl, problem.xu, rng, eta=20.0)

```



```

        Q.append(child)
    Q = np.asarray(Q)
    QF = problem.evaluate(Q)
    evals += pop_size
    archive.update(Q, QF)
    gen += 1
    progress = min(gen / max_gen, 1.0)
    X, F = _rvea_environmental_selection(
        np.vstack([X, Q]), np.vstack([F, QF]), pop_size, W, progress, alpha=alpha
    )
    if gen % adapt_every == 0:
        span = np.maximum(F.max(axis=0) - F.min(axis=0), 1e-12)
        W = W0 * span[None, :]
        W /= np.linalg.norm(W, axis=1, keepdims=True) + 1e-12
    fronts, ranks = fast_nondominated_sort(F)
    crowd = np.zeros(len(F))
    for front in fronts:
        crowd[front] = crowding_distance(F[front])
    if gen % trace_every == 0:
        trace.append({"gen": gen, "evals": evals, "archive_size": len(archive.F)})
    outX, outF = archive.output(pop_size)
    return RunResult("RVEA", problem.name, seed, outX, outF, evals, trace, {})

```

```

# -----
# Proposed algorithm: MOSAIC-MOEA
# -----

```

```

class ContextualThompsonPortfolio:

```

```

    """Hierarchical fractional Beta-Bernoulli Thompson sampler.

```

```

    Local state learns context-specific success while a global posterior prevents sparse
    states from becoming brittle. Rewards are exact post-evaluation credits in [0,1].
    """

```

```

    def __init__(self, n_states: int, n_ops: int):
        self.a = np.ones((n_states, n_ops), dtype=float)
        self.b = np.ones((n_states, n_ops), dtype=float)
        self.ga = np.ones(n_ops, dtype=float)
        self.gb = np.ones(n_ops, dtype=float)
        self.counts = np.zeros((n_states, n_ops), dtype=int)
        self.reward_sum = np.zeros((n_states, n_ops), dtype=float)

```

```

    def choose(self, state: int, rng: np.random.Generator, epsilon: float = 0.08) -> int:
        n_ops = self.a.shape[1]
        if rng.random() < epsilon:
            # Prioritize least-tested arms inside the current state.
            c = self.counts[state]
            minc = c.min()
            return int(rng.choice(np.flatnonzero(c == minc)))
        local = rng.beta(self.a[state], self.b[state])
        global_ = rng.beta(self.ga, self.gb)
        # Small deterministic optimism for low-count operators.
        bonus = 0.08 / np.sqrt(1.0 + self.counts[state])
        score = 0.70 * local + 0.30 * global_ + bonus
        return int(np.argmax(score))

```

```

    def update(self, state: int, op: int, reward: float) -> None:
        r = float(np.clip(reward, 0.0, 1.0))

```

```

        self.a[state, op] += r
        self.b[state, op] += 1.0 - r
        self.ga[op] += 0.25 * r
        self.gb[op] += 0.25 * (1.0 - r)
        self.counts[state, op] += 1
        self.reward_sum[state, op] += r

def _smooth_tchebycheff(F: Array, W: Array, ideal: Array, nadir: Array, mu: float) -> Array:
    Y = np.maximum((F - ideal) / (nadir - ideal + 1e-12), 0.0)
    ww = np.maximum(W, 1e-4)
    A = (ww * Y) / mu
    amax = np.max(A, axis=-1, keepdims=True)
    return mu * (amax[...], 0) + np.log(np.sum(np.exp(A - amax), axis=-1)))

def _mosaic_scalar(F: Array, W: Array, ideal: Array, nadir: Array, mu: float,
                  theta: float = 0.18, blend: float = 0.0) -> Array:
    """Adaptive blend of preference-form STCH and reference-ray STCH-PBI.

    blend=0 uses the smooth multiplication-form Tchebycheff scalarization;
    blend=1 aligns the scalarization with the geometric objective ray W via
    reciprocal coefficients and a perpendicular-distance penalty.
    """
    legacy = _smooth_tchebycheff(F, W, ideal, nadir, mu)
    if blend <= 1e-12:
        return legacy
    Y = np.maximum((F - ideal) / (nadir - ideal + 1e-12), 0.0)
    coeff = 1.0 / np.maximum(W, 0.02)
    coeff = coeff / np.mean(coeff, axis=-1, keepdims=True)
    A = (coeff * Y) / mu
    amax = np.max(A, axis=-1, keepdims=True)
    aligned = mu * (amax[...], 0) + np.log(np.sum(np.exp(A - amax), axis=-1)))
    Wn = W / (np.linalg.norm(W, axis=-1, keepdims=True) + 1e-12)
    d1 = np.sum(Y * Wn, axis=-1, keepdims=True)
    d2 = np.linalg.norm(Y - d1 * Wn, axis=-1)
    aligned = aligned + theta * d2
    b = float(np.clip(blend, 0.0, 1.0))
    return (1.0 - b) * legacy + b * aligned

def _local_density(archive_F: Array, f: Array) -> Tuple[float, float]:
    if len(archive_F) < 3:
        return 0.0, 0.0
    lo = archive_F.min(axis=0)
    hi = archive_F.max(axis=0)
    G = (archive_F - lo) / (hi - lo + 1e-12)
    y = (f - lo) / (hi - lo + 1e-12)
    dist = np.linalg.norm(G - y, axis=1)
    dist.sort()
    nearest = float(dist[1] if dist[0] < 1e-10 and len(dist) > 1 else dist[0])
    k = min(4, len(dist) - 1)
    density = float(1.0 / (np.mean(dist[1 : k + 1]) + 1e-9)) if len(dist) > 1 else 0.0
    return density, nearest

def _mosaic_state(progress: float, stagnation: int, density: float, median_density: float) -> int:
    phase = 0 if progress < 0.33 else (1 if progress < 0.72 else 2)
    stuck = int(stagnation >= 5)

```

```

crowded = int(density > median_density and median_density > 0)
return phase * 4 + stuck * 2 + crowded

def _mosaic_operator(
    op: int,
    i: int,
    X: Array,
    F: Array,
    W: Array,
    B: Array,
    archive: Archive,
    successful_steps: List[List[Array]],
    progress: float,
    rng: np.random.Generator,
    problem: Problem,
    sparse_target_x: Optional[Array] = None,
    scalar_blend: float = 0.0,
) -> Array:
    n, d = X.shape
    neigh = B[i]
    pool = neigh if rng.random() < 0.85 else np.arange(n)
    if len(pool) < 3:
        pool = np.arange(n)
    # Base current-to-pbest/l move, shared by exploit and residual variants.
    local_F = F[pool]
    ideal, nadir = F.min(axis=0), F.max(axis=0)
    mu = 0.02 + 0.13 * (1.0 - progress) ** 2
    scores = _mosaic_scalar(local_F, np.repeat(W[i][None, :], len(pool), axis=0), ideal, nadir, mu, blend=scalar_blend)
    top = pool[np.argsort(scores)[: max(2, int(math.ceil(0.2 * len(pool))))]]
    pbest = int(rng.choice(top))
    r1, r2 = rng.choice(pool, size=2, replace=False)
    Fscale = float(np.clip(0.55 + 0.20 * rng.standard_cauchy(), 0.15, 1.0))
    cr = float(np.clip(rng.normal(0.85, 0.10), 0.1, 1.0))
    base_mutant = X[i] + Fscale * (X[pbest] - X[i]) + Fscale * (X[r1] - X[r2])
    base = de_binomial(X[i], base_mutant, cr, rng)

    if op == 0: # current-to-pbest exploitation
        child = base

    elif op == 1: # global/niche DE exploration
        pool2 = np.arange(n)
        r = rng.choice(pool2, size=3, replace=False)
        F2 = float(np.clip(rng.normal(0.75, 0.18), 0.2, 1.2))
        mutant = X[r[0]] + F2 * (X[r[1]] - X[r[2]])
        child = de_binomial(X[i], mutant, float(np.clip(rng.normal(0.55, 0.2), 0.05, 1.0)), rng)

    elif op == 2: # archive-manifold residual (PCA tangent + replay memory)
        ramp = np.clip((progress - 0.10) / 0.45, 0.0, 1.0)
        delta = np.zeros(d)
        if len(archive.F) >= min(6, d + 2):
            lo = archive.F.min(axis=0)
            hi = archive.F.max(axis=0)
            AF = (archive.F - lo) / (hi - lo + 1e-12)
            fi = (F[i] - lo) / (hi - lo + 1e-12)
            idx = np.argsort(np.linalg.norm(AF - fi, axis=1))[: min(10, len(AF))]
            AX = archive.X[idx]
            center = AX.mean(axis=0)

```

```

Z = (AX - center) / (problem.xu - problem.xl + 1e-12)
try:
    _, s, vt = np.linalg.svd(Z, full_matrices=False)
    k = max(1, min(problem.n_obj - 1, len(s)))
    coeff = rng.normal(size=k) * (s[:k] + 0.02)
    tangent = coeff @ vt[:k]
    delta += 0.20 * (problem.xu - problem.xl) * tangent
except np.linalg.LinAlgError:
    pass

if sparse_target_x is not None:
    # Coverage residual: transport a successful local proposal toward the
    # nearest currently under-occupied reference cell.
    delta += 0.30 * (sparse_target_x - X[i])
if successful_steps[2] and rng.random() < 0.45:
    delta += 0.50 * successful_steps[2][rng.integers(len(successful_steps[2]))]
child = base + ramp * delta

elif op == 3: # heavy-tail residual for valleys / stagnation
    ramp = np.clip((progress - 0.05) / 0.35, 0.0, 1.0)
    # Sparse component-wise Cauchy residual; scale shrinks over time but never vanishes.
    mask = rng.random(d) < min(0.25, 3.0 / d)
    if not mask.any():
        mask[rng.integers(d)] = True
    scale = (0.12 * (1.0 - progress) + 0.015) * (problem.xu - problem.xl)
    residual = np.zeros(d)
    residual[mask] = np.clip(rng.standard_cauchy(mask.sum()), -5.0, 5.0) * scale[mask]
    child = base + ramp * residual

elif op == 4: # SBX + polynomial mutation as a structurally different expert
    mate = int(rng.choice(pool))
    child = sbx_one(X[i], X[mate], problem.xl, problem.xu, rng, eta=15.0)
else:
    raise ValueError(op)

# Reachability-Consistent Projection (RCP): every expert is evaluated
# under the same feasible proposal map, so contextual credit compares
# operators rather than operator-by-repair confounding.
child = np.clip(child, problem.xl, problem.xu)
# All experts share a light mutation floor; this prevents frozen coordinates.
child = polynomial_mutation(child, problem.xl, problem.xu, rng, eta=25.0, prob=0.5 / d)
return child

def run_mosaic(problem: Problem, seed: int, pop_size: int = 100, max_evals: int = 10100,
               T: int = 20, nr: int = 3, trace_every: int = 10,
               ablation: Optional[str] = None) -> RunResult:
    """MOSAIC-MOEA v4: Multi-Operator Speculation with Archive/Indicator-Calibrated verification.

    All operator proposals use Reachability-Consistent Projection (RCP) before
    exact evaluation, eliminating repair-induced confounding in bandit credit.

    Ablations:
    - fixed: uniform random operator choice (no contextual Thompson portfolio)
    - no_stch: classical max Tchebycheff replacement
    - no_manifold: disables archive-manifold residual (maps op2 to exploit)
    - no_heavy: disables heavy-tail residual (maps op3 to exploit)
    - exploit_only: only current-to-pbest operator
    - force_coverage: enables periodic reference-cell population repair

```

```

- legacy_scalar: multiplication-form STCH used by the v1 prototype
"""
flags = set() if ablation is None else set(str(ablation).split("+"))
rng, X, F = initialize(problem, pop_size, seed)
W = _dirichlet_reference_points(pop_size, problem.n_obj, seed=601)
dW = ((W[:, None, :] - W[None, :, :]) ** 2).sum(axis=2)
B = np.argsort(dW, axis=1)[: , : min(T, pop_size)]
init_assoc, _ = associate_to_reference(F, W)
init_occupancy = float(np.count_nonzero(np.bincount(init_assoc, minlength=pop_size)) / pop_size)
# Some decision-to-objective mappings (e.g. strong power-law bias) collapse
# almost the entire initial population into a few reference cells. That is a
# structural signal, so latch ray-aligned scalarization from the start.
alignment_latch = bool(problem.n_obj > 2 and init_occupancy < 0.25)
ideal = F.min(axis=0)
nadir = F.max(axis=0)
archive = Archive(max_size=5 * pop_size)
archive.update(X, F)
bandit = ContextualThompsonPortfolio(n_states=12, n_ops=5)
stagnation = np.zeros(pop_size, dtype=int)
successful_steps: List[List[Array]] = [[] for _ in range(5)]
evals = pop_size
gen = 0
trace: List[Dict[str, float]] = []
op_counts = np.zeros(5, dtype=int)
op_rewards = np.zeros(5, dtype=float)

while evals < max_evals:
    progress = evals / max_evals
    # Estimate current density distribution once per generation.
    densities = np.zeros(pop_size)
    for j in range(pop_size):
        densities[j] = _local_density(archive.F, F[j])[0] if len(archive.F) else 0.0
    med_density = float(np.median(densities))
    # Reference-cell occupancy is computed on the working population. It is a
    # deterministic state signal, not a learned surrogate.
    assoc_pop, _ = associate_to_reference(F, W)
    cell_counts = np.bincount(assoc_pop, minlength=pop_size)
    sparse_cells = np.flatnonzero(cell_counts == cell_counts.min())
    occupancy_ratio = float(np.count_nonzero(cell_counts) / pop_size)
    if problem.n_obj <= 2 or "legacy_scalar" in flags:
        scalar_blend = 0.0
    elif "aligned_scalar" in flags or alignment_latch:
        scalar_blend = 1.0
    else:
        # Conservative geometry gate: do not switch scalarization merely because
        # a healthy search temporarily occupies fewer cells. The initial
        # occupancy fingerprint is fixed before any optimization feedback, so it
        # cannot chase benchmark noise or create mid-run mode switching.
        scalar_blend = 0.0

    for i in rng.permutation(pop_size):
        if evals >= max_evals:
            break
        state = _mosaic_state(progress, int(stagnation[i]), float(densities[i]), med_density)
        if "exploit_only" in flags:
            op = 0
        elif "fixed" in flags:
            op = int(rng.integers(5))

```

```

else:
    op = bandit.choose(state, rng, epsilon=0.10 if progress < 0.5 else 0.06)
if "no_manifold" in flags and op == 2:
    op = 0
if "no_heavy" in flags and op == 3:
    op = 0

parent = X[i].copy()
parentF = F[i].copy()
sparse_target_x = None
if op == 2 and len(archive.F) >= 3 and len(sparse_cells) > 0:
    target_cell = int(rng.choice(sparse_cells))
    # Select the archive design whose normalized objective direction is
    # closest to the under-filled reference vector.
    az = archive.F.min(axis=0)
    an = archive.F.max(axis=0)
    AY = np.maximum((archive.F - az) / (an - az + 1e-12), 0.0)
    AYn = AY / (np.linalg.norm(AY, axis=1, keepdims=True) + 1e-12)
    wt = W[target_cell] / (np.linalg.norm(W[target_cell]) + 1e-12)
    sparse_target_x = archive.X[int(np.argmax(AYn @ wt))]
child = _mosaic_operator(
    op, i, X, F, W, B, archive, successful_steps, progress, rng, problem,
    sparse_target_x=sparse_target_x, scalar_blend=scalar_blend
)
childF = problem.evaluate(child)[0]
evals += 1

old_ideal, old_nadir = ideal.copy(), nadir.copy()
ideal = np.minimum(ideal, childF)
# Robust nadir tracks current + archive rather than monotonic historical worst.
pooled_F = np.vstack([F, archive.F, childF[None, :]]) if len(archive.F) else np.vstack([F, childF])
nadir = np.quantile(pooled_F, 0.95, axis=0)
nadir = np.maximum(nadir, ideal + 1e-9)
entered, novelty = archive.update(child, childF)

neigh = B[i]
order = rng.permutation(neigh)
replaced = 0
max_rel_improve = 0.0
for j in order:
    if "no_stch" in flags:
        oldg = _tchebycheff(F[j][None, :], W[j][None, :], ideal, nadir)[0]
        newg = _tchebycheff(childF[None, :], W[j][None, :], ideal, nadir)[0]
    else:
        mu = 0.008 + 0.10 * (1.0 - progress) ** 2
        theta = 0.10 + 0.22 * progress
        oldg = _mosaic_scalar(F[j][None, :], W[j][None, :], ideal, nadir, mu, theta=theta, blend=scalar_blend)[0]
        newg = _mosaic_scalar(childF[None, :], W[j][None, :], ideal, nadir, mu, theta=theta, blend=scalar_blend)[0]
    rel = (oldg - newg) / (abs(oldg) + 1e-12)
    # Exact verifier: a speculative expert receives control only when the exact
    # scalarized objective improves. Near-ties may pass if the offspring is newly
    # nondominated and fills a sparse region.
    verify = newg <= oldg or (entered and novelty > 0.45 and newg <= oldg * 1.015)
    if verify:
        X[j], F[j] = child.copy(), childF.copy()
        stagnation[j] = 0
        replaced += 1
        max_rel_improve = max(max_rel_improve, float(max(rel, 0.0)))

```

```

        if replaced >= nr:
            break

    if replaced == 0:
        stagnation[i] += 1
    else:
        step = child - parent
        successful_steps[op].append(step)
        if len(successful_steps[op]) > 64:
            successful_steps[op] = successful_steps[op][-64:]

    # Exact post-evaluation credit. No surrogate claim enters the posterior.
    r_conv = float(np.clip(max_rel_improve / 0.08, 0.0, 1.0))
    r_rep = float(np.clip(replaced / max(nr, 1), 0.0, 1.0))
    r_arc = 0.25 + 0.75 * novelty if entered else 0.0
    cz = np.minimum(F.min(axis=0), childF)
    cn = np.maximum(F.max(axis=0), childF)
    cy = np.maximum((childF - cz) / (cn - cz + 1e-12), 0.0)
    cyn = cy / (np.linalg.norm(cy) + 1e-12)
    child_cell = int(np.argmax((W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)) @ cyn))
    r_cover = 1.0 if entered and cell_counts[child_cell] == 0 else 0.0
    reward = float(np.clip(0.42 * r_conv + 0.20 * r_rep + 0.23 * r_arc + 0.15 * r_cover, 0.0, 1.0))
    if "fixed" not in flags and "exploit_only" not in flags:
        bandit.update(state, op, reward)
    op_counts[op] += 1
    op_rewards[op] += reward

gen += 1
if "force_coverage" in flags and gen % 4 == 0 and len(archive.F) >= pop_size // 2:
    # Exact coverage repair: combine the working population with a compact
    # nondominated archive sample, then apply NSGA-III reference niching.
    ax, af = archive.output(min(2 * pop_size, len(archive.F)))
    X, F = nsga3_environmental_selection(
        np.vstack([X, ax]), np.vstack([F, af]), pop_size, W, rng
    )
    stagnation = np.minimum(stagnation, 3)
    ideal = np.minimum(ideal, F.min(axis=0))
    nadir = np.maximum(np.quantile(np.vstack([F, af]), 0.95, axis=0), ideal + 1e-9)
if gen % trace_every == 0:
    trace.append({
        "gen": gen,
        "evals": evals,
        "archive_size": len(archive.F),
        "mean_stagnation": float(stagnation.mean()),
        "mean_reward": float(op_rewards.sum() / max(op_counts.sum(), 1)),
        "occupancy_ratio": occupancy_ratio,
        "scalar_blend": scalar_blend,
    })

outX, outF = archive.output(pop_size)
name = "MOSAIC-MOEA" if ablation is None else f"MOSAIC[{ablation}]"
diagnostics: Dict[str, object] = {
    "version": "MOSAIC-MOEA-v4-RCP",
    "repair": "reachability-consistent box projection",
    "operator_counts": op_counts.tolist(),
    "operator_mean_rewards": (op_rewards / np.maximum(op_counts, 1)).tolist(),
    "bandit_counts": bandit.counts.tolist(),
    "bandit_mean": (bandit.a / (bandit.a + bandit.b)).tolist(),
}

```

```

    }

    return RunResult(name, problem.name, seed, outX, outF, evals, trace, diagnostics)

ALGORITHMS = {
    "nsga2": run_nsga2,
    "moea": run_moea,
    "nsga3": run_nsga3,
    "rdex": run_rdex,
    "rvea": run_rvea,
    "mosaic": run_mosaic,
}

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/legacy/mosaic_lmqoe.py

```
from __future__ import annotations
```

```
"""MOSAIC-ST research branch.
```

This module keeps the frozen MOSAIC-MOEA v4 evaluator, scalarization and exact replacement semantics, and adds three isolated mechanisms:

1. Exact Coverage Memory (ECM): a bounded bank of *actually evaluated* designs, indexed by reference direction. It is separate from the Pareto archive so potentially useful stepping stones are not erased merely because they are temporarily dominated.
2. Archive-Reuse Transport Expert (ART, operator 5): a failure-mode specialist activated only when reference-cell coverage is distressed.
3. Secant-Repulsive Transport (SRT, optional upgrade of operator 2): a local ridge secant model maps a Smooth-Tchebycheff descent/repulsion direction in objective space back to a trust-region decision-space residual. The residual is bypassed when its cross-validated local fit is unreliable.
4. Role-Gated Coordinate Basin Tunneling (RCBT, operator 6): when reference coverage is healthy but scalar progress stalls, low-discrepancy one-coordinate probes are applied only to variables inferred to control convergence rather than Pareto direction. Successful coordinate values are composed into later probes, but every candidate still requires exact evaluation and STCH acceptance.

The default callable exposes ablations so every mechanism can be evaluated under identical seeds and function-evaluation budgets. No surrogate prediction is allowed to update the population or archive without exact objective evaluation.

```
"""
```

```

from dataclasses import dataclass
from typing import Dict, List, Optional, Sequence, Tuple
import math
import numpy as np

```

```
import moo_core as core
```

```
Array = np.ndarray
```

```
def _associate_fixed(F: Array, W: Array, ideal: Array, nadir: Array) -> Tuple[Array, Array]:
```



```

"""Reference association in one shared longitudinal normalization frame."""
F = np.asarray(F, dtype=float)
Y = np.maximum((F - ideal) / (nadir - ideal + 1e-12), 0.0)
Yn = Y / (np.linalg.norm(Y, axis=1, keepdims=True) + 1e-12)
Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)
cos = np.clip(Yn @ Wn.T, -1.0, 1.0)
assoc = np.argmax(cos, axis=1)
angle = np.arccos(cos[np.arange(len(F)), assoc])
return assoc.astype(int), angle

def _occupancy_entropy(assoc: Array, n_cells: int) -> Tuple[float, float, Array]:
    counts = np.bincount(np.asarray(assoc, dtype=int), minlength=n_cells)
    occupancy = float(np.count_nonzero(counts) / max(n_cells, 1))
    p = counts[counts > 0].astype(float)
    p /= max(p.sum(), 1.0)
    if n_cells <= 1 or len(p) == 0:
        entropy = 0.0
    else:
        entropy = float(-np.sum(p * np.log(p + 1e-15)) / np.log(n_cells))
    return occupancy, entropy, counts

@dataclass
class MemoryCandidate:
    x: Array
    f: Array
    born: int

class ExactCoverageMemory:
    """Bounded exact-evaluation memory with reference-cell-aware compaction.

    Pareto archives are intentionally selective in objective space. This memory
    has a different role: preserve historically evaluated stepping stones that
    may be needed to reconstruct a missing reference region. It never invents
    objective values and never bypasses the exact verifier for newly generated
    offspring.
    """

    def __init__(self, capacity: int, per_cell: int = 3):
        self.capacity = int(max(capacity, 8))
        self.per_cell = int(max(per_cell, 1))
        self.X = np.empty((0, 0), dtype=float)
        self.F = np.empty((0, 0), dtype=float)
        self.born = np.empty(0, dtype=int)

    def add(self, X: Array, F: Array, generation: int, W: Array, ideal: Array, nadir: Array) -> None:
        X = np.asarray(X, dtype=float)
        F = np.asarray(F, dtype=float)
        if X.ndim == 1:
            X = X[None, :]
        if F.ndim == 1:
            F = F[None, :]
        if self.X.size == 0:
            self.X = X.copy()
            self.F = F.copy()
            self.born = np.full(len(X), generation, dtype=int)

```

```

else:
    self.X = np.vstack([self.X, X])
    self.F = np.vstack([self.F, F])
    self.born = np.r_[self.born, np.full(len(X), generation, dtype=int)]
if len(self.F) > int(1.20 * self.capacity):
    self.compact(W, ideal, nadir)

def compact(self, W: Array, ideal: Array, nadir: Array) -> None:
    if len(self.F) <= self.capacity:
        return
    assoc, _ = _associate_fixed(self.F, W, ideal, nadir)
    selected: List[int] = []
    mu = 0.02
    span_x = np.ptp(self.X, axis=0) + 1e-12
    for c in range(len(W)):
        idx = np.flatnonzero(assoc == c)
        if len(idx) == 0:
            continue
        Wi = np.repeat(W[c][None, :], len(idx), axis=0)
        scores = core._mosaic_scalar(self.F[idx], Wi, ideal, nadir, mu, theta=0.22, blend=0.0)
        order = idx[np.argsort(scores, kind="mergesort")]
        keep: List[int] = [int(order[0])]
        # Preserve a recent candidate and then decision-diverse candidates.
        recent = int(idx[np.argmax(self.born[idx])])
        if recent not in keep and len(keep) < self.per_cell:
            keep.append(recent)
        while len(keep) < min(self.per_cell, len(idx)):
            remaining = [int(j) for j in idx if int(j) not in keep]
            if not remaining:
                break
            D = (self.X[remaining] - self.X[keep][:, None, :]).transpose(1, 0, 2)
            # min normalized decision distance to an already kept point
            d = np.linalg.norm(D / span_x[None, None, :], axis=2).min(axis=1)
            keep.append(remaining[int(np.argmax(d))])
        selected.extend(keep)
    # If per-cell retention is still too large, use objective-space maximin.
    selected = list(dict.fromkeys(selected))
    if len(selected) > self.capacity:
        local = core.select_indices_farthest(self.F[selected], self.capacity)
        selected = [selected[int(j)] for j in local]
    # If cells are sparse and capacity remains, retain recent evaluated points.
    if len(selected) < self.capacity:
        rest = [i for i in np.argsort(-self.born, kind="mergesort") if int(i) not in set(selected)]
        selected.extend([int(i) for i in rest[: self.capacity - len(selected)]])
    idx = np.asarray(selected[: self.capacity], dtype=int)
    self.X, self.F, self.born = self.X[idx], self.F[idx], self.born[idx]

def candidates_for_cell(
    self, cell: int, W: Array, ideal: Array, nadir: Array, max_candidates: int = 6
) -> Array:
    if len(self.F) == 0:
        return np.empty(0, dtype=int)
    assoc, angle = _associate_fixed(self.F, W, ideal, nadir)
    exact = np.flatnonzero(assoc == int(cell))
    if len(exact) == 0:
        # Fallback to closest objective direction; this is still exact memory.
        return np.argsort(angle, kind="mergesort")[:max_candidates]
    Wi = np.repeat(W[int(cell)][None, :], len(exact), axis=0)

```

```

score = core._mosaic_scalar(self.F[exact], Wi, ideal, nadir, 0.018, theta=0.25, blend=0.0)
return exact[np.argsort(score, kind="mergesort")[:max_candidates]]

def best_anchor_advantage(
    self, cell: int, current_F: Array, W: Array, ideal: Array, nadir: Array
) -> float:
    """Exact scalar advantage of the best memory anchor for one cell."""
    idx = self.candidates_for_cell(cell, W, ideal, nadir, max_candidates=3)
    if len(idx) == 0:
        return 0.0
    wc = W[int(cell)][None, :]
    mem = core._mosaic_scalar(
        self.F[idx], np.repeat(wc, len(idx), axis=0), ideal, nadir, 0.018,
        theta=0.25, blend=0.0
    ).min()
    cur = core._mosaic_scalar(
        current_F, np.repeat(wc, len(current_F), axis=0), ideal, nadir, 0.018,
        theta=0.25, blend=0.0
    ).min()
    return float((cur - mem) / (abs(cur) + 1e-12))

def choose_anchor(
    self, cell: int, W: Array, ideal: Array, nadir: Array, rng: np.random.Generator
) -> Optional[Tuple[Array, Array]]:
    idx = self.candidates_for_cell(cell, W, ideal, nadir)
    if len(idx) == 0:
        return None
    # Mostly choose the best scalar candidate but retain slight diversity.
    j = int(idx[0] if len(idx) == 1 or rng.random() < 0.75 else rng.choice(idx[: min(3, len(idx))]))
    return self.X[j].copy(), self.F[j].copy()

def reconstruct(
    self,
    X: Array,
    F: Array,
    W: Array,
    ideal: Array,
    nadir: Array,
    fraction: float,
    rng: np.random.Generator,
) -> Tuple[Array, Array, int, List[int]]:
    """Inject exact historical candidates into missing cells without new FEs."""
    if len(self.F) == 0:
        return X, F, 0, []
    assoc, _ = _associate_fixed(F, W, ideal, nadir)
    _, _, counts = _occupancy_entropy(assoc, len(W))
    missing = np.flatnonzero(counts == 0)
    if len(missing) == 0:
        return X, F, 0, []
    n_inject = min(max(1, int(math.ceil(fraction * len(X)))), len(missing))
    # Largest angular holes first: directions farthest from occupied directions.
    occupied = np.flatnonzero(counts > 0)
    Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)
    if len(occupied):
        proximity = np.max(Wn[missing] @ Wn[occupied].T, axis=1)
        targets = missing[np.argsort(proximity, kind="mergesort")[:n_inject]]
    else:
        targets = missing[:n_inject]

```

```

# Victims: poor members of over-populated cells. Do not force a target
# into the same array index; the frozen MOEA/D update will reassociate it.
redundancy = counts[assoc]
ownW = W[assoc]
scalar = core._mosaic_scalar(F, ownW, ideal, nadir, 0.012, theta=0.25, blend=0.0)
s_norm = (scalar - scalar.min()) / (np.ptp(scalar) + 1e-12)
victim_score = redundancy.astype(float) + s_norm
victim_order = np.argsort(-victim_score, kind="mergesort")

Xn, Fn = X.copy(), F.copy()
used_mem: set[int] = set()
used_victims: set[int] = set()
restored_cells: List[int] = []
for cell in targets:
    cand = self.candidates_for_cell(int(cell), W, ideal, nadir)
    cand = np.asarray([j for j in cand if int(j) not in used_mem], dtype=int)
    if len(cand) == 0:
        continue
    mem_j = int(cand[0])
    victim = next((int(j) for j in victim_order if int(j) not in used_victims), None)
    if victim is None:
        break
    # Skip exact duplicate designs.
    if np.any(np.all(np.isclose(Xn, self.X[mem_j], atol=1e-12, rtol=1e-10), axis=1)):
        used_mem.add(mem_j)
        continue
    Xn[victim], Fn[victim] = self.X[mem_j].copy(), self.F[mem_j].copy()
    used_mem.add(mem_j)
    used_victims.add(victim)
    restored_cells.append(int(cell))
return Xn, Fn, len(restored_cells), restored_cells

class DiscountedPortfolio(core.ContextualThompsonPortfolio):
    """Optional non-stationary extension; kept as an ablation, not assumed useful."""

    def decay(self, gamma: float) -> None:
        g = float(np.clip(gamma, 0.90, 1.0))
        self.a = 1.0 + g * (self.a - 1.0)
        self.b = 1.0 + g * (self.b - 1.0)
        self.ga = 1.0 + g * (self.ga - 1.0)
        self.gb = 1.0 + g * (self.gb - 1.0)
        self.reward_sum *= g

    def _secant_repulsive_delta(
        i: int,
        X: Array,
        F: Array,
        archive: core.Archive,
        W: Array,
        ideal: Array,
        nadir: Array,
        progress: float,
        problem: core.Problem,
        rng: np.random.Generator,
        use_repulsion: bool = True,

```

```

) -> Tuple[Array, float, float]:
    """Map an objective-space descent/repulsion vector through a local secant model.

    Returns (decision residual, reliability, normalized local fit error). The
    caller must ignore the residual when reliability is low.
    """
    d, m = problem.n_var, problem.n_obj
    if len(archive.F) < max(8, m + 4):
        return np.zeros(d), 0.0, 1.0
    span_x = problem.xu - problem.xl + 1e-12
    span_f = nadir - ideal + 1e-12
    AF = np.maximum((archive.F - ideal) / span_f, 0.0)
    yi = np.maximum((F[i] - ideal) / span_f, 0.0)
    dist = np.linalg.norm(AF - yi, axis=1)
    k = min(max(10, 2 * m + 4), len(AF))
    idx = np.argsort(dist, kind="mergesort")[:k]
    DX = (archive.X[idx] - X[i]) / span_x
    DY = AF[idx] - yi
    # Remove nearly identical rows, which make local secants unidentifiable.
    keep = np.linalg.norm(DX, axis=1) > 1e-8
    DX, DY = DX[keep], DY[keep]
    if len(DX) < m + 3:
        return np.zeros(d), 0.0, 1.0
    ridge = 2e-3 + 2e-2 * (1.0 - progress)
    try:
        # Dual ridge fit:  $J^T = DX^T (DX DX^T + \lambda I)^{-1} DY$ .
        gram = DX @ DX.T + ridge * np.eye(len(DX))
        Jt = DX.T @ np.linalg.solve(gram, DY)
        J = Jt.T
    except np.linalg.LinAlgError:
        return np.zeros(d), 0.0, 1.0
    pred = DX @ Jt
    sse = float(np.sum((DY - pred) ** 2))
    sst = float(np.sum((DY - DY.mean(axis=0, keepdims=True)) ** 2)) + 1e-12
    fit_error = float(np.clip(sse / sst, 0.0, 2.0))
    # Condition reliability in objective row-space.
    try:
        cond = float(np.linalg.cond(J @ J.T + 1e-4 * np.eye(m)))
    except np.linalg.LinAlgError:
        cond = 1e12
    reliability = float(np.clip((1.0 - min(fit_error, 1.0)) * np.exp(-max(np.log10(cond) - 3.0, 0.0)), 0.0, 1.0))
    if reliability < 0.28:
        return np.zeros(d), reliability, fit_error

    # Smooth-Tchebycheff gradient in normalized objective coordinates.
    w = np.maximum(W[i], 1e-4)
    mu = 0.012 + 0.07 * (1.0 - progress) ** 2
    logits = w * yi / mu
    logits -= logits.max()
    soft = np.exp(logits)
    soft /= soft.sum() + 1e-12
    grad = soft * w
    dy = -grad / (np.linalg.norm(grad) + 1e-12)

    if use_repulsion:
        D = yi[None, :] - AF[idx]
        r = np.linalg.norm(D, axis=1)
        positive = r[r > 1e-8]

```

```

h = float(np.median(positive)) if len(positive) else 0.15
h = max(h, 0.05)
weight = np.exp(-(r * r) / (2.0 * h * h))
rep = np.sum(weight[:, None] * D, axis=0) / (weight.sum() + 1e-12)
if np.linalg.norm(rep) > 1e-10:
    rep /= np.linalg.norm(rep)
    # Repulsion matters mainly in crowded regions and later in the run.
    beta = 0.12 + 0.22 * progress
    dy = dy + beta * rep
    dy /= np.linalg.norm(dy) + 1e-12

try:
    dx = J.T @ np.linalg.solve(J @ J.T + (4e-3 + 1e-2 * (1.0 - progress)) * np.eye(m), dy)
except np.linalg.LinAlgError:
    return np.zeros(d), 0.0, fit_error
nrm = np.linalg.norm(dx)
if not np.isfinite(nrm) or nrm < 1e-12:
    return np.zeros(d), 0.0, fit_error
dx /= nrm
trust = 0.10 * (1.0 - progress) + 0.018
# Small random tangent jitter prevents deterministic secant lock-in.
jitter = rng.normal(size=d)
jitter -= dx * float(np.dot(jitter, dx))
jitter /= np.linalg.norm(jitter) + 1e-12
residual = span_x * trust * (0.90 * dx + 0.10 * jitter)
return residual, reliability, fit_error

def _infer_variable_roles(
    memory: ExactCoverageMemory,
    problem: core.Problem,
    ideal: Array,
    nadir: Array,
    W: Optional[Array] = None,
) -> Tuple[Optional[Array], float, Dict[str, float]]:
    """Infer position-vs-distance roles by recent reference-cell separation.

    Let  $c(n)$  be the reference cell associated with evaluated design  $n$ . For each
    decision variable  $j$ , we compute the one-way ANOVA effect size

    
$$\eta_j^2 = \text{between-cell variance} / \text{total variance}.$$


    A high  $\eta^2$  means the variable changes systematically with Pareto direction
    (a position/diversity variable). A low  $\eta^2$  means it is better supplied by
    a currently converged donor. Only the recent half of exact memory is used;
    early random correlations otherwise make unrelated variables look like
    position variables after the population has already contracted.
    """
    if W is None:
        W = core._dirichlet_reference_points(max(32, problem.n_obj * 16), problem.n_obj, seed=601)
    if len(memory.F) < max(40, 2 * problem.n_var):
        return None, 0.0, {"occupied_cells": 0.0, "eta_gap": 0.0}
    cutoff = float(np.quantile(memory.born, 0.50))
    idx = np.flatnonzero(memory.born >= cutoff)
    if len(idx) < max(40, 2 * problem.n_var):
        idx = np.argsort(memory.born, kind="mergesort")[-min(len(memory.F), max(80, 4 * problem.n_var)):]
    X, F = memory.X[idx], memory.F[idx]

```

```

Xn = (X - problem.xl) / (problem.xu - problem.xl + 1e-12)
assoc, _ = _associate_fixed(F, W, ideal, nadir)
occupied = np.unique(assoc)
mean = Xn.mean(axis=0)
total = np.sum((Xn - mean) ** 2, axis=0) + 1e-12
between = np.zeros(problem.n_var)
for cell in occupied:
    j = np.flatnonzero(assoc == cell)
    if len(j):
        between += len(j) * (Xn[j].mean(axis=0) - mean) ** 2
eta = np.clip(between / total, 0.0, 1.0)

# Deterministic 1-D two-cluster split of variable effect sizes.
c0, c1 = float(eta.min()), float(eta.max())
for _ in range(20):
    d0, d1 = np.abs(eta - c0), np.abs(eta - c1)
    hi = d1 < d0
    if not np.any(hi) or np.all(hi):
        break
    n0, n1 = float(eta[~hi].mean()), float(eta[hi].mean())
    if abs(n0 - c0) + abs(n1 - c1) < 1e-10:
        c0, c1 = n0, n1
        break
    c0, c1 = n0, n1
low, high = sorted((c0, c1))
gap = high - low
midpoint = 0.5 * (high + low)
temperature = max(0.035, 0.18 * gap)
role_prob = 1.0 / (1.0 + np.exp(-(eta - midpoint) / temperature))
# Require at least one clear position variable; cap uncertain probabilities.
role_prob = np.clip(role_prob, 0.02, 0.98)
if gap < 0.06:
    reliability = 0.0
else:
    cell_factor = min(1.0, len(occupied) / max(problem.n_obj + 2, 4))
    reliability = float(np.clip((gap / (np.std(eta) + 1e-12)) / 4.0, 0.0, 1.0) * cell_factor)
return role_prob, reliability, {
    "occupied_cells": float(len(occupied)),
    "eta_gap": float(gap),
    "eta_max": float(eta.max()),
    "eta_second": float(np.partition(eta, -2)[-2]) if len(eta) > 1 else float(eta.max()),
    "mean_position_probability": float(role_prob.mean()),
}

```

```

def _archive_transport_child(
    i: int,
    target_cell: int,
    X: Array,
    F: Array,
    W: Array,
    B: Array,
    memory: ExactCoverageMemory,
    ideal: Array,
    nadir: Array,
    progress: float,
    rng: np.random.Generator,
    problem: core.Problem,

```

```

    role_probability: Optional[Array] = None,
    role_reliability: float = 0.0,
) -> Optional[Array]:
    anchor = memory.choose_anchor(target_cell, W, ideal, nadir, rng)
    if anchor is None:
        return None
    xa, _ = anchor
    n, d = X.shape
    pool = B[i] if len(B[i]) >= 3 else np.arange(n)
    r1, r2 = rng.choice(pool, size=2, replace=False)

    if role_probability is not None and role_reliability >= 0.18:
        # Donor minimizes the target scalarization among the current population:
        # it supplies variables that have already learned convergence, while the
        # memory anchor supplies variables that control objective direction.
        wt = np.repeat(W[int(target_cell)][None, :], n, axis=0)
        score = core._mosaic_scalar(F, wt, ideal, nadir, 0.015, theta=0.22, blend=0.0)
        elite = np.argsort(score, kind="mergesort")[: max(2, int(math.ceil(0.12 * n)))]
        donor = X[int(rng.choice(elite))]
        ppos = np.clip(role_probability, 0.05, 0.95)
        position_mask = rng.random(d) < ppos
        if not np.any(position_mask):
            position_mask[int(np.argmax(ppos))] = True
        if np.all(position_mask):
            position_mask[int(np.argmin(ppos))] = False
        child = donor.copy()
        child[position_mask] = 0.92 * xa[position_mask] + 0.08 * donor[position_mask]
        # Refine distance variables only; perturbing inferred position variables
        # would immediately destroy the rescued reference direction.
        conv_mask = ~position_mask
        scale = float(np.clip(rng.normal(0.18 - 0.08 * progress, 0.05), 0.04, 0.28))
        child[conv_mask] += scale * (X[r1, conv_mask] - X[r2, conv_mask])
    else:
        # Reliability fallback: ordinary archive transport, still exact-verified.
        Fscale = float(np.clip(rng.normal(0.30 - 0.12 * progress, 0.08), 0.08, 0.45))
        mutant = xa + Fscale * (X[r1] - X[r2])
        cr = float(np.clip(rng.normal(0.82, 0.10), 0.35, 1.0))
        child = core.de_binomial(xa, mutant, cr, rng)

    child = np.clip(child, problem.xl, problem.xu)
    child = core.polynomial_mutation(child, problem.xl, problem.xu, rng, eta=30.0, prob=0.35 / d)
    return child

def _van_der_corput(index: int, base: int = 2) -> float:
    """Deterministic low-discrepancy scalar in (0,1)."""
    n = max(int(index), 1)
    denom = 1.0
    out = 0.0
    while n:
        n, rem = divmod(n, base)
        denom *= base
        out += rem / denom
    return float(out)

```



```

def _coordinate_expert_scores(
    candidates: Array,
    role_probability: Array,
    memory: ExactCoverageMemory,
    problem: core.Problem,
    ideal: Array,
    nadir: Array,
    probe_trials: Array,
    probe_success: Array,
) -> Array:
    """Score coordinate experts from exact memory, role confidence and UCB.

    No objective call is made here. The sensitivity term measures how strongly
    elite exact-memory points concentrate a variable relative to the whole exact
    memory, while the role prior suppresses variables that encode PF position.
    """
    cand = np.asarray(candidates, dtype=int)
    if len(cand) == 0:
        return np.empty(0)
    role = 1.0 - np.clip(role_probability[cand], 0.0, 1.0)
    sensitivity = np.zeros(len(cand), dtype=float)
    if len(memory.F) >= max(30, 2 * len(cand)):
        recent_cut = float(np.quantile(memory.born, 0.45))
        idx = np.flatnonzero(memory.born >= recent_cut)
        if len(idx) < 30:
            idx = np.arange(len(memory.F))
        MX = (memory.X[idx] - problem.xl) / (problem.xu - problem.xl + 1e-12)
        MF = memory.F[idx]
        Y = np.maximum((MF - ideal) / (nadir - ideal + 1e-12), 0.0)
        quality = np.sum(Y, axis=1)
        elite_n = max(8, int(math.ceil(0.20 * len(quality))))
        elite_idx = np.argsort(quality, kind="mergesort")[:elite_n]
        for k, j in enumerate(cand):
            all_std = float(np.std(MX[:, j])) + 1e-9
            elite_std = float(np.std(MX[elite_idx, j]))
            center_shift = abs(float(np.mean(MX[elite_idx, j]) - np.mean(MX[:, j]))) / all_std
            concentration = float(np.clip(1.0 - elite_std / all_std, 0.0, 1.0))
            sensitivity[k] = float(np.clip(0.55 * concentration + 0.45 * np.tanh(center_shift / 2.0), 0.0, 1.0))
    post_mean = (probe_success[cand] + 1.0) / (probe_trials[cand] + 2.0)
    uncertainty = 1.0 / np.sqrt(1.0 + probe_trials[cand])
    return 0.42 * role + 0.28 * sensitivity + 0.20 * post_mean + 0.10 * uncertainty


def _dynamic_coordinate_batch(
    candidates: Array,
    scores: Array,
    probe_trials: Array,
    max_batch: int = 4,
) -> Tuple[Array, int, float]:
    """Entropy-calibrated dynamic-k with a score-quantile cutoff."""
    cand = np.asarray(candidates, dtype=int)
    scores = np.asarray(scores, dtype=float)
    if len(cand) == 0:
        return np.empty(0, dtype=int), 0, 0.0
    z = scores - np.max(scores)
    p = np.exp(z / 0.12)
    p /= max(float(p.sum()), 1e-12)
    entropy = float(-np.sum(p * np.log(p + 1e-15)) / max(np.log(len(p)), 1e-12)) if len(p) > 1 else 0.0

```

```

# High uncertainty/entropy buys more coordinate experts; confident routing
# spends only one. The overall FE budget remains unchanged.
k = int(np.clip(1 + math.ceil((max_batch - 1) * entropy), 1, min(max_batch, len(cand))))
# Prefer untested experts, then apply the score quantile within that set.
untested = probe_trials[cand] == 0
pool_idx = np.flatnonzero(untested) if np.any(untested) else np.arange(len(cand))
pool_scores = scores[pool_idx]
take = min(k, len(pool_idx))
cutoff = float(np.quantile(pool_scores, max(0.0, 1.0 - take / max(len(pool_scores), 1))))
eligible = pool_idx[pool_scores >= cutoff - 1e-12]
order = eligible[np.argsort(-scores[eligible], kind="mergesort")]
chosen = cand[order[:take]]
return chosen.astype(int), k, entropy

def _coordinate_tunnel_child(
    target_cell: int,
    variable: int,
    X: Array,
    F: Array,
    W: Array,
    ideal: Array,
    nadir: Array,
    problem: core.Problem,
    probe_index: Array,
    learned_values: Array,
    role_probability: Array,
) -> Tuple[Array, float]:
    """Generate one exact coordinate intervention without moving PF position vars."""
    n, d = X.shape
    wt = np.repeat(W[int(target_cell)][None, :], n, axis=0)
    score = core._mosaic_scalar(F, wt, ideal, nadir, 0.015, theta=0.22, blend=0.0)
    donor = X[int(np.argmin(score))]
    child = donor.copy()
    # Compose previously verified convergence alleles. This is an exact-memory
    # analogue of optimal mixing: only values with prior positive verifier credit
    # are broadcast, and inferred position variables remain untouched.
    known = np.isfinite(learned_values) & (role_probability < 0.35)
    child[known] = learned_values[known]
    j = int(variable)
    span = float(problem.xu[j] - problem.xl[j])
    # Advance the base-2 low-discrepancy line-search sequence until the probe is
    # not numerically identical to the donor/known value.
    value = child[j]
    for _ in range(8):
        probe_index[j] += 1
        u = _van_der_corput(int(probe_index[j]), 2)
        value = float(problem.xl[j] + u * span)
        if abs(value - child[j]) > 0.015 * max(span, 1e-12):
            break
    child[j] = value
    return np.clip(child, problem.xl, problem.xu), value

# -----
# MOSAIC-LQMoE v8: Shared backbone + latent quantile-routed micro-experts
# -----

```

```

from scipy.optimize import linear_sum_assignment

def _safe_unit_rows(A: Array) -> Array:
    A = np.asarray(A, dtype=float)
    return A / (np.linalg.norm(A, axis=1, keepdims=True) + 1e-12)

def _build_latent_basis(
    X: Array,
    successful_steps: Sequence[Sequence[Array]],
    span: Array,
    n_obj: int,
    max_rank: int = 10,
) -> Array:
    """Build a stable low-rank routed subspace from population and verified steps.

    The always-on shared proposal remains full dimensional. Only routed residuals
    are compressed into this basis, mirroring the shared-full-width / routed-latent
    separation of modern LatentMoE designs.
    """
    blocks: List[Array] = []
    span = np.maximum(np.asarray(span, dtype=float), 1e-12)
    Xn = (np.asarray(X, dtype=float) - np.mean(X, axis=0, keepdims=True)) / span
    if len(Xn):
        blocks.append(Xn)
    verified: List[Array] = []
    for seq in successful_steps:
        if seq:
            verified.extend(np.asarray(s, dtype=float) / span for s in seq[-24:])
    if verified:
        blocks.append(np.vstack(verified))
    Z = np.vstack(blocks) if blocks else np.eye(len(span))
    # Remove numerically empty rows and apply light shrinkage toward coordinate axes.
    nz = np.linalg.norm(Z, axis=1) > 1e-10
    Z = Z[nz]
    if len(Z) == 0:
        return np.eye(len(span))[:1]
    try:
        _, s, vt = np.linalg.svd(Z, full_matrices=False)
    except np.linalg.LinAlgError:
        return np.eye(len(span))[: min(len(span), max(1, n_obj))]
    numerical_rank = int(np.count_nonzero(s > max(s[0], 1e-12) * 1e-7))
    rank = max(1, min(max_rank, len(span), max(n_obj + 1, numerical_rank)))
    return vt[:rank].copy()

def _project_routed_residual(delta: Array, basis: Array, span: Array, retain: float = 0.18) -> Array:
    """Project a residual into the latent search subspace with an escape floor."""
    span = np.maximum(np.asarray(span, dtype=float), 1e-12)
    z = np.asarray(delta, dtype=float) / span
    if basis.size == 0:
        return np.asarray(delta, dtype=float)
    projected = (z @ basis.T) @ basis
    # A small full-space component prevents a stale basis from making dimensions
    # permanently unreachable.
    return span * ((1.0 - retain) * projected + retain * z)

```

```

def _proposal_energy_clip(delta: Array, span: Array, cap: float) -> Tuple[Array, float]:
    """RMS-cap the routed proposal energy; return clipped delta and scale factor."""
    span = np.maximum(np.asarray(span, dtype=float), 1e-12)
    energy = float(np.sqrt(np.mean((np.asarray(delta, dtype=float) / span) ** 2)))
    if not np.isfinite(energy) or energy <= 1e-15:
        return np.zeros_like(delta, dtype=float), 1.0
    scale = min(1.0, float(cap) / energy)
    return np.asarray(delta, dtype=float) * scale, scale

class LatentQuantileRouter:
    """Continuous Bayesian router with memory and loss-free quantile balancing.

    Balance biases affect assignment only. They never enter the exact reward or
    posterior target, preserving a clean separation between task utility and load
    control, analogous to auxiliary-loss-free routing.
    """

    def __init__(self, n_experts: int, n_features: int, ridge: float = 2.0):
        self.n_experts = int(n_experts)
        self.n_features = int(n_features)
        self.ridge = float(ridge)
        eye = np.eye(self.n_features, dtype=float)
        self.A = np.repeat((self.ridge * eye)[None, :, :], self.n_experts, axis=0)
        self.b = np.zeros((self.n_experts, self.n_features), dtype=float)
        self.calls = np.zeros(self.n_experts, dtype=int)
        self.reward_ema = np.full(self.n_experts, 0.30, dtype=float)
        self.advantage_ema = np.zeros(self.n_experts, dtype=float)
        self.load_ema = np.full(self.n_experts, 1.0 / self.n_experts, dtype=float)
        self.default_reward = np.full(self.n_experts, 0.30, dtype=float)
        self.prototype = np.zeros((self.n_experts, self.n_features), dtype=float)
        self.prototype_mass = np.zeros(self.n_experts, dtype=float)
        self.last_target = np.full(self.n_experts, 1.0 / self.n_experts, dtype=float)
        self.last_bias = np.zeros(self.n_experts, dtype=float)
        self.last_load = np.zeros(self.n_experts, dtype=float)
        self.score_clip_events = 0

    def _posterior_scores(self, Phi: Array, rng: np.random.Generator, uncertainty: float) -> Array:
        Phi = np.asarray(Phi, dtype=float)
        scores = np.empty((len(Phi), self.n_experts), dtype=float)
        for e in range(self.n_experts):
            try:
                inv = np.linalg.inv(self.A[e])
            except np.linalg.LinAlgError:
                inv = np.linalg.pinv(self.A[e])
            mean = inv @ self.b[e]
            cov = 0.10 ** 2 * inv
            try:
                theta = rng.multivariate_normal(mean, cov, check_valid="ignore")
            except (ValueError, np.linalg.LinAlgError):
                theta = mean
            pred = Phi @ theta
            unc = np.sqrt(np.maximum(np.einsum("nd,dd,nd->n", Phi, inv, Phi), 0.0))
            scores[:, e] = pred + float(uncertainty) * unc
        return scores

    def _memory_affinity(self, Phi: Array) -> Array:

```

```

P = _safe_unit_rows(np.asarray(Phi, dtype=float))
Q = _safe_unit_rows(self.prototypes + 1e-12)
sim = P @ Q.T
active = self.prototype_mass >= 2.0
sim[:, ~active] = 0.0
return sim

def _target_distribution(self, n: int, progress: float) -> Array:
    # The null residual (expert 0) is an always-available shared-only route.
    q0 = float(np.clip(0.42 + 0.18 * progress, 0.42, 0.60))
    quality = np.clip(self.advantage_ema[1:] / 0.10, -4.0, 4.0)
    quality -= np.max(quality) if len(quality) else 0.0
    expq = np.exp(quality / 1.35)
    soft = expq / max(float(expq.sum()), 1e-12)
    # Early in training every fine-grained expert receives a small probe quota;
    # later, proven specialists get the capacity while weak experts can go idle.
    if np.min(self.calls[1:]) < 2:
        floor = min(1.0 / max(n, 1), (1.0 - q0) / max(self.n_experts - 1, 1))
    else:
        floor = 0.0
    routed = (1.0 - q0) * (0.18 / max(self.n_experts - 1, 1) + 0.82 * soft)
    if floor > 0:
        routed = np.maximum(routed, floor)
        routed *= (1.0 - q0) / max(float(routed.sum()), 1e-12)
    q = np.r_[q0, routed]
    q /= q.sum()
    return q

@staticmethod
def _integer_quotas(q: Array, n: int) -> Array:
    raw = np.asarray(q, dtype=float) * int(n)
    quota = np.floor(raw).astype(int)
    remainder = int(n - quota.sum())
    if remainder > 0:
        order = np.argsort(-(raw - quota), kind="mergesort")
        quota[order[:remainder]] += 1
    elif remainder < 0:
        order = np.argsort(raw - quota, kind="mergesort")
        for e in order:
            take = min(quota[e], -remainder)
            quota[e] -= take
            remainder += take
            if remainder == 0:
                break
    return quota

def _quantile_bias(self, raw: Array, q: Array, sweeps: int = 4) -> Array:
    """Top-1 specialization of Kimi K3-style score-quantile balancing."""
    n, E = raw.shape
    bias = self.last_bias.copy()
    for _ in range(max(int(sweeps), 1)):
        adjusted = raw + bias[None, :]
        for e in range(E):
            other = adjusted.copy()
            other[:, e] = -np.inf
            cutoff = np.max(other, axis=1)
            margins = cutoff - raw[:, e]
            target = float(np.clip(q[e], 0.0, 1.0))

```

```

        if target <= 0.0:
            bias[e] = float(np.min(margins) - 2.0)
        elif target >= 1.0:
            bias[e] = float(np.max(margins) + 2.0)
        else:
            bias[e] = float(np.quantile(margins, target))
    bias -= float(np.median(bias))
    bias = np.clip(bias, -2.5, 2.5)
    return bias

def assign(
    self,
    Phi: Array,
    rng: np.random.Generator,
    progress: float,
    use_quantile: bool = True,
    use_memory: bool = True,
    use_specialization: bool = False,
) -> Tuple[Array, Array, Array, Array]:
    Phi = np.asarray(Phi, dtype=float)
    raw = self._posterior_scores(Phi, rng, uncertainty=0.08 if progress < 0.55 else 0.045)
    if use_memory:
        raw += 0.11 * self._memory_affinity(Phi)
    if use_specialization and np.count_nonzero(self.prototype_mass >= 2.0) >= 2:
        U = _safe_unit_rows(self.prototypes + 1e-12)
        overlap = np.maximum(U @ U.T - np.eye(self.n_experts), 0.0)
        redundancy = overlap.max(axis=1)
        raw -= 0.06 * redundancy[None, :]
    # Shared/null route receives a small reliability prior, but routed experts
    # can readily overtake it when their conditional evidence is stronger.
    raw[:, 0] += 0.11 + 0.08 * progress

    # MuonClip-inspired robust score clipping: only extreme router logits are
    # capped; ordinary ranking geometry is unchanged.
    med = np.median(raw, axis=1, keepdims=True)
    mad = np.median(np.abs(raw - med), axis=1, keepdims=True) + 1e-9
    lo, hi = med - 5.0 * mad, med + 5.0 * mad
    clipped = np.clip(raw, lo, hi)
    self.score_clip_events += int(np.count_nonzero(np.abs(clipped - raw) > 1e-12))
    raw = clipped

    q = self._target_distribution(len(Phi), progress)
    quota = self._integer_quotas(q, len(Phi))
    if use_quantile:
        bias = self._quantile_bias(raw, q)
        adjusted = raw + bias[None, :]
        # Exact capacity assignment after quantile correction. This prevents a
        # few near-tied contexts from violating the global FE allocation plan.
        slots = np.repeat(np.arange(self.n_experts), quota)
        if len(slots) != len(Phi):
            raise RuntimeError("quantile quota does not match population size")
        rows, cols = linear_sum_assignment(-adjusted[:, slots])
        assignment = np.empty(len(Phi), dtype=int)
        assignment[rows] = slots[cols]
    else:
        bias = np.zeros(self.n_experts, dtype=float)
        adjusted = raw
        assignment = np.argmax(adjusted, axis=1).astype(int)

```

```

# Confidence controls routed residual amplitude, not FE count.
sorted_scores = np.sort(adjusted, axis=1)
margin = sorted_scores[:, -1] - sorted_scores[:, -2] if self.n_experts > 1 else np.ones(len(Phi))
gate = 1.0 / (1.0 + np.exp(-np.clip(margin, -8.0, 8.0)))
gate = np.clip(0.18 + 0.42 * gate, 0.18, 0.60)
gate[assignment == 0] = 0.0

load = np.bincount(assignment, minlength=self.n_experts).astype(float) / max(len(Phi), 1)
self.load_ema = 0.92 * self.load_ema + 0.08 * load
self.last_target = q
self.last_bias = bias
self.last_load = load
return assignment, gate, raw, bias

def update(
    self,
    phi: Array,
    expert: int,
    reward: float,
    baseline: float,
    dense_default: bool = False,
) -> float:
    phi = np.asarray(phi, dtype=float)
    e = int(expert)
    r = float(np.clip(reward, 0.0, 1.0))
    advantage = float(np.clip(r - baseline, -1.0, 1.0))
    target = float(np.clip(0.5 + advantage / 0.24, 0.0, 1.0))
    decay = 0.994
    eye = np.eye(self.n_features)
    self.A[e] = decay * self.A[e] + np.outer(phi, phi) + (1.0 - decay) * self.ridge * eye
    self.b[e] = decay * self.b[e] + phi * target
    self.calls[e] += 1
    self.reward_ema[e] = 0.94 * self.reward_ema[e] + 0.06 * r
    self.advantage_ema[e] = 0.94 * self.advantage_ema[e] + 0.06 * advantage
    self.default_reward[e] = 0.97 * self.default_reward[e] + 0.03 * r
    if r >= 0.42 or advantage >= 0.08:
        if self.prototype_mass[e] == 0:
            self.prototype[e] = phi
        else:
            self.prototype[e] = 0.92 * self.prototype[e] + 0.08 * phi
            self.prototype_mass[e] += 1.0

# Default-MoE-inspired weak dense feedback: unselected experts receive
# only their EMA default target. The small weight regularizes the router
# without pretending these are exact counterfactual evaluations.
if dense_default:
    w = 0.018
    for j in range(self.n_experts):
        if j == e:
            continue
        pseudo = float(np.clip(self.default_reward[j], 0.0, 1.0))
        self.A[j] = decay * self.A[j] + w * np.outer(phi, phi) + (1.0 - decay) * self.ridge * eye
        self.b[j] = decay * self.b[j] + w * phi * pseudo
    return advantage

def _lq_context_matrix(

```

```

progress: float,
stagnation: Array,
densities: Array,
med_density: float,
distress: float,
entropy: float,
assigned_scalar: Array,
F: Array,
W: Array,
ideal: Array,
nadir: Array,
role_reliability: float,
archive_fill: float,
) -> Array:
    n = len(F)
    stag = np.clip(np.loglp(np.asarray(stagnation, dtype=float)) / np.log(12.0), 0.0, 1.0)
    density = np.asarray(densities, dtype=float)
    density_ratio = np.clip(density / (med_density + 1e-9), 0.0, 4.0) / 4.0 if med_density > 0 else np.zeros(n)
    scalar_rel = np.clip(np.asarray(assigned_scalar, dtype=float) / (np.median(assigned_scalar) + 1e-12), 0.0, 4.0) / 4.0
    Y = np.maximum((np.asarray(F) - ideal) / (nadir - ideal + 1e-12), 0.0)
    Yn = _safe_unit_rows(Y + 1e-12)
    Wn = _safe_unit_rows(W + 1e-12)
    alignment = np.clip(np.sum(Yn * Wn, axis=1), 0.0, 1.0)
    Phi = np.column_stack([
        np.ones(n),
        np.full(n, progress),
        np.full(n, progress * progress),
        stag,
        density_ratio,
        np.full(n, distress),
        np.full(n, entropy),
        scalar_rel,
        alignment,
        np.full(n, float(np.clip(role_reliability, 0.0, 1.0))),
        np.full(n, float(np.clip(archive_fill, 0.0, 1.0))),
        stag * scalar_rel,
    ])
    return Phi.astype(float)

def _choose_distinct(rng: np.random.Generator, pool: Array, exclude: Sequence[int], n: int) -> Array:
    pool = np.asarray(pool, dtype=int)
    mask = ~np.isin(pool, np.asarray(list(exclude), dtype=int))
    cand = pool[mask]
    if len(cand) < n:
        cand = pool
    return np.asarray(rng.choice(cand, size=n, replace=len(cand) < n), dtype=int)

def _micro_residual(
    micro: int,
    i: int,
    X: Array,
    F: Array,
    W: Array,
    B: Array,
    archive: core.Archive,
    residual_steps: Sequence[Sequence[Array]],

```



```

sparse_target_x: Optional[Array],
progress: float,
rng: np.random.Generator,
problem: core.Problem,
ideal: Array,
nadir: Array,
) -> Array:
    """Fine-grained residual bank; expert 0 is the exact shared-only route."""
    micro = int(micro)
    d = problem.n_var
    span = problem.xu - problem.xl
    if micro == 0:
        return np.zeros(d, dtype=float)

    neigh = np.asarray(B[int(i)], dtype=int)
    local = neigh if len(neigh) >= 3 else np.arange(len(X))
    mu = 0.015 + 0.08 * (1.0 - progress) ** 2
    scores = core._mosaic_scalar(
        F[local], np.repeat(W[int(i)][None, :], len(local), axis=0),
        ideal, nadir, mu, theta=0.18, blend=0.0,
    )

    elite = local[np.argsort(scores, kind="mergesort")[: max(2, int(math.ceil(0.18 * len(local))))]]
    pbest = int(rng.choice(elite))

    if micro == 1: # pbest micro
        return (0.030 + 0.020 * (1.0 - progress)) * (X[pbest] - X[i])
    if micro == 2: # pbest macro
        return (0.080 + 0.040 * (1.0 - progress)) * (X[pbest] - X[i])
    if micro == 3: # local differential micro
        r1, r2 = _choose_distinct(rng, local, [i], 2)
        return 0.050 * (X[r1] - X[r2])
    if micro == 4: # global differential
        r1, r2 = _choose_distinct(rng, np.arange(len(X)), [i], 2)
        return (0.070 + 0.035 * (1.0 - progress)) * (X[r1] - X[r2])
    if micro == 5: # local archive tangent
        if len(archive.F) >= max(5, problem.n_obj + 1):
            AF = (archive.F - ideal) / (nadir - ideal + 1e-12)
            fi = (F[i] - ideal) / (nadir - ideal + 1e-12)
            idx = np.argsort(np.linalg.norm(AF - fi, axis=1), kind="mergesort")[: min(14, len(AF))]
            Z = (archive.X[idx] - np.mean(archive.X[idx], axis=0)) / (span + 1e-12)
            try:
                _, s, vt = np.linalg.svd(Z, full_matrices=False)
                k = max(1, min(problem.n_obj - 1, len(s)))
                return span * ((rng.normal(size=k) * (s[:k] + 0.01)) @ vt[:k]) * 0.050
            except np.linalg.LinAlgError:
                pass
            return np.zeros(d)
    if micro == 6: # reference-gap transport
        if sparse_target_x is not None:
            return (0.055 + 0.030 * (1.0 - progress)) * (sparse_target_x - X[i])
        return np.zeros(d)
    if micro == 7: # sparse Gaussian
        mask = rng.random(d) < min(0.20, 3.0 / max(d, 1))
        if not mask.any():
            mask[int(rng.integers(d))] = True
        out = np.zeros(d)
        out[mask] = rng.normal(size=int(mask.sum())) * span[mask] * (0.012 + 0.018 * (1.0 - progress))
        return out

```

```

if micro == 8: # sparse Cauchy
    mask = rng.random(d) < min(0.16, 2.5 / max(d, 1))
    if not mask.any():
        mask[int(rng.integers(d))] = True
    out = np.zeros(d)
    out[mask] = np.clip(rng.standard_cauchy(int(mask.sum())) , -4.0, 4.0) * span[mask] * (0.010 + 0.022 * (1.0 - progress))
    return out
if micro == 9: # exact-verified residual replay
    bank = [s for seq in residual_steps for s in seq[-16:]]
    if bank:
        return 0.35 * np.asarray(bank[int(rng.integers(len(bank)))], dtype=float)
    return np.zeros(d)
if micro == 10: # SBX residual relative to parent
    mate = int(rng.choice(local))
    sbx = core.sbx_one(X[i], X[mate], problem.xl, problem.xu, rng, eta=25.0)
    return 0.12 * (sbx - X[i])
if micro == 11: # low-variance / orthogonal escape
    Z = (X - np.mean(X, axis=0)) / (span + 1e-12)
    try:
        _, _, vt = np.linalg.svd(Z, full_matrices=False)
        q = vt[-min(2, len(vt)):]
        direction = rng.normal(size=len(q)) @ q
        return span * direction * (0.010 + 0.018 * (1.0 - progress))
    except np.linalg.LinAlgError:
        return np.zeros(d)
raise ValueError(f"unknown micro expert {micro}")

def run_mosaic_lqmoe(
    problem: core.Problem,
    seed: int,
    pop_size: int = 100,
    max_evals: int = 10100,
    T: int = 20,
    nr: int = 3,
    trace_every: int = 5,
    ablation: Optional[str] = None,
    recovery_fraction: float = 0.22,
    recovery_cooldown: int = 10,
    max_recoveries: int = 3,
    collapse_ratio: float = 0.35,
    collapse_start: float = 0.70,
    collapse_patience: int = 4,
) -> core.RunResult:
    """MOSAIC-LQMoE with a frozen R3 shared backbone and routed micro-experts.

    Flags (join with '+'):
    no_memory      disable Exact Coverage Memory entirely
    no_recovery     keep memory/ART but disable collapse intervention
    no_o5          disable Archive-Reuse Transport expert entirely
    no_routine_o5  disable routine ART; allow it only through soft recovery
    no_role_factorization use ordinary archive-DE instead of role-separated crossover
    soft_recovery  convert collapse intervention into an exact-verified ART queue
    hard_recovery  directly inject exact memory points (diagnostic comparator)
    secant         enable reliable Secant-Repulsive Transport in operator 2
    no_repulsion   with secant, remove objective-space RBF repulsion
    discount_router decay posterior evidence toward its prior each generation
    fixed_router   uniformly sample standard experts 0..4

```

```

force_coverage frozen v4 periodic NSGA-III archive repair comparator
coordinate_tunnel enable role-gated coordinate basin tunneling (operator 6)
moe_residual enable shared-backbone + fine-grained residual MoE
no_qb disable loss-free quantile balancing in residual router
no_memory_route disable successful-context expert memory
no_latent disable low-rank latent routed residuals
no_energy_clip disable routed proposal-energy clipping
dense_default weak EMA default feedback to non-selected micro-experts
specialization penalize overlapping expert context prototypes
coordinate_qb quantile-balanced dynamic-k routing over 06 variable experts
coordinate_dynamic_k preserve R3 pilots, then evidence-scale 06 expert batch size
coordinate_shared_qb preserve R3 shared pilots, then quantile-route residual coordinate experts
coordinate_evidence_qb preserve R3 except under low-occupancy evidence, then quantile-route residual experts
coordinate_geometry_qb additionally requires non-planar PF evidence before quantile residual routing
coordinate_consensus_qb requires two consistent exact pilots before geometry-aware quantile routing
"""

flags = set() if ablation is None else {s for s in str(ablation).split("+") if s}
rng, X, F = core.initialize(problem, pop_size, seed)
# Independent MoE stream: adding a router must not perturb the frozen R3
# random trajectory when the shared/null route is selected.
moe_rng = np.random.default_rng(int(seed) + 104729)
W = core.dirichlet_reference_points(pop_size, problem.n_obj, seed=601)
dW = ((W[:, None, :] - W[None, :, :]) ** 2).sum(axis=2)
B = np.argsort(dW, axis=1)[: , : min(T, pop_size)]
init_assoc, _ = core.associate_to_reference(F, W)
init_occ = float(np.count_nonzero(np.bincount(init_assoc, minlength=pop_size)) / pop_size)
alignment_latch = bool(problem.n_obj > 2 and init_occ < 0.25)

ideal = F.min(axis=0)
nadir = np.maximum(F.max(axis=0), ideal + 1e-9)
archive = core.Archive(max_size=5 * pop_size)
archive.update(X, F)
memory = ExactCoverageMemory(capacity=8 * pop_size, per_cell=3)
if "no_memory" not in flags:
    memory.add(X, F, 0, W, ideal, nadir)

# Preserve the frozen five-expert router exactly. ART (05) is a
# conservative challenger with its own posterior; adding an expert must not
# perturb the core router merely by changing the dimension of a Beta draw.
bandit = DiscountedPortfolio(n_states=12, n_ops=5)
o5_a = np.ones(12, dtype=float)
o5_b = np.full(12, 3.0, dtype=float)
o6_a = np.ones(12, dtype=float)
o6_b = np.full(12, 2.5, dtype=float)
stagnation = np.zeros(pop_size, dtype=int)
successful_steps: List[List[Array]] = [[] for _ in range(7)]
evals, gen = pop_size, 0
trace: List[Dict[str, float]] = []
op_counts = np.zeros(7, dtype=int)
op_rewards = np.zeros(7, dtype=float)
secant_calls = secant_used = 0
secant_rel_sum = 0.0
role_probability: Optional[Array] = None
role_reliability = 0.0
role_diagnostics: Dict[str, float] = {}
recovery_events: List[Dict[str, object]] = []
rescue_queue: List[int] = []
last_recovery = -10**9

```

```

collapse_streak = 0
max_occupancy = max(init_occ, 1.0 / pop_size)
occupancy_history: List[float] = []
archive_history: List[int] = []
scalar_history: List[float] = []
probe_index = np.zeros(problem.n_var, dtype=int)
probe_trials = np.zeros(problem.n_var, dtype=int)
probe_success = np.zeros(problem.n_var, dtype=int)
learned_values = np.full(problem.n_var, np.nan, dtype=float)
coordinate_events: List[Dict[str, object]] = []
coordinate_candidates: Optional[Array] = None
coordinate_pilot_rewards: List[float] = []
coordinate_expand_threshold = 0.35
coordinate_pilot_budget_current = 2
coordinate_batch_history: List[Dict[str, object]] = []

# DeepSeek/Kimi-inspired second routing layer. The legacy R3 proposal is the
# always-on shared path; one of 11 fine-grained residual experts, or the exact
# null residual, is selected without additional objective evaluations.
n_micro = 12
micro_router = LatentQuantileRouter(n_experts=n_micro, n_features=12)
micro_counts = np.zeros(n_micro, dtype=int)
micro_rewards = np.zeros(n_micro, dtype=float)
micro_advantages = np.zeros(n_micro, dtype=float)
micro_clip_scales: List[float] = []
residual_successful_steps: List[List[Array]] = [[] for _ in range(n_micro)]
shared_reward_ema = np.full(12, 0.30, dtype=float)
micro_assignment_history: List[List[int]] = []
micro_target_history: List[List[float]] = []
micro_load_history: List[List[float]] = []
radial_dispersion_history: List[float] = []
geometry_score_history: List[float] = []
radial_failure_streak = 0

while evals < max_evals:
    progress = evals / max_evals
    densities = np.zeros(pop_size)
    for j in range(pop_size):
        densities[j] = core._local_density(archive.F, F[j])[0] if len(archive.F) else 0.0
    med_density = float(np.median(densities))

    # Frozen v4 search association is preserved for standard experts and
    # reward credit. A separate shared longitudinal frame is used only by
    # the collapse detector and exact-memory reconstruction.
    assoc_search, _ = core.associate_to_reference(F, W)
    search_counts = np.bincount(assoc_search, minlength=pop_size)
    sparse_cells = np.flatnonzero(search_counts == search_counts.min())
    assoc_detect, _ = _associate_fixed(F, W, ideal, nadir)
    occupancy, entropy, detect_counts = _occupancy_entropy(assoc_detect, pop_size)
    max_occupancy = max(max_occupancy, occupancy)
    threshold = max(0.08, collapse_ratio * max_occupancy)
    distress = float(np.clip((threshold - occupancy) / (threshold + 1e-12), 0.0, 1.0))

    # Predict ART's exact anchor contribution once per generation. This is
    # the optimization analogue of contribution regression in D2DMoE: the
    # router asks whether its specialist has a measurable job to do before
    # spending samples on it.
    o5_target_cell: Optional[int] = None

```

```

o5_advantage_gen = 0.0
if (
    "no_o5" not in flags
    and "no_routine_o5" not in flags
    and "no_memory" not in flags
    and len(memory.F) > pop_size
    and progress > 0.22
    and len(sparse_cells) > 0
):
    advantages = np.asarray([
        memory.best_anchor_advantage(int(c), F, W, ideal, nadir)
        for c in sparse_cells
    ])
    bp = int(np.argmax(advantages))
    o5_target_cell = int(sparse_cells[bp])
    o5_advantage_gen = float(advantages[bp])

if (
    "no_o5" not in flags
    and "no_role_factorization" not in flags
    and "no_memory" not in flags
    and len(memory.F) >= max(40, 2 * problem.n_var)
    # RCBT needs role information earlier than coverage recovery.
    # Without the coordinate_tunnel ablation, preserve R2 timing.
    and progress >= (0.28 if "coordinate_tunnel" in flags else min(0.60, max(collapse_start - 0.10, 0.0)))
    and (gen % 4 == 0 or role_probability is None)
):
    role_probability, role_reliability, role_diagnostics = _infer_variable_roles(
        memory, problem, ideal, nadir, W
    )

# Direction-conditioned convergence diagnostic. Unlike occupancy
# distress, this detects a healthy spread whose assigned STCH values have
# stopped improving. It controls a separate specialist and never changes
# the frozen five-expert posterior.
mu_diag = 0.008 + 0.10 * (1.0 - progress) ** 2
theta_diag = 0.10 + 0.22 * progress
assigned_scalar = core.mosaic_scalar(F, W, ideal, nadir, mu_diag, theta=theta_diag, blend=0.0)
scalar_level = float(np.median(assigned_scalar))
scalar_history.append(scalar_level)
if len(scalar_history) >= 7:
    old_level = float(np.median(scalar_history[-7:-4]))
    new_level = float(np.median(scalar_history[-3:]))
    scalar_progress = (old_level - new_level) / (abs(old_level) + 1e-12)
else:
    scalar_progress = 1.0

# Scale-free local PF geometry diagnostic. A positive value means an
# L2/spherical shell explains the current front better than an L1/simplex
# shell; a negative value favors a flat simplex-like front. This is only
# computed from already evaluated points.
Y_geom = np.maximum((F - ideal) / (nadir - ideal + 1e-12), 0.0)
l1_geom = np.sum(Y_geom, axis=1)
l2_geom = np.linalg.norm(Y_geom, axis=1)
cv1_geom = float(np.std(l1_geom) / (np.mean(l1_geom) + 1e-12))
cv2_geom = float(np.std(l2_geom) / (np.mean(l2_geom) + 1e-12))
geometry_score = float(cv1_geom - cv2_geom)
geometry_score_history.append(geometry_score)
coordinate_plans: List[Tuple[int, int]] = []

```

```

if (
    "coordinate_tunnel" in flags
    and role_probability is not None
    and role_reliability >= 0.28
    and progress >= 0.40
    and gen % 2 == 0
    and not rescue_queue
):
    # Freeze one role-derived candidate set. Recomputing the role mask
    # could otherwise restart pilots and silently spend more than promised.
    if coordinate_candidates is None:
        conv_all = np.flatnonzero(role_probability <= 0.50)
        if len(conv_all) >= max(2, problem.n_var // 4):
            order_conv = conv_all[np.argsort(role_probability[conv_all], kind="mergesort")]
            coordinate_candidates = order_conv[: min(12, len(order_conv))].copy()

    conv = np.empty(0, dtype=int) if coordinate_candidates is None else coordinate_candidates
    if len(conv):
        target_score = assigned_scalar / (np.median(assigned_scalar) + 1e-12)
        target_order = np.argsort(-(stagnation + 3.0 * target_score), kind="mergesort")
        if "coordinate_evidence_qb" in flags or "coordinate_geometry_qb" in flags or "coordinate_consensus_qb" in
            ↪ flags:
            # Evidence-triggered expert expansion. The two exact R3
            # shared pilots always keep their original order. When the
            # exact-memory reference occupancy is healthy, execution
            # falls back bit-for-bit to the frozen R3 coordinate route.
            # Only a low-occupancy + strong-pilot state unlocks the
            # quantile-balanced residual experts.
            untested = conv[probe_trials[conv] == 0]
            total_trials = int(probe_trials[conv].sum())
            pilot_budget = min(2, len(conv))
            coordinate_pilot_budget_current = pilot_budget
            pilot_due = total_trials < pilot_budget
            pilot_ready = (
                total_trials >= pilot_budget
                and len(coordinate_pilot_rewards) >= pilot_budget
            )
            strong_rewards = (
                np.asarray(coordinate_pilot_rewards[:pilot_budget], dtype=float)
                if pilot_ready else np.empty(0, dtype=float)
            )
            occ_ratio = float(role_diagnostics.get("occupied_cells", pop_size)) / max(pop_size, 1)
            structural_evidence = occ_ratio < 0.80
            if "coordinate_geometry_qb" in flags or "coordinate_consensus_qb" in flags:
                # Kimi-style expert expansion is reserved for a curved or
                # irregular shell. On a clearly planar/simplex front, the
                # frozen role order is more reliable than memory re-ranking.
                structural_evidence = structural_evidence and geometry_score > -0.02
            # The frozen shared path uses its original max-pilot gate.
            # Routed residual experts may use a stricter consensus gate,
            # but a failed residual gate must never disable the shared
            # expert's own continuation (DeepSeek Keep-Routing analogue).
            r3_expand_evidence = bool(
                pilot_ready
                and len(untested) > 0
                and np.max(strong_rewards) >= coordinate_expand_threshold
            )
            if "coordinate_consensus_qb" in flags:

```

```

        reward_evidence = bool(
            pilot_ready
            and len(untested) > 0
            and np.min(strong_rewards) >= coordinate_expand_threshold
        )
    else:
        reward_evidence = r3_expand_evidence
    strong_evidence = reward_evidence
    if pilot_due and len(untested):
        coordinate_plans.append((int(target_order[0]), int(untested[0])))
        k_now = 1
        coord_entropy = 0.0
        mode_now = "shared_pilot"
    elif structural_evidence and strong_evidence:
        rs = _coordinate_expert_scores(
            untested, role_probability, memory, problem, ideal, nadir,
            probe_trials, probe_success,
        )
        rs = rs + 1e-8 * moe_rng.normal(size=len(rs))
        ranked, k_entropy, coord_entropy = _dynamic_coordinate_batch(
            untested, rs, probe_trials, max_batch=min(3, len(untested))
        )
        n_strong = int(np.count_nonzero(strong_rewards >= coordinate_expand_threshold))
        evidence_cap = 1 + int(n_strong >= 2 or np.max(strong_rewards) >= 0.60)
        k_now = int(min(len(ranked), k_entropy, evidence_cap, 2))
        for qidx, jvar in enumerate(ranked[:k_now]):
            target = int(target_order[qidx % len(target_order)])
            coordinate_plans.append((target, int(jvar)))
        mode_now = "quantile_residual"
    elif r3_expand_evidence:
        # Residual gate closed: preserve the original R3 route and
        # spend exactly one coordinate FE on the first unttested
        # role-ordered variable.
        coordinate_plans.append((int(target_order[0]), int(unttested[0])))
        k_now = 1
        coord_entropy = 0.0
        mode_now = "frozen_r3_fallback"
    else:
        k_now = 0
        coord_entropy = 0.0
        mode_now = "null_route"
    coordinate_batch_history.append({
        "gen": int(gen), "evals": int(evals),
        "dynamic_k": int(k_now), "entropy": float(coord_entropy),
        "scheduled": [int(j) for _, j in coordinate_plans],
        "pilot_budget": int(coordinate_pilot_budget_current),
        "mode": mode_now, "keep_route": True,
        "occupancy_ratio": float(occ_ratio),
        "structural_evidence": bool(structural_evidence),
        "geometry_score": float(geometry_score),
        "consensus_gate": bool("coordinate_consensus_qb" in flags),
    })
elif "coordinate_shared_qb" in flags:
    # DeepSeek shared-expert isolation + Kimi quantile balancing:
    # the frozen R3 two-pilot path is always executed first, in
    # exactly the same variable order. Routed coordinate experts
    # are allowed only as a residual after exact pilot evidence.
    unttested = conv[probe_trials[conv] == 0]

```

```

total_trials = int(probe_trials[conv].sum())
pilot_budget = min(2, len(conv))
coordinate_pilot_budget_current = pilot_budget
pilot_due = total_trials < pilot_budget
pilot_ready = (
    total_trials >= pilot_budget
    and len(coordinate_pilot_rewards) >= pilot_budget
)
strong_rewards = (
    np.asarray(coordinate_pilot_rewards[:pilot_budget], dtype=float)
    if pilot_ready else np.empty(0, dtype=float)
)
residual_gate = bool(
    pilot_ready
    and len(untested) > 0
    and np.max(strong_rewards) >= coordinate_expand_threshold
)
if pilot_due and len(untested):
    # One exact shared pilot at a time; this preserves the
    # R3 route and its random/evaluation trajectory.
    coordinate_plans.append((int(target_order[0]), int(untested[0])))
    k_now = 1
    coord_entropy = 0.0
elif residual_gate:
    rs = _coordinate_expert_scores(
        untested, role_probability, memory, problem, ideal, nadir,
        probe_trials, probe_success,
    )
    rs = rs + 1e-8 * moe_rng.normal(size=len(rs))
    ranked, k_entropy, coord_entropy = _dynamic_coordinate_batch(
        untested, rs, probe_trials, max_batch=min(3, len(untested))
    )
    # Evidence-triggered capacity. One strong pilot buys one
    # routed expert; two strong pilots can buy a second. A
    # very high single reward may also buy the second. This
    # is a strict FE budget, not free parallel computation.
    n_strong = int(np.count_nonzero(strong_rewards >= coordinate_expand_threshold))
    evidence_cap = 1 + int(n_strong >= 2 or np.max(strong_rewards) >= 0.60)
    k_now = int(min(len(ranked), k_entropy, evidence_cap, 2))
    for qidx, jvar in enumerate(ranked[:k_now]):
        target = int(target_order[qidx % len(target_order)])
        coordinate_plans.append((target, int(jvar)))
else:
    k_now = 0
    coord_entropy = 0.0
coordinate_batch_history.append({
    "gen": int(gen), "evals": int(eval),
    "dynamic_k": int(k_now), "entropy": float(coord_entropy),
    "scheduled": [int(j) for _, j in coordinate_plans],
    "pilot_budget": int(coordinate_pilot_budget_current),
    "mode": "shared_pilot_then_quantile_residual",
    "keep_route": True,
})
elif "coordinate_dynamic_k" in flags:
    # Preserve the exact frozen R3 first-two pilot order. Only
    # after those exact rewards prove usefulness do we expand the
    # per-generation coordinate-expert budget. This mirrors
    # Kimi-style dynamic-k while protecting the shared path.

```



```

untested = conv[probe_trials[conv] == 0]
total_trials = int(probe_trials[conv].sum())
pilot_budget = min(2, len(conv))
coordinate_pilot_budget_current = pilot_budget
pilot_due = total_trials < pilot_budget
expand_due = (
    total_trials >= pilot_budget
    and len(coordinate_pilot_rewards) >= pilot_budget
    and max(coordinate_pilot_rewards[:pilot_budget]) >= coordinate_expand_threshold
    and len(untested) > 0
)
if pilot_due and len(untested):
    coordinate_plans.append((int(target_order[0]), int(untested[0])))
    k_now = 1
elif expand_due:
    strong = int(np.count_nonzero(
        np.asarray(coordinate_pilot_rewards[:pilot_budget]) >= coordinate_expand_threshold
    ))
    k_now = min(len(untested), 1 + strong)
    # Role ordering is the frozen R3 ordering; quantile-based
    # capacity changes how many, not which, experts are trusted.
    for qidx, jvar in enumerate(untested[:k_now]):
        target = int(target_order[qidx % len(target_order)])
        coordinate_plans.append((target, int(jvar)))
else:
    k_now = 0
coordinate_batch_history.append({
    "gen": int(gen), "evals": int(evals),
    "dynamic_k": int(k_now), "entropy": None,
    "scheduled": [int(j) for _, j in coordinate_plans],
    "pilot_budget": int(coordinate_pilot_budget_current),
    "mode": "shared_pilot_then_dynamic_k",
})
elif "coordinate_qb" in flags:
    cs = _coordinate_expert_scores(
        conv, role_probability, memory, problem, ideal, nadir,
        probe_trials, probe_success,
    )
    # Independent, tiny jitter only resolves exact score ties.
    cs = cs + 1e-8 * moe_rng.normal(size=len(cs))
    chosen_vars, dynamic_k, coord_entropy = _dynamic_coordinate_batch(
        conv, cs, probe_trials, max_batch=min(4, len(conv))
    )
    coordinate_pilot_budget_current = max(2, dynamic_k)
    total_trials = int(probe_trials[conv].sum())
    pilot_due = total_trials < coordinate_pilot_budget_current
    expand_due = (
        total_trials >= coordinate_pilot_budget_current
        and len(coordinate_pilot_rewards) >= coordinate_pilot_budget_current
        and max(coordinate_pilot_rewards[:coordinate_pilot_budget_current]) >= coordinate_expand_threshold
    )
    if pilot_due or expand_due:
        if not pilot_due:
            # After a successful pilot phase, continue one expert
            # at a time, ranked by exact posterior contribution.
            remaining = conv[probe_trials[conv] == 0]
            if len(remaining):
                rs = _coordinate_expert_scores(

```

```

        remaining, role_probability, memory, problem,
        ideal, nadir, probe_trials, probe_success,
    )
    chosen_vars = remaining[np.argsort(-rs, kind="mergesort")[:1]]
    else:
        chosen_vars = np.empty(0, dtype=int)
    else:
        need = max(0, coordinate_pilot_budget_current - total_trials)
        chosen_vars = chosen_vars[:need]
        for qidx, jvar in enumerate(chosen_vars):
            target = int(target_order[qidx % len(target_order)])
            coordinate_plans.append((target, int(jvar)))
        coordinate_batch_history.append({
            "gen": int(gen), "evals": int(evals),
            "dynamic_k": int(dynamic_k), "entropy": float(coord_entropy),
            "scheduled": [int(j) for _, j in coordinate_plans],
            "pilot_budget": int(coordinate_pilot_budget_current),
        })
    else:
        untested = conv[probe_trials[conv] == 0]
        total_trials = int(probe_trials[conv].sum())
        pilot_budget = min(2, len(conv))
        coordinate_pilot_budget_current = pilot_budget
        pilot_due = total_trials < pilot_budget
        expand_due = (
            total_trials >= pilot_budget
            and len(coordinate_pilot_rewards) >= pilot_budget
            and max(coordinate_pilot_rewards[:pilot_budget]) >= coordinate_expand_threshold
            and len(untested) > 0
        )
        if (pilot_due or expand_due) and len(untested):
            coordinate_plans.append((int(target_order[0]), int(untested[0])))

if problem.n_obj <= 2:
    scalar_blend = 0.0
elif alignment_latch:
    scalar_blend = 1.0
else:
    scalar_blend = 0.0

# Scale-free radial-dispersion diagnostic. A large ratio means a few
# directions have reached a much better radial basin while most of the
# population remains trapped far away -- the exact failure RCBT and the
# routed residual bank are intended to solve.
# Use a shift- and scale-equivariant radial mass ratio rather than the
# current min-max normalized radius. The latter can hide severe multimodal
# distance-variable traps by normalizing the trapped front to [0,1].
span_obj = np.maximum(nadir - ideal, 1e-12)
radial = np.sum(np.maximum(F - ideal, 0.0) + 0.01 * span_obj, axis=1)
radial_dispersion = float(np.median(radial) / (np.quantile(radial, 0.10) + 1e-12))
radial_dispersion_history.append(radial_dispersion)
radial_failure_streak = radial_failure_streak + 1 if radial_dispersion >= 3.0 else 0

micro_assign = np.zeros(pop_size, dtype=int)
micro_gate = np.zeros(pop_size, dtype=float)
micro_context = np.zeros((pop_size, 12), dtype=float)
latent_basis = np.empty((0, problem.n_var), dtype=float)
if "moe_residual" in flags:

```

```

micro_context = _lq_context_matrix(
    progress, stagnation, densities, med_density, distress, entropy,
    assigned_scalar, F, W, ideal, nadir, role_reliability,
    len(archive.F) / max(archive.max_size, 1),
)
# A routed residual is a challenger, never a compulsory replacement
# for the reliable shared path. Quantile balancing is performed only
# across contexts for which a diagnosed radial failure exists; this
# prevents capacity targets from spending residual routes on easy rows.
eligible = (progress >= 0.25) & (
    (radial_failure_streak >= 3) & (stagnation >= 3)
)
idx_eligible = np.flatnonzero(eligible)
if len(idx_eligible):
    ae, ge, _, _ = micro_router.assign(
        micro_context[idx_eligible], moe_rng, progress,
        use_quantile="no_qb" not in flags,
        use_memory="no_memory_route" not in flags,
        use_specialization="specialization" in flags,
    )
    micro_assign[idx_eligible] = ae
    micro_gate[idx_eligible] = ge
if "no_latent" not in flags:
    latent_basis = _build_latent_basis(
        X, list(successful_steps[:5]) + list(residual_successful_steps),
        problem.xu - problem.xl, problem.n_obj, max_rank=10,
    )
micro_assignment_history.append(np.bincount(micro_assign, minlength=n_micro).astype(int).tolist())
micro_target_history.append(micro_router.last_target.tolist())
micro_load_history.append(micro_router.last_load.tolist())

for i in rng.permutation(pop_size):
    if evals >= max_evals:
        break
    state = core._mosaic_state(progress, int(stagnation[i]), float(densities[i]), med_density)
    forced_rescue = bool("soft_recovery" in flags and rescue_queue and "no_o5" not in flags)
    forced_target = int(rescue_queue.pop(0)) if forced_rescue else None
    forced_coordinate = bool(coordinate_plans and not forced_rescue)
    coordinate_variable: Optional[int] = None
    coordinate_value: Optional[float] = None
    if forced_rescue:
        op = 5
    elif forced_coordinate:
        op = 6
        forced_target, coordinate_variable = coordinate_plans.pop(0)
    elif "fixed_router" in flags:
        op = int(rng.integers(5))
    else:
        op = bandit.choose(state, rng, epsilon=0.10 if progress < 0.5 else 0.06)

    # D2DMoE-inspired contribution routing: ART is not selected merely
    # because its load is low. Its best exact memory anchor must have a
    # measurable scalar advantage for a currently sparse cell, and its
    # conservative posterior must beat the selected core expert.
    target_cell = int(forced_target) if forced_target is not None else int(i)
    if (
        not forced_rescue
        and o5_target_cell is not None

```

```

    and (distress > 0.08 or stagnation[i] >= 8)
    and o5_advantage_gen > 0.025
):
    target_cell = int(o5_target_cell)
    q5 = float(rng.beta(o5_a[state], o5_b[state]))
    core_mean = float(bandit.a[state, op] / (bandit.a[state, op] + bandit.b[state, op]))
    predicted = float(np.clip(o5_advantage_gen / 0.12, 0.0, 1.0))
    o5_score = 0.55 * q5 + 0.45 * predicted
    if o5_score > core_mean + 0.06:
        op = 5

# A specialist routed to reference cell c must also be verified in
# c's neighborhood. Earlier prototypes generated for c but compared
# only against the arbitrary loop index i, which silently discarded
# otherwise useful rescue children.
search_i = int(target_cell) if op in (5, 6) else int(i)
parent = X[search_i].copy()
parentF = F[search_i].copy()
if op == 2 and len(archive.F) >= 3 and len(sparse_cells):
    # Preserve v4's stochastic sparse target for the original 02.
    target_cell = int(rng.choice(sparse_cells))
sparse_target_x: Optional[Array] = None
if op == 2 and len(archive.F) >= 3 and len(sparse_cells):
    az, an = archive.F.min(axis=0), archive.F.max(axis=0)
    AY = np.maximum((archive.F - az) / (an - az + 1e-12), 0.0)
    AYn = AY / (np.linalg.norm(AY, axis=1, keepdims=True) + 1e-12)
    wt = W[target_cell] / (np.linalg.norm(W[target_cell]) + 1e-12)
    sparse_target_x = archive.X[int(np.argmax(AYn @ wt))]

micro_op = 0
micro_phi = micro_context[int(i)].copy() if "moe_residual" in flags else np.zeros(12)
shared_child: Optional[Array] = None
routed_delta = np.zeros(problem.n_var, dtype=float)
clip_scale = 1.0

if op == 5:
    child = _archive_transport_child(
        search_i, target_cell, X, F, W, B, memory, ideal, nadir, progress, rng, problem,
        role_probability=role_probability, role_reliability=role_reliability
    )
    if child is None:
        op = 0
        search_i = int(i)
        parent = X[search_i].copy()
elif op == 6 and role_probability is not None and coordinate_variable is not None:
    child, coordinate_value = _coordinate_tunnel_child(
        target_cell, coordinate_variable, X, F, W, ideal, nadir, problem,
        probe_index, learned_values, role_probability
    )
else:
    child = None

if child is None:
    child = core._mosaic_operator(
        int(op), search_i, X, F, W, B, archive, successful_steps, progress,
        rng, problem, sparse_target_x=sparse_target_x, scalar_blend=scalar_blend
    )
    if op == 2 and "secant" in flags:

```

```

        secant_calls += 1
        residual, reliability, _ = _secant_repulsive_delta(
            search_i, X, F, archive, W, ideal, nadir, progress, problem, rng,
            use_repulsion="no_repulsion" not in flags,
        )
        secant_rel_sum += reliability
        if reliability >= 0.28:
            ramp = float(np.clip((progress - 0.12) / 0.35, 0.0, 1.0))
            child = np.clip(child + ramp * reliability * residual, problem.xl, problem.xu)
            secant_used += 1

    shared_child = child.copy()
    # Fine-grained residual MoE is applied only to the frozen core experts.
    # 05/06 already are evidence-gated failure specialists with separate FE
    # semantics and are deliberately left untouched.
    if "moe_residual" in flags and op < 5:
        micro_op = int(micro_assign[int(i)])
        routed_delta = _micro_residual(
            micro_op, search_i, X, F, W, B, archive, residual_successful_steps,
            sparse_target_x, progress, moe_rng, problem, ideal, nadir,
        )
        if micro_op != 0 and "no_latent" not in flags:
            routed_delta = _project_routed_residual(
                routed_delta, latent_basis, problem.xu - problem.xl, retain=0.18
            )
        if micro_op != 0 and "no_energy_clip" not in flags:
            cap = 0.010 + 0.025 * (1.0 - progress) + 0.012 * float(stagnation[search_i] >= 6)
            routed_delta, clip_scale = _proposal_energy_clip(
                routed_delta, problem.xu - problem.xl, cap
            )
        child = np.clip(
            shared_child + float(micro_gate[int(i)]) * routed_delta,
            problem.xl, problem.xu,
        )
        micro_clip_scales.append(float(clip_scale))

    childF = problem.evaluate(child)[0]
    evals += 1
    ideal = np.minimum(ideal, childF)
    pooled = np.vstack([F, archive.F, childF[None, :]]) if len(archive.F) else np.vstack([F, childF])
    nadir = np.maximum(np.quantile(pooled, 0.95, axis=0), ideal + 1e-9)
    entered, novelty = archive.update(child, childF)
    if "no_memory" not in flags:
        memory.add(child, childF, gen, W, ideal, nadir)

    neigh = B[search_i]
    replaced, max_rel_improve = 0, 0.0
    for j in rng.permutation(neigh):
        mu = 0.008 + 0.10 * (1.0 - progress) ** 2
        theta = 0.10 + 0.22 * progress
        oldg = core._mosaic_scalar(
            F[j][None, :], W[j][None, :], ideal, nadir, mu, theta=theta, blend=scalar_blend
        )[0]
        newg = core._mosaic_scalar(
            childF[None, :], W[j][None, :], ideal, nadir, mu, theta=theta, blend=scalar_blend
        )[0]
        rel = float((oldg - newg) / (abs(oldg) + 1e-12))
        verify = bool(newg <= oldg or (entered and novelty > 0.45 and newg <= oldg * 1.015))

```

```

    if verify:
        X[j], F[j] = child.copy(), childF.copy()
        stagnation[j] = 0
        replaced += 1
        max_rel_improve = max(max_rel_improve, max(rel, 0.0))
        if replaced >= nr:
            break
    if replaced == 0:
        stagnation[search_i] += 1
    else:
        core_step = (shared_child if shared_child is not None else child) - parent
        successful_steps[op].append(core_step)
        if len(successful_steps[op]) > 64:
            successful_steps[op] = successful_steps[op][-64:]
        if "moe_residual" in flags and op < 5 and micro_op != 0:
            residual_successful_steps[micro_op].append(child - shared_child)
            if len(residual_successful_steps[micro_op]) > 48:
                residual_successful_steps[micro_op] = residual_successful_steps[micro_op][-48:]

    r_conv = float(np.clip(max_rel_improve / 0.08, 0.0, 1.0))
    r_rep = float(np.clip(replaced / max(nr, 1), 0.0, 1.0))
    r_arc = 0.25 + 0.75 * novelty if entered else 0.0
    # Preserve frozen v4 coverage credit so the no-05/no-recovery
    # ablation is behavior-identical before a reconstruction event.
    cz = np.minimum(F.min(axis=0), childF)
    cn = np.maximum(F.max(axis=0), childF)
    cy = np.maximum((childF - cz) / (cn - cz + 1e-12), 0.0)
    cyn = cy / (np.linalg.norm(cy) + 1e-12)
    c = int(np.argmax((W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)) @ cyn))
    r_cover = 1.0 if entered and search_counts[c] == 0 else 0.0
    reward = float(np.clip(0.42*r_conv + 0.20*r_rep + 0.23*r_arc + 0.15*r_cover, 0.0, 1.0))
    if "fixed_router" not in flags:
        if op == 5:
            o5_a[state] += reward
            o5_b[state] += 1.0 - reward
        elif op == 6:
            o6_a[state] += reward
            o6_b[state] += 1.0 - reward
        else:
            bandit.update(state, op, reward)

    if "moe_residual" in flags and op < 5:
        baseline = float(shared_reward_ema[state])
        advantage = micro_router.update(
            micro_phi, micro_op, reward, baseline,
            dense_default="dense_default" in flags,
        )
        # Only actual null/shared-only routes update the counterfactual
        # baseline; routed outcomes cannot rewrite the control estimate.
        if micro_op == 0:
            shared_reward_ema[state] = 0.94 * shared_reward_ema[state] + 0.06 * reward
        micro_counts[micro_op] += 1
        micro_rewards[micro_op] += reward
        micro_advantages[micro_op] += advantage

    if op == 6 and coordinate_variable is not None and coordinate_value is not None:
        jv = int(coordinate_variable)
        probe_trials[jv] += 1

```

```

        accepted_probe = bool(replaced > 0 and reward >= 0.08)
        if accepted_probe:
            probe_success[jv] += 1
            learned_values[jv] = float(coordinate_value)
        if len(coordinate_pilot_rewards) < max(coordinate_pilot_budget_current, 1):
            coordinate_pilot_rewards.append(float(reward))
        coordinate_events.append({
            "gen": gen, "evals": evals, "cell": int(search_i), "variable": jv,
            "value": float(coordinate_value), "accepted": accepted_probe,
            "reward": reward, "replaced": int(replaced),
        })
        op_counts[op] += 1
        op_rewards[op] += reward

    gen += 1
    if "discount_router" in flags:
        bandit.decay(0.992)
        o5_a = 1.0 + 0.992 * (o5_a - 1.0)
        o5_b = 1.0 + 0.992 * (o5_b - 1.0)
        o6_a = 1.0 + 0.992 * (o6_a - 1.0)
        o6_b = 1.0 + 0.992 * (o6_b - 1.0)

    # Compare against v4's periodic repair as an explicit ablation.
    if "force_coverage" in flags and gen % 4 == 0 and len(archive.F) >= pop_size // 2:
        ax, af = archive.output(min(2 * pop_size, len(archive.F)))
        X, F = core.nsga3_environmental_selection(
            np.vstack([X, ax]), np.vstack([F, af]), pop_size, W, rng
        )
        stagnation = np.minimum(stagnation, 3)

    # Evidence-triggered reconstruction. It does not run for an initially
    # collapsed many-objective mapping, where low occupancy is structural.
    if (
        "no_memory" not in flags
        and "no_recovery" not in flags
        and "force_coverage" not in flags
        and not alignment_latch
        and progress >= collapse_start
    ):
        recent = np.asarray(occupancy_history[-6:], dtype=float)
        if len(recent) >= 4:
            xh = np.arange(len(recent), dtype=float)
            slope = float(np.polyfit(xh, recent, 1)[0])
        else:
            slope = 0.0

        # A temporary collapse is common and may self-recover. Intervene
        # only when the low-entropy state persists late in the run and the
        # longitudinal occupancy is flat or declining.
        severe = occupancy < threshold and entropy < 0.62 and slope <= 0.003
        collapse_streak = collapse_streak + 1 if severe else 0
        if (
            collapse_streak >= max(int(collapse_patience), 1)
            and gen - last_recovery >= recovery_cooldown
            and len(recovery_events) < max_recoveries
        ):
            if "soft_recovery" in flags:
                assoc_now, _ = _associate_fixed(F, W, ideal, nadir)
                _, cnt_now = _occupancy_entropy(assoc_now, pop_size)

```

```

missing = np.flatnonzero(cnt_now == 0)
quota = min(max(1, int(math.ceil(recovery_fraction * pop_size))), len(missing))
occupied = np.flatnonzero(cnt_now > 0)
Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)
if len(missing) and len(occupied):
    proximity = np.max(Wn[missing] @ Wn[occupied].T, axis=1)
    cells = missing[np.argsort(proximity, kind="mergesort")[:quota]].astype(int).tolist()
else:
    cells = missing[:quota].astype(int).tolist()
# Only cells backed by an exact memory anchor enter the queue.
cells = [c for c in cells if len(memory.candidates_for_cell(c, W, ideal, nadir)) > 0]
rescue_queue.extend(cells)
restored = len(cells)
mode = "soft_ART_queue"
else:
    X, F, restored, cells = memory.reconstruct(
        X, F, W, ideal, nadir, recovery_fraction, rng
    )
    mode = "hard_exact_injection"
if restored > 0:
    recovery_events.append({
        "gen": gen,
        "evals": evals,
        "occupancy_before": occupancy,
        "entropy_before": entropy,
        "restored": restored,
        "cells": cells,
        "mode": mode,
    })
    last_recovery = gen
    collapse_streak = 0
    if mode == "hard_exact_injection":
        stagnation = np.minimum(stagnation, 3)
        ideal = np.minimum(ideal, F.min(axis=0))
        nadir = np.maximum(np.quantile(np.vstack([F, memory.F]), 0.95, axis=0), ideal + 1e-9)

if gen % trace_every == 0:
    trace.append({
        "gen": gen,
        "evals": evals,
        "archive_size": len(archive.F),
        "memory_size": len(memory.F),
        "mean_stagnation": float(stagnation.mean()),
        "mean_reward": float(op_rewards.sum() / max(op_counts.sum(), 1)),
        "occupancy_ratio": occupancy,
        "occupancy_entropy": entropy,
        "coverage_distress": distress,
        "recoveries": len(recovery_events),
    })

outX, outF = archive.output(pop_size)
label = "MOSAIC-LQMoE" if "moe_residual" in flags else "MOSAIC-R3"
if flags:
    label += "[" + "+".join(sorted(flags)) + "]"
diagnostics: Dict[str, object] = {
    "version": "MOSAIC-LQMoE-v8-research",
    "flags": sorted(flags),
    "operator_counts": op_counts.tolist(),

```



```

"operator_mean_rewards": (op_rewards / np.maximum(op_counts, 1)).tolist(),
"bandit_counts": bandit.counts.tolist(),
"micro_expert_names": [
    "shared_null", "pbest_micro", "pbest_macro", "local_diff",
    "global_diff", "archive_tangent", "gap_transport",
    "sparse_gaussian", "sparse_cauchy", "verified_replay",
    "sbx_residual", "orthogonal_escape",
],
"micro_counts": micro_counts.tolist(),
"micro_mean_rewards": (micro_rewards / np.maximum(micro_counts, 1)).tolist(),
"micro_mean_advantages": (micro_advantages / np.maximum(micro_counts, 1)).tolist(),
"micro_router_calls": micro_router.calls.tolist(),
"micro_router_reward_ema": micro_router.reward_ema.tolist(),
"micro_router_advantage_ema": micro_router.advantage_ema.tolist(),
"micro_router_load_ema": micro_router.load_ema.tolist(),
"micro_router_last_target": micro_router.last_target.tolist(),
"micro_router_last_load": micro_router.last_load.tolist(),
"micro_router_last_bias": micro_router.last_bias.tolist(),
"micro_router_prototype_mass": micro_router.prototype_mass.tolist(),
"micro_router_score_clip_events": int(micro_router.score_clip_events),
"micro_assignment_history": micro_assignment_history,
"micro_target_history": micro_target_history,
"micro_load_history": micro_load_history,
"micro_mean_clip_scale": float(np.mean(micro_clip_scales)) if micro_clip_scales else 1.0,
"radial_dispersion_history": radial_dispersion_history,
"geometry_score_history": geometry_score_history,
"radial_failure_streak_final": int(radial_failure_streak),
"o5_posterior_mean": (o5_a / (o5_a + o5_b)).tolist(),
"o6_posterior_mean": (o6_a / (o6_a + o6_b)).tolist(),
"coordinate_events": coordinate_events,
"coordinate_probe_trials": probe_trials.tolist(),
"coordinate_probe_success": probe_success.tolist(),
"coordinate_learned_values": learned_values.tolist(),
"coordinate_candidates": None if coordinate_candidates is None else coordinate_candidates.tolist(),
"coordinate_pilot_rewards": coordinate_pilot_rewards,
"coordinate_expand_threshold": coordinate_expand_threshold,
"coordinate_pilot_budget": int(coordinate_pilot_budget_current),
"coordinate_batch_history": coordinate_batch_history,
"recoveries": recovery_events,
"unspent_rescue_queue": rescue_queue,
"secant_calls": secant_calls,
"secant_used": secant_used,
"mean_secant_reliability": float(secant_rel_sum / max(secant_calls, 1)),
"role_reliability": role_reliability,
"role_diagnostics": role_diagnostics,
"role_probability": None if role_probability is None else role_probability.tolist(),
"initial_occupancy": init_occ,
"max_occupancy": max_occupancy,
"scalar_history": scalar_history,
"occupancy_history": occupancy_history,
"recovery_config": {
    "fraction": recovery_fraction,
    "cooldown": recovery_cooldown,
    "max_recoveries": max_recoveries,
    "collapse_ratio": collapse_ratio,
    "collapse_start": collapse_start,
    "collapse_patience": collapse_patience,
},

```

```

    }
    return core.RunResult(label, problem.name, seed, outX, outF, evals, trace, diagnostics)

__all__ = [
    "ExactCoverageMemory",
    "DiscountedPortfolio",
    "run_mosaic_lqmoe",
    "_van_der_corput",
    "_coordinate_tunnel_child",
]

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/legacy/mosaic_sqmoe_v9.py

"""Frozen entry point for MOSAIC-SQMoE v9-CGR.

Shared-backbone Quantile-routed Mixture of Search Experts with
Consensus-Gated Residual routing.

The implementation deliberately wraps the research module with one immutable
configuration. Every routed proposal is evaluated through the original
objective function and counts against ``max\_evals``.

"""

from __future__ import annotations

from typing import Optional

import moo_core as core

from mosaic_lqmoe import run_mosaic_lqmoe

VERSION = "MOSAIC-SQMoE-v9-CGR"

ABBREVIATION = "SQMoE-CGR"

FROZEN_FLAGS = (

"no_routine_o5+soft_recovery+coordinate_tunnel+coordinate_consensus_qb"

)

def run_mosaic_sqmoe(

problem: core.Problem,

seed: int,

pop_size: int = 100,

max_evals: int = 10100,

T: int = 20,

nr: int = 3,

trace_every: int = 5,

) -> core.RunResult:

"""Run the frozen SQMoE-CGR configuration.

Parameters follow the existing MOSAIC runner. The returned diagnostics
retain the full route, pilot, memory, geometry, and FE audit trail.

"""

result = run_mosaic_lqmoe(

problem=problem,

seed=seed,

```

        pop_size=pop_size,
        max_evals=max_evals,
        T=T,
        nr=nr,
        trace_every=trace_every,
        ablation=FROZEN_FLAGS,
        recovery_cooldown=12,
        max_recoveries=2,
    )
    result.algorithm = VERSION
    result.diagnostics["release_version"] = VERSION
    result.diagnostics["frozen_flags"] = FROZEN_FLAGS
    result.diagnostics["strict_fe_pass"] = bool(result.evaluations == max_evals)
    return result

```

```
__all__ = ["VERSION", "ABBREVIATION", "FROZEN_FLAGS", "run_mosaic_sqmoe"]
```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/__init__.py

```

"""MOSAIC v10: audited legacy shared core with typed proposal extensions."""
__version__ = '10.0.0-research'

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/api.py

```

from __future__ import annotations
from pathlib import Path
from dataclasses import dataclass
import sys
import numpy as np

# Isolate the old public imports while preserving file contents and RNG behavior.
LEGACY=Path(__file__).resolve().parents[1]/'legacy'
if str(LEGACY) not in sys.path:sys.path.insert(0,str(LEGACY))
import moo_core as core
from mosaic_sqmoe_v9 import run_mosaic_sqmoe,FROZEN_FLAGS
from .extended_core import run_mosaic_lqmoe
from .extensions import TypedRouter,ExtensionConfig

@dataclass(frozen=True)
class ProblemContract:
    representation:str='continuous_box'
    codec:object|None=None
    constraints:str='box' # box or feasible_decoder; unsupported -> explicit error
    deterministic:bool=True

class EvaluationLedger:
    """Count evaluated vectors, not Python batch calls. No caching in benchmarks."""
    def __init__(self,problem,budget:int):
        self._problem=problem;self.budget=int(budget);self.spent=0;self.batch_calls=0

```

```

        self.name=problem.name;self.n_var=problem.n_var;self.n_obj=problem.n_obj
        self.xl=np.array(problem.xl);self.xu=np.array(problem.xu)
        self.X=[];self.F=[];self.checkpoints=[]
    def evaluate(self,X):
        X=np.atleast_2d(np.asarray(X,float))
        if X.shape[1]!=self.n_var or not np.isfinite(X).all():raise ValueError('Invalid designs')
        if np.any(X<self.xl-1e-10) or np.any(X>self.xu+1e-10):raise ValueError('Out-of-bounds proposal')
        if self.spent+len(X)>self.budget:raise RuntimeError('Evaluation budget exceeded')
        # Charge attempted evaluations before calling the oracle; exceptions are not free.
        self.spent+=len(X);self.batch_calls+=1
        F=np.asarray(self._problem.evaluate(X),float)
        if F.shape!=(len(X),self.n_obj) or not np.isfinite(F).all():raise ValueError('Invalid evaluator output')
        self.X.extend(X.copy());self.F.extend(F.copy());return F
    def pareto_front(self,*args,**kwargs):
        raise RuntimeError('Test front must never be used during optimization')

@dataclass
class AuditedRun:
    result:object
    ledger:EvaluationLedger

def optimize(problem, seed=1, budget=2080, population=32,
             contract=ProblemContract(), extension_config=ExtensionConfig(), algorithm='mosaic_v10'):
    if budget<population or population<8:raise ValueError('Require budget >= population >= 8')
    if not contract.deterministic or contract.constraints not in ('box','feasible_decoder'):
        raise NotImplementedError('Noise and general nonlinear constraints require independent certified adapters')
    ledger=EvaluationLedger(problem,budget);ext=None
    if algorithm=='mosaic_v9':
        result=run_mosaic_sqmoe(ledger,seed,population,budget,T=min(12,population))
    elif algorithm=='typed_residual_research':
        if contract.representation=='continuous_box' and contract.codec is None:
            result=run_mosaic_sqmoe(ledger,seed,population,budget,T=min(12,population))
        elif contract.codec is not None:
            ext=TypedRouter(contract.codec,seed,budget,extension_config)
            result=run_mosaic_lqmoe(ledger,seed,population,budget,T=min(12,population),
                                   ablation=FROZEN_FLAGS,recovery_cooldown=12,max_recoveries=2,extension=ext)
        else:raise ValueError('Structured representation needs a pure codec')
    elif algorithm=='mosaic_v10':
        if contract.representation=='continuous_box' and contract.codec is None:
            result=run_mosaic_sqmoe(ledger,seed,population,budget,T=min(12,population))
            result.diagnostics['backend']='frozen_v9'
        elif contract.representation=='ordered_subset' and contract.codec is not None:
            from .graph_search import run_graph
            result=run_graph(ledger,contract.codec,seed,population,budget,mode='uniform',ties_per_objective=4)
            result.diagnostics['backend']='behavior_archive_structured'
        else:
            raise NotImplementedError('Representation is not yet validated; provide a tested adapter')
        result.algorithm='MOSAIC-v10'
    elif algorithm in ['nsga2_port','moead_port','rdex_port','rvea_port','nsga3_port']:
        if budget%population:raise ValueError('Legacy generation baselines need budget multiple of population')
        funcs={'nsga2_port':core.run_nsga2,'moead_port':core.run_moead,'rdex_port':core.run_rdex,
              'rvea_port':core.run_rvea,'nsga3_port':core.run_nsga3}
        result=funcs[algorithm](ledger,seed,population,budget)
    elif algorithm in ('typed_nsga2','typed_nsga2_unique'):
        from .typed_baseline import run_typed_nsga2,run_typed_nsga2_unique
        if contract.codec is None:raise ValueError('typed_nsga2 needs codec')
        result=(run_typed_nsga2 if algorithm=='typed_nsga2' else
        ↪ run_typed_nsga2_unique)(ledger,contract.codec,seed,population,budget)

```

```

elif algorithm=='random':
    rng=np.random.default_rng(seed);X=ledger.xl+rng.random((budget,ledger.n_var))*(ledger.xu-ledger.xl)
    F=ledger.evaluate(X);archive=core.Archive(max_size=population);archive.update(X,F)
    x,f=archive.output(population)
    result=core.RunResult('random',ledger.name,seed,x,f,budget,[],{})
else:raise ValueError('Unknown algorithm')
if result.evaluations!=ledger.spent or ledger.spent!=budget:
    raise AssertionError((result.evaluations,ledger.spent,budget))
if ext is not None:result.diagnostics['typed_extension']=ext.diagnostics()
result.diagnostics.update(actual_evaluated_vectors=ledger.spent,batch_calls=ledger.batch_calls,
    official_baseline=False, implementation='inherited source, not official upstream binaries')
return AuditedRun(result,ledger)

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/extended_core.py

```
from __future__ import annotations
```

```
"""MOSAIC-ST research branch.
```

This module keeps the frozen MOSAIC-MOEA v4 evaluator, scalarization and exact replacement semantics, and adds three isolated mechanisms:

1. Exact Coverage Memory (ECM): a bounded bank of *actually evaluated* designs, indexed by reference direction. It is separate from the Pareto archive so potentially useful stepping stones are not erased merely because they are temporarily dominated.
2. Archive-Reuse Transport Expert (ART, operator 5): a failure-mode specialist activated only when reference-cell coverage is distressed.
3. Secant-Repulsive Transport (SRT, optional upgrade of operator 2): a local ridge secant model maps a Smooth-Tchebycheff descent/repulsion direction in objective space back to a trust-region decision-space residual. The residual is bypassed when its cross-validated local fit is unreliable.
4. Role-Gated Coordinate Basin Tunneling (RCBT, operator 6): when reference coverage is healthy but scalar progress stalls, low-discrepancy one-coordinate probes are applied only to variables inferred to control convergence rather than Pareto direction. Successful coordinate values are composed into later probes, but every candidate still requires exact evaluation and STCH acceptance.

The default callable exposes ablations so every mechanism can be evaluated under identical seeds and function-evaluation budgets. No surrogate prediction is allowed to update the population or archive without exact objective evaluation.

```
"""
```

```
from dataclasses import dataclass
```

```
from typing import Dict, List, Optional, Sequence, Tuple
```

```
import math
```

```
import numpy as np
```

```
import moo_core as core
```

```
Array = np.ndarray
```

```
def _associate_fixed(F: Array, W: Array, ideal: Array, nadir: Array) -> Tuple[Array, Array]:
```

```

"""Reference association in one shared longitudinal normalization frame."""
F = np.asarray(F, dtype=float)
Y = np.maximum((F - ideal) / (nadir - ideal + 1e-12), 0.0)
Yn = Y / (np.linalg.norm(Y, axis=1, keepdims=True) + 1e-12)
Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)
cos = np.clip(Yn @ Wn.T, -1.0, 1.0)
assoc = np.argmax(cos, axis=1)
angle = np.arccos(cos[np.arange(len(F)), assoc])
return assoc.astype(int), angle

def _occupancy_entropy(assoc: Array, n_cells: int) -> Tuple[float, float, Array]:
    counts = np.bincount(np.asarray(assoc, dtype=int), minlength=n_cells)
    occupancy = float(np.count_nonzero(counts) / max(n_cells, 1))
    p = counts[counts > 0].astype(float)
    p /= max(p.sum(), 1.0)
    if n_cells <= 1 or len(p) == 0:
        entropy = 0.0
    else:
        entropy = float(-np.sum(p * np.log(p + 1e-15)) / np.log(n_cells))
    return occupancy, entropy, counts

@dataclass
class MemoryCandidate:
    x: Array
    f: Array
    born: int

class ExactCoverageMemory:
    """Bounded exact-evaluation memory with reference-cell-aware compaction.

    Pareto archives are intentionally selective in objective space. This memory
    has a different role: preserve historically evaluated stepping stones that
    may be needed to reconstruct a missing reference region. It never invents
    objective values and never bypasses the exact verifier for newly generated
    offspring.
    """

    def __init__(self, capacity: int, per_cell: int = 3):
        self.capacity = int(max(capacity, 8))
        self.per_cell = int(max(per_cell, 1))
        self.X = np.empty((0, 0), dtype=float)
        self.F = np.empty((0, 0), dtype=float)
        self.born = np.empty(0, dtype=int)

    def add(self, X: Array, F: Array, generation: int, W: Array, ideal: Array, nadir: Array) -> None:
        X = np.asarray(X, dtype=float)
        F = np.asarray(F, dtype=float)
        if X.ndim == 1:
            X = X[None, :]
        if F.ndim == 1:
            F = F[None, :]
        if self.X.size == 0:
            self.X = X.copy()
            self.F = F.copy()
            self.born = np.full(len(X), generation, dtype=int)

```

```

else:
    self.X = np.vstack([self.X, X])
    self.F = np.vstack([self.F, F])
    self.born = np.r_[self.born, np.full(len(X), generation, dtype=int)]
if len(self.F) > int(1.20 * self.capacity):
    self.compact(W, ideal, nadir)

def compact(self, W: Array, ideal: Array, nadir: Array) -> None:
    if len(self.F) <= self.capacity:
        return
    assoc, _ = _associate_fixed(self.F, W, ideal, nadir)
    selected: List[int] = []
    mu = 0.02
    span_x = np.ptp(self.X, axis=0) + 1e-12
    for c in range(len(W)):
        idx = np.flatnonzero(assoc == c)
        if len(idx) == 0:
            continue
        Wi = np.repeat(W[c][None, :], len(idx), axis=0)
        scores = core._mosaic_scalar(self.F[idx], Wi, ideal, nadir, mu, theta=0.22, blend=0.0)
        order = idx[np.argsort(scores, kind="mergesort")]
        keep: List[int] = [int(order[0])]
        # Preserve a recent candidate and then decision-diverse candidates.
        recent = int(idx[np.argmax(self.born[idx])])
        if recent not in keep and len(keep) < self.per_cell:
            keep.append(recent)
        while len(keep) < min(self.per_cell, len(idx)):
            remaining = [int(j) for j in idx if int(j) not in keep]
            if not remaining:
                break
            D = (self.X[remaining] - self.X[keep][:, None, :]).transpose(1, 0, 2)
            # min normalized decision distance to an already kept point
            d = np.linalg.norm(D / span_x[None, None, :], axis=2).min(axis=1)
            keep.append(remaining[int(np.argmax(d))])
        selected.extend(keep)
    # If per-cell retention is still too large, use objective-space maximin.
    selected = list(dict.fromkeys(selected))
    if len(selected) > self.capacity:
        local = core.select_indices_farthest(self.F[selected], self.capacity)
        selected = [selected[int(j)] for j in local]
    # If cells are sparse and capacity remains, retain recent evaluated points.
    if len(selected) < self.capacity:
        rest = [i for i in np.argsort(-self.born, kind="mergesort") if int(i) not in set(selected)]
        selected.extend([int(i) for i in rest[: self.capacity - len(selected)]])
    idx = np.asarray(selected[: self.capacity], dtype=int)
    self.X, self.F, self.born = self.X[idx], self.F[idx], self.born[idx]

def candidates_for_cell(
    self, cell: int, W: Array, ideal: Array, nadir: Array, max_candidates: int = 6
) -> Array:
    if len(self.F) == 0:
        return np.empty(0, dtype=int)
    assoc, angle = _associate_fixed(self.F, W, ideal, nadir)
    exact = np.flatnonzero(assoc == int(cell))
    if len(exact) == 0:
        # Fallback to closest objective direction; this is still exact memory.
        return np.argsort(angle, kind="mergesort")[:max_candidates]
    Wi = np.repeat(W[int(cell)][None, :], len(exact), axis=0)

```

```

score = core._mosaic_scalar(self.F[exact], Wi, ideal, nadir, 0.018, theta=0.25, blend=0.0)
return exact[np.argsort(score, kind="mergesort")[:max_candidates]]

def best_anchor_advantage(
    self, cell: int, current_F: Array, W: Array, ideal: Array, nadir: Array
) -> float:
    """Exact scalar advantage of the best memory anchor for one cell."""
    idx = self.candidates_for_cell(cell, W, ideal, nadir, max_candidates=3)
    if len(idx) == 0:
        return 0.0
    wc = W[int(cell)][None, :]
    mem = core._mosaic_scalar(
        self.F[idx], np.repeat(wc, len(idx), axis=0), ideal, nadir, 0.018,
        theta=0.25, blend=0.0
    ).min()
    cur = core._mosaic_scalar(
        current_F, np.repeat(wc, len(current_F), axis=0), ideal, nadir, 0.018,
        theta=0.25, blend=0.0
    ).min()
    return float((cur - mem) / (abs(cur) + 1e-12))

def choose_anchor(
    self, cell: int, W: Array, ideal: Array, nadir: Array, rng: np.random.Generator
) -> Optional[Tuple[Array, Array]]:
    idx = self.candidates_for_cell(cell, W, ideal, nadir)
    if len(idx) == 0:
        return None
    # Mostly choose the best scalar candidate but retain slight diversity.
    j = int(idx[0] if len(idx) == 1 or rng.random() < 0.75 else rng.choice(idx[: min(3, len(idx))]))
    return self.X[j].copy(), self.F[j].copy()

def reconstruct(
    self,
    X: Array,
    F: Array,
    W: Array,
    ideal: Array,
    nadir: Array,
    fraction: float,
    rng: np.random.Generator,
) -> Tuple[Array, Array, int, List[int]]:
    """Inject exact historical candidates into missing cells without new FEs."""
    if len(self.F) == 0:
        return X, F, 0, []
    assoc, _ = _associate_fixed(F, W, ideal, nadir)
    _, _, counts = _occupancy_entropy(assoc, len(W))
    missing = np.flatnonzero(counts == 0)
    if len(missing) == 0:
        return X, F, 0, []
    n_inject = min(max(1, int(math.ceil(fraction * len(X)))), len(missing))
    # Largest angular holes first: directions farthest from occupied directions.
    occupied = np.flatnonzero(counts > 0)
    Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)
    if len(occupied):
        proximity = np.max(Wn[missing] @ Wn[occupied].T, axis=1)
        targets = missing[np.argsort(proximity, kind="mergesort")[:n_inject]]
    else:
        targets = missing[:n_inject]

```



```

# Victims: poor members of over-populated cells. Do not force a target
# into the same array index; the frozen MOEA/D update will reassociate it.
redundancy = counts[assoc]
ownW = W[assoc]
scalar = core._mosaic_scalar(F, ownW, ideal, nadir, 0.012, theta=0.25, blend=0.0)
s_norm = (scalar - scalar.min()) / (np.ptp(scalar) + 1e-12)
victim_score = redundancy.astype(float) + s_norm
victim_order = np.argsort(-victim_score, kind="mergesort")

Xn, Fn = X.copy(), F.copy()
used_mem: set[int] = set()
used_victims: set[int] = set()
restored_cells: List[int] = []
for cell in targets:
    cand = self.candidates_for_cell(int(cell), W, ideal, nadir)
    cand = np.asarray([j for j in cand if int(j) not in used_mem], dtype=int)
    if len(cand) == 0:
        continue
    mem_j = int(cand[0])
    victim = next((int(j) for j in victim_order if int(j) not in used_victims), None)
    if victim is None:
        break
    # Skip exact duplicate designs.
    if np.any(np.all(np.isclose(Xn, self.X[mem_j], atol=1e-12, rtol=1e-10), axis=1)):
        used_mem.add(mem_j)
        continue
    Xn[victim], Fn[victim] = self.X[mem_j].copy(), self.F[mem_j].copy()
    used_mem.add(mem_j)
    used_victims.add(victim)
    restored_cells.append(int(cell))
return Xn, Fn, len(restored_cells), restored_cells

class DiscountedPortfolio(core.ContextualThompsonPortfolio):
    """Optional non-stationary extension; kept as an ablation, not assumed useful."""

    def decay(self, gamma: float) -> None:
        g = float(np.clip(gamma, 0.90, 1.0))
        self.a = 1.0 + g * (self.a - 1.0)
        self.b = 1.0 + g * (self.b - 1.0)
        self.ga = 1.0 + g * (self.ga - 1.0)
        self.gb = 1.0 + g * (self.gb - 1.0)
        self.reward_sum *= g

    def _secant_repulsive_delta(
        i: int,
        X: Array,
        F: Array,
        archive: core.Archive,
        W: Array,
        ideal: Array,
        nadir: Array,
        progress: float,
        problem: core.Problem,
        rng: np.random.Generator,
        use_repulsion: bool = True,

```

```

) -> Tuple[Array, float, float]:
    """Map an objective-space descent/repulsion vector through a local secant model.

    Returns (decision residual, reliability, normalized local fit error). The
    caller must ignore the residual when reliability is low.
    """
    d, m = problem.n_var, problem.n_obj
    if len(archive.F) < max(8, m + 4):
        return np.zeros(d), 0.0, 1.0
    span_x = problem.xu - problem.xl + 1e-12
    span_f = nadir - ideal + 1e-12
    AF = np.maximum((archive.F - ideal) / span_f, 0.0)
    yi = np.maximum((F[i] - ideal) / span_f, 0.0)
    dist = np.linalg.norm(AF - yi, axis=1)
    k = min(max(10, 2 * m + 4), len(AF))
    idx = np.argsort(dist, kind="mergesort")[:k]
    DX = (archive.X[idx] - X[i]) / span_x
    DY = AF[idx] - yi
    # Remove nearly identical rows, which make local secants unidentifiable.
    keep = np.linalg.norm(DX, axis=1) > 1e-8
    DX, DY = DX[keep], DY[keep]
    if len(DX) < m + 3:
        return np.zeros(d), 0.0, 1.0
    ridge = 2e-3 + 2e-2 * (1.0 - progress)
    try:
        # Dual ridge fit:  $J^T = DX^T (DX DX^T + \lambda I)^{-1} DY$ .
        gram = DX @ DX.T + ridge * np.eye(len(DX))
        Jt = DX.T @ np.linalg.solve(gram, DY)
        J = Jt.T
    except np.linalg.LinAlgError:
        return np.zeros(d), 0.0, 1.0
    pred = DX @ Jt
    sse = float(np.sum((DY - pred) ** 2))
    sst = float(np.sum((DY - DY.mean(axis=0, keepdims=True)) ** 2)) + 1e-12
    fit_error = float(np.clip(sse / sst, 0.0, 2.0))
    # Condition reliability in objective row-space.
    try:
        cond = float(np.linalg.cond(J @ J.T + 1e-4 * np.eye(m)))
    except np.linalg.LinAlgError:
        cond = 1e12
    reliability = float(np.clip((1.0 - min(fit_error, 1.0)) * np.exp(-max(np.log10(cond) - 3.0, 0.0)), 0.0, 1.0))
    if reliability < 0.28:
        return np.zeros(d), reliability, fit_error

    # Smooth-Tchebycheff gradient in normalized objective coordinates.
    w = np.maximum(W[i], 1e-4)
    mu = 0.012 + 0.07 * (1.0 - progress) ** 2
    logits = w * yi / mu
    logits -= logits.max()
    soft = np.exp(logits)
    soft /= soft.sum() + 1e-12
    grad = soft * w
    dy = -grad / (np.linalg.norm(grad) + 1e-12)

    if use_repulsion:
        D = yi[None, :] - AF[idx]
        r = np.linalg.norm(D, axis=1)
        positive = r[r > 1e-8]

```

```

h = float(np.median(positive)) if len(positive) else 0.15
h = max(h, 0.05)
weight = np.exp(-(r * r) / (2.0 * h * h))
rep = np.sum(weight[:, None] * D, axis=0) / (weight.sum() + 1e-12)
if np.linalg.norm(rep) > 1e-10:
    rep /= np.linalg.norm(rep)
    # Repulsion matters mainly in crowded regions and later in the run.
    beta = 0.12 + 0.22 * progress
    dy = dy + beta * rep
    dy /= np.linalg.norm(dy) + 1e-12

try:
    dx = J.T @ np.linalg.solve(J @ J.T + (4e-3 + 1e-2 * (1.0 - progress)) * np.eye(m), dy)
except np.linalg.LinAlgError:
    return np.zeros(d), 0.0, fit_error
nrm = np.linalg.norm(dx)
if not np.isfinite(nrm) or nrm < 1e-12:
    return np.zeros(d), 0.0, fit_error
dx /= nrm
trust = 0.10 * (1.0 - progress) + 0.018
# Small random tangent jitter prevents deterministic secant lock-in.
jitter = rng.normal(size=d)
jitter -= dx * float(np.dot(jitter, dx))
jitter /= np.linalg.norm(jitter) + 1e-12
residual = span_x * trust * (0.90 * dx + 0.10 * jitter)
return residual, reliability, fit_error

def _infer_variable_roles(
    memory: ExactCoverageMemory,
    problem: core.Problem,
    ideal: Array,
    nadir: Array,
    W: Optional[Array] = None,
) -> Tuple[Optional[Array], float, Dict[str, float]]:
    """Infer position-vs-distance roles by recent reference-cell separation.

    Let  $c(n)$  be the reference cell associated with evaluated design  $n$ . For each
    decision variable  $j$ , we compute the one-way ANOVA effect size

    
$$\eta_j^2 = \text{between-cell variance} / \text{total variance}.$$


    A high  $\eta^2$  means the variable changes systematically with Pareto direction
    (a position/diversity variable). A low  $\eta^2$  means it is better supplied by
    a currently converged donor. Only the recent half of exact memory is used;
    early random correlations otherwise make unrelated variables look like
    position variables after the population has already contracted.
    """
    if W is None:
        W = core._dirichlet_reference_points(max(32, problem.n_obj * 16), problem.n_obj, seed=601)
    if len(memory.F) < max(40, 2 * problem.n_var):
        return None, 0.0, {"occupied_cells": 0.0, "eta_gap": 0.0}
    cutoff = float(np.quantile(memory.born, 0.50))
    idx = np.flatnonzero(memory.born >= cutoff)
    if len(idx) < max(40, 2 * problem.n_var):
        idx = np.argsort(memory.born, kind="mergesort")[-min(len(memory.F), max(80, 4 * problem.n_var)):]
    X, F = memory.X[idx], memory.F[idx]

```

```

Xn = (X - problem.xl) / (problem.xu - problem.xl + 1e-12)
assoc, _ = _associate_fixed(F, W, ideal, nadir)
occupied = np.unique(assoc)
mean = Xn.mean(axis=0)
total = np.sum((Xn - mean) ** 2, axis=0) + 1e-12
between = np.zeros(problem.n_var)
for cell in occupied:
    j = np.flatnonzero(assoc == cell)
    if len(j):
        between += len(j) * (Xn[j].mean(axis=0) - mean) ** 2
eta = np.clip(between / total, 0.0, 1.0)

# Deterministic 1-D two-cluster split of variable effect sizes.
c0, c1 = float(eta.min()), float(eta.max())
for _ in range(20):
    d0, d1 = np.abs(eta - c0), np.abs(eta - c1)
    hi = d1 < d0
    if not np.any(hi) or np.all(hi):
        break
    n0, n1 = float(eta[~hi].mean()), float(eta[hi].mean())
    if abs(n0 - c0) + abs(n1 - c1) < 1e-10:
        c0, c1 = n0, n1
        break
    c0, c1 = n0, n1
low, high = sorted((c0, c1))
gap = high - low
midpoint = 0.5 * (high + low)
temperature = max(0.035, 0.18 * gap)
role_prob = 1.0 / (1.0 + np.exp(-(eta - midpoint) / temperature))
# Require at least one clear position variable; cap uncertain probabilities.
role_prob = np.clip(role_prob, 0.02, 0.98)
if gap < 0.06:
    reliability = 0.0
else:
    cell_factor = min(1.0, len(occupied) / max(problem.n_obj + 2, 4))
    reliability = float(np.clip((gap / (np.std(eta) + 1e-12)) / 4.0, 0.0, 1.0) * cell_factor)
return role_prob, reliability, {
    "occupied_cells": float(len(occupied)),
    "eta_gap": float(gap),
    "eta_max": float(eta.max()),
    "eta_second": float(np.partition(eta, -2)[-2]) if len(eta) > 1 else float(eta.max()),
    "mean_position_probability": float(role_prob.mean()),
}

```

```

def _archive_transport_child(
    i: int,
    target_cell: int,
    X: Array,
    F: Array,
    W: Array,
    B: Array,
    memory: ExactCoverageMemory,
    ideal: Array,
    nadir: Array,
    progress: float,
    rng: np.random.Generator,
    problem: core.Problem,

```

```

    role_probability: Optional[Array] = None,
    role_reliability: float = 0.0,
) -> Optional[Array]:
    anchor = memory.choose_anchor(target_cell, W, ideal, nadir, rng)
    if anchor is None:
        return None
    xa, _ = anchor
    n, d = X.shape
    pool = B[i] if len(B[i]) >= 3 else np.arange(n)
    r1, r2 = rng.choice(pool, size=2, replace=False)

    if role_probability is not None and role_reliability >= 0.18:
        # Donor minimizes the target scalarization among the current population:
        # it supplies variables that have already learned convergence, while the
        # memory anchor supplies variables that control objective direction.
        wt = np.repeat(W[int(target_cell)][None, :], n, axis=0)
        score = core._mosaic_scalar(F, wt, ideal, nadir, 0.015, theta=0.22, blend=0.0)
        elite = np.argsort(score, kind="mergesort")[: max(2, int(math.ceil(0.12 * n)))]
        donor = X[int(rng.choice(elite))]
        ppos = np.clip(role_probability, 0.05, 0.95)
        position_mask = rng.random(d) < ppos
        if not np.any(position_mask):
            position_mask[int(np.argmax(ppos))] = True
        if np.all(position_mask):
            position_mask[int(np.argmin(ppos))] = False
        child = donor.copy()
        child[position_mask] = 0.92 * xa[position_mask] + 0.08 * donor[position_mask]
        # Refine distance variables only; perturbing inferred position variables
        # would immediately destroy the rescued reference direction.
        conv_mask = ~position_mask
        scale = float(np.clip(rng.normal(0.18 - 0.08 * progress, 0.05), 0.04, 0.28))
        child[conv_mask] += scale * (X[r1, conv_mask] - X[r2, conv_mask])
    else:
        # Reliability fallback: ordinary archive transport, still exact-verified.
        Fscale = float(np.clip(rng.normal(0.30 - 0.12 * progress, 0.08), 0.08, 0.45))
        mutant = xa + Fscale * (X[r1] - X[r2])
        cr = float(np.clip(rng.normal(0.82, 0.10), 0.35, 1.0))
        child = core.de_binomial(xa, mutant, cr, rng)

    child = np.clip(child, problem.xl, problem.xu)
    child = core.polynomial_mutation(child, problem.xl, problem.xu, rng, eta=30.0, prob=0.35 / d)
    return child

def _van_der_corput(index: int, base: int = 2) -> float:
    """Deterministic low-discrepancy scalar in (0,1)."""
    n = max(int(index), 1)
    denom = 1.0
    out = 0.0
    while n:
        n, rem = divmod(n, base)
        denom *= base
        out += rem / denom
    return float(out)

```

```

def _coordinate_expert_scores(
    candidates: Array,
    role_probability: Array,
    memory: ExactCoverageMemory,
    problem: core.Problem,
    ideal: Array,
    nadir: Array,
    probe_trials: Array,
    probe_success: Array,
) -> Array:
    """Score coordinate experts from exact memory, role confidence and UCB.

    No objective call is made here. The sensitivity term measures how strongly
    elite exact-memory points concentrate a variable relative to the whole exact
    memory, while the role prior suppresses variables that encode PF position.
    """
    cand = np.asarray(candidates, dtype=int)
    if len(cand) == 0:
        return np.empty(0)
    role = 1.0 - np.clip(role_probability[cand], 0.0, 1.0)
    sensitivity = np.zeros(len(cand), dtype=float)
    if len(memory.F) >= max(30, 2 * len(cand)):
        recent_cut = float(np.quantile(memory.born, 0.45))
        idx = np.flatnonzero(memory.born >= recent_cut)
        if len(idx) < 30:
            idx = np.arange(len(memory.F))
        MX = (memory.X[idx] - problem.xl) / (problem.xu - problem.xl + 1e-12)
        MF = memory.F[idx]
        Y = np.maximum((MF - ideal) / (nadir - ideal + 1e-12), 0.0)
        quality = np.sum(Y, axis=1)
        elite_n = max(8, int(math.ceil(0.20 * len(quality))))
        elite_idx = np.argsort(quality, kind="mergesort")[:elite_n]
        for k, j in enumerate(cand):
            all_std = float(np.std(MX[:, j])) + 1e-9
            elite_std = float(np.std(MX[elite_idx, j]))
            center_shift = abs(float(np.mean(MX[elite_idx, j]) - np.mean(MX[:, j]))) / all_std
            concentration = float(np.clip(1.0 - elite_std / all_std, 0.0, 1.0))
            sensitivity[k] = float(np.clip(0.55 * concentration + 0.45 * np.tanh(center_shift / 2.0), 0.0, 1.0))
    post_mean = (probe_success[cand] + 1.0) / (probe_trials[cand] + 2.0)
    uncertainty = 1.0 / np.sqrt(1.0 + probe_trials[cand])
    return 0.42 * role + 0.28 * sensitivity + 0.20 * post_mean + 0.10 * uncertainty


def _dynamic_coordinate_batch(
    candidates: Array,
    scores: Array,
    probe_trials: Array,
    max_batch: int = 4,
) -> Tuple[Array, int, float]:
    """Entropy-calibrated dynamic-k with a score-quantile cutoff."""
    cand = np.asarray(candidates, dtype=int)
    scores = np.asarray(scores, dtype=float)
    if len(cand) == 0:
        return np.empty(0, dtype=int), 0, 0.0
    z = scores - np.max(scores)
    p = np.exp(z / 0.12)
    p /= max(float(p.sum()), 1e-12)
    entropy = float(-np.sum(p * np.log(p + 1e-15)) / max(np.log(len(p)), 1e-12)) if len(p) > 1 else 0.0

```

```

# High uncertainty/entropy buys more coordinate experts; confident routing
# spends only one. The overall FE budget remains unchanged.
k = int(np.clip(1 + math.ceil((max_batch - 1) * entropy), 1, min(max_batch, len(cand))))
# Prefer untested experts, then apply the score quantile within that set.
untested = probe_trials[cand] == 0
pool_idx = np.flatnonzero(untested) if np.any(untested) else np.arange(len(cand))
pool_scores = scores[pool_idx]
take = min(k, len(pool_idx))
cutoff = float(np.quantile(pool_scores, max(0.0, 1.0 - take / max(len(pool_scores), 1))))
eligible = pool_idx[pool_scores >= cutoff - 1e-12]
order = eligible[np.argsort(-scores[eligible], kind="mergesort")]
chosen = cand[order[:take]]
return chosen.astype(int), k, entropy

def _coordinate_tunnel_child(
    target_cell: int,
    variable: int,
    X: Array,
    F: Array,
    W: Array,
    ideal: Array,
    nadir: Array,
    problem: core.Problem,
    probe_index: Array,
    learned_values: Array,
    role_probability: Array,
) -> Tuple[Array, float]:
    """Generate one exact coordinate intervention without moving PF position vars."""
    n, d = X.shape
    wt = np.repeat(W[int(target_cell)][None, :], n, axis=0)
    score = core._mosaic_scalar(F, wt, ideal, nadir, 0.015, theta=0.22, blend=0.0)
    donor = X[int(np.argmin(score))]
    child = donor.copy()
    # Compose previously verified convergence alleles. This is an exact-memory
    # analogue of optimal mixing: only values with prior positive verifier credit
    # are broadcast, and inferred position variables remain untouched.
    known = np.isfinite(learned_values) & (role_probability < 0.35)
    child[known] = learned_values[known]
    j = int(variable)
    span = float(problem.xu[j] - problem.xl[j])
    # Advance the base-2 low-discrepancy line-search sequence until the probe is
    # not numerically identical to the donor/known value.
    value = child[j]
    for _ in range(8):
        probe_index[j] += 1
        u = _van_der_corput(int(probe_index[j]), 2)
        value = float(problem.xl[j] + u * span)
        if abs(value - child[j]) > 0.015 * max(span, 1e-12):
            break
    child[j] = value
    return np.clip(child, problem.xl, problem.xu), value

# -----
# MOSAIC-LQMoE v8: Shared backbone + latent quantile-routed micro-experts
# -----

```

```

from scipy.optimize import linear_sum_assignment

def _safe_unit_rows(A: Array) -> Array:
    A = np.asarray(A, dtype=float)
    return A / (np.linalg.norm(A, axis=1, keepdims=True) + 1e-12)

def _build_latent_basis(
    X: Array,
    successful_steps: Sequence[Sequence[Array]],
    span: Array,
    n_obj: int,
    max_rank: int = 10,
) -> Array:
    """Build a stable low-rank routed subspace from population and verified steps.

    The always-on shared proposal remains full dimensional. Only routed residuals
    are compressed into this basis, mirroring the shared-full-width / routed-latent
    separation of modern LatentMoE designs.
    """
    blocks: List[Array] = []
    span = np.maximum(np.asarray(span, dtype=float), 1e-12)
    Xn = (np.asarray(X, dtype=float) - np.mean(X, axis=0, keepdims=True)) / span
    if len(Xn):
        blocks.append(Xn)
    verified: List[Array] = []
    for seq in successful_steps:
        if seq:
            verified.extend(np.asarray(s, dtype=float) / span for s in seq[-24:])
    if verified:
        blocks.append(np.vstack(verified))
    Z = np.vstack(blocks) if blocks else np.eye(len(span))
    # Remove numerically empty rows and apply light shrinkage toward coordinate axes.
    nz = np.linalg.norm(Z, axis=1) > 1e-10
    Z = Z[nz]
    if len(Z) == 0:
        return np.eye(len(span))[:1]
    try:
        _, s, vt = np.linalg.svd(Z, full_matrices=False)
    except np.linalg.LinAlgError:
        return np.eye(len(span))[: min(len(span), max(1, n_obj))]
    numerical_rank = int(np.count_nonzero(s > max(s[0], 1e-12) * 1e-7))
    rank = max(1, min(max_rank, len(span), max(n_obj + 1, numerical_rank)))
    return vt[:rank].copy()

def _project_routed_residual(delta: Array, basis: Array, span: Array, retain: float = 0.18) -> Array:
    """Project a residual into the latent search subspace with an escape floor."""
    span = np.maximum(np.asarray(span, dtype=float), 1e-12)
    z = np.asarray(delta, dtype=float) / span
    if basis.size == 0:
        return np.asarray(delta, dtype=float)
    projected = (z @ basis.T) @ basis
    # A small full-space component prevents a stale basis from making dimensions
    # permanently unreachable.
    return span * ((1.0 - retain) * projected + retain * z)

```



```

def _proposal_energy_clip(delta: Array, span: Array, cap: float) -> Tuple[Array, float]:
    """RMS-cap the routed proposal energy; return clipped delta and scale factor."""
    span = np.maximum(np.asarray(span, dtype=float), 1e-12)
    energy = float(np.sqrt(np.mean((np.asarray(delta, dtype=float) / span) ** 2)))
    if not np.isfinite(energy) or energy <= 1e-15:
        return np.zeros_like(delta, dtype=float), 1.0
    scale = min(1.0, float(cap) / energy)
    return np.asarray(delta, dtype=float) * scale, scale

class LatentQuantileRouter:
    """Continuous Bayesian router with memory and loss-free quantile balancing.

    Balance biases affect assignment only. They never enter the exact reward or
    posterior target, preserving a clean separation between task utility and load
    control, analogous to auxiliary-loss-free routing.
    """

    def __init__(self, n_experts: int, n_features: int, ridge: float = 2.0):
        self.n_experts = int(n_experts)
        self.n_features = int(n_features)
        self.ridge = float(ridge)
        eye = np.eye(self.n_features, dtype=float)
        self.A = np.repeat((self.ridge * eye)[None, :, :], self.n_experts, axis=0)
        self.b = np.zeros((self.n_experts, self.n_features), dtype=float)
        self.calls = np.zeros(self.n_experts, dtype=int)
        self.reward_ema = np.full(self.n_experts, 0.30, dtype=float)
        self.advantage_ema = np.zeros(self.n_experts, dtype=float)
        self.load_ema = np.full(self.n_experts, 1.0 / self.n_experts, dtype=float)
        self.default_reward = np.full(self.n_experts, 0.30, dtype=float)
        self.prototype = np.zeros((self.n_experts, self.n_features), dtype=float)
        self.prototype_mass = np.zeros(self.n_experts, dtype=float)
        self.last_target = np.full(self.n_experts, 1.0 / self.n_experts, dtype=float)
        self.last_bias = np.zeros(self.n_experts, dtype=float)
        self.last_load = np.zeros(self.n_experts, dtype=float)
        self.score_clip_events = 0

    def _posterior_scores(self, Phi: Array, rng: np.random.Generator, uncertainty: float) -> Array:
        Phi = np.asarray(Phi, dtype=float)
        scores = np.empty((len(Phi), self.n_experts), dtype=float)
        for e in range(self.n_experts):
            try:
                inv = np.linalg.inv(self.A[e])
            except np.linalg.LinAlgError:
                inv = np.linalg.pinv(self.A[e])
            mean = inv @ self.b[e]
            cov = 0.10 ** 2 * inv
            try:
                theta = rng.multivariate_normal(mean, cov, check_valid="ignore")
            except (ValueError, np.linalg.LinAlgError):
                theta = mean
            pred = Phi @ theta
            unc = np.sqrt(np.maximum(np.einsum("nd,dd,nd->n", Phi, inv, Phi), 0.0))
            scores[:, e] = pred + float(uncertainty) * unc
        return scores

    def _memory_affinity(self, Phi: Array) -> Array:

```

```

P = _safe_unit_rows(np.asarray(Phi, dtype=float))
Q = _safe_unit_rows(self.prototypes + 1e-12)
sim = P @ Q.T
active = self.prototype_mass >= 2.0
sim[:, ~active] = 0.0
return sim

def _target_distribution(self, n: int, progress: float) -> Array:
    # The null residual (expert 0) is an always-available shared-only route.
    q0 = float(np.clip(0.42 + 0.18 * progress, 0.42, 0.60))
    quality = np.clip(self.advantage_ema[1:] / 0.10, -4.0, 4.0)
    quality -= np.max(quality) if len(quality) else 0.0
    expq = np.exp(quality / 1.35)
    soft = expq / max(float(expq.sum()), 1e-12)
    # Early in training every fine-grained expert receives a small probe quota;
    # later, proven specialists get the capacity while weak experts can go idle.
    if np.min(self.calls[1:]) < 2:
        floor = min(1.0 / max(n, 1), (1.0 - q0) / max(self.n_experts - 1, 1))
    else:
        floor = 0.0
    routed = (1.0 - q0) * (0.18 / max(self.n_experts - 1, 1) + 0.82 * soft)
    if floor > 0:
        routed = np.maximum(routed, floor)
        routed *= (1.0 - q0) / max(float(routed.sum()), 1e-12)
    q = np.r_[q0, routed]
    q /= q.sum()
    return q

@staticmethod
def _integer_quotas(q: Array, n: int) -> Array:
    raw = np.asarray(q, dtype=float) * int(n)
    quota = np.floor(raw).astype(int)
    remainder = int(n - quota.sum())
    if remainder > 0:
        order = np.argsort(-(raw - quota), kind="mergesort")
        quota[order[:remainder]] += 1
    elif remainder < 0:
        order = np.argsort(raw - quota, kind="mergesort")
        for e in order:
            take = min(quota[e], -remainder)
            quota[e] -= take
            remainder += take
            if remainder == 0:
                break
    return quota

def _quantile_bias(self, raw: Array, q: Array, sweeps: int = 4) -> Array:
    """Top-1 specialization of Kimi K3-style score-quantile balancing."""
    n, E = raw.shape
    bias = self.last_bias.copy()
    for _ in range(max(int(sweeps), 1)):
        adjusted = raw + bias[None, :]
        for e in range(E):
            other = adjusted.copy()
            other[:, e] = -np.inf
            cutoff = np.max(other, axis=1)
            margins = cutoff - raw[:, e]
            target = float(np.clip(q[e], 0.0, 1.0))

```

```

        if target <= 0.0:
            bias[e] = float(np.min(margins) - 2.0)
        elif target >= 1.0:
            bias[e] = float(np.max(margins) + 2.0)
        else:
            bias[e] = float(np.quantile(margins, target))
    bias -= float(np.median(bias))
    bias = np.clip(bias, -2.5, 2.5)
    return bias

def assign(
    self,
    Phi: Array,
    rng: np.random.Generator,
    progress: float,
    use_quantile: bool = True,
    use_memory: bool = True,
    use_specialization: bool = False,
) -> Tuple[Array, Array, Array, Array]:
    Phi = np.asarray(Phi, dtype=float)
    raw = self._posterior_scores(Phi, rng, uncertainty=0.08 if progress < 0.55 else 0.045)
    if use_memory:
        raw += 0.11 * self._memory_affinity(Phi)
    if use_specialization and np.count_nonzero(self.prototype_mass >= 2.0) >= 2:
        U = _safe_unit_rows(self.prototype + 1e-12)
        overlap = np.maximum(U @ U.T - np.eye(self.n_experts), 0.0)
        redundancy = overlap.max(axis=1)
        raw -= 0.06 * redundancy[None, :]
    # Shared/null route receives a small reliability prior, but routed experts
    # can readily overtake it when their conditional evidence is stronger.
    raw[:, 0] += 0.11 + 0.08 * progress

    # MuonClip-inspired robust score clipping: only extreme router logits are
    # capped; ordinary ranking geometry is unchanged.
    med = np.median(raw, axis=1, keepdims=True)
    mad = np.median(np.abs(raw - med), axis=1, keepdims=True) + 1e-9
    lo, hi = med - 5.0 * mad, med + 5.0 * mad
    clipped = np.clip(raw, lo, hi)
    self.score_clip_events += int(np.count_nonzero(np.abs(clipped - raw) > 1e-12))
    raw = clipped

    q = self._target_distribution(len(Phi), progress)
    quota = self._integer_quotas(q, len(Phi))
    if use_quantile:
        bias = self._quantile_bias(raw, q)
        adjusted = raw + bias[None, :]
        # Exact capacity assignment after quantile correction. This prevents a
        # few near-tied contexts from violating the global FE allocation plan.
        slots = np.repeat(np.arange(self.n_experts), quota)
        if len(slots) != len(Phi):
            raise RuntimeError("quantile quota does not match population size")
        rows, cols = linear_sum_assignment(-adjusted[:, slots])
        assignment = np.empty(len(Phi), dtype=int)
        assignment[rows] = slots[cols]
    else:
        bias = np.zeros(self.n_experts, dtype=float)
        adjusted = raw
        assignment = np.argmax(adjusted, axis=1).astype(int)

```

```

# Confidence controls routed residual amplitude, not FE count.
sorted_scores = np.sort(adjusted, axis=1)
margin = sorted_scores[:, -1] - sorted_scores[:, -2] if self.n_experts > 1 else np.ones(len(Phi))
gate = 1.0 / (1.0 + np.exp(-np.clip(margin, -8.0, 8.0)))
gate = np.clip(0.18 + 0.42 * gate, 0.18, 0.60)
gate[assignment == 0] = 0.0

load = np.bincount(assignment, minlength=self.n_experts).astype(float) / max(len(Phi), 1)
self.load_ema = 0.92 * self.load_ema + 0.08 * load
self.last_target = q
self.last_bias = bias
self.last_load = load
return assignment, gate, raw, bias

def update(
    self,
    phi: Array,
    expert: int,
    reward: float,
    baseline: float,
    dense_default: bool = False,
) -> float:
    phi = np.asarray(phi, dtype=float)
    e = int(expert)
    r = float(np.clip(reward, 0.0, 1.0))
    advantage = float(np.clip(r - baseline, -1.0, 1.0))
    target = float(np.clip(0.5 + advantage / 0.24, 0.0, 1.0))
    decay = 0.994
    eye = np.eye(self.n_features)
    self.A[e] = decay * self.A[e] + np.outer(phi, phi) + (1.0 - decay) * self.ridge * eye
    self.b[e] = decay * self.b[e] + phi * target
    self.calls[e] += 1
    self.reward_ema[e] = 0.94 * self.reward_ema[e] + 0.06 * r
    self.advantage_ema[e] = 0.94 * self.advantage_ema[e] + 0.06 * advantage
    self.default_reward[e] = 0.97 * self.default_reward[e] + 0.03 * r
    if r >= 0.42 or advantage >= 0.08:
        if self.prototype_mass[e] == 0:
            self.prototype[e] = phi
        else:
            self.prototype[e] = 0.92 * self.prototype[e] + 0.08 * phi
            self.prototype_mass[e] += 1.0

# Default-MoE-inspired weak dense feedback: unselected experts receive
# only their EMA default target. The small weight regularizes the router
# without pretending these are exact counterfactual evaluations.
if dense_default:
    w = 0.018
    for j in range(self.n_experts):
        if j == e:
            continue
        pseudo = float(np.clip(self.default_reward[j], 0.0, 1.0))
        self.A[j] = decay * self.A[j] + w * np.outer(phi, phi) + (1.0 - decay) * self.ridge * eye
        self.b[j] = decay * self.b[j] + w * phi * pseudo
    return advantage

def _lq_context_matrix(

```

```

progress: float,
stagnation: Array,
densities: Array,
med_density: float,
distress: float,
entropy: float,
assigned_scalar: Array,
F: Array,
W: Array,
ideal: Array,
nadir: Array,
role_reliability: float,
archive_fill: float,
) -> Array:
    n = len(F)
    stag = np.clip(np.loglp(np.asarray(stagnation, dtype=float)) / np.log(12.0), 0.0, 1.0)
    density = np.asarray(densities, dtype=float)
    density_ratio = np.clip(density / (med_density + 1e-9), 0.0, 4.0) / 4.0 if med_density > 0 else np.zeros(n)
    scalar_rel = np.clip(np.asarray(assigned_scalar, dtype=float) / (np.median(assigned_scalar) + 1e-12), 0.0, 4.0) / 4.0
    Y = np.maximum((np.asarray(F) - ideal) / (nadir - ideal + 1e-12), 0.0)
    Yn = _safe_unit_rows(Y + 1e-12)
    Wn = _safe_unit_rows(W + 1e-12)
    alignment = np.clip(np.sum(Yn * Wn, axis=1), 0.0, 1.0)
    Phi = np.column_stack([
        np.ones(n),
        np.full(n, progress),
        np.full(n, progress * progress),
        stag,
        density_ratio,
        np.full(n, distress),
        np.full(n, entropy),
        scalar_rel,
        alignment,
        np.full(n, float(np.clip(role_reliability, 0.0, 1.0))),
        np.full(n, float(np.clip(archive_fill, 0.0, 1.0))),
        stag * scalar_rel,
    ])
    return Phi.astype(float)

def _choose_distinct(rng: np.random.Generator, pool: Array, exclude: Sequence[int], n: int) -> Array:
    pool = np.asarray(pool, dtype=int)
    mask = ~np.isin(pool, np.asarray(list(exclude), dtype=int))
    cand = pool[mask]
    if len(cand) < n:
        cand = pool
    return np.asarray(rng.choice(cand, size=n, replace=len(cand) < n), dtype=int)

def _micro_residual(
    micro: int,
    i: int,
    X: Array,
    F: Array,
    W: Array,
    B: Array,
    archive: core.Archive,
    residual_steps: Sequence[Sequence[Array]],

```

```

sparse_target_x: Optional[Array],
progress: float,
rng: np.random.Generator,
problem: core.Problem,
ideal: Array,
nadir: Array,
) -> Array:
    """Fine-grained residual bank; expert 0 is the exact shared-only route."""
    micro = int(micro)
    d = problem.n_var
    span = problem.xu - problem.xl
    if micro == 0:
        return np.zeros(d, dtype=float)

    neigh = np.asarray(B[int(i)], dtype=int)
    local = neigh if len(neigh) >= 3 else np.arange(len(X))
    mu = 0.015 + 0.08 * (1.0 - progress) ** 2
    scores = core._mosaic_scalar(
        F[local], np.repeat(W[int(i)][None, :], len(local), axis=0),
        ideal, nadir, mu, theta=0.18, blend=0.0,
    )

    elite = local[np.argsort(scores, kind="mergesort")[: max(2, int(math.ceil(0.18 * len(local))))]]
    pbest = int(rng.choice(elite))

    if micro == 1: # pbest micro
        return (0.030 + 0.020 * (1.0 - progress)) * (X[pbest] - X[i])
    if micro == 2: # pbest macro
        return (0.080 + 0.040 * (1.0 - progress)) * (X[pbest] - X[i])
    if micro == 3: # local differential micro
        r1, r2 = _choose_distinct(rng, local, [i], 2)
        return 0.050 * (X[r1] - X[r2])
    if micro == 4: # global differential
        r1, r2 = _choose_distinct(rng, np.arange(len(X)), [i], 2)
        return (0.070 + 0.035 * (1.0 - progress)) * (X[r1] - X[r2])
    if micro == 5: # local archive tangent
        if len(archive.F) >= max(5, problem.n_obj + 1):
            AF = (archive.F - ideal) / (nadir - ideal + 1e-12)
            fi = (F[i] - ideal) / (nadir - ideal + 1e-12)
            idx = np.argsort(np.linalg.norm(AF - fi, axis=1), kind="mergesort")[: min(14, len(AF))]
            Z = (archive.X[idx] - np.mean(archive.X[idx], axis=0)) / (span + 1e-12)
            try:
                _, s, vt = np.linalg.svd(Z, full_matrices=False)
                k = max(1, min(problem.n_obj - 1, len(s)))
                return span * ((rng.normal(size=k) * (s[:k] + 0.01)) @ vt[:k]) * 0.050
            except np.linalg.LinAlgError:
                pass
            return np.zeros(d)
    if micro == 6: # reference-gap transport
        if sparse_target_x is not None:
            return (0.055 + 0.030 * (1.0 - progress)) * (sparse_target_x - X[i])
        return np.zeros(d)
    if micro == 7: # sparse Gaussian
        mask = rng.random(d) < min(0.20, 3.0 / max(d, 1))
        if not mask.any():
            mask[int(rng.integers(d))] = True
        out = np.zeros(d)
        out[mask] = rng.normal(size=int(mask.sum())) * span[mask] * (0.012 + 0.018 * (1.0 - progress))
        return out

```

```

if micro == 8: # sparse Cauchy
    mask = rng.random(d) < min(0.16, 2.5 / max(d, 1))
    if not mask.any():
        mask[int(rng.integers(d))] = True
    out = np.zeros(d)
    out[mask] = np.clip(rng.standard_cauchy(int(mask.sum())) , -4.0, 4.0) * span[mask] * (0.010 + 0.022 * (1.0 - progress))
    return out
if micro == 9: # exact-verified residual replay
    bank = [s for seq in residual_steps for s in seq[-16:]]
    if bank:
        return 0.35 * np.asarray(bank[int(rng.integers(len(bank)))], dtype=float)
    return np.zeros(d)
if micro == 10: # SBX residual relative to parent
    mate = int(rng.choice(local))
    sbx = core.sbx_one(X[i], X[mate], problem.xl, problem.xu, rng, eta=25.0)
    return 0.12 * (sbx - X[i])
if micro == 11: # low-variance / orthogonal escape
    Z = (X - np.mean(X, axis=0)) / (span + 1e-12)
    try:
        _, _, vt = np.linalg.svd(Z, full_matrices=False)
        q = vt[-min(2, len(vt)):]
        direction = rng.normal(size=len(q)) @ q
        return span * direction * (0.010 + 0.018 * (1.0 - progress))
    except np.linalg.LinAlgError:
        return np.zeros(d)
raise ValueError(f"unknown micro expert {micro}")

def run_mosaic_lqmoe(
    problem: core.Problem,
    seed: int,
    pop_size: int = 100,
    max_evals: int = 10100,
    T: int = 20,
    nr: int = 3,
    trace_every: int = 5,
    ablation: Optional[str] = None,
    recovery_fraction: float = 0.22,
    recovery_cooldown: int = 10,
    max_recoveries: int = 3,
    collapse_ratio: float = 0.35,
    collapse_start: float = 0.70,
    collapse_patience: int = 4,
    extension=None,
) -> core.RunResult:
    """MOSAIC-LQMoe with a frozen R3 shared backbone and routed micro-experts.

    Flags (join with '+'):
    no_memory      disable Exact Coverage Memory entirely
    no_recovery     keep memory/ART but disable collapse intervention
    no_o5           disable Archive-Reuse Transport expert entirely
    no_routine_o5  disable routine ART; allow it only through soft recovery
    no_role_factorization use ordinary archive-DE instead of role-separated crossover
    soft_recovery   convert collapse intervention into an exact-verified ART queue
    hard_recovery   directly inject exact memory points (diagnostic comparator)
    secant          enable reliable Secant-Repulsive Transport in operator 2
    no_repulsion    with secant, remove objective-space RBF repulsion
    discount_router decay posterior evidence toward its prior each generation

```

```

fixed_router    uniformly sample standard experts 0..4
force_coverage  frozen v4 periodic NSGA-III archive repair comparator
coordinate_tunnel enable role-gated coordinate basin tunneling (operator 6)
moe_residual    enable shared-backbone + fine-grained residual MoE
no_qb           disable loss-free quantile balancing in residual router
no_memory_route disable successful-context expert memory
no_latent       disable low-rank latent routed residuals
no_energy_clip  disable routed proposal-energy clipping
dense_default   weak EMA default feedback to non-selected micro-experts
specialization  penalize overlapping expert context prototypes
coordinate_qb   quantile-balanced dynamic-k routing over 06 variable experts
coordinate_dynamic_k preserve R3 pilots, then evidence-scale 06 expert batch size
coordinate_shared_qb preserve R3 shared pilots, then quantile-route residual coordinate experts
coordinate_evidence_qb preserve R3 except under low-occupancy evidence, then quantile-route residual experts
coordinate_geometry_qb additionally requires non-planar PF evidence before quantile residual routing
coordinate_consensus_qb requires two consistent exact pilots before geometry-aware quantile routing
"""

flags = set() if ablation is None else {s for s in str(ablation).split("+") if s}
rng, X, F = core.initialize(problem, pop_size, seed)
# Independent MoE stream: adding a router must not perturb the frozen R3
# random trajectory when the shared/null route is selected.
moe_rng = np.random.default_rng(int(seed) + 104729)
W = core.dirichlet_reference_points(pop_size, problem.n_obj, seed=601)
dW = ((W[:, None, :] - W[None, :, :]) ** 2).sum(axis=2)
B = np.argsort(dW, axis=1)[: , : min(T, pop_size)]
init_assoc, _ = core.associate_to_reference(F, W)
init_occ = float(np.count_nonzero(np.bincount(init_assoc, minlength=pop_size)) / pop_size)
alignment_latch = bool(problem.n_obj > 2 and init_occ < 0.25)

ideal = F.min(axis=0)
nadir = np.maximum(F.max(axis=0), ideal + 1e-9)
archive = core.Archive(max_size=5 * pop_size)
archive.update(X, F)
memory = ExactCoverageMemory(capacity=8 * pop_size, per_cell=3)
if "no_memory" not in flags:
    memory.add(X, F, 0, W, ideal, nadir)

# Preserve the frozen five-expert router exactly. ART (05) is a
# conservative challenger with its own posterior; adding an expert must not
# perturb the core router merely by changing the dimension of a Beta draw.
bandit = DiscountedPortfolio(n_states=12, n_ops=5)
o5_a = np.ones(12, dtype=float)
o5_b = np.full(12, 3.0, dtype=float)
o6_a = np.ones(12, dtype=float)
o6_b = np.full(12, 2.5, dtype=float)
stagnation = np.zeros(pop_size, dtype=int)
successful_steps: List[List[Array]] = [[] for _ in range(7)]
evals, gen = pop_size, 0
trace: List[Dict[str, float]] = []
op_counts = np.zeros(7, dtype=int)
op_rewards = np.zeros(7, dtype=float)
secant_calls = secant_used = 0
secant_rel_sum = 0.0
role_probability: Optional[Array] = None
role_reliability = 0.0
role_diagnostics: Dict[str, float] = {}
recovery_events: List[Dict[str, object]] = []
rescue_queue: List[int] = []

```



```

last_recovery = -10**9
collapse_streak = 0
max_occupancy = max(init_occ, 1.0 / pop_size)
occupancy_history: List[float] = []
archive_history: List[int] = []
scalar_history: List[float] = []
probe_index = np.zeros(problem.n_var, dtype=int)
probe_trials = np.zeros(problem.n_var, dtype=int)
probe_success = np.zeros(problem.n_var, dtype=int)
learned_values = np.full(problem.n_var, np.nan, dtype=float)
coordinate_events: List[Dict[str, object]] = []
coordinate_candidates: Optional[Array] = None
coordinate_pilot_rewards: List[float] = []
coordinate_expand_threshold = 0.35
coordinate_pilot_budget_current = 2
coordinate_batch_history: List[Dict[str, object]] = []

# DeepSeek/Kimi-inspired second routing layer. The legacy R3 proposal is the
# always-on shared path; one of 11 fine-grained residual experts, or the exact
# null residual, is selected without additional objective evaluations.
n_micro = 12
micro_router = LatentQuantileRouter(n_experts=n_micro, n_features=12)
micro_counts = np.zeros(n_micro, dtype=int)
micro_rewards = np.zeros(n_micro, dtype=float)
micro_advantages = np.zeros(n_micro, dtype=float)
micro_clip_scales: List[float] = []
residual_successful_steps: List[List[Array]] = [[] for _ in range(n_micro)]
shared_reward_ema = np.full(12, 0.30, dtype=float)
micro_assignment_history: List[List[int]] = []
micro_target_history: List[List[float]] = []
micro_load_history: List[List[float]] = []
radial_dispersion_history: List[float] = []
geometry_score_history: List[float] = []
radial_failure_streak = 0

while evals < max_evals:
    progress = evals / max_evals
    densities = np.zeros(pop_size)
    for j in range(pop_size):
        densities[j] = core._local_density(archive.F, F[j])[0] if len(archive.F) else 0.0
    med_density = float(np.median(densities))

    # Frozen v4 search association is preserved for standard experts and
    # reward credit. A separate shared longitudinal frame is used only by
    # the collapse detector and exact-memory reconstruction.
    assoc_search, _ = core.associate_to_reference(F, W)
    search_counts = np.bincount(assoc_search, minlength=pop_size)
    sparse_cells = np.flatnonzero(search_counts == search_counts.min())
    assoc_detect, _ = _associate_fixed(F, W, ideal, nadir)
    occupancy, entropy, detect_counts = _occupancy_entropy(assoc_detect, pop_size)
    max_occupancy = max(max_occupancy, occupancy)
    threshold = max(0.08, collapse_ratio * max_occupancy)
    distress = float(np.clip((threshold - occupancy) / (threshold + 1e-12), 0.0, 1.0))

    # Predict ART's exact anchor contribution once per generation. This is
    # the optimization analogue of contribution regression in D2DMoE: the
    # router asks whether its specialist has a measurable job to do before
    # spending samples on it.

```

```

o5_target_cell: Optional[int] = None
o5_advantage_gen = 0.0
if (
    "no_o5" not in flags
    and "no_routine_o5" not in flags
    and "no_memory" not in flags
    and len(memory.F) > pop_size
    and progress > 0.22
    and len(sparse_cells) > 0
):
    advantages = np.asarray([
        memory.best_anchor_advantage(int(c), F, W, ideal, nadir)
        for c in sparse_cells
    ])
    bp = int(np.argmax(advantages))
    o5_target_cell = int(sparse_cells[bp])
    o5_advantage_gen = float(advantages[bp])

if (
    "no_o5" not in flags
    and "no_role_factorization" not in flags
    and "no_memory" not in flags
    and len(memory.F) >= max(40, 2 * problem.n_var)
    # RCBT needs role information earlier than coverage recovery.
    # Without the coordinate_tunnel ablation, preserve R2 timing.
    and progress >= (0.28 if "coordinate_tunnel" in flags else min(0.60, max(collapse_start - 0.10, 0.0)))
    and (gen % 4 == 0 or role_probability is None)
):
    role_probability, role_reliability, role_diagnostics = _infer_variable_roles(
        memory, problem, ideal, nadir, W
    )

# Direction-conditioned convergence diagnostic. Unlike occupancy
# distress, this detects a healthy spread whose assigned STCH values have
# stopped improving. It controls a separate specialist and never changes
# the frozen five-expert posterior.
mu_diag = 0.008 + 0.10 * (1.0 - progress) ** 2
theta_diag = 0.10 + 0.22 * progress
assigned_scalar = core._mosaic_scalar(F, W, ideal, nadir, mu_diag, theta=theta_diag, blend=0.0)
scalar_level = float(np.median(assigned_scalar))
scalar_history.append(scalar_level)
if len(scalar_history) >= 7:
    old_level = float(np.median(scalar_history[-7:-4]))
    new_level = float(np.median(scalar_history[-3:]))
    scalar_progress = (old_level - new_level) / (abs(old_level) + 1e-12)
else:
    scalar_progress = 1.0

# Scale-free local PF geometry diagnostic. A positive value means an
# L2/spherical shell explains the current front better than an L1/simplex
# shell; a negative value favors a flat simplex-like front. This is only
# computed from already evaluated points.
Y_geom = np.maximum((F - ideal) / (nadir - ideal + 1e-12), 0.0)
l1_geom = np.sum(Y_geom, axis=1)
l2_geom = np.linalg.norm(Y_geom, axis=1)
cv1_geom = float(np.std(l1_geom) / (np.mean(l1_geom) + 1e-12))
cv2_geom = float(np.std(l2_geom) / (np.mean(l2_geom) + 1e-12))
geometry_score = float(cv1_geom - cv2_geom)
geometry_score_history.append(geometry_score)

```

```

coordinate_plans: List[Tuple[int, int]] = []
if (
    "coordinate_tunnel" in flags
    and role_probability is not None
    and role_reliability >= 0.28
    and progress >= 0.40
    and gen % 2 == 0
    and not rescue_queue
):
    # Freeze one role-derived candidate set. Recomputing the role mask
    # could otherwise restart pilots and silently spend more than promised.
    if coordinate_candidates is None:
        conv_all = np.flatnonzero(role_probability <= 0.50)
        if len(conv_all) >= max(2, problem.n_var // 4):
            order_conv = conv_all[np.argsort(role_probability[conv_all], kind="mergesort")]
            coordinate_candidates = order_conv[: min(12, len(order_conv))].copy()

conv = np.empty(0, dtype=int) if coordinate_candidates is None else coordinate_candidates
if len(conv):
    target_score = assigned_scalar / (np.median(assigned_scalar) + 1e-12)
    target_order = np.argsort(-(stagnation + 3.0 * target_score), kind="mergesort")
    if "coordinate_evidence_qb" in flags or "coordinate_geometry_qb" in flags or "coordinate_consensus_qb" in
↳ flags:
        # Evidence-triggered expert expansion. The two exact R3
        # shared pilots always keep their original order. When the
        # exact-memory reference occupancy is healthy, execution
        # falls back bit-for-bit to the frozen R3 coordinate route.
        # Only a low-occupancy + strong-pilot state unlocks the
        # quantile-balanced residual experts.
        untested = conv[probe_trials[conv] == 0]
        total_trials = int(probe_trials[conv].sum())
        pilot_budget = min(2, len(conv))
        coordinate_pilot_budget_current = pilot_budget
        pilot_due = total_trials < pilot_budget
        pilot_ready = (
            total_trials >= pilot_budget
            and len(coordinate_pilot_rewards) >= pilot_budget
        )
        strong_rewards = (
            np.asarray(coordinate_pilot_rewards[:pilot_budget], dtype=float)
            if pilot_ready else np.empty(0, dtype=float)
        )
    occ_ratio = float(role_diagnostics.get("occupied_cells", pop_size)) / max(pop_size, 1)
    structural_evidence = occ_ratio < 0.80
    if "coordinate_geometry_qb" in flags or "coordinate_consensus_qb" in flags:
        # Kimi-style expert expansion is reserved for a curved or
        # irregular shell. On a clearly planar/simplex front, the
        # frozen role order is more reliable than memory re-ranking.
        structural_evidence = structural_evidence and geometry_score > -0.02
    # The frozen shared path uses its original max-pilot gate.
    # Routed residual experts may use a stricter consensus gate,
    # but a failed residual gate must never disable the shared
    # expert's own continuation (DeepSeek Keep-Routing analogue).
    r3_expand_evidence = bool(
        pilot_ready
        and len(untested) > 0
        and np.max(strong_rewards) >= coordinate_expand_threshold
    )

```

```

if "coordinate_consensus_qb" in flags:
    reward_evidence = bool(
        pilot_ready
        and len(untested) > 0
        and np.min(strong_rewards) >= coordinate_expand_threshold
    )
else:
    reward_evidence = r3_expand_evidence
strong_evidence = reward_evidence
if pilot_due and len(untested):
    coordinate_plans.append((int(target_order[0]), int(untested[0])))
    k_now = 1
    coord_entropy = 0.0
    mode_now = "shared_pilot"
elif structural_evidence and strong_evidence:
    rs = _coordinate_expert_scores(
        untested, role_probability, memory, problem, ideal, nadir,
        probe_trials, probe_success,
    )
    rs = rs + 1e-8 * moe_rng.normal(size=len(rs))
    ranked, k_entropy, coord_entropy = _dynamic_coordinate_batch(
        untested, rs, probe_trials, max_batch=min(3, len(untested))
    )
    n_strong = int(np.count_nonzero(strong_rewards >= coordinate_expand_threshold))
    evidence_cap = 1 + int(n_strong >= 2 or np.max(strong_rewards) >= 0.60)
    k_now = int(min(len(ranked), k_entropy, evidence_cap, 2))
    for qidx, jvar in enumerate(ranked[:k_now]):
        target = int(target_order[qidx % len(target_order)])
        coordinate_plans.append((target, int(jvar)))
    mode_now = "quantile_residual"
elif r3_expand_evidence:
    # Residual gate closed: preserve the original R3 route and
    # spend exactly one coordinate FE on the first unttested
    # role-ordered variable.
    coordinate_plans.append((int(target_order[0]), int(unttested[0])))
    k_now = 1
    coord_entropy = 0.0
    mode_now = "frozen_r3_fallback"
else:
    k_now = 0
    coord_entropy = 0.0
    mode_now = "null_route"
coordinate_batch_history.append({
    "gen": int(gen), "evals": int(evals),
    "dynamic_k": int(k_now), "entropy": float(coord_entropy),
    "scheduled": [int(j) for _, j in coordinate_plans],
    "pilot_budget": int(coordinate_pilot_budget_current),
    "mode": mode_now, "keep_route": True,
    "occupancy_ratio": float(occ_ratio),
    "structural_evidence": bool(structural_evidence),
    "geometry_score": float(geometry_score),
    "consensus_gate": bool("coordinate_consensus_qb" in flags),
})
elif "coordinate_shared_qb" in flags:
    # DeepSeek shared-expert isolation + Kimi quantile balancing:
    # the frozen R3 two-pilot path is always executed first, in
    # exactly the same variable order. Routed coordinate experts
    # are allowed only as a residual after exact pilot evidence.

```

```

    untested = conv[probe_trials[conv] == 0]
    total_trials = int(probe_trials[conv].sum())
    pilot_budget = min(2, len(conv))
    coordinate_pilot_budget_current = pilot_budget
    pilot_due = total_trials < pilot_budget
    pilot_ready = (
        total_trials >= pilot_budget
        and len(coordinate_pilot_rewards) >= pilot_budget
    )
    strong_rewards = (
        np.asarray(coordinate_pilot_rewards[:pilot_budget], dtype=float)
        if pilot_ready else np.empty(0, dtype=float)
    )
    residual_gate = bool(
        pilot_ready
        and len(untested) > 0
        and np.max(strong_rewards) >= coordinate_expand_threshold
    )
    if pilot_due and len(untested):
        # One exact shared pilot at a time; this preserves the
        # R3 route and its random/evaluation trajectory.
        coordinate_plans.append((int(target_order[0]), int(untested[0])))
        k_now = 1
        coord_entropy = 0.0
    elif residual_gate:
        rs = _coordinate_expert_scores(
            untested, role_probability, memory, problem, ideal, nadir,
            probe_trials, probe_success,
        )
        rs = rs + 1e-8 * moe_rng.normal(size=len(rs))
        ranked, k_entropy, coord_entropy = _dynamic_coordinate_batch(
            untested, rs, probe_trials, max_batch=min(3, len(untested))
        )
        # Evidence-triggered capacity. One strong pilot buys one
        # routed expert; two strong pilots can buy a second. A
        # very high single reward may also buy the second. This
        # is a strict FE budget, not free parallel computation.
        n_strong = int(np.count_nonzero(strong_rewards >= coordinate_expand_threshold))
        evidence_cap = 1 + int(n_strong >= 2 or np.max(strong_rewards) >= 0.60)
        k_now = int(min(len(ranked), k_entropy, evidence_cap, 2))
        for qidx, jvar in enumerate(ranked[k_now:]):
            target = int(target_order[qidx % len(target_order)])
            coordinate_plans.append((target, int(jvar)))
    else:
        k_now = 0
        coord_entropy = 0.0
    coordinate_batch_history.append({
        "gen": int(gen), "evals": int(evals),
        "dynamic_k": int(k_now), "entropy": float(coord_entropy),
        "scheduled": [int(j) for _, j in coordinate_plans],
        "pilot_budget": int(coordinate_pilot_budget_current),
        "mode": "shared_pilot_then_quantile_residual",
        "keep_route": True,
    })
elif "coordinate_dynamic_k" in flags:
    # Preserve the exact frozen R3 first-two pilot order. Only
    # after those exact rewards prove usefulness do we expand the
    # per-generation coordinate-expert budget. This mirrors

```

```

# Kimi-style dynamic-k while protecting the shared path.
untested = conv[probe_trials[conv] == 0]
total_trials = int(probe_trials[conv].sum())
pilot_budget = min(2, len(conv))
coordinate_pilot_budget_current = pilot_budget
pilot_due = total_trials < pilot_budget
expand_due = (
    total_trials >= pilot_budget
    and len(coordinate_pilot_rewards) >= pilot_budget
    and max(coordinate_pilot_rewards[:pilot_budget]) >= coordinate_expand_threshold
    and len(untested) > 0
)
if pilot_due and len(untested):
    coordinate_plans.append((int(target_order[0]), int(untested[0])))
    k_now = 1
elif expand_due:
    strong = int(np.count_nonzero(
        np.asarray(coordinate_pilot_rewards[:pilot_budget]) >= coordinate_expand_threshold
    ))
    k_now = min(len(untested), 1 + strong)
    # Role ordering is the frozen R3 ordering; quantile-based
    # capacity changes how many, not which, experts are trusted.
    for qidx, jvar in enumerate(untested[:k_now]):
        target = int(target_order[qidx % len(target_order)])
        coordinate_plans.append((target, int(jvar)))
else:
    k_now = 0
coordinate_batch_history.append({
    "gen": int(gen), "evals": int(eval),
    "dynamic_k": int(k_now), "entropy": None,
    "scheduled": [int(j) for _, j in coordinate_plans],
    "pilot_budget": int(coordinate_pilot_budget_current),
    "mode": "shared_pilot_then_dynamic_k",
})
elif "coordinate_qb" in flags:
    cs = _coordinate_expert_scores(
        conv, role_probability, memory, problem, ideal, nadir,
        probe_trials, probe_success,
    )
    # Independent, tiny jitter only resolves exact score ties.
    cs = cs + 1e-8 * moe_rng.normal(size=len(cs))
    chosen_vars, dynamic_k, coord_entropy = _dynamic_coordinate_batch(
        conv, cs, probe_trials, max_batch=min(4, len(conv))
    )
    coordinate_pilot_budget_current = max(2, dynamic_k)
    total_trials = int(probe_trials[conv].sum())
    pilot_due = total_trials < coordinate_pilot_budget_current
    expand_due = (
        total_trials >= coordinate_pilot_budget_current
        and len(coordinate_pilot_rewards) >= coordinate_pilot_budget_current
        and max(coordinate_pilot_rewards[:coordinate_pilot_budget_current]) >= coordinate_expand_threshold
    )
    if pilot_due or expand_due:
        if not pilot_due:
            # After a successful pilot phase, continue one expert
            # at a time, ranked by exact posterior contribution.
            remaining = conv[probe_trials[conv] == 0]
            if len(remaining):

```

```

        rs = _coordinate_expert_scores(
            remaining, role_probability, memory, problem,
            ideal, nadir, probe_trials, probe_success,
        )
        chosen_vars = remaining[np.argsort(-rs, kind="mergesort")[:1]]
    else:
        chosen_vars = np.empty(0, dtype=int)
    else:
        need = max(0, coordinate_pilot_budget_current - total_trials)
        chosen_vars = chosen_vars[:need]
        for qidx, jvar in enumerate(chosen_vars):
            target = int(target_order[qidx % len(target_order)])
            coordinate_plans.append((target, int(jvar)))
        coordinate_batch_history.append({
            "gen": int(gen), "evals": int(evals),
            "dynamic_k": int(dynamic_k), "entropy": float(coord_entropy),
            "scheduled": [int(j) for _, j in coordinate_plans],
            "pilot_budget": int(coordinate_pilot_budget_current),
        })
    else:
        untested = conv[probe_trials[conv] == 0]
        total_trials = int(probe_trials[conv].sum())
        pilot_budget = min(2, len(conv))
        coordinate_pilot_budget_current = pilot_budget
        pilot_due = total_trials < pilot_budget
        expand_due = (
            total_trials >= pilot_budget
            and len(coordinate_pilot_rewards) >= pilot_budget
            and max(coordinate_pilot_rewards[:pilot_budget]) >= coordinate_expand_threshold
            and len(untested) > 0
        )
        if (pilot_due or expand_due) and len(untested):
            coordinate_plans.append((int(target_order[0]), int(untested[0])))

if problem.n_obj <= 2:
    scalar_blend = 0.0
elif alignment_latch:
    scalar_blend = 1.0
else:
    scalar_blend = 0.0

# Scale-free radial-dispersion diagnostic. A large ratio means a few
# directions have reached a much better radial basin while most of the
# population remains trapped far away -- the exact failure RCBT and the
# routed residual bank are intended to solve.
# Use a shift- and scale-equivariant radial mass ratio rather than the
# current min-max normalized radius. The latter can hide severe multimodal
# distance-variable traps by normalizing the trapped front to [0,1].
span_obj = np.maximum(nadir - ideal, 1e-12)
radial = np.sum(np.maximum(F - ideal, 0.0) + 0.01 * span_obj, axis=1)
radial_dispersion = float(np.median(radial) / (np.quantile(radial, 0.10) + 1e-12))
radial_dispersion_history.append(radial_dispersion)
radial_failure_streak = radial_failure_streak + 1 if radial_dispersion >= 3.0 else 0

micro_assign = np.zeros(pop_size, dtype=int)
micro_gate = np.zeros(pop_size, dtype=float)
micro_context = np.zeros((pop_size, 12), dtype=float)
latent_basis = np.empty((0, problem.n_var), dtype=float)

```

```

if "moe_residual" in flags:
    micro_context = _lq_context_matrix(
        progress, stagnation, densities, med_density, distress, entropy,
        assigned_scalar, F, W, ideal, nadir, role_reliability,
        len(archive.F) / max(archive.max_size, 1),
    )
    # A routed residual is a challenger, never a compulsory replacement
    # for the reliable shared path. Quantile balancing is performed only
    # across contexts for which a diagnosed radial failure exists; this
    # prevents capacity targets from spending residual routes on easy rows.
    eligible = (progress >= 0.25) & (
        (radial_failure_streak >= 3) & (stagnation >= 3)
    )
    idx_eligible = np.flatnonzero(eligible)
    if len(idx_eligible):
        ae, ge, _, _ = micro_router.assign(
            micro_context[idx_eligible], moe_rng, progress,
            use_quantile="no_qb" not in flags,
            use_memory="no_memory_route" not in flags,
            use_specialization="specialization" in flags,
        )
        micro_assign[idx_eligible] = ae
        micro_gate[idx_eligible] = ge
    if "no_latent" not in flags:
        latent_basis = _build_latent_basis(
            X, list(successful_steps[:5]) + list(residual_successful_steps),
            problem.xu - problem.xl, problem.n_obj, max_rank=10,
        )
    micro_assignment_history.append(np.bincount(micro_assign, minlength=n_micro).astype(int).tolist())
    micro_target_history.append(micro_router.last_target.tolist())
    micro_load_history.append(micro_router.last_load.tolist())

for i in rng.permutation(pop_size):
    if evals >= max_evals:
        break
    state = core._mosaic_state(progress, int(stagnation[i]), float(densities[i]), med_density)
    forced_rescue = bool("soft_recovery" in flags and rescue_queue and "no_o5" not in flags)
    forced_target = int(rescue_queue.pop(0)) if forced_rescue else None
    forced_coordinate = bool(coordinate_plans and not forced_rescue)
    coordinate_variable: Optional[int] = None
    coordinate_value: Optional[float] = None
    if forced_rescue:
        op = 5
    elif forced_coordinate:
        op = 6
        forced_target, coordinate_variable = coordinate_plans.pop(0)
    elif "fixed_router" in flags:
        op = int(rng.integers(5))
    else:
        op = bandit.choose(state, rng, epsilon=0.10 if progress < 0.5 else 0.06)

    # D2DMoE-inspired contribution routing: ART is not selected merely
    # because its load is low. Its best exact memory anchor must have a
    # measurable scalar advantage for a currently sparse cell, and its
    # conservative posterior must beat the selected core expert.
    target_cell = int(forced_target) if forced_target is not None else int(i)
    if (
        not forced_rescue

```



```

    and o5_target_cell is not None
    and (distress > 0.08 or stagnation[i] >= 8)
    and o5_advantage_gen > 0.025
):
    target_cell = int(o5_target_cell)
    q5 = float(rng.beta(o5_a[state], o5_b[state]))
    core_mean = float(bandit.a[state, op] / (bandit.a[state, op] + bandit.b[state, op]))
    predicted = float(np.clip(o5_advantage_gen / 0.12, 0.0, 1.0))
    o5_score = 0.55 * q5 + 0.45 * predicted
    if o5_score > core_mean + 0.06:
        op = 5

# A specialist routed to reference cell c must also be verified in
# c's neighborhood. Earlier prototypes generated for c but compared
# only against the arbitrary loop index i, which silently discarded
# otherwise useful rescue children.
search_i = int(target_cell) if op in (5, 6) else int(i)
parent = X[search_i].copy()
parentF = F[search_i].copy()
if op == 2 and len(archive.F) >= 3 and len(sparse_cells):
    # Preserve v4's stochastic sparse target for the original 02.
    target_cell = int(rng.choice(sparse_cells))
sparse_target_x: Optional[Array] = None
if op == 2 and len(archive.F) >= 3 and len(sparse_cells):
    az, an = archive.F.min(axis=0), archive.F.max(axis=0)
    AY = np.maximum((archive.F - az) / (an - az + 1e-12), 0.0)
    AYn = AY / (np.linalg.norm(AY, axis=1, keepdims=True) + 1e-12)
    wt = W[target_cell] / (np.linalg.norm(W[target_cell]) + 1e-12)
    sparse_target_x = archive.X[int(np.argmax(AYn @ wt))]

micro_op = 0
micro_phi = micro_context[int(i)].copy() if "moe_residual" in flags else np.zeros(12)
shared_child: Optional[Array] = None
routed_delta = np.zeros(problem.n_var, dtype=float)
clip_scale = 1.0

if op == 5:
    child = _archive_transport_child(
        search_i, target_cell, X, F, W, B, memory, ideal, nadir, progress, rng, problem,
        role_probability=role_probability, role_reliability=role_reliability
    )
    if child is None:
        op = 0
        search_i = int(i)
        parent = X[search_i].copy()
elif op == 6 and role_probability is not None and coordinate_variable is not None:
    child, coordinate_value = _coordinate_tunnel_child(
        target_cell, coordinate_variable, X, F, W, ideal, nadir, problem,
        probe_index, learned_values, role_probability
    )
else:
    child = None

if child is None:
    child = core._mosaic_operator(
        int(op), search_i, X, F, W, B, archive, successful_steps, progress,
        rng, problem, sparse_target_x=sparse_target_x, scalar_blend=scalar_blend
    )

```

```

if op == 2 and "secant" in flags:
    secant_calls += 1
    residual, reliability, _ = _secant_repulsive_delta(
        search_i, X, F, archive, W, ideal, nadir, progress, problem, rng,
        use_repulsion="no_repulsion" not in flags,
    )
    secant_rel_sum += reliability
    if reliability >= 0.28:
        ramp = float(np.clip((progress - 0.12) / 0.35, 0.0, 1.0))
        child = np.clip(child + ramp * reliability * residual, problem.xl, problem.xu)
        secant_used += 1

shared_child = child.copy()
# Fine-grained residual MoE is applied only to the frozen core experts.
# 05/06 already are evidence-gated failure specialists with separate FE
# semantics and are deliberately left untouched.
if "moe_residual" in flags and op < 5:
    micro_op = int(micro_assign[int(i)])
    routed_delta = _micro_residual(
        micro_op, search_i, X, F, W, B, archive, residual_successful_steps,
        sparse_target_x, progress, moe_rng, problem, ideal, nadir,
    )
    if micro_op != 0 and "no_latent" not in flags:
        routed_delta = _project_routed_residual(
            routed_delta, latent_basis, problem.xu - problem.xl, retain=0.18
        )
    if micro_op != 0 and "no_energy_clip" not in flags:
        cap = 0.010 + 0.025 * (1.0 - progress) + 0.012 * float(stagnation[search_i] >= 6)
        routed_delta, clip_scale = _proposal_energy_clip(
            routed_delta, problem.xu - problem.xl, cap
        )
    child = np.clip(
        shared_child + float(micro_gate[int(i)]) * routed_delta,
        problem.xl, problem.xu,
    )
    micro_clip_scales.append(float(clip_scale))

extension_ticket = None
if extension is not None:
    child, extension_ticket = extension.propose(
        child, parent, parentF, X, F, W, search_i,
        ideal, nadir, progress, evals, op, stagnation[search_i]
    )
    child = np.clip(child, problem.xl, problem.xu)
childF = problem.evaluate(child)[0]
evals += 1
ideal = np.minimum(ideal, childF)
pooled = np.vstack([F, archive.F, childF[None, :]]) if len(archive.F) else np.vstack([F, childF])
nadir = np.maximum(np.quantile(pooled, 0.95, axis=0), ideal + 1e-9)
entered, novelty = archive.update(child, childF)
if "no_memory" not in flags:
    memory.add(child, childF, gen, W, ideal, nadir)

neigh = B[search_i]
replaced, max_rel_improve = 0, 0.0
for j in rng.permutation(neigh):
    mu = 0.008 + 0.10 * (1.0 - progress) ** 2
    theta = 0.10 + 0.22 * progress

```

```

oldg = core._mosaic_scalar(
    F[j][None, :], W[j][None, :], ideal, nadir, mu, theta=theta, blend=scalar_blend
)[0]
newg = core._mosaic_scalar(
    childF[None, :], W[j][None, :], ideal, nadir, mu, theta=theta, blend=scalar_blend
)[0]
rel = float((oldg - newg) / (abs(oldg) + 1e-12))
verify = bool(newg <= oldg or (entered and novelty > 0.45 and newg <= oldg * 1.015))
if verify:
    X[j], F[j] = childF.copy(), childF.copy()
    stagnation[j] = 0
    replaced += 1
    max_rel_improve = max(max_rel_improve, max(rel, 0.0))
    if replaced >= nr:
        break
if replaced == 0:
    stagnation[search_i] += 1
elif extension_ticket is None:
    core_step = (shared_child if shared_child is not None else child) - parent
    successful_steps[op].append(core_step)
    if len(successful_steps[op]) > 64:
        successful_steps[op] = successful_steps[op][-64:]
    if "moe_residual" in flags and op < 5 and micro_op != 0:
        residual_successful_steps[micro_op].append(child - shared_child)
        if len(residual_successful_steps[micro_op]) > 48:
            residual_successful_steps[micro_op] = residual_successful_steps[micro_op][-48:]

r_conv = float(np.clip(max_rel_improve / 0.08, 0.0, 1.0))
r_rep = float(np.clip(replaced / max(nr, 1), 0.0, 1.0))
r_arc = 0.25 + 0.75 * novelty if entered else 0.0
# Preserve frozen v4 coverage credit so the no-05/no-recovery
# ablation is behavior-identical before a reconstruction event.
cz = np.minimum(F.min(axis=0), childF)
cn = np.maximum(F.max(axis=0), childF)
cy = np.maximum((childF - cz) / (cn - cz + 1e-12), 0.0)
cyn = cy / (np.linalg.norm(cy) + 1e-12)
c = int(np.argmax((W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)) @ cyn))
r_cover = 1.0 if entered and search_counts[c] == 0 else 0.0
reward = float(np.clip(0.42*r_conv + 0.20*r_rep + 0.23*r_arc + 0.15*r_cover, 0.0, 1.0))
if extension_ticket is not None:
    extension.observe(extension_ticket, childF, entered, replaced)
if "fixed_router" not in flags and extension_ticket is None:
    if op == 5:
        o5_a[state] += reward
        o5_b[state] += 1.0 - reward
    elif op == 6:
        o6_a[state] += reward
        o6_b[state] += 1.0 - reward
    else:
        bandit.update(state, op, reward)

if "moe_residual" in flags and op < 5:
    baseline = float(shared_reward_ema[state])
    advantage = micro_router.update(
        micro_phi, micro_op, reward, baseline,
        dense_default="dense_default" in flags,
    )
# Only actual null/shared-only routes update the counterfactual

```

```

        # baseline; routed outcomes cannot rewrite the control estimate.
        if micro_op == 0:
            shared_reward_ema[state] = 0.94 * shared_reward_ema[state] + 0.06 * reward
            micro_counts[micro_op] += 1
            micro_rewards[micro_op] += reward
            micro_advantages[micro_op] += advantage

    if op == 6 and coordinate_variable is not None and coordinate_value is not None:
        jv = int(coordinate_variable)
        probe_trials[jv] += 1
        accepted_probe = bool(replaced > 0 and reward >= 0.08)
        if accepted_probe:
            probe_success[jv] += 1
            learned_values[jv] = float(coordinate_value)
        if len(coordinate_pilot_rewards) < max(coordinate_pilot_budget_current, 1):
            coordinate_pilot_rewards.append(float(reward))
        coordinate_events.append({
            "gen": gen, "evals": evals, "cell": int(search_i), "variable": jv,
            "value": float(coordinate_value), "accepted": accepted_probe,
            "reward": reward, "replaced": int(replaced),
        })
    if extension_ticket is None:
        op_counts[op] += 1
        op_rewards[op] += reward

    gen += 1
    if "discount_router" in flags:
        bandit.decay(0.992)
        o5_a = 1.0 + 0.992 * (o5_a - 1.0)
        o5_b = 1.0 + 0.992 * (o5_b - 1.0)
        o6_a = 1.0 + 0.992 * (o6_a - 1.0)
        o6_b = 1.0 + 0.992 * (o6_b - 1.0)

    # Compare against v4's periodic repair as an explicit ablation.
    if "force_coverage" in flags and gen % 4 == 0 and len(archive.F) >= pop_size // 2:
        ax, af = archive.output(min(2 * pop_size, len(archive.F)))
        X, F = core.nsga3_environmental_selection(
            np.vstack([X, ax]), np.vstack([F, af]), pop_size, W, rng
        )
        stagnation = np.minimum(stagnation, 3)

    # Evidence-triggered reconstruction. It does not run for an initially
    # collapsed many-objective mapping, where low occupancy is structural.
    if (
        "no_memory" not in flags
        and "no_recovery" not in flags
        and "force_coverage" not in flags
        and not alignment_latch
        and progress >= collapse_start
    ):
        recent = np.asarray(occupancy_history[-6:], dtype=float)
        if len(recent) >= 4:
            xh = np.arange(len(recent), dtype=float)
            slope = float(np.polyfit(xh, recent, 1)[0])
        else:
            slope = 0.0

    # A temporary collapse is common and may self-recover. Intervene
    # only when the low-entropy state persists late in the run and the

```

```

# longitudinal occupancy is flat or declining.
severe = occupancy < threshold and entropy < 0.62 and slope <= 0.003
collapse_streak = collapse_streak + 1 if severe else 0
if (
    collapse_streak >= max(int(collapse_patience), 1)
    and gen - last_recovery >= recovery_cooldown
    and len(recovery_events) < max_recoveries
):
    if "soft_recovery" in flags:
        assoc_now, _ = _associate_fixed(F, W, ideal, nadir)
        _, _, cnt_now = _occupancy_entropy(assoc_now, pop_size)
        missing = np.flatnonzero(cnt_now == 0)
        quota = min(max(1, int(math.ceil(recovery_fraction * pop_size))), len(missing))
        occupied = np.flatnonzero(cnt_now > 0)
        Wn = W / (np.linalg.norm(W, axis=1, keepdims=True) + 1e-12)
        if len(missing) and len(occupied):
            proximity = np.max(Wn[missing] @ Wn[occupied].T, axis=1)
            cells = missing[np.argsort(proximity, kind="mergesort")[:quota]].astype(int).tolist()
        else:
            cells = missing[:quota].astype(int).tolist()
        # Only cells backed by an exact memory anchor enter the queue.
        cells = [c for c in cells if len(memory.candidates_for_cell(c, W, ideal, nadir)) > 0]
        rescue_queue.extend(cells)
        restored = len(cells)
        mode = "soft_ART_queue"
    else:
        X, F, restored, cells = memory.reconstruct(
            X, F, W, ideal, nadir, recovery_fraction, rng
        )
        mode = "hard_exact_injection"
    if restored > 0:
        recovery_events.append({
            "gen": gen,
            "evals": evals,
            "occupancy_before": occupancy,
            "entropy_before": entropy,
            "restored": restored,
            "cells": cells,
            "mode": mode,
        })
        last_recovery = gen
        collapse_streak = 0
        if mode == "hard_exact_injection":
            stagnation = np.minimum(stagnation, 3)
            ideal = np.minimum(ideal, F.min(axis=0))
            nadir = np.maximum(np.quantile(np.vstack([F, memory.F]), 0.95, axis=0), ideal + 1e-9)

if gen % trace_every == 0:
    trace.append({
        "gen": gen,
        "evals": evals,
        "archive_size": len(archive.F),
        "memory_size": len(memory.F),
        "mean_stagnation": float(stagnation.mean()),
        "mean_reward": float(op_rewards.sum() / max(op_counts.sum(), 1)),
        "occupancy_ratio": occupancy,
        "occupancy_entropy": entropy,
        "coverage_distress": distress,
    })

```

```

        "recoveries": len(recovery_events),
    })

outX, outF = archive.output(pop_size)
label = "MOSAIC-LQMoE" if "moe_residual" in flags else "MOSAIC-R3"
if flags:
    label += "[" + "+".join(sorted(flags)) + "]"
diagnostics: Dict[str, object] = {
    "version": "MOSAIC-LQMoE-v8-research",
    "flags": sorted(flags),
    "operator_counts": op_counts.tolist(),
    "operator_mean_rewards": (op_rewards / np.maximum(op_counts, 1)).tolist(),
    "bandit_counts": bandit.counts.tolist(),
    "micro_expert_names": [
        "shared_null", "pbest_micro", "pbest_macro", "local_diff",
        "global_diff", "archive_tangent", "gap_transport",
        "sparse_gaussian", "sparse_cauchy", "verified_replay",
        "sbx_residual", "orthogonal_escape",
    ],
    "micro_counts": micro_counts.tolist(),
    "micro_mean_rewards": (micro_rewards / np.maximum(micro_counts, 1)).tolist(),
    "micro_mean_advantages": (micro_advantages / np.maximum(micro_counts, 1)).tolist(),
    "micro_router_calls": micro_router.calls.tolist(),
    "micro_router_reward_ema": micro_router.reward_ema.tolist(),
    "micro_router_advantage_ema": micro_router.advantage_ema.tolist(),
    "micro_router_load_ema": micro_router.load_ema.tolist(),
    "micro_router_last_target": micro_router.last_target.tolist(),
    "micro_router_last_load": micro_router.last_load.tolist(),
    "micro_router_last_bias": micro_router.last_bias.tolist(),
    "micro_router_prototype_mass": micro_router.prototype_mass.tolist(),
    "micro_router_score_clip_events": int(micro_router.score_clip_events),
    "micro_assignment_history": micro_assignment_history,
    "micro_target_history": micro_target_history,
    "micro_load_history": micro_load_history,
    "micro_mean_clip_scale": float(np.mean(micro_clip_scales)) if micro_clip_scales else 1.0,
    "radial_dispersion_history": radial_dispersion_history,
    "geometry_score_history": geometry_score_history,
    "radial_failure_streak_final": int(radial_failure_streak),
    "o5_posterior_mean": (o5_a / (o5_a + o5_b)).tolist(),
    "o6_posterior_mean": (o6_a / (o6_a + o6_b)).tolist(),
    "coordinate_events": coordinate_events,
    "coordinate_probe_trials": probe_trials.tolist(),
    "coordinate_probe_success": probe_success.tolist(),
    "coordinate_learned_values": learned_values.tolist(),
    "coordinate_candidates": None if coordinate_candidates is None else coordinate_candidates.tolist(),
    "coordinate_pilot_rewards": coordinate_pilot_rewards,
    "coordinate_expand_threshold": coordinate_expand_threshold,
    "coordinate_pilot_budget": int(coordinate_pilot_budget_current),
    "coordinate_batch_history": coordinate_batch_history,
    "recoveries": recovery_events,
    "unspent_rescue_queue": rescue_queue,
    "secant_calls": secant_calls,
    "secant_used": secant_used,
    "mean_secant_reliability": float(secant_rel_sum / max(secant_calls, 1)),
    "role_reliability": role_reliability,
    "role_diagnostics": role_diagnostics,
    "role_probability": None if role_probability is None else role_probability.tolist(),
    "initial_occupancy": init_occ,

```

```

        "max_occupancy": max_occupancy,
        "scalar_history": scalar_history,
        "occupancy_history": occupancy_history,
        "recovery_config": {
            "fraction": recovery_fraction,
            "cooldown": recovery_cooldown,
            "max_recoveries": max_recoveries,
            "collapse_ratio": collapse_ratio,
            "collapse_start": collapse_start,
            "collapse_patience": collapse_patience,
        },
    }
}
return core.RunResult(label, problem.name, seed, outX, outF, evals, trace, diagnostics)

__all__ = [
    "ExactCoverageMemory",
    "DiscountedPortfolio",
    "run_mosaic_lqmoe",
    "_van_der_corput",
    "_coordinate_tunnel_child",
]

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/extensions.py

"""Pure proposal extensions: no evaluator, oracle front, or objective callback.

The quantile correction is an online routing heuristic inspired by expert-load calibration. It is not Kimi's neural router, a pretrained policy, or a guarantee that equal expert utilization is optimal for black-box optimization.

"""

```

from __future__ import annotations
from dataclasses import dataclass
from collections import deque
from typing import Protocol
import numpy as np
from scipy.special import logsumexp, softmax

```

```

class Codec(Protocol):
    def mutate(self, parent, donor, expert:int, rng:np.random.Generator)->np.ndarray: ...
    def features(self, parent, donor)->np.ndarray: ...

```

```

@dataclass(frozen=True)
class ExtensionConfig:
    mode:str='quantile' # quantile, contextual, uniform, disabled
    fraction:float=.50
    warmup:float=.10
    forgetting:float=.995
    uncertainty:float=.18
    quantile_cap:float=.08

```

```

@dataclass
class Ticket:
    expert:int

```

```

phi:np.ndarray
parent_f:np.ndarray
ideal:np.ndarray
scale:np.ndarray
weight:np.ndarray
mu:float
selected_at:int

class TypedRouter:
    def __init__(self, codec:Codec, seed:int, budget:int, config:ExtensionConfig=ExtensionConfig()):
        self.codec=codec; self.config=config; self.budget=budget
        self.rng=np.random.default_rng(np.random.SeedSequence([int(seed), 991081]))
        self.n_experts=4; self.dim=7
        self.A=np.repeat(np.eye(self.dim)[None, :, :], 4, axis=0)
        self.b=np.zeros((4, self.dim)); self.bias=np.zeros(4)
        self.count=np.zeros(4, int); self.reward_sum=np.zeros(4); self.ema=np.full(4, .5)
        self.contexts=deque(maxlen=48); self.executed=0; self.null_routes=0; self.log=[]

    @staticmethod
    def scalar(f, w, ideal, scale, mu):
        y=np.maximum((f-ideal)/scale, 0.)
        return float(mu*logsumexp(np.maximum(w, 1e-4)*y/mu))

    def _quantile_bias(self, means):
        # S[i,e]: predicted utility for several recent compatible contexts.
        # Nonuniform target load tracks empirical utility, with a small floor.
        S=np.asarray(self.contexts).T
        q=.30/4+.70*softmax((self.ema-self.ema.max())/ .15)
        bias=self.bias.copy()
        for _ in range(3):
            proposal=[]
            for e in range(4):
                others=np.delete(S+bias[None, :], e, axis=1)
                margin=np.max(others, axis=1)-S[:, e]
                proposal.append(np.quantile(margin, 1-q[e]))
            proposal=np.asarray(proposal); proposal-=proposal.mean()
            bias=.7*bias+.3*proposal
        self.bias=np.clip(bias, -self.config.quantile_cap, self.config.quantile_cap)

    def propose(self, shared, parent, parent_f, X, F, W, cell, ideal, nadir,
                progress, evals, base_op, stagnation):
        if (self.config.mode=='disabled' or base_op>=5 or progress<self.config.warmup
            or self.executed>=int(self.config.fraction*self.budget)
            or self.rng.random()>=self.config.fraction):
            self.null_routes+=1
            return shared, None
        # Donor selection uses only observed population values, not new calls.
        dist=np.sum((F-parent_f)**2/(np.ptp(F, axis=0)**2+1e-12), axis=1)
        near=np.argsort(dist, kind='stable')[:max(4, len(X)//3)]
        donor=X[int(self.rng.choice(near))]
        feats=np.clip(np.asarray(self.codec.features(parent, donor), float), 0, 1)
        phi=np.r_[1., progress, min(float(stagnation)/8, 1.), feats, progress*feats[0]]
        if phi.shape!=(7,): raise ValueError('Codec must return exactly three context features')
        if self.config.mode=='uniform': e=int(self.rng.integers(4))
        else:
            inv=np.linalg.inv(self.A)
            means=np.einsum('eij,ej->ei', inv, self.b)
            if self.config.mode=='quantile' and len(self.contexts)>=16 and self.executed%12==0:

```



```

        self._quantile_bias(means)
        std=np.sqrt(np.maximum(np.einsum('i,eij,j->e',phi,inv,phi),1e-12))
        score=means@phi+self.config.uncertainty*std*self.rng.standard_normal(4)
        if self.config.mode=='quantile':score+=self.bias
        # Deterministic finite warm-up ensures each compatible expert gets data.
        e=int(self.executed%4) if self.executed<8 else int(np.argmax(score))
        child=self.codec.mutate(parent,donor,e,self.rng)
        self.contexts.append(phi.copy());self.executed+=1
        ticket=Ticket(e,phi,parent_f.copy(),ideal.copy(),np.maximum(nadir-ideal,1e-9),
                      W[cell].copy(),.008+.10*(1-progress)**2,int(evals))
        return child,ticket

def observe(self,ticket,child_f,entered,replaced):
    if ticket is None:return
    old=self.scalar(ticket.parent_f,ticket.weight,ticket.ideal,ticket.scale,ticket.mu)
    new=self.scalar(child_f,ticket.weight,ticket.ideal,ticket.scale,ticket.mu)
    relative=np.clip((old-new)/(abs(old)+.05),-1.,1.)
    # Evaluated parent vs evaluated child: NOT an unobserved counterfactual.
    reward=float(.8*(.5+.5*relative)+.2*bool(entered))
    e=ticket.expert;p=ticket.phi;g=self.config.forgetting
    self.A[e]=np.eye(7)+g*(self.A[e]-np.eye(7))+np.outer(p,p)
    self.b[e]=g*self.b[e]+p*reward
    self.count[e]+=1;self.reward_sum[e]+=reward
    self.ema[e]=.9*self.ema[e]+.1*reward
    self.log.append({'fe':ticket.selected_at+1,'expert':int(e),'reward':reward,
                    'entered':bool(entered),'replaced':int(replaced),
                    'bias':self.bias.tolist()})

def diagnostics(self):
    return {'mode':self.config.mode,'executed':self.executed,'null_routes':self.null_routes,
            'counts':self.count.tolist(),'reward_sum':self.reward_sum.tolist(),
            'final_bias':self.bias.tolist(),'route_log':self.log}

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/final.py

"""MOSAIC v10-R1: final interface with explicit, tested capability boundaries.

The held-out failure of equal-objective behavior retention is preserved in the research branch. R1 keeps only one representative per objective in structured search, and uses an eligibility/novelty router rather than the unproven quantile controller. Continuous inputs dispatch to the original frozen SQMoE v9.

"""

```

from dataclasses import dataclass
from typing import Iterable
import numpy as np
from .api import optimize, ProblemContract, EvaluationLedger, AuditedRun,core
from .graph_search import run_graph

```

VERSION='MOSAIC-v10-R1'

```

def solve(problem, seed:int=1, budget:int=2080, population:int=32,
          contract:ProblemContract=ProblemContract()->AuditedRun:
    if budget<population or population<8:
        raise ValueError('Require budget >= population >= 8')

```

```

if not contract.deterministic or contract.constraints not in ('box', 'feasible_decoder'):
    raise NotImplementedError('Noisy or general nonlinear-constrained models need validated adapters')
if contract.representation=='continuous_box' and contract.codec is None:
    result=optimize(problem, seed, budget, population, contract, algorithm='mosaic_v9')
    result.result.diagnostics['backend']='frozen_v9_exact_fallback'
elif contract.representation=='ordered_subset' and contract.codec is not None:
    ledger=EvaluationLedger(problem, budget)
    r=run_graph(ledger, contract.codec, seed, population, budget, mode='no_memory', ties_per_objective=1)
    assert r.evaluations==ledger.spent==budget
    r.diagnostics['backend']='typed_archive_novelty_router'
    result=AuditedRun(r, ledger)
else:
    raise NotImplementedError('Unsupported representation; silent rounding is forbidden')
result.result.algorithm=VERSION
return result

@dataclass
class ExhaustiveResult:
    X:np.ndarray
    F:np.ndarray
    evaluations:int
    certificate:dict

def enumerate_finite(problem, encoded_designs:Iterable[np.ndarray], cardinality:int,
                    budget:int, scope:str, chunk_size:int=2048)->ExhaustiveResult:
    """Exhaustively evaluate a caller-declared finite domain through the ledger.

    A certificate is conditional on the generator enumerating that full domain;
    it does not certify the correctness of the mathematical model or a larger
    policy space. No Pareto oracle is supplied to this routine.
    """
    if cardinality<=0 or budget<cardinality or chunk_size<1:
        raise ValueError('Insufficient budget for full enumeration')
    ledger=EvaluationLedger(problem, budget)
    xs=[]; fs=[]; batch=[]
    def consume(batch):
        X=np.asarray(batch, float); F=ledger.evaluate(X)
        xs.extend(X); fs.extend(F)
    for x in encoded_designs:
        batch.append(np.asarray(x, float))
        if len(batch)==chunk_size:consume(batch); batch=[]
    if batch:consume(batch)
    if ledger.spent!=cardinality:raise AssertionError('Generator cardinality mismatch')
    X=np.asarray(xs); F=np.asarray(fs)
    unique, index=np.unique(F, axis=0, return_index=True)
    nd=core.nondominated_mask(unique); index=index[nd]
    return ExhaustiveResult(X[index], F[index], ledger.spent,
        {'scope':scope, 'declared_cardinality':cardinality, 'evaluations':ledger.spent,
         'domain_exhausted':True, 'unique_objective_vectors':len(unique),
         'front_size':len(index), 'model_correctness_certified':False,
         'larger_policy_space_certified':False})

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/graph_search.py

```

"""Behavior-diverse archive search with sparse, real-evaluation-only expert credit.

Preserves distinct decision policies even at exactly equal objective vectors.
The idea is a structured search extension, not a claim to reproduce a neural MoE.
"""

from __future__ import annotations
from collections import defaultdict, deque
import numpy as np
from scipy.special import softmax
import moo_core as core

def key(codec, x): return tuple(map(int, codec.decode(x)))

def run_graph(problem, codec, seed, pop_size, max_evals, mode='uniform', ties_per_objective=4):
    rng, X, F = core.initialize(problem, pop_size, seed)
    # Index by exact phenotype, NOT by floating-point random-key genotype.
    seen = set(key(codec, x) for x in X)
    visits = defaultdict(int)
    a = np.ones((3, 4)); b = np.ones((3, 4)); counts = np.zeros(4, int); rewards = np.zeros(4)
    bias = np.zeros(4); history = deque(maxlen=48); score_history = deque(maxlen=48)
    X = [x.copy() for x in X]; F = [f.copy() for f in F]
    evals = pop_size; log = []; rejected_duplicates = 0

    def trim_pool(X, F):
        arr = np.asarray(F); unique, inv = np.unique(arr, axis=0, return_inverse=True)
        valid = core.nondominated_mask(unique)
        kept = []
        for u in np.flatnonzero(valid):
            ids = np.flatnonzero(inv == u)
            # Retain underexplored phenotypes; zero visits to newly admitted point.
            if mode == 'no_memory': limit = 1
            else: limit = ties_per_objective
            ids = sorted(ids, key=lambda i: (visits[key(codec, X[i])], rng.random()))
            distinct = []; local_seen = set()
            for i in ids:
                k = key(codec, X[i])
                if k not in local_seen:
                    distinct.append(i); local_seen.add(k)
                if len(distinct) >= limit: break
            kept.extend(distinct)
        if len(kept) > 4 * pop_size:
            loc = core.select_indices_farthest(arr[kept], 4 * pop_size)
            kept = [kept[i] for i in loc]
        return [X[i] for i in kept], [F[i] for i in kept]

    X, F = trim_pool(X, F)
    while evals < max_evals:
        arr = np.asarray(F); unique, inv = np.unique(arr, axis=0, return_inverse=True)
        # Regions (objective vectors) have equal parent probability, irrespective
        # of how many behavior variants happen to have accumulated there.
        group = int(rng.integers(len(unique))); ids = np.flatnonzero(inv == group)
        nvis = np.array([visits[key(codec, X[i])] for i in ids], float)
        parent_id = int(rng.choice(ids, p=(1/(1+nvis))/(1/(1+nvis)).sum()))
        donor_id = int(rng.integers(len(X))); parent = X[parent_id]; pf = F[parent_id]; donor = X[donor_id]
        state = min(2, int(3 * evals / max_evals))
        sample = rng.beta(a[state], b[state])
        if mode in ('uniform', 'no_memory', 'no_novelty'): sample = rng.random(4)
        if mode == 'single_expert': sample = np.array([1., 0., 0., 0.])

```

```

if mode=='quantile' and len(score_history)>=16 and evals%16==0:
    S=np.asarray(score_history)
    quality=(a.sum(0)/(a+b).sum(0))
    q=.20/4+.80*softmax((quality-quality.max())/(.20))
    candidates=[]
    for e in range(4):
        margin=np.max(np.delete(S+bias,e,axis=1),axis=1)-S[:,e]
        candidates.append(np.quantile(margin,1-q[e]))
    correction=np.asarray(candidates);correction-=correction.mean()
    bias=np.clip(.75*bias+.25*correction,-.08,.08)
score_history.append(sample.copy())
if mode=='quantile':sample+=bias
ranked=np.argsort(-sample,kind='stable')
child=None;chosen=None
# Pure generation can be retried without a function call. The same rule
# is provided to the duplicate-aware NSGA-II control below.
for e in ranked:
    attempts=1 if mode=='no_novelty' else 12
    for _ in range(attempts):
        cand=codec.mutate(parent,donor,int(e),rng)
        if mode=='no_novelty' or key(codec,cand) not in seen:
            child=cand;chosen=int(e);break
        rejected_duplicates+=1
    if child is not None:break
if child is None:
    child=problem.xl+rng.random(problem.n_var)*(problem.xu-problem.xl);chosen=4
cf=problem.evaluate(child)[0];evals+=1
visits[key(codec,parent)]+=1;seen.add(key(codec,child))
nd=not np.any(np.all(arr<=cf,axis=1)&np.any(arr<cf,axis=1))
dominates=bool(np.all(cf<=pf) and np.any(cf<pf))
novelty=not np.any(np.all(arr==cf,axis=1))
reward=float(.50*dominates+.35*(nd and novelty)+.15*nd)
if chosen<4:
    # Only the actually evaluated expert receives evidence.
    a[state,chosen]=1+.995*(a[state,chosen]-1)+reward
    b[state,chosen]=1+.995*(b[state,chosen]-1)+1-reward
    counts[chosen]+=1;rewards[chosen]+=reward
if nd:
    X.append(child.copy());F.append(cf.copy());X,F=trim_pool(X,F)
log.append({'fe':evals,'expert':chosen,'reward':reward,'pool_size':len(X)})
AX=np.asarray(X);AF=np.asarray(F)
_,idx=np.unique(AF,axis=0,return_index=True);AX=AX[idx];AF=AF[idx]
if len(AF)>pop_size:
    idx=core.select_indices_farthest(AF,pop_size);AX,AF=AX[idx],AF[idx]
return core.RunResult('graph_'+mode,problem.name,seed,AX,AF,evals,[],
    {'counts':counts.tolist(),'reward_sum':rewards.tolist(),'route_log':log,
    'duplicate_proposals_rejected':rejected_duplicates,'unique_evaluated':len(seen),
    'behavior_memory':mode!='no_memory','novelty_filter':mode!='no_novelty'})

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/rgv.py

""2018 CUMCM B, deterministic one-process cyclic-policy benchmark.

The input parameters are transcribed from the published problem. The policy

```

restriction, event ordering and added movement objective are documented in data.
Only the small finite policy class is exhaustively certifiable, NOT the whole
original dynamic scheduling problem.
"""
from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
from itertools import permutations
import json
import numpy as np
try:
    from numba import njit
except ImportError:
    def njit(*args, **kwargs):
        return lambda f: f

DATA = json.loads((Path(__file__).resolve().parents[1]/'data/rgv2018_parameters.json').read_text())

@njit(cache=True)
def simulate_fast(order, travel, process, loads, wash, horizon):
    ready = np.zeros(8, dtype=np.float64)
    loaded = np.zeros(8, dtype=np.bool_)
    t=0.; position=0; completed=0; movement=0.; p=0
    # Every operation advances time by >=27; bounded even for singleton cycles.
    while True:
        j = order[p % len(order)]
        pos=j//2
        move=travel[abs(pos-position)]
        arrival=t+move
        start=max(arrival,ready[j]) if loaded[j] else arrival
        service_end=start+loads[j]
        end=service_end+(wash if loaded[j] else 0.)
        if end>horizon+1e-10:
            break
        completed+=int(loaded[j]); movement+=move
        ready[j]=service_end+process; loaded[j]=True
        position=pos; t=end; p+=1
    return completed,movement

@njit(cache=True)
def batch_simulate(orders, sizes, travel, process, loads, wash, horizon):
    F=np.empty((len(orders),2))
    for k in range(len(orders)):
        n,m=simulate_fast(orders[k,:sizes[k]],travel,process,loads,wash,horizon)
        F[k,0]=-n; F[k,1]=m
    return F

class RGVCodec:
    """Real random keys -> nonempty ordered subset; surjective over our class."""
    n_var=16
    def decode(self, x):
        x=np.asarray(x)
        active=x[:8]>=.5
        if not np.any(active):
            active[int(np.argmax(x[:8]))]=True
        ordered=np.argsort(x[8:],kind='stable')
        return ordered[active[ordered]].astype(np.int64)
    def encode(self, order):

```

```

order=np.asarray(order,dtype=np.int64)
if len(order)==0 or len(np.unique(order))!=len(order) or np.any(order<0) or np.any(order>=8):
    raise ValueError('Not a nonempty ordered CNC subset')
x=np.zeros(16); x[order]=.75; x[8:]=.99
for k,j in enumerate(order): x[8+j]=(k+.5)/8
return x
def mutate(self, parent, donor, expert, rng):
    p=list(map(int,self.decode(parent))); q=list(map(int,self.decode(donor)))
    absent=[j for j in range(8) if j not in p]
    if expert==0: # swap: change visit order, preserve active set
        if len(p)>1:
            a,b=rng.choice(len(p),2,replace=False);p[a],p[b]=p[b],p[a]
            elif absent:p.append(int(rng.choice(absent)))
    elif expert==1: # relocate / orientation: structural local search
        if len(p)>1:
            a,b=map(int,rng.choice(len(p),2,replace=False));j=p.pop(a);p.insert(b,j)
            elif absent:p.insert(0,int(rng.choice(absent)))
    elif expert==2: # activate / deactivate: cross subset cardinalities
        if absent and (len(p)==1 or rng.random()<.5):
            p.insert(int(rng.integers(len(p)+1)),int(rng.choice(absent)))
            elif len(p)>1:p.pop(int(rng.integers(len(p))))
    elif expert==3: # order-preserving donor segment, no numeric averaging
        if len(q)>1:
            a,b=sorted(map(int,rng.choice(len(q)+1,2,replace=False)))
            piece=q[a:b]
            else:piece=q[:]
            p=[j for j in p if j not in piece]
            at=int(rng.integers(len(p)+1));p[at:at]=piece
            else:raise ValueError('Unknown expert')
    return self.encode(p)
def features(self,parent,donor):
    p=self.decode(parent);q=self.decode(donor)
    return np.array([len(p)/8, np.mean(np.abs(np.diff(p//2)))/3 if len(p)>1 else 0.,len(set(p)&set(q))/8])

@dataclass
class RGVProblem:
    group:int=1
    horizon:float=28800.
    def __post_init__(self):
        g=DATA['groups'][self.group-1]
        self.travel=np.array(g['travel'],dtype=np.float64)
        self.process=float(g['process']);self.wash=float(g['wash'])
        self.loads=np.array([g['odd_load'] if j%2==0 else g['even_load'] for j in range(8)],float)
        self.name=f'CUMCM2018B-G{self.group}-cyclic'
        self.n_var=16;self.n_obj=2;self.xl=np.zeros(16);self.xu=np.ones(16)
        self.codec=RGVCodec();self.calls=0
    def evaluate(self,X):
        X=np.atleast_2d(X)
        if X.shape[1]!=16 or not np.isfinite(X).all():raise ValueError('Invalid input')
        orders=np.zeros((len(X),8),dtype=np.int64); sizes=np.zeros(len(X),dtype=np.int64)
        for i,x in enumerate(X):
            o=self.codec.decode(x);orders[i,:len(o)]=o;sizes[i]=len(o)
        self.calls+=len(X)
        return batch_simulate(orders,sizes,self.travel,self.process,self.loads,self.wash,self.horizon)
    def pareto_front(self,n=1000):
        raise RuntimeError('Oracle front deliberately unavailable to optimizer')
    def simulate_log(self,order):
        ready=np.zeros(8);loaded=np.zeros(8,bool);t=0.;pos=0;rows=[];p=0

```

```

while True:
    j=int(order[p%len(order)]);move=float(self.travel[abs(j//2-pos)])
    arrival=t+move;start=max(arrival,ready[j]) if loaded[j] else arrival
    endservice=start+self.loads[j];end=endservice+(self.wash if loaded[j] else 0.)
    if end>self.horizon+1e-10:break
    rows.append(dict(visit=p,cnc=j+1,depart=t,arrival=arrival,service_start=start,
        service_end=endservice,end=end,wash_loaded=bool(loaded[j]),travel=move,
        processing_ready_before=float(ready[j]),next_ready=float(endservice+self.process)))
    ready[j]=endservice+self.process;loaded[j]=True;t=end;pos=j//2;p+=1
return rows

def all_policies():
    orders=[];sizes=[]
    for k in range(1,9):
        for p in permutations(range(8),k):
            orders.append(list(p)+[-1]*(8-k));sizes.append(k)
    return np.asarray(orders,dtype=np.int64),np.asarray(sizes,dtype=np.int64)

def nondominated_2d(F):
    # Unique nondominated objective vectors for minimization, exact integer values.
    ids=np.lexsort((F[:,1],F[:,0]));best=np.inf;out=[]
    for i in ids:
        if F[i,1]<best:
            out.append(int(i));best=F[i,1]
    return np.array(out,dtype=int)

def exhaustive(group):
    problem=RGVProblem(group); orders,sizes=all_policies()
    F=batch_simulate(orders,sizes,problem.travel,problem.process,problem.loads,problem.wash,problem.horizon)
    idx=nondominated_2d(F)
    return {'F':F[idx], 'orders':orders[idx], 'sizes':sizes[idx], 'all_F':F, 'num_policies':len(F)}

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/baseline_v10/mosaic/typed_baseline.py

```

"""NSGA-II-style comparator using EXACTLY the same structural operator library.
Independent variation controller; environmental selection reused from legacy.
This is a documented implementation, NOT the authors' official executable.
"""

import numpy as np
import moo_core as core

def run_typed_nsga2(problem,codec,seed,pop_size,max_evals):
    rng,X,F=core.initialize(problem,pop_size,seed)
    A=core.Archive(max_size=5*pop_size);A.update(X,F)
    fronts,ranks=core.fast_nondominated_sort(F);crowd=np.zeros(pop_size)
    for front in fronts:crowd[front]=core.crowding_distance(F[front])
    count=pop_size
    while count<max_evals:
        n=min(pop_size,max_evals-count);offs=[]
        for _ in range(n):
            a=core.tournament(ranks,crowd,rng);b=core.tournament(ranks,crowd,rng)
            offs.append(codec.mutate(X[a],X[b],int(rng.integers(4)),rng))
        Q=np.asarray(offs);G=problem.evaluate(Q);count+=n;A.update(Q,G)
        X,F,ranks,crowd=core.nsga2_environmental_selection(np.vstack([X,Q]),np.vstack([F,G]),pop_size)

```

```

xx,ff=A.output(pop_size)
return core.RunResult('typed_nsga2',problem.name,seed,xx,ff,count,[],{}))

def run_typed_nsga2_unique(problem,codec,seed,pop_size,max_evals):
    """Stronger control: same 4 operators AND same phenotype novelty filter.

    No evaluator call is made while testing key novelty. Pending batch keys count
    as seen immediately to prevent duplicate queries within the same batch.
    """
    rng,X,F=core.initialize(problem,pop_size,seed)
    to_key=lambda x:tuple(map(int,codec.decode(x)))
    seen=set(to_key(x) for x in X)
    A=core.Archive(max_size=5*pop_size);A.update(X,F)
    fronts,ranks=core.fast_nondominated_sort(F);crowd=np.zeros(pop_size)
    for front in fronts:crowd[front]=core.crowding_distance(F[front])
    count=pop_size;duplicates=0
    while count<max_evals:
        n=min(pop_size,max_evals-count);offs=[]
        for _ in range(n):
            a=core.tournament(ranks,crowd,rng);b=core.tournament(ranks,crowd,rng)
            child=None
            for e in rng.permutation(4):
                for _ in range(12):
                    cand=codec.mutate(X[a],X[b],int(e),rng)
                    if to_key(cand) not in seen:
                        child=cand;break
                    duplicates+=1
                if child is not None:break
            if child is None:
                child=problem.xl+rng.random(problem.n_var)*(problem.xu-problem.xl)
                seen.add(to_key(child));offs.append(child)
        Q=np.asarray(offs);G=problem.evaluate(Q);count+=n;A.update(Q,G)
        X,F,ranks,crowd=core.nsga2_environmental_selection(np.vstack([X,Q]),np.vstack([F,G]),pop_size)
    xx,ff=A.output(pop_size)
    return core.RunResult('typed_nsga2_unique',problem.name,seed,xx,ff,count,[],
        {'duplicate_proposals_rejected':duplicates,'unique_evaluated':len(seen)})

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/__init__.py

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/bootstrap.py

```

from pathlib import Path
import sys
BASE=Path(__file__).resolve().parents[1]/"baseline_v10"
if str(BASE) not in sys.path: sys.path.insert(0,str(BASE))
from mosaic.api import core, EvaluationLedger, ProblemContract, AuditedRun

```


源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/fast_api.py

```
"""Optional engineering upgrade, not an untested universal optimizer.

Only the deterministic ordered-subset branch is changed. Continuous problems
and all unsupported contracts are delegated to the unchanged v10 entry point.
"""

from .bootstrap import EvaluationLedger, AuditedRun, ProblemContract
from .fast_v10 import run_fast_v10
from mosaic.final import solve as solve_v10

VERSION='MOSAIC-v10-R1-Fast-branch'

def solve_fast(problem,seed:int=1,budget:int=2080,population:int=32,
               contract:ProblemContract=ProblemContract()->AuditedRun:
    if budget<population or population<8:
        raise ValueError('Require budget >= population >=8')
    if not contract.deterministic or contract.constraints not in ('box','feasible_decoder'):
        raise NotImplementedError('A validated noise/constraint adapter is required')
    if contract.representation=='ordered_subset' and contract.codec is not None:
        ledger=EvaluationLedger(problem,budget)
        result=run_fast_v10(ledger,contract.codec,seed,population,budget)
        assert result.evaluations==ledger.spent==budget
        result.algorithm=VERSION;result.diagnostics['search_distribution_changed']=False
        return AuditedRun(result,ledger)
    return solve_v10(problem,seed,budget,population,contract)
```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/fast_v10.py

```
"""Behavior-diverse archive search with sparse, real-evaluation-only expert credit.

Preserves distinct decision policies even at exactly equal objective vectors.
The idea is a structured search extension, not a claim to reproduce a neural MoE.
"""

from __future__ import annotations
from collections import defaultdict,deque
import numpy as np
from scipy.special import softmax
from .bootstrap import core

def key(codec,x): return tuple(map(int,codec.decode(x)))

def run_fast_v10(problem,codec,seed,pop_size,max_evals,mode='no_memory',ties_per_objective=1):
    assert mode=='no_memory' and ties_per_objective==1, 'Fast path certified only for R1 single representative'
    key_cache={}
    def key(codec,x):
        raw=np.asarray(x,dtype=float).tobytes()
        if raw not in key_cache:key_cache[raw]=tuple(map(int,codec.decode(x)))
        return key_cache[raw]
    rng,X,F=core.initialize(problem,pop_size,seed)
    # Index by exact phenotype, NOT by floating-point random-key genotype.
```

```

seen=set(key(codec,x) for x in X)
visits=defaultdict(int)
a=np.ones((3,4));b=np.ones((3,4));counts=np.zeros(4,int);rewards=np.zeros(4)
bias=np.zeros(4);history=deque(maxlen=48);score_history=deque(maxlen=48)
X=[x.copy() for x in X];F=[f.copy() for f in F]
evals=pop_size;log=[];rejected_duplicates=0

def trim_pool(X,F):
    arr=np.asarray(F)
    groups={}
    for i,f in enumerate(F):groups.setdefault(tuple(f),[]).append(i)
    ordered=sorted(groups);unique=np.asarray(ordered)
    valid=core.nondominated_mask(unique)
    kept=[]
    for u in np.flatnonzero(valid):
        ids=groups[ordered[u]]
        # Retain underexplored phenotypes; zero visits to newly admitted point.
        if mode=='no_memory':limit=1
        else:limit=ties_per_objective
        ids=sorted(ids,key=lambda i:(visits[key(codec,X[i])],rng.random()))
        distinct=[];local_seen=set()
        for i in ids:
            k=key(codec,X[i])
            if k not in local_seen:
                distinct.append(i);local_seen.add(k)
            if len(distinct)>=limit:break
        kept.extend(distinct)
    if len(kept)>4*pop_size:
        loc=core.select_indices_farthest(arr[kept],4*pop_size)
        kept=[kept[i] for i in loc]
    return [X[i] for i in kept],[F[i] for i in kept]

X,F=trim_pool(X,F)
while evals<max_evals:
    arr=np.asarray(F);unique=arr;inv=np.arange(len(F))
    # Regions (objective vectors) have equal parent probability, irrespective
    # of how many behavior variants happen to have accumulated there.
    group=int(rng.integers(len(unique))); ids=np.flatnonzero(inv==group)
    nvis=np.array([visits[key(codec,X[i])] for i in ids],float)
    parent_id=int(rng.choice(ids,p=(1/(1+nvis))/(1/(1+nvis)).sum()))
    donor_id=int(rng.integers(len(X)));parent=X[parent_id];pf=F[parent_id];donor=X[donor_id]
    state=min(2,int(3*evals/max_evals))
    sample=rng.beta(a[state],b[state])
    if mode in ('uniform','no_memory','no_novelty'):sample=rng.random(4)
    if mode=='single_expert':sample=np.array([1.,0.,0.,0.])
    if mode=='quantile' and len(score_history)>=16 and evals%16==0:
        S=np.asarray(score_history)
        quality=(a.sum(0)/(a+b).sum(0))
        q=.20/4+.80*softmax((quality-quality.max())/-.20)
        candidates=[]
        for e in range(4):
            margin=np.max(np.delete(S+bias,e,axis=1),axis=1)-S[:,e]
            candidates.append(np.quantile(margin,1-q[e]))
        correction=np.asarray(candidates);correction-=correction.mean()
        bias=np.clip(.75*bias+.25*correction,-.08,.08)
    score_history.append(sample.copy())
    if mode=='quantile':sample+=bias
    ranked=np.argsort(-sample,kind='stable')

```

```

child=None;chosen=None
# Pure generation can be retried without a function call. The same rule
# is provided to the duplicate-aware NSGA-II control below.
for e in ranked:
    attempts=1 if mode=='no_novelty' else 12
    for _ in range(attempts):
        cand=codec.mutate(parent,donor,int(e),rng)
        if mode=='no_novelty' or key(codec,cand) not in seen:
            child=cand;chosen=int(e);break
        rejected_duplicates+=1
    if child is not None:break
if child is None:
    child=problem.xl+rng.random(problem.n_var)*(problem.xu-problem.xl);chosen=4
cf=problem.evaluate(child)[0];evals+=1
visits[key(codec,parent)]+=1;seen.add(key(codec,child))
nd=not np.any(np.all(arr<=cf,axis=1)&np.any(arr<cf,axis=1))
dominates=bool(np.all(cf<=pf) and np.any(cf<pf))
novelty=not np.any(np.all(arr==cf,axis=1))
reward=float(.50*dominates+.35*(nd and novelty)+.15*nd)
if chosen<4:
    # Only the actually evaluated expert receives evidence.
    a[state,chosen]=1+.995*(a[state,chosen]-1)+reward
    b[state,chosen]=1+.995*(b[state,chosen]-1)+1-reward
    counts[chosen]+=1;rewards[chosen]+=reward
if nd:
    X.append(child.copy());F.append(cf.copy());X,F=trim_pool(X,F)
log.append({'fe':evals,'expert':chosen,'reward':reward,'pool_size':len(X)})
AX=np.asarray(X);AF=np.asarray(F)
_,idx=np.unique(AF,axis=0,return_index=True);AX=AX[idx];AF=AF[idx]
if len(AF)>pop_size:
    idx=core.select_indices_farthest(AF,pop_size);AX,AF=AX[idx],AF[idx]
return core.RunResult('graph_'+mode,problem.name,seed,AX,AF,evals,[],
    {'counts':counts.tolist(),'reward_sum':rewards.tolist(),'route_log':log,
    'duplicate_proposals_rejected':rejected_duplicates,'unique_evaluated':len(seen),
    'behavior_memory':False,'novelty_filter':True,'key_cache_size':len(key_cache)})

```

源程序与运行资源

code/dependencies/vendor/mosaic_v14/vendor/v13/vendor/src/problems.py

```

"""Known v10 RGV model + preregistered perturbations + synthetic transfer tasks."""
from __future__ import annotations
import numpy as np
from .bootstrap import core
from mosaic.rgv import RGVProblem, RGVCodec, all_policies, batch_simulate, nondominated_2d
from numba import njit

@njit(cache=True)
def tour_batch(orders,sizes,dist,values,service,discount):
    out=np.zeros((len(orders),2))
    for i in range(len(orders)):
        clock=0.;reward=0.;prev=8
        for q in range(sizes[i]):
            j=orders[i,q];clock+=dist[prev,j]+service[j]
            reward+=values[j]*np.exp(-clock/discount[j]);prev=j
        clock+=dist[prev,8]

```

```

        out[i,0]=-reward;out[i,1]=clock
    return out

class TourProblem:
    def __init__(self,spec):
        self.name=spec['id'];self.n_var=16;self.n_obj=2;self.xl=np.zeros(16);self.xu=np.ones(16)
        self.codec=RGVCodec();self.calls=0
        self.dist=np.array(spec['dist']);self.values=np.array(spec['values']);self.service=np.array(spec['service']);self.discount=
        ← out=np.array(spec['discount'])
    def evaluate(self,X):
        X=np.atleast_2d(X);orders=np.zeros((len(X),8),np.int64);sizes=np.zeros(len(X),np.int64)
        for i,x in enumerate(X):
            p=self.codec.decode(x);orders[i,:len(p)]=p;sizes[i]=len(p)
            self.calls+=len(X)
            return tour_batch(orders,sizes,self.dist,self.values,self.service,self.discount)
    def pareto_front(self,*a,**kw):raise RuntimeError('No oracle during search')

def make_problem(spec):
    if spec['kind']=='tour':return TourProblem(spec)
    p=RGVProblem(spec.get('group',1));p.name=spec['id']
    for name in ['process','wash','horizon']:
        if name in spec:setattr(p,name,float(spec[name]))
    for name in ['loads','travel']:
        if name in spec:setattr(p,name,np.array(spec[name],float))
    return p

def generate_specs():
    specs=[dict(id=f'rgv_g{g}',kind='rgv',group=g,partition='known_confirmation') for g in [1,2,3]]
    for i,seed in enumerate([5411,6781,7823,8317,9719]):
        rng=np.random.default_rng(seed);g=(i%3)+1;p=RGVProblem(g)
        specs.append(dict(id='rgv_dev' if i==0 else f'rgv_hold{i}',kind='rgv',group=g,
            partition='development' if i==0 else 'heldout_parameter',
            process=float(np.round(p.process*rng.uniform(.60,1.50))),
            wash=float(np.round(p.wash*rng.uniform(.6,1.7))),
            loads=np.round(p.loads*rng.uniform(.6,1.7,8)).tolist(),
            travel=(np.r_[0.,np.cumsum(rng.uniform(10.,40.,3))]).round().tolist(),
            horizon=float(rng.choice([7200,14400,21600,36000]))))
    for i,seed in enumerate([4133,5129,6163,7283,8297]):
        rng=np.random.default_rng(seed);xy=rng.uniform(0,100,(9,2));xy[8]=[50,50]
        d=np.sqrt(np.sum((xy[:,None]-xy[None,:])**2,axis=2))
        specs.append(dict(id='tour_dev' if i==0 else f'tour_hold{i}',kind='tour',
            partition='development' if i==0 else 'heldout_synthetic_transfer',
            dist=d.tolist(),values=rng.uniform(15,100,8).tolist(),service=rng.uniform(1,10,8).tolist(),
            discount=rng.uniform(80,350,8).tolist()))
    return specs

def enumerate_oracle(spec):
    p=make_problem(spec);o,n=all_policies()
    if spec['kind']=='tour':F=tour_batch(o,n,p.dist,p.values,p.service,p.discount)
    else:F=batch_simulate(o,n,p.travel,p.process,p.loads,p.wash,p.horizon)
    p.calls+=len(o) # Certification calls are counted separately from search FE.
    ids=nondominated_2d(F)
    return F[ids],o[ids],n[ids],len(o)

```

源程序与运行资源

code/reproduce.py

```
#!/usr/bin/env python3
"""Reproduce a frozen solver and evaluator from a support bundle.
Use an isolated environment. This script does not run unless explicitly trusted.
"""

from __future__ import annotations
import argparse, hashlib, json, os, shutil, subprocess, sys, tempfile
from pathlib import Path

def sha(p): return hashlib.sha256(p.read_bytes()).hexdigest()

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--candidate',default='baseline');ap.add_argument('--phase',choices=['develop',
↪ ment','confirmation'],default='confirmation')
    ap.add_argument('--seed',type=int,default=None);ap.add_argument('--variant',default=None);ap.add_argument('--original-input',
↪ t',type=Path)
    ap.add_argument('--out',type=Path,default=Path('reproduced'));ap.add_argument('--trust-reviewed-code',action='store_true')
↪ ;args=ap.parse_args()
    root=Path(__file__).resolve().parent
    if not args.trust_reviewed_code:ap.error('Review code first; run in an isolated environment and explicitly pass
↪ --trust-reviewed-code')
    manifest=json.loads((root/'manifest.json').read_text('utf-8'))
    for rel,expected in manifest['files'].items():
        p=root/rel
        if not p.is_file() or sha(p)!=expected:raise SystemExit('Integrity mismatch: '+rel)
    protocol=json.loads((root/'protocol.json').read_text('utf-8'))
    seed=args.seed if args.seed is not None else protocol[args.phase+'_seeds'][0]
    if seed not in protocol[args.phase+'_seeds']:raise SystemExit('Seed is outside frozen phase protocol')
    code=root/'code'/args.candidate
    if not code.is_dir() or not code.resolve().is_relative_to((root/'code').resolve()):raise SystemExit('Unknown candidate')
    data=root/'inputs'/args.phase
    if not data.is_dir():
        if args.original_input is None:raise SystemExit('Original contest input excluded. Supply --original-input DIR matching
↪ input_manifest.json')
        data=args.original_input
    inputs=json.loads((root/'input_manifest.json').read_text('utf-8'))[args.phase]
    for rel,h in inputs.items():
        if not (data/rel).is_file() or sha(data/rel)!=h:raise SystemExit('Original input mismatch: '+rel)
    if args.out.exists() and any(args.out.iterdir()):raise SystemExit('Output must be a new/empty directory')
    args.out.mkdir(parents=True,exist_ok=True);out=args.out.resolve()
    env={k:v for k,v in os.environ.items() if k in ('PATH','HOME','LANG','SYSTEMROOT','TMPDIR')}
    env.update(PYTHONPATH=str(root/'code'/dependencies'),PYTHONHASHSEED='0',PYTHONDONTWRITEBYTECODE='1',
        OMP_NUM_THREADS='1',MKL_NUM_THREADS='1',OPENBLAS_NUM_THREADS='1',NUMBA_CACHE_DIR=str(out/'numba-cache'))
    variant=args.variant or ('baseline' if args.candidate=='baseline' else 'full')
    common=['--seed',str(seed),'--budget',str(protocol['fe_budget']), '--variant',variant]
    (out/'solver').mkdir();(out/'evaluation').mkdir()
    subprocess.run([sys.executable,'-B','-m','cumcm_harness.worker',str(code/'main.py'),'--input',str(data.resolve()), '--out',
↪ str(out/'solver'),*common],env=env,check=True,timeout=600)
    with tempfile.TemporaryDirectory(prefix='cumcm-eval-data-') as td:
        evaluation_data=Path(td);shutil.copytree(data,evaluation_data/'public')
        private=root/'private'/args.phase
        if private.exists():shutil.copytree(private,evaluation_data/'private')
        else:(evaluation_data/'private').mkdir()
        subprocess.run([sys.executable,'-B','-m','cumcm_harness.worker',str(root/'code'/verifier/evaluate.py'),
            '--input',str(evaluation_data),'--out',str(out/'evaluation'),'--answer',str(out/'solver'),*common],env
↪ =env,check=True,timeout=600)
```

```

    print((out/'evaluation/evaluation.json').read_text('utf-8'))
if __name__=='__main__':main()

```

源程序与运行资源

code/verifier/evaluate.py

```

"""Independent evaluator: no imports from MOSAIC or its simulator/metrics.
The exact reference enumerates this six-job finite domain only.
"""

from pathlib import Path
from itertools import permutations
import argparse,json
import numpy as np

def decode(x,n):
    active=[i for i in range(n) if x[i]>=.5]
    if not active:active=[max(range(n),key=lambda i:x[i])]
    return [i for i in sorted(range(n),key=lambda i:(x[n+i],i)) if i in active]

def objective(spec,order):
    n=len(spec['values']);previous=n;t=0.;reward=0.
    for j in order:
        t+=spec['setup'][previous][j]+spec['process'][j]
        reward+=max(0.,spec['values'][j]-spec['penalty'][j]*max(t-spec['dues'][j],0.));previous=j
    return [-reward,t]

def hv(F,r):
    y=r[1];area=0.
    for x,v in sorted(set(tuple(map(float,f)) for f in F)):
        if x>r[0] or v>r[1]:raise ValueError('reference does not dominate point')
        if v<y:area+=(r[0]-x)*(y-v);y=v
    return area

def evaluate_payload(spec,a,budget):
    n=len(spec['values']);X=np.asarray(a['X'],float);F=np.asarray(a['F'],float)
    LX=np.asarray(a['ledger_X'],float);LF=np.asarray(a['ledger_F'],float)
    if X.ndim!=2 or X.shape[1]!=2*n or F.shape!=(len(X),2) or not len(X):raise ValueError('bad front shapes')
    if LX.shape!=(budget,2*n) or LF.shape!=(budget,2) or a['budget_spent']!=budget:raise ValueError('paid ledger mismatch')
    if any(not np.isfinite(v).all() for v in (X,F,LX,LF)) or np.any(X<0) or np.any(X>1) or np.any(LX<0) or np.any(LX>1):raise
    ↪ ValueError('invalid values/bounds')
    expected=np.array([objective(spec,decode(x,n)) for x in LX])
    if not np.allclose(expected,LF,rtol=0,atol=1e-9):raise ValueError('paid objectives not independently reproducible')
    checks=[{'name':'independent_objective_recompute','passed':True,'detail':'all paid objective vectors recomputed in
    ↪ independent Python implementation'}}]
    for x,f in zip(X,F):
        if not np.allclose(objective(spec,decode(x,n)),f,rtol=0,atol=1e-9):raise ValueError('fabricated front objective')
        if not np.any(np.all(LX==x,axis=1)&np.all(np.isclose(LF,f,rtol=0,atol=1e-9),axis=1)):raise ValueError('front includes
        ↪ unpaid point')
        if np.any(np.all(LF<=f,axis=1)&np.any(LF<f-1e-9,axis=1)):raise ValueError('front contains dominated paid observation')
    checks.append({'name':'budget_and_archive','passed':True,'detail':'FE count and paid-observation archive validated'})
    T=sum(spec['process'])+n*max(max(row) for row in spec['setup'])+1;r=[1.,float(T)]
    exact=[objective(spec,p) for k in range(1,n+1) for p in permutations(range(n),k)]
    score=hv(F,r)/hv(exact,r)
    if not 0<=score<=1+1e-10:raise ValueError('impossible normalized hypervolume')
    index=a['selected_index']
    if type(index)!=int or not 0<=index<len(F):raise ValueError('invalid selection')

```

```

utility=-F[:,0]/sum(spec['values'])-F[:,1]/T
if index!=int(np.argmax(utility)):raise ValueError('recommendation violates frozen utility rule')
reward=-float(F[index,0]);duration=float(F[index,1])
checks.append({'name':'exact_finite_domain','passed':True,'detail':f'enumerated {len(exact)} feasible nonempty ordered
↳ subsets; not a global domain claim'})
return {'score':float(score),'valid':True,'metric':'normalized_hv','checks':checks,'question_coverage':['q1','q2'],
        'measurements': [{'id':'hv','value':float(score),'unit':'ratio','question_id':'q1','description':'normalized
↳ dominated hypervolume'},
        {'id':'reward','value':reward,'unit':'reward_units','question_id':'q2','description':'reward of representative
↳ selected schedule'},
        {'id':'duration','value':duration,'unit':'minutes','question_id':'q2','description':'duration of representative
↳ selected schedule'}]}

def invalid(message):
    return {'score':0.,'valid':False,'metric':'normalized_hv','checks': [{'name':'input_validation','passed':False,'detail':mes
↳ sage}],
        'question_coverage':['q1','q2'],'measurements': [{'id':'invalid','value':0.,'unit':None,'question_id':'q1','descr
↳ iption':'invalid answer, never accepted as a measured result'}]}

def main():
    p=argparse.ArgumentParser();p.add_argument('--input',type=Path,required=True);p.add_argument('--out',type=Path,required=Tr
↳ ue);p.add_argument('--answer',type=Path,required=True)
    p.add_argument('--seed',type=int,required=True);p.add_argument('--budget',type=int,required=True);p.add_argument('--varian
↳ t',required=True);a=p.parse_args()
    try:
        spec=json.loads((a.input/'public/problem.json').read_text());ans=json.loads((a.answer/'answer.json').read_text());resu
↳ lt=evaluate_payload(spec,ans,a.budget)
    except (KeyError,ValueError,TypeError,IndexError,FileNotFoundError) as e:result=invalid(type(e).__name__+' '+str(e))
    a.out.mkdir(parents=True,exist_ok=True);(a.out/'evaluation.json').write_text(json.dumps(result,allow_nan=False))
if __name__=='__main__':main()

```

源程序与运行资源

code/verifier/test_solver.py

```

"""Independent executable tests for the supplied synthetic demo evaluator."""
from pathlib import Path
import argparse,json,copy
from evaluate import evaluate_payload,decode,objective

def main():
    p=argparse.ArgumentParser();p.add_argument('--input',type=Path,required=True);p.add_argument('--out',type=Path,required=Tr
↳ ue);p.add_argument('--answer',type=Path,required=True)
    p.add_argument('--seed',type=int,required=True);p.add_argument('--budget',type=int,required=True);p.add_argument('--varian
↳ t',required=True);a=p.parse_args()
    spec=json.loads((a.input/'public/problem.json').read_text());ans=json.loads((a.answer/'answer.json').read_text());cases=[]
    assert
    ↳ evaluate_payload(spec,ans,a.budget)['valid'];cases.append({'name':'valid_paid_front','passed':True,'detail':'accepted
↳ independently recomputed baseline'})
    bad=copy.deepcopy(ans);bad['F'][0][0]=-10000
    try:evaluate_payload(spec,bad,a.budget)
    except ValueError:pass
    else:raise AssertionError('fabricated front accepted')
    cases.append({'name':'fabricated_objective_rejected','passed':True,'detail':'injected false reward triggers failure'})
    bad=copy.deepcopy(ans);bad['budget_spent']-=1

```

```

try:evaluate_payload(spec,bad,a.budget)
except ValueError:pass
else:raise AssertionError('false FE count accepted')
cases.append({'name':'wrong_FE_rejected','passed':True,'detail':'ledger accounting cannot be changed by a reported score'})
n=len(spec['values']);order=decode([0.]*(2*n),n)
assert order==[0] and objective(spec,order)[1]>0
cases.append({'name':'empty_mask_is_one_job','passed':True,'detail':'decoder feasible nonempty subset boundary verified'})
a.out.mkdir(parents=True,exist_ok=True);(a.out/'tests.json').write_text(json.dumps({'cases':cases,'all_passed':True}))
if __name__=='__main__':main()

```